

เกี่ยวกับหนังสือเล่มนี้

หนังสือเล่มนี้เรียบเรียงขึ้นเพื่อใช้เป็นเอกสารประกอบการเรียน และตำราสำหรับรายวิชา Discrete Mathematics ในบริบทของนักศึกษา ด้านคอมพิวเตอร์และวิศวกรรมคอมพิวเตอร์ โดยเน้นการอธิบายอย่างเป็นระบบ ตั้งแต่นิยาม ภาษาเชิงคณิตศาสตร์ เทคนิคการพิสูจน์ การเวียนเกิด ทฤษฎีกราฟ ไปจนถึงคอมบินาทอริกส์

เป้าหมายของหนังสือไม่ใช่เพียงให้ผู้อ่านจดจำสูตรหรือเทคนิคเฉพาะข้อ แต่เพื่อฝึกการมองเห็นโครงสร้างของปัญหา การให้เหตุผลอย่างรัดกุม และการออกแบบกระบวนการแก้ปัญหาอย่างตรวจสอบได้ ซึ่งเป็นทักษะสำคัญทั้งในคณิตศาสตร์ดิสครีตและในวิทยาการคอมพิวเตอร์โดยทั่วไป.

คณิตศาสตร์ดิสครีต
สำหรับคอมพิวเตอร์

Discrete Mathematics for Computer Study

ภาษาเชิงคณิตศาสตร์สำหรับคอมพิวเตอร์
การพิสูจน์และการให้เหตุผล
การเวียนเกิดและอุปนัย
ทฤษฎีกราฟ
คอมบินาทอริกส์

Final Draft
Handout version
Phaphontee Yamchote

คณิตศาสตร์ดิสครีตสำหรับการเขียนโปรแกรม
(Discrete Mathematics for Programming)

Phaphontee Yamchote

Final Draft
Handout version
Phaphontee Yamchote

คำนำ

หนังสือเล่มนี้เรียบเรียงขึ้นจากเอกสารประกอบการสอน lecture notes และ handouts ที่ใช้ในการสอนรายวิชาวิทยาการคอมพิวเตอร์ (Discrete Mathematics) สำหรับนักศึกษาสาขาวิศวกรรมคอมพิวเตอร์มหาวิทยาลัยเอเชียอาคเนย์ จุดเริ่มต้นของหนังสือไม่ได้มาจากความตั้งใจจะรวบรวมสูตรหรือทำสรุปย่อของเนื้อหา แต่เกิดจากความต้องการที่จะจัดระเบียบสิ่งที่สอนในชั้นเรียน ให้กลายเป็นข้อความที่อ่านต่อเนื่องได้ด้วยตนเอง มีลำดับการเล่าที่ชัด และสะท้อนวิธีคิดที่ผู้เรียนควรค่อย ๆ พัฒนาไปพร้อมกับเนื้อหา

ผู้เขียนเชื่อว่าคณิตศาสตร์ดีสครีตมีความสำคัญต่อผู้เรียนสายคอมพิวเตอร์ ไม่ใช่เพียงเพราะมันเป็นรายวิชาพื้นฐาน แต่เพราะมันฝึกสิ่งที่สำคัญมากกว่านั้น นั่นคือการนิยามสิ่งที่กำลังพูดให้ชัด การมองเห็นโครงสร้างที่ซ่อนอยู่ในปัญหา และการให้เหตุผลอย่างตรวจสอบได้ในหลายบริบทของการเรียนก่อนหน้า ผู้เรียนอาจคุ้นกับการแทนค่าสูตรเพื่อหาคำตอบ แต่เมื่อเข้าสู่คณิตศาสตร์ดีสครีต คำถามที่สำคัญมักไม่ใช่เพียงว่า “คำตอบคืออะไร” หากเป็นว่า “เรารู้ได้อย่างไรว่าคำตอบนั้นถูกต้อง” หรือ “เหตุใดนิยามนี้จึงเหมาะกับปัญหานี้” ดังนั้นหนังสือเล่มนี้จึงพยายามวางน้ำหนักไว้ที่ ภาษา โครงสร้าง และการพิสูจน์ มากกว่าการรวบรวมสูตรสำเร็จรูปเพียงอย่างเดียว

หนังสือเล่มนี้ไม่ได้ตั้งใจทำหน้าที่เป็นสารานุกรมที่เก็บรายละเอียดทุกหัวข้อให้ครบถ้วนที่สุด แต่ตั้งใจเป็นตำราที่พาผู้อ่านค่อย ๆ เข้าใจวิธีคิดของคณิตศาสตร์ดีสครีตทีละก้าวตามขั้นตอนการเรียนการสอนในห้องเรียน หลายแนวคิดจะปรากฏในระดับปูพื้นฐานก่อน แล้วจึงกลับมาลงลึกอีกครั้งในบทที่เฉพาะทางกว่า ด้วยเหตุนี้ บางคำหรือบางโครงสร้างอาจถูกกล่าวถึงในช่วงต้นอย่างย่อ ก่อนจะได้รับขยายความอย่างจริงจังภายหลัง วิธีจัดวางเช่นนี้เกิดจากความตั้งใจให้หนังสือมีจังหวะการเรียนรู้ที่เป็นธรรมชาติ มากกว่าการพยายามปิดทุกเรื่องให้จบในจุดที่มันปรากฏครั้งแรก

ในด้านการเขียน ผู้เขียนพยายามรักษาสมดุลระหว่างความเป็นทางการกับความอ่านรู้เรื่อง นิยามและทฤษฎีบทจะถูกเขียนให้รัดกุมเท่าที่จำเป็น แต่ในขณะเดียวกันก็พยายามมีคำอธิบายเชื่อมโยง มี intuition และมีการชี้จุดที่ผู้เรียนมักสับสน เพราะประสบการณ์จากการสอนจริงแสดงให้เห็นว่า อุปสรรคของผู้เรียนจำนวนมากไม่ได้อยู่ที่ความยากของสัญลักษณ์เพียงอย่างเดียว แต่อยู่ที่การไม่เห็นวามเชื่อมโยงของแต่ละนิยามมีไว้เพื่ออะไร แต่ละวิธีพิสูจน์เหมาะกับสถานการณ์แบบใด และแต่ละสูตรเกิดขึ้นจากโครงสร้างของปัญหาอย่างไร

ผู้เขียนหวังว่าหนังสือเล่มนี้จะช่วยให้ผู้อ่านค่อย ๆ เปลี่ยนวิธีมองคณิตศาสตร์ จากการมองว่าเป็นชุดของเทคนิคที่ต้องจำให้ถูกข้อ ไปสู่การมองว่าเป็นระบบของความคิดที่มีระเบียบ ถ้าผู้อ่านอ่านหนังสือเล่มนี้แล้ว เริ่มตั้งคำถาม

กับนิยามมากขึ้น อ่านโครงสร้างของข้อความได้ดีขึ้น มองการนับว่าเป็นการวางแผน หรือเริ่มเห็นว่าการพิสูจน์ไม่ใช่การเขียนข้อความยาว ๆ แต่เป็นการเชื่อมสิ่งที่รู้ไปสู่สิ่งที่ต้องการอย่างมีหลักการ ผู้เขียนก็จะถือว่าหนังสือเล่มนี้ทำหน้าที่ของมันได้แล้ว

ท้ายที่สุดนี้ ผู้เขียนหวังว่าหนังสือเล่มนี้จะเป็นประโยชน์ทั้งต่อผู้ที่กำลังเรียนรายวิชานี้เป็นครั้งแรก และต่อผู้ที่ต้องการทบทวนรากฐานของการให้เหตุผลเชิงตรรกศาสตร์อีกครั้ง หากมีส่วนใดที่ยังอธิบายได้ชัดกว่านี้ หรือมีตัวอย่างใดที่ควรเพิ่มเติม ผู้เขียนยินดีอย่างยิ่งหากหนังสือเล่มนี้仍将คงถูกปรับปรุงต่อไป เพราะโดยธรรมชาติแล้ว หนังสือที่เติบโตมาจากการสอนจริง ย่อมควรเติบโตต่อไปพร้อมกับคำถามและประสบการณ์ของผู้อ่านด้วย

ปพนธีร์ แยมโชติ

สารบัญ

คำนำ

i

I	คณิตศาสตร์พื้นฐานทั้งหมดสำหรับดิสครีต	1
1	Mathematical Language, Logic, and Proof	3
1.1	คณิตศาสตร์ในฐานภาษา	4
1.2	เซต: การพูดถึงกลุ่มและความเป็นสมาชิก	4
1.3	ตรรกศาสตร์: โครงสร้างของประโยคและการให้เหตุผล	8
1.4	ตัวบ่งปริมาณ บริบท และตรรกศาสตร์อันดับหนึ่ง	13
1.5	ความสัมพันธ์และฟังก์ชัน: การอธิบายการเชื่อมโยงระหว่างสิ่งของ	14
1.5.1	ความสัมพันธ์	14
1.5.2	ฟังก์ชัน	14
1.6	การอ่านข้อความคณิตศาสตร์ให้เป็นโครงสร้าง	15
1.7	การให้เหตุผลทางคณิตศาสตร์และการพิสูจน์	16
1.8	บทบาทของบทนี้ต่อบทถัดไป	20
II	พื้นฐานการพิสูจน์และการให้เหตุผลทางคณิตศาสตร์	23
2	ตรรกศาสตร์ประพจน์และโครงสร้างของการพิสูจน์	25
2.1	ประพจน์และประโยคเปิด	25
2.1.1	ประพจน์	25
2.1.2	Predicate หรือประโยคเปิด	26
2.1.3	จากประโยคเปิดสู่ประพจน์	26
2.2	เอกภพสัมพัทธ์และข้อตกลงพื้นฐาน	27
2.3	การพิสูจน์คืออะไร	28
2.3.1	ทำไมการผ่าน test cases ไม่ใช่การพิสูจน์	28

2.4	นิยามพื้นฐานที่ใช้ฝึกพิสูจน์	29
2.5	โครงสร้างมาตรฐานของการพิสูจน์แบบอิมพลีเคชัน	29
2.5.1	ตัวอย่างต้นแบบ: ถ้าจำนวนคี่แล้วยกกำลังสองยังคงเป็นคี่	30
2.5.2	อ่านพิสูจน์อย่างนักเขียนพิสูจน์	31
2.5.3	อีกหนึ่งตัวอย่างสั้น: ถ้าจำนวนคู่แล้วยกกำลังสองยังคงเป็นคู่	31
2.6	ตัวอย่างค่านและการปฏิเสธข้อความแบบ "สำหรับทุก"	31
2.7	การเขียนพิสูจน์ให้เหมือนหนังสือ มากกว่าเหมือนแบบเติมคำ	32
2.8	ตัวเชื่อมตรรกะและสูตรตรรกะ	32
2.9	อิมพลีเคชัน: ความหมายของข้อความรูป $P \rightarrow Q$	34
2.9.1	อิมพลีเคชันในฐานะคำสัญญา	34
2.9.2	อิมพลีเคชันเป็นเท็จได้เมื่อใด	34
2.9.3	ความจริงแบบว่าง	35
2.9.4	ถ้อยคำที่มักแปลผิด	35
2.10	จากรูปประโยคสู่รูปแบบการพิสูจน์	36
2.11	สมมูลตรรกะที่มีประโยชน์ต่อการพิสูจน์	36
2.11.1	Contrapositive	37
2.11.2	De Morgan	37
2.11.3	อิมพลีเคชันในรูปดิสจังก์ชัน	38
2.12	การพิสูจน์แบบ contradiction	38
2.13	การใช้อิมพลีเคชันที่พิสูจน์แล้ว: Modus Ponens	39
2.14	สรุปส่งท้ายอิมพลีเคชัน	40
2.15	ดิสจังก์ชันและการพิสูจน์แบบแยกกรณี	41
2.15.1	ความหมายของ $P \vee Q$	41
2.15.2	การพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง	41
2.15.3	การใช้ $P \vee Q$: proof by cases	42
2.16	ไบคอนดิชันนัลและการพิสูจน์สองทิศ	43
2.17	ข้อผิดพลาดที่พบบ่อยในการเขียนพิสูจน์	44
2.17.1	พิสูจน์ converse แทนที่จะพิสูจน์ข้อความเดิม	45
2.17.2	พิสูจน์ iff เพียงทิศเดียว	45
2.17.3	ใช้ proof by cases แบบไม่ครบทุกกรณี	45
2.17.4	พิสูจน์แบบ contradiction โดยสมมติผิดตัว	45
2.18	สรุป	45
3	ตัวบ่งปริมาณและแพตเทิร์นพื้นฐานของการพิสูจน์	49
3.1	ตัวบ่งปริมาณและเอกภพสัมพัทธ์	49
3.2	เมื่อเป้าหมายเป็น $\forall$: เลือกสมาชิกแบบ arbitrary	50
3.3	เมื่อเป้าหมายเป็น $\exists$: ระบุพยาน	51

3.4	หลายตัวบ่งปริมาณ: ลำดับมีผล	52
3.4.1	แนวคิดเชิงภาษา	52
3.4.2	ตัวอย่างเชิงจำนวน	53
3.4.3	ตัวอย่างเชิงระบบคอมพิวเตอร์	54
3.5	การปฏิเสธตัวบ่งปริมาณ	54
3.6	ตัวอย่างทฤษฎีบทผสม $\forall \exists$	55
3.7	ตัวอย่างทฤษฎีบทแบบ $\exists \forall$	57
3.8	การหักล้างข้อความที่มีตัวบ่งปริมาณ	57
3.9	ตัวอย่างเสริมเกี่ยวกับการแปลความหมาย	58
3.10	สรุปประเด็นของบทนี้	59
3.11	ข้อผิดพลาดที่พบบ่อยก่อนเริ่มพิสูจน์จริง	59
4	โครงสร้างทฤษฎี: เซต ความสัมพันธ์ และฟังก์ชัน	65
4.1	เซต: ภาษาของการเป็นสมาชิกและการรวมกลุ่ม	66
4.1.1	ความเท่ากันของเซต	66
4.1.2	เซตย่อย	67
4.1.3	ข้อควรระวังเรื่องการนิยามเซต	68
4.1.4	เซตว่าง	68
4.1.5	เซตกำลัง	69
4.1.6	การดำเนินการของเซต	70
4.1.7	ผลคูณคาร์ทีเซียน	72
4.2	ความสัมพันธ์: การอธิบายว่าอะไรเกี่ยวข้องกับอะไร	72
4.2.1	สมบัติสำคัญของความสัมพันธ์	73
4.2.2	ความสัมพันธ์สมมูล	74
4.2.3	ความสัมพันธ์อันดับบางส่วน	75
4.3	ฟังก์ชัน: ความสัมพันธ์ที่กำหนดผลลัพธ์อย่างแน่นอน	75
4.3.1	ฟังก์ชันหนึ่งต่อหนึ่ง ทั้งถึง และพหุภาพ	76
4.3.2	ฟังก์ชันประกอบและฟังก์ชันผกผัน	77
4.4	จากความสัมพันธ์สู่การนับ: ขนาดของเซต	78
4.4.1	ทฤษฎีบทของแคนเทอร์	78
4.5	สรุปภาพรวมของบท	80
III	การแก้ปัญหาเชิงกระบวนการและการเวียนเกิด	81
5	แนวคิดพื้นฐานของการเวียนเกิด	83
5.1	Recursion	83

5.1.1	Recursion ในฐานะรูปแบบของการแก้ปัญหา	84
5.1.2	เปรียบเทียบกับการทำงานแบบวนซ้ำ	85
5.2	ตัวอย่างอุปกรณ์: การนิยามลำดับแบบเวียนเกิด	85
5.2.1	ลำดับเลขคณิต	85
5.2.2	ลำดับฟีโบนากชี	86
5.3	Recursion ในฐานะขั้นตอนวิธี: Tower of Hanoi	87
5.3.1	กติกาของปัญหา	87
5.3.2	เริ่มจากกรณี 4 แผ่น	87
5.3.3	รูปแบบทั่วไปสำหรับ n แผ่น	87
5.3.4	จำนวนครั้งที่ต้องย้าย	88
5.4	อีกตัวอย่างหนึ่ง: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น	89
5.5	การนับจำนวนวิธีด้วยการแยกกรณี: จำนวนเซตย่อย	89
5.6	สรุปภาพรวมของการตั้งความสัมพันธ์เวียนเกิด	90
5.7	ข้อควรระวัง	91
5.7.1	มี recursive step แต่ไม่มี base case	91
5.7.2	มี base case แต่ยังไม่เพียงพอ	92
5.7.3	ปัญหาไม่เล็กลงจริง	93
5.7.4	ลดขนาดลงแล้ว แต่ไม่รับประกันว่าจะไปถึงฐาน	93
5.7.5	ไม่ระบุโดเมนหรือเงื่อนไขของ input ให้ชัด	94
5.7.6	สูตรไม่ครอบคลุมทุกกรณีที่ควรเกิดขึ้น	94
5.7.7	สูตรไม่สะท้อนโครงสร้างจริงของปัญหา	95
5.7.8	สับสนระหว่าง recursion กับสูตรปิด	95
5.7.9	คิดว่าเขียน recursion ได้แล้ว แปลว่าถูกต้องแล้ว	96
6	Mathematical Induction	99
6.1	Ordinary Induction	100
6.1.1	หลักการพื้นฐาน	100
6.1.2	สิ่งที่ $P(n)$ เป็น และสิ่งที่มันไม่ใช่	100
6.1.3	ข้อผิดพลาดที่พบบ่อย	101
6.2	ตัวอย่างแรก: ผลบวกของจำนวนเต็มตัวแรก	104
6.2.1	สิ่งที่ตัวอย่างนี้สอนเรา	106
6.2.2	ผลบวกกำลังสอง	106
6.3	อีกสองตัวอย่างของ ordinary induction	108
6.3.1	ผลบวกของจำนวนคี่	108
6.3.2	ตัวอย่างสมการ	109
6.4	Induction กับสูตรปิดของลำดับที่นิยามแบบเวียนเกิด	110
6.4.1	ลำดับเลขคณิต	110

6.4.2	ลำดับเรขาคณิต	111
6.5	อีกตัวอย่างหนึ่งที่มีโครงสร้างมากขึ้น: การปูบอร์ดด้วย L-tromino	112
6.6	จาก ordinary induction สู่ structural induction	113
6.7	Structural Induction	114
6.7.1	หลักทั่วไป	114
6.7.2	การพิสูจน์ความเท่ากันของเซต $R = A$	114
6.7.3	ตัวอย่าง: เซตของกำลังของ 2	115
6.7.4	ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 จากกฎ $x \mapsto x + 5$	116
6.7.5	ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 ที่ปิดภายใต้การบวก	117
6.7.6	ตัวอย่าง: คุณสมบัติของสมาชิกทุกตัวในเซตที่สร้างแบบ recursion	119
6.7.7	ตัวอย่างกับ bit string	121
6.8	จาก structural induction สู่ strong induction	122
6.9	Strong Induction	122
6.9.1	หลักการ	122
6.9.2	เมื่อไรควรใช้ strong induction	123
6.9.3	ตัวอย่าง: recurrence ที่ย้อนสองขั้น	123
6.9.4	ตัวอย่าง: หักข้อโกแลตบาร์	124
6.9.5	ตัวอย่าง: แพ็กเหรียญขนาด 4 และ 5	125
6.9.6	ตัวอย่างเพิ่ม: การแยกตัวประกอบเป็นจำนวนเฉพาะ	126
6.10	สรุปความสัมพันธ์ของสามแนวคิด	128
6.11	ความสมมูลกันของ ordinary and strong induction	128
6.11.1	Strong induction $\Rightarrow$ Ordinary induction	129
6.11.2	Ordinary induction $\Rightarrow$ Strong induction	130
6.11.3	แล้วเหตุใดเรายังแยก ordinary กับ strong induction ออกจากกัน?	132
6.11.4	ความสัมพันธ์กับ structural induction	132
6.12	บทสรุปของ chapter นี้	133
7	Recursive Correctness Proof	137
7.1	Specification, Termination, and Correctness	137
7.2	Divide-and-Conquer: Merge Sort	138
7.2.1	แนวคิด Divide--Conquer--Combine	138
7.2.2	นิยาม sorted และสเปกของ MergeSort	139
7.2.3	Pseudocode ของ MergeSort และ Merge	140
7.3	พิสูจน์ความถูกต้องของ Merge	140
7.4	พิสูจน์ความถูกต้องของ MergeSort	142
7.5	Recursive Binary Search	144
7.5.1	นิยามและสเปก	144

7.5.2	Pseudocode	145
7.5.3	Termination	145
7.5.4	Correctness	146
7.6	ตัวอย่าง recursive algorithm เพิ่มเติม	149
7.6.1	Power	149
7.6.2	SumList	151
7.7	Pattern Summary	152
8	Recurrent Relation	155
8.1	Introduction to Recurrence Relations	155
8.1.1	ตัวอย่างที่ 1: recurrence แบบง่าย	156
8.1.2	ตัวอย่างที่ 2: Merge Sort recurrence (กรณี n เป็นกำลังของ 2)	157
8.1.3	Plug-and-chug สำหรับ Merge Sort	158
8.2	Binary Search recurrence	161
8.2.1	การวิเคราะห์	161
8.2.2	Plug-and-chug	162
8.3	From Recurrence to Closed Form and Back Again	163
IV	ทฤษฎีกราฟเบื้องต้น	165
9	ทฤษฎีกราฟเบื้องต้น	167
9.1	กราฟอย่างง่ายและภาษาพื้นฐาน	167
9.1.1	Simple graph, vertex, edge	168
9.1.2	ดีกรีของจุดยอด	169
9.1.3	Handshake Lemma และการนับแบบสองทาง	170
9.2	Walks, Paths, Cycles, and Connectivity	172
9.2.1	Walk, path, closed walk, cycle	172
9.2.2	จาก walk ไปสู่ path	174
9.2.3	Connected graph และ connected component	175
9.3	Bridges	178
9.3.1	Bridge และความสัมพันธ์กับ cycle	178
10	Trees	181
10.1	Forests และ Trees	181
10.1.1	Forest และ tree	182
10.1.2	คุณสมบัติพื้นฐานของต้นไม้	183

10.2	Characterizations of Trees	186
10.2.1	Path uniqueness	187
10.2.2	$ E = V - 1$	187
10.2.3	Minimal connected graphs	189
10.2.4	Maximal acyclic graphs	190
10.2.5	ความสมมูลของ characterization ต่าง ๆ ของ tree	191
11	Algebra ของ Graph	195
11.1	Adjacency Matrices	195
11.1.1	Adjacency matrix ของกราฟไม่มีทิศทาง	196
11.1.2	การอ่านข้อมูลกราฟจากเมทริกซ์	198
11.1.3	กำลังของ adjacency matrix และการนับจำนวน walks	199
11.2	Directed Graphs	202
11.2.1	Directed graph และ ordered pairs	203
11.2.2	Adjacency matrix ของ directed graph	204
11.2.3	Directed walks และการตีความเมทริกซ์	205
11.3	ความสัมพันธ์ในมุมมองของกราฟมีทิศทาง	208
11.3.1	Relation as a subset of $A \times A$	209
11.3.2	Directed graph representation ของ relation	210
11.3.3	0--1 matrix representation ของ relation	211
11.3.4	Composition และ powers of relations	212
11.3.5	Boolean product กับความหมายแบบมี/ไม่มี	214
V	คอมบินาทอริกเบื้องต้น	217
12	กฎการนับเบื้องต้น	219
12.1	บทนำ: การนับในฐานะการวางแผน	220
12.1.1	การนับไม่ใช่การจำสูตร	220
12.1.2	มุมมองแบบ set-theoretic และการนับจำนวนสมาชิก	220
12.1.3	โจทย์การนับกับการออกแบบกระบวนการนับ	221
12.2	หลักการนับพื้นฐาน	222
12.2.1	หลักการบวก	222
12.2.2	หลักการคูณ	223
12.2.3	การนับผ่าน Cartesian product	224
12.2.4	การแยกกรณีและการนับแบบเป็นขั้นตอน	225
12.2.5	ข้อควรระวัง: การนับซ้ำและการแบ่งกรณีไม่ครบ	225

12.3	Permutations: การเรียงสับเปลี่ยน	226
12.3.1	การเรียงสับเปลี่ยนเชิงเส้นของสิ่งที่ไม่ซ้ำกัน	226
12.3.2	จำนวนวิธีเรียงสับเปลี่ยน $P(n, r)$	227
12.3.3	การเรียงสับเปลี่ยนเต็มชุดและแพกทอเรียล	228
12.3.4	การเรียงสับเปลี่ยนแบบวงกลม	229
12.3.5	การเรียงสับเปลี่ยนของสิ่งที่มีของซ้ำ	230
12.3.6	โจทย์ประยุกต์ของ permutations	231
12.4	Combinations: การจัดกลุ่ม	231
12.4.1	ความแตกต่างระหว่างการเรียงกับการเลือก	232
12.4.2	จำนวนวิธีจัดกลุ่ม $\binom{n}{r}$	232
12.4.3	ความสัมพันธ์ระหว่าง permutations และ combinations	233
12.4.4	การเลือกภายใต้เงื่อนไข	233
12.4.5	การแบ่งคนหรือสิ่งของออกเป็นกลุ่ม	234
12.4.6	ข้อควรระวัง: กลุ่มที่มีป้ายชื่อและไม่มีป้ายชื่อ	235
12.5	Combinatorial Proofs and Double Counting	235
12.5.1	การพิสูจน์ด้วยการนับสองวิธี	236
12.5.2	$\binom{n}{r} = \binom{n}{n-r}$	237
12.5.3	Pascal identity	238
12.5.4	เอกลักษณ์ที่เกิดจากการตีความโจทย์เดียวกันสองแบบ	239
12.5.5	ตัวอย่างของ double counting ในปัญหาการจัดเรียงและการเลือก	240
13	หัวข้อเพิ่มเติมเกี่ยวกับการนับ	243
13.1	Binomial Coefficients and the Binomial Theorem	244
13.1.1	สัมประสิทธิ์ทวินามในฐานะจำนวนวิธีเลือก	244
13.1.2	ทฤษฎีบททวินาม	245
13.1.3	การอ่านสัมประสิทธิ์จากการกระจายพหุนาม	246
13.1.4	เอกลักษณ์พื้นฐานของสัมประสิทธิ์ทวินาม	248
13.1.5	การใช้ทฤษฎีบททวินามในการพิสูจน์เอกลักษณ์เชิงการจัด	249
13.2	เส้นทางบนตารางและการตีความแบบ combinations	251
13.2.1	การนับ lattice paths	252
13.2.2	การบังคับให้ผ่านจุดที่กำหนด	253
13.2.3	การบังคับให้ผ่านเส้นหรือช่วงที่กำหนด	254
13.2.4	การแปลปัญหาเส้นทางให้เป็นปัญหาการเลือกตำแหน่ง	255
13.3	Distribution Problems: โจทย์การแจกของ	256
13.3.1	ทำไมโจทย์การแจกจึงเป็นหัวข้อถัดจาก combinations	257
13.3.2	มุมมองของโจทย์การแจก: ฟังก์ชัน เซตย่อย ลำดับ และคำตอบจำนวนเต็ม	257
13.3.3	แจกของที่แตกต่างกันให้คนที่แตกต่างกัน	258

13.3.4	แจกของที่แตกต่างกันโดยมีเงื่อนไขเรื่องคนต้องได้ของหรือห้ามได้ของ	259
13.3.5	แจกของที่เหมือนกันให้คนที่แตกต่างกัน	260
13.3.6	Stars and Bars และการนับคำตอบจำนวนเต็มไม่ลบ	261
13.3.7	กรณีที่ต้องมีอย่างน้อยหนึ่งชิ้น: การลดรูปด้วยการหัก quota ขั้นต่ำ	262
13.3.8	กรณีมีขอบเขตบน: การแยกกรณีและการตีความใหม่	263
13.3.9	แจกของที่เหมือนกันลงกลุ่มที่ไม่ distinguish กัน	265
13.3.10	Multisets, weak compositions, และมุมมองที่เชื่อมโยงกัน	265
13.3.11	โจทย์ประยุกต์ของการแจกของ	267
13.4	สรุปภาพรวมของบท	268
13.4.1	แผนที่ความคิด: บวก คูณ เรียง เลือก ตัดซ้ำ รับประกันการมีอยู่	269
13.4.2	เทคนิคที่ควรเลือกใช้ในสถานการณ์ต่าง ๆ	269
VI	ส่งท้าย	273
	บทส่งท้าย	275
	เกี่ยวกับผู้เขียน	281

Final Draft
Handout version
Phaphontee Yamchote

ภาค I

คณิตศาสตร์พื้นฐานทั้งหมดสำหรับดีสคริต

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 1

Mathematical Language, Logic, and Proof

บทนี้มีหน้าที่เป็นบทปูพื้นก่อนเข้าสู่เนื้อหาหลักของรายวิชา เป้าหมายไม่ใช่การลงรายละเอียดทุกเรื่องให้จบในบทเดียว แต่เป็นการทำให้ผู้อ่านมองเห็นว่า คณิตศาสตร์ในหนังสือเล่มนี้เป็นเรื่องของ ภาษา โครงสร้าง และ การให้เหตุผล มากกว่าการแทนค่าสูตรเพื่อหาคำตอบเชิงคำนวณเพียงอย่างเดียว

ในห้องเรียนจริง ผู้สอนอาจไม่ได้หยุดอธิบายละเอียดตั้งแต่ต้นว่า เซตคืออะไร ประพจน์คืออะไร หรือความหมายของตัวเชื่อมตรรกศาสตร์แต่ละตัวคืออะไร เพราะเวลาในชั้นเรียนต้องถูกใช้กับการฝึกคิด ฝึกโต้แย้ง และฝึกพิสูจน์เป็นหลัก ดังนั้นบทนี้จึงทำหน้าที่เป็น ``พื้นหลังที่ควรรู้'' เพื่อให้ผู้อ่านอ่านนิยาม อ่านข้อความทางคณิตศาสตร์ และตามการพิสูจน์ในบทถัด ๆ ไปได้คล่องขึ้น

อีกประการหนึ่ง บางหัวข้อในบทนี้จะถูกนำกลับไปใช้และลงรายละเอียดอีกครั้งในบทภายหลัง เช่น เรื่องการพิสูจน์จะกลับมาอย่างจริงจังในบทที่เน้น *proof techniques*, เรื่อง induction จะมีบทของมันเป็นเอง, ส่วน recurrence, recursive correctness, relation, function, และโครงสร้างอื่น ๆ ก็จะกลับมาในบริบทที่สำคัญกว่าเดิมภายหลังเช่นกัน ดังนั้นเวลาที่อ่านบทนี้ ให้มองว่ากำลังเรียนรู้ ``ภาพรวมของภาษา" ที่เราจะใช้ทั้งหมด

เป้าหมายของบทนี้ เมื่ออ่านบทนี้จบ ผู้อ่านควรจะ

- มองออกว่าคณิตศาสตร์ใช้สัญลักษณ์เพื่ออธิบายสิ่งต่าง ๆ อย่างไร
- อ่านข้อความที่มีเซต ตัวเชื่อมตรรกศาสตร์ และตัวบ่งปริมาณได้ดีขึ้น
- เห็นความต่างระหว่าง ``การยกตัวอย่าง" กับ ``การพิสูจน์"
- เข้าใจโครงสร้างพื้นฐานของการให้เหตุผลทางคณิตศาสตร์
- พร้อมสำหรับบทถัดไปที่เน้นการพิสูจน์จริงมากขึ้น

1.1 คณิตศาสตร์ในฐานะภาษา

ผู้เรียนจำนวนไม่น้อยเติบโตมากับภาพจำว่า คณิตศาสตร์คือวิชาของการคิดเลข การจำสูตร และการแทนค่าตามแบบฝึกหัด ภาพจำเช่นนี้ไม่ได้ผิดทั้งหมด แต่ก็ไม่ใช่ภาพรวมของคณิตศาสตร์เช่นกัน โดยเฉพาะเมื่อเราเข้าสู่คณิตศาสตร์ไม่ต่อเนื่อง หรือวิชาที่เกี่ยวกับโครงสร้างเชิงนามธรรมและการพิสูจน์ เราจะพบว่า “การคำนวณ” ไม่ใช่หัวใจของเรื่องอีกต่อไป สิ่งที่สำคัญกว่าคือการอธิบายให้ชัดว่า เรากำลังพูดถึงอะไร เราอนุญาตให้ใช้อะไรได้บ้าง และเหตุใดข้อสรุปหนึ่งจึงตามมาจากสิ่งที่รู้อยู่แล้วได้อย่างสมเหตุสมผล

กล่าวอีกแบบหนึ่ง คณิตศาสตร์เป็นภาษาชนิดหนึ่ง เป็นภาษาที่ใช้บรรยายวัตถุ โครงสร้าง ความสัมพันธ์ และกฎเกณฑ์อย่างรัดกุม เมื่อเราบอกว่า “ a เป็นสมาชิกของเซต A ” หรือ “ถ้า n เป็นจำนวนคู่แล้ว n^2 เป็นจำนวนคู่” สิ่งที่เรากำลังทำไม่ใช่การคำนวณ แต่คือการใช้ภาษาเชิงสัญลักษณ์เพื่อสื่อความหมายอย่างชัดเจน และเมื่อภาษาเริ่มชัดเจน การให้เหตุผลและการพิสูจน์ก็จะชัดเจนตามไปด้วย

ในบทนี้ เราจะค่อย ๆ แนะนำองค์ประกอบสำคัญของภาษานี้ทีละส่วน เริ่มจาก เซต ซึ่งใช้พูดถึง “ความเป็นกลุ่ม” และ “ความเป็นสมาชิก” ต่อด้วย ตรรกศาสตร์ ซึ่งใช้ควบคุมโครงสร้างของประโยคและการอนุมาน จากนั้นจึงพูดถึง ความสัมพันธ์ และ ฟังก์ชัน ซึ่งเป็นเครื่องมือมาตรฐานในการอธิบายการเชื่อมโยงระหว่างวัตถุ และปิดท้ายด้วยภาพรวมของ การให้เหตุผลและการพิสูจน์ ซึ่งเป็นทักษะหลักของหนังสือเล่มนี้

1.2 เซต: การพูดถึงกลุ่มและความเป็นสมาชิก

หนึ่งในเหตุผลที่ทำให้คณิตศาสตร์ถูกมองว่าเป็น “ภาษา” ก็คือ มันพยายามหาวิธีอธิบายสิ่งต่าง ๆ ให้กระชับ ชัดเจน และลดความกำกวมของภาษามนุษย์ แนวคิดเรื่อง เซต เป็นตัวอย่างที่ดีมากของเรื่องนี้ เพราะเซตคือเครื่องมือพื้นฐานที่ใช้พูดถึง “กลุ่มของสิ่งที่เราสนใจ” รวมไปถึงการบอกว่า “อะไรอยู่ในกลุ่มนั้น” และ “กลุ่มหนึ่งอยู่ข้างในอีกกลุ่มหนึ่งหรือไม่”

ในภาษามนุษย์ เรามักพูดประโยคอย่างเช่น “ a เป็นนักเรียน” หรือ “นักเรียนทุกคนเป็นบุคลากรของโรงเรียน” ซึ่งในชีวิตประจำวันเราพูดกันได้โดยไม่ต้องระวังมากนัก แต่ถ้าต้องการเขียนให้เป็นภาษาคณิตศาสตร์ เราจะพยายามทำให้ข้อความเหล่านี้อยู่ในรูปที่มีโครงสร้างชัดเจน

การเป็นสมาชิกของเซต

สมมติว่าเรามีเซต S แทน “กลุ่มของนักเรียน” ถ้าเราต้องการกล่าวว่า “ a เป็นนักเรียน” เราจะเขียนเป็นสัญลักษณ์ว่า

$$a \in S.$$

สัญลักษณ์ $\in$ อ่านว่า “เป็นสมาชิกของ” ดังนั้นประโยค $a \in S$ จึงเป็นวิธีเขียนแทนข้อความ “ a เป็นสมาชิกของกลุ่มนักเรียน” หรือก็คือ “ a เป็นนักเรียน” นั่นเอง

ในทำนองเดียวกัน ถ้า $a \notin S$ ก็แปลว่า a ไม่ได้อยู่ในกลุ่มนักเรียน

สิ่งสำคัญที่ควรเห็นตั้งแต่แรกคือ เซตไม่ได้เกิดขึ้นมาเพื่อให้เราคำนวณหาคำตอบแบบตัวเลขเสมอไป แต่เกิดขึ้นมาเพื่อช่วยจัดระเบียบความคิดว่า ตอนนี้เรากำลังพูดถึง “กลุ่มของใคร” อยู่ และสิ่งที่เรากล่าวถึงนั้น “อยู่ในกลุ่มนั้นหรือไม่”

เซตย่อย: การอธิบายว่ากลุ่มหนึ่งอยู่ในอีกกลุ่มหนึ่ง

นอกจากการบอกว่าสิ่งใดเป็นสมาชิกของกลุ่มแล้ว เรายังต้องการบอกด้วยว่า บางกลุ่มเป็นส่วนหนึ่งของอีกกลุ่มหนึ่ง ตัวอย่างเช่น ถ้าเรามีเซต X แทน “กลุ่มของบุคลากรของโรงเรียน” และมีเซต S แทน “กลุ่มของนักเรียน” เรามักต้องการอธิบายว่า นักเรียนทุกคนก็เป็นบุคลากรของโรงเรียนในความหมายกว้างเช่นกัน ซึ่งเขียนแทนได้ว่า

$$S \subseteq X.$$

สัญลักษณ์ $\subseteq$ อ่านว่า “เป็นเซตย่อยของ”

เพื่อให้ความหมายของสัญลักษณ์นี้ชัดเจน เรานิยามอย่างเป็นทางการดังนี้

Definition 1.2.1. เซตย่อย ให้ A และ B เป็นเซต เราจะกล่าวว่า A เป็นเซตย่อยของ B และเขียนว่า $A \subseteq B$ ก็ต่อเมื่อ สำหรับทุก x ถ้า $x \in A$ แล้ว $x \in B$

แม้นิยามนี้จะดูเหมือนเป็นเพียงนิยามของเรื่องเซต แต่แท้จริงแล้วมันเริ่มเผยให้เห็นบทบาทของตรรกศาสตร์แล้ว เพราะคำว่า “สำหรับทุก” และ “ถ้า...แล้ว...” เป็นโครงสร้างทางตรรกศาสตร์โดยตรง กล่าวอีกแบบคือ แม้เราจะกำลังเรียนเรื่องเซต แต่วิธีนิยามของคณิตศาสตร์ก็เริ่มพึ่งตรรกศาสตร์ตั้งแต่ตรงนี้แล้ว

จากนิยามข้างต้น ถ้าเรารู้สองอย่างคือ

$$a \in S \quad \text{และ} \quad S \subseteq X,$$

เราก็สามารถสรุปต่อได้ว่า

$$a \in X.$$

ในภาษามนุษย์ก็คือ ถ้า a เป็นนักเรียน และนักเรียนทุกคนเป็นบุคลากรของโรงเรียน ดังนั้น a ก็เป็นบุคลากรของโรงเรียนด้วย

ตัวอย่างนี้สำคัญเพราะแสดงให้เห็นว่า เซตไม่ได้เป็นเพียงเครื่องมือสำหรับ “เก็บสมาชิก” แต่ยังช่วยให้เราพูดถึงการอนุมานเกี่ยวกับความเป็นสมาชิกได้ด้วย อย่างไรก็ตาม กลไกที่ทำให้การอนุมานนี้สมเหตุสมผลจริง ๆ จะเป็น

หน้าที่ของตรรกศาสตร์ ซึ่งเราจะกล่าวถึงในหัวข้อถัดไป

การใช้เซตเพื่อเขียนข้อความให้กระชับขึ้น

ในหลายสถานการณ์ เราไม่ได้สนใจเพียงการเป็นสมาชิกของกลุ่มเดียว แต่สนใจการอยู่ร่วมกันของหลายกลุ่มพร้อมกัน ตัวอย่างเช่น สมมติว่า S เป็นเซตของนักเรียน ให้ C เป็นเซตของนักศึกษาที่ลงทะเบียนเรียนวิชาทฤษฎีสารสนเทศ และ D เป็นเซตของนักศึกษาที่ลงทะเบียนเรียนวิชาโครงสร้างข้อมูล

ถ้าเราต้องการกล่าวว่า “ a เป็นนักศึกษาที่ลงทะเบียนทั้งวิชาทฤษฎีสารสนเทศและวิชาโครงสร้างข้อมูล” เราสามารถเขียนตรง ๆ ว่า

$$a \in C \text{ และ } a \in D.$$

แต่ในโลกของเซต เรามีเครื่องมือที่ทำให้ข้อความนี้สั้นและเป็นระบบมากขึ้น นั่นคือ อินเตอร์เซกชัน (intersection) ซึ่งเขียนว่า

$$C \cap D.$$

และตีความว่าเป็น “กลุ่มของสมาชิกที่อยู่ในทั้ง C และ D พร้อมกัน”

ดังนั้นข้อความข้างต้นจึงเขียนได้กระชับขึ้นเป็น

$$a \in C \cap D.$$

ตรงนี้เองที่นำสังเกตว่า ข้อความในภาษามนุษย์ที่มีคำว่า “และ” ค่อย ๆ ถูกเปลี่ยนให้มาอยู่ในรูปของเซตได้ และในรูปใหม่นี้ ตัวหลักของประโยคไม่ใช่คำเชื่อมแล้ว แต่กลายเป็น “การเป็นสมาชิกของเซต” แทน ซึ่งเป็นรูปแบบที่เหมาะสมกับการทำงานทางคณิตศาสตร์มากกว่า

ในทำนองเดียวกัน ถ้าเราต้องการพูดถึงสมาชิกที่อยู่ในอย่างน้อยหนึ่งในสองกลุ่ม เราจะใช้ ยูเนียน (union) เขียนเป็น

$$C \cup D.$$

เซตนี้หมายถึงสมาชิกที่อยู่ใน C หรืออยู่ใน D หรืออยู่ทั้งสองเซต

เซตในฐานะภาษาสำหรับจัดระเบียบความคิด

จากตัวอย่างที่ผ่านมา จะเห็นว่าคำศัพท์สำคัญในเรื่องเซต เช่น สมาชิก เซตย่อย อินเตอร์เซกชัน และยูเนียน ล้วนเกิดขึ้นมาเพื่อช่วยอธิบายความสัมพันธ์เชิง “การอยู่ในกลุ่ม” แทบทั้งสิ้น กล่าวคือ เซตเป็นภาษาที่ช่วยให้เราพูดถึงสิ่งต่าง ๆ อย่างเป็นกลุ่มเป็นก้อน แทนที่จะต้องเล่าแบบกระจายกระจายทีละประโยค

ตัวอย่างเช่น ถ้าเราพูดว่า

นักศึกษาที่ลงเรียนวิชาดีสคริตทุกคนเป็นนักศึกษาในมหาวิทยาลัย

ในภาษามนุษย์อาจฟังดูเป็นประโยคธรรมดา แต่ในภาษาคณิตศาสตร์ เราจะพยายามเปลี่ยนให้เป็นความสัมพันธ์ระหว่างเซตทันที เช่น

$$C \subseteq U,$$

เมื่อ C คือเซตของนักศึกษาที่ลงเรียนวิชาดีสคริต และ U คือเซตของนักศึกษาทั้งหมดในมหาวิทยาลัย

เมื่อเราเขียนให้อยู่ในรูปนี้ ข้อความจะไม่เพียงสั้นลง แต่ยังชัดเจนด้วยว่า เรากำลังกล่าวถึง “ทั้งกลุ่ม” ไม่ใช่เพียงสมาชิกบางตัวแบบไม่เป็นระบบ

ข้อสังเกตสำคัญ

แม้เซตจะช่วยให้เราจัดรูปประโยคและจัดระเบียบสิ่งที่พูดถึงได้ดีมาก แต่เซตเพียงอย่างเดียวไม่ได้ทำหน้าที่อธิบายว่า “จากข้อความหนึ่ง ทำไมจึงสรุปอีกข้อความหนึ่งได้” ยกตัวอย่างเช่น การที่เรารู้ว่า

$$a \in S \quad \text{และ} \quad S \subseteq X$$

แล้วจึงสรุปว่า

$$a \in X$$

นั้นไม่ได้เกิดจากเซตเพียงลำพัง แต่เกิดจากโครงสร้างการให้เหตุผลที่อยู่เบื้องหลังนิยามของเซตย่อย ซึ่งตรงนี้คือบทบาทของ *ตรรกศาสตร์*

ดังนั้น ถ้าจะกล่าวอย่างสั้นที่สุด เซตมีหน้าที่หลักในการบอกว่า

- เรากำลังพูดถึงกลุ่มของอะไร
- ใครเป็นสมาชิกของกลุ่มนั้น
- กลุ่มหนึ่งเกี่ยวข้องกับอีกกลุ่มหนึ่งอย่างไร

ส่วนเรื่องของการสรุปผลอย่างมีหลักการจากข้อความเหล่านี้ จะเป็นเรื่องที่เราจะค่อย ๆ เห็นชัดเจนเมื่อเข้าสู่หัวข้อตรรกศาสตร์และการพิสูจน์ในลำดับถัดไป

1.3 ตรรกศาสตร์: โครงสร้างของประโยคและการให้เหตุผล

ถ้าเซตเป็นภาษาสำหรับพูดถึง ``กลุ่ม" และ ``ความเป็นสมาชิก" ตรรกศาสตร์ก็เป็นภาษาสำหรับพูดถึง ``ประโยค" และ ``ความสัมพันธ์เชิงเหตุผล" ระหว่างประโยคเหล่านั้น กล่าวอย่างง่ายที่สุด เซตช่วยให้เราทราบว่าเรากำลังพูดถึงใครหรือสิ่งใดอยู่ แต่ตรรกศาสตร์ช่วยให้เราเห็นว่า จากสิ่งที่รู้อยู่แล้ว เรามีสิทธิ์สรุปอะไรต่อไปได้บ้าง

ในชีวิตประจำวัน มนุษย์ใช้เหตุผลกันตลอดเวลา เช่น

ถ้าฝนตก ถนนจะเปียก
ตอนนี้ฝนตก
ดังนั้นถนนเปียก

หรือ

นักศึกษาทุกคนในรายวิชานี้ต้องลงทะเบียนเรียนให้เรียบร้อย
a เป็นนักศึกษาในรายวิชานี้
ดังนั้น a ต้องลงทะเบียนเรียนให้เรียบร้อย

สิ่งที่น่าสนใจคือ แม้เนื้อหาของสองตัวอย่างนี้ต่างกันมาก แต่ ``รูปแบบการให้เหตุผล" กลับคล้ายกัน ตรรกศาสตร์จึงไม่ได้สนใจเพียงว่าเราพูดเรื่องอะไร แต่สนใจด้วยว่าเรากำลัง *เชื่อมประโยค* และ *สรุปผล* กันอย่างไร

ประโยคที่มีค่าความจริง

จุดเริ่มต้นของตรรกศาสตร์คือการแยกให้ออกก่อนว่า ข้อความแบบใดเป็นข้อความที่เราสามารถพูดได้ชัดเจนว่า ``จริง" หรือ ``เท็จ" ข้อความประเภทนี้เรียกว่า *ประพจน์*

Definition 1.3.1. ประพจน์ ประพจน์คือข้อความที่มีค่าความจริงแน่นอนอย่างใดอย่างหนึ่ง คือเป็นจริง หรือเป็นเท็จ

ตัวอย่างเช่น

- `` $2 + 3 = 5$ " เป็นประพจน์ และเป็นจริง
- ``7 เป็นจำนวนคู่" เป็นประพจน์ และเป็นเท็จ
- ``วันนี้อากาศดี" ในหลายบริบทอาจไม่เหมาะสมจะถือเป็นประพจน์ เพราะคำว่า ``ดี" คลุมเครือเกินไป

ตรงนี้ควรสังเกตว่า ตรรกศาสตร์ต้องการประโยคที่มีความหมายชัดเจนจะพิจารณาค่าความจริงได้ ดังนั้นมันจึงเป็นวิชาที่บังคับให้เราเขียนอย่างรัดกุมมากขึ้น ในแง่นี้ตรรกศาสตร์จึงทำหน้าที่คล้าย ``กฎไวยากรณ์" ของการให้เหตุผลทางคณิตศาสตร์

ตัวเชื่อมตรรกศาสตร์: การสร้างประโยคที่ซับซ้อนขึ้น

เมื่อเรามีประพจน์พื้นฐานแล้ว เราย่อมต้องการนำหลายประโยคมาเชื่อมกันเพื่อสื่อสารความคิดที่ซับซ้อนกว่าเดิม ในภาษามนุษย์ เราใช้คำเชื่อมอย่าง “และ” “หรือ” “ถ้า...แล้ว...” “ก็ต่อเมื่อ” และ “ไม่” ในคณิตศาสตร์ก็เช่นเดียวกัน เพียงแต่เราทำให้คำเชื่อมเหล่านี้มีความหมายที่แน่นอนและใช้อย่างสม่ำเสมอ

เพื่อความสะดวก มักใช้ตัวอักษร $P, Q, R, \dots$ แทนประพจน์ต่าง ๆ แล้วสร้างประโยคใหม่จากประพจน์เหล่านั้น เช่น

$$P \wedge Q, \quad P \vee Q, \quad P \rightarrow Q, \quad P \leftrightarrow Q, \quad \neg P.$$

1) “และ” ($\wedge$) ประโยค $P \wedge Q$ หมายถึง “ P และ Q ” จะเป็นจริงก็ต่อเมื่อทั้ง P และ Q เป็นจริงพร้อมกัน ตัวเชื่อมนี้สอดคล้องกับความรู้สึกตามธรรมชาติของมนุษย์มาก ถ้าเราบอกว่า

“a เป็นนักศึกษา และ a ลงทะเบียนเรียนเรียบร้อยแล้ว”

เราย่อมต้องการให้ทั้งสองส่วนเป็นจริงพร้อมกันจริง ๆ

2) “หรือ” ($\vee$) ประโยค $P \vee Q$ หมายถึง “ P หรือ Q ” ในตรรกศาสตร์ คำว่า “หรือ” มักใช้ในความหมายแบบ อย่างน้อยหนึ่งอย่างเป็นจริง นั่นคืออาจเป็นจริงเพียง P อย่างเดียว หรือ Q อย่างเดียว หรือจริงทั้งคู่ก็ได้

จุดนี้ต่างจากภาษาไทยในชีวิตประจำวันอยู่บ้าง เพราะเวลาพูดว่า

“จะกินชาเขียวหรือโกโก้”

หลายครั้งเราหมายถึงให้เลือกอย่างใดอย่างหนึ่ง แต่ในตรรกศาสตร์ มักตีความ “หรือ” แบบรวมกรณีที่ทั้งสองจริง พร้อมกันด้วย ดังนั้นเวลาอ่านโจทย์คณิตศาสตร์ เราจึงต้องระวังว่าคำว่า “หรือ” กำลังถูกใช้ในความหมายใด

3) “ไม่” ($\neg$) ประโยค $\neg P$ หมายถึง “ไม่ใช่ P ” หรือ “เป็นการปฏิเสธ P ” ถ้า P เป็นจริง $\neg P$ ก็เป็นเท็จ และถ้า P เป็นเท็จ $\neg P$ ก็เป็นจริง

ตัวเชื่อมนี้ดูเหมือนง่าย แต่มีบทบาทสำคัญมากในคณิตศาสตร์ โดยเฉพาะเวลาพิสูจน์แบบขัดแย้ง หรือเวลาต้องการเขียน “ข้อความตรงข้าม” ของประโยคหนึ่งให้ถูกต้อง

4) “ถ้า...แล้ว...” ($\rightarrow$) ประโยค $P \rightarrow Q$ อ่านว่า “ถ้า P แล้ว Q ” หรือ “ P เป็นเงื่อนไขเพียงพอของ Q ” ตัวเชื่อมนี้มีความสำคัญมากที่สุดตัวหนึ่ง เพราะประโยคคณิตศาสตร์จำนวนมากอยู่ในรูปนี้ รวมถึงบทนิยาม ทฤษฎีบท และข้อพิสูจน์จำนวนมากด้วย

อย่างไรก็ดี นี่ก็เป็นตัวเชื่อมที่นักศึกษามักรู้สึกขัดใจมากที่สุดเช่นกัน เพราะในตรรกศาสตร์ ประโยค $P \rightarrow Q$ จะถือว่า “ผิด” เพียงกรณีเดียวเท่านั้น คือกรณีที่ P จริง แต่ Q เท็จ ส่วนกรณีที่ P ยังไม่เกิดขึ้น เรายังไม่ถือว่าผิดสัญญาอะไร

เหตุผลเชิงสัญชาตญาณคือ ประโยค “ถ้า P แล้ว Q ” กำลังตั้งกติกาไว้ว่า เมื่อใดก็ตามที่ P เกิด Q ต้องเกิดตาม ดังนั้นกรณีที่ทำให้กติกานี้พังคือ มี P เกิดขึ้นจริง แต่ Q ไม่เกิด นอกนั้นยังไม่ถือว่าแหกกติกา

ตัวอย่างเช่น ประโยค

“ถ้าสอบผ่านวิชานี้แล้วจะได้หน่วยกิต”

จะมีปัญหาต่อเมื่อมีคนสอบผ่าน แต่กลับไม่ได้หน่วยกิต แต่ถ้านักศึกษาคนนั้นยังสอบไม่ผ่าน ประโยคนี้อย่างไรก็ไม่ถูกหักล้างจากกรณีนั้น

ด้วยเหตุนี้ เวลาพิสูจน์ข้อความรูป “ถ้า...แล้ว...” เราจึงมักเริ่มจากการ สมมติส่วนหน้า ว่าเป็นจริงก่อน แล้วพยายามพิสูจน์ว่าส่วนหลังต้องจริงตามมา

5) “ก็ต่อเมื่อ” ($\leftrightarrow$) ประโยค $P \leftrightarrow Q$ หมายถึง “ P ก็ต่อเมื่อ Q ” หรือในภาษาพูดมักอ่านว่า “ P ก็ต่อเมื่อ และต่อเมื่อ Q ” แนวคิดสำคัญของคำเชื่อมนี้คือ สองข้อความต้องจริงเท่ากัน กล่าวคือ P จริงเมื่อ Q จริง และ P เท็จเมื่อ Q เท็จ

ในทางพิสูจน์ การจะพิสูจน์ $P \leftrightarrow Q$ มักเท่ากับการพิสูจน์สองทิศพร้อมกัน คือ

$$P \rightarrow Q \quad \text{และ} \quad Q \rightarrow P.$$

ดังนั้นข้อความประเภท “ก็ต่อเมื่อ” จึงมักยาวกว่าที่เห็นในตอนแรก เพราะจริง ๆ แล้วมันซ่อนการพิสูจน์ไว้สองชั้น

ตรรกศาสตร์ไม่ใช่แค่ตารางค่าความจริง

ในหลายบริบทของการเรียนระดับต้น ตรรกศาสตร์มักถูกสอนในภาพของการ “คำนวณค่าความจริง” หรือเติมตารางว่า ถ้า P จริง Q เท็จ แล้ว $P \rightarrow Q$ เป็นอะไร วิธีฝึกแบบนี้มีประโยชน์ เพราะช่วยให้เราคุ้นกับความหมายที่แน่นอนของตัวเชื่อมต่าง ๆ แต่ถ้าหยุดอยู่แค่นั้น เราอาจเข้าใจตรรกศาสตร์เพียงผิวเผินเกินไป

แก่นจริง ๆ ของตรรกศาสตร์คือ มันเป็นเครื่องมือสำหรับควบคุมการให้เหตุผล กล่าวคือมันช่วยตอบคำถามว่า

จากสิ่งที่สมมติอยู่ เราควรเดินไปข้างหน้าอย่างไรจึงจะสรุปสิ่งที่ต้องการได้อย่างถูกต้อง

ตัวอย่างเช่น ถ้าโจทย์ต้องการพิสูจน์ว่า

$$P \rightarrow Q,$$

ตรรกศาสตร์บอกเราว่า วิธีธรรมชาติที่สุดคือสมมติ P แล้วพยายามไปให้ถึง Q

ถ้าโจทย์ต้องการพิสูจน์ว่า

$$P \wedge Q,$$

เราต้องพิสูจน์ให้ครบทั้งสองส่วน

ถ้าโจทย์ต้องการพิสูจน์ว่า

$$P \vee Q,$$

เราต้องแสดงให้เห็นว่าอย่างน้อยหนึ่งในสองส่วนนั้นเป็นจริง

และถ้าโจทย์ต้องการพิสูจน์ว่า

$$P \leftrightarrow Q,$$

เราต้องไม่ลืมว่ามันแยกออกเป็นสองทิศ

จะเห็นว่า ตรรกศาสตร์กำลังบอก “รูปทรง” ของการพิสูจน์ หรืออย่างน้อยก็บอกได้ว่า โครงร่างคร่าว ๆ ของข้อพิสูจน์ควรเริ่มอย่างไรและจบอย่างไร

ประโยคเปิด: เมื่อเรายังไม่ระบุว่าเป็นใคร

นอกจากประโยคที่มีค่าความจริงแน่นอนแล้ว ในคณิตศาสตร์ยังมีข้อความอีกประเภทหนึ่งที่สำคัญมาก คือข้อความที่ยังมีตัวแปรเปิดอยู่ เช่น

$$x \text{ เป็นจำนวนคู่} \quad \text{หรือ} \quad P(x).$$

ข้อความลักษณะนี้ยังบอกไม่ได้ทันทีว่าจริงหรือเท็จ เพราะเราไม่รู้ว่ x คืออะไร ถ้าแทน $x = 4$ อาจจริง แต่ถ้าแทน $x = 5$ อาจเท็จ ข้อความแบบนี้มักเรียกว่า *ประโยคเปิด*

ประโยคเปิดมีความสำคัญมาก เพราะนิยามและทฤษฎีบทจำนวนมากในคณิตศาสตร์ไม่ได้พูดถึงสมาชิกตัวเดียว แต่พูดถึงสมาชิกใด ๆ ในโดเมนทั้งหมด หรือพูดถึงการมีอยู่ของสมาชิกบางตัวที่ทำให้เงื่อนไขเป็นจริง ซึ่งต่อไปจะนำเราไปสู่เรื่อง “สำหรับทุก” และ “มีอยู่” ในตรรกศาสตร์อันดับหนึ่ง

ในบทนี้เรายังไม่ลงรายละเอียดเรื่องนั้นมากนัก แต่ควรจำไว้ก่อนว่า ตรรกศาสตร์ไม่ได้หยุดอยู่แค่ประโยคปิดที่จริงหรือเท็จเท่านั้น มันยังขยายไปสู่ประโยคที่มีตัวแปร และการอธิบายว่าเรากำลังพูดถึง “ทุกตัว” หรือ “บางตัว” อย่างไรด้วย

ตรรกศาสตร์กับการพิสูจน์

ท้ายที่สุด สิ่งที่ทำให้ตรรกศาสตร์สำคัญมากในวิชาคณิตศาสตร์ก็คือ มันเป็นพื้นฐานของการพิสูจน์แทบทุกแบบ ไม่ว่าจะเป็น

- การพิสูจน์ตรง
- การพิสูจน์แบบสลับนัย
- การพิสูจน์แบบแย้ง
- การพิสูจน์แบบแยกกรณี
- การพิสูจน์ด้วยอุปนัย

ล้วนมีโครงสร้างที่อธิบายได้ด้วยตรรกศาสตร์ทั้งสิ้น

เมื่อเราอ่านข้อพิสูจน์หนึ่งชิ้น สิ่งที่เราควรถามไม่ใช่เพียงว่า “คำนวณถูกหรือไม่” แต่ควรถามด้วยว่า

ตอนนี้ผู้เขียนกำลังสมมติอะไรอยู่
เราต้องการพิสูจน์อะไร
และเหตุใดประโยคถัดไปจึงตามมาจากระโยคก่อนหน้านี้ได้

คำถามเหล่านี้เป็นคำถามเชิงตรรกศาสตร์ทั้งสิ้น

ดังนั้นถ้าเซตช่วยให้เราจัดระเบียบว่าใครอยู่ในกลุ่มใด ตรรกศาสตร์ก็ช่วยให้เราจัดระเบียบว่า ประโยคใดกำลังถูกใช้เป็นสมมติฐาน ประโยคใดคือข้อสรุป และเราควรเดินจากจุดหนึ่งไปอีกจุดหนึ่งอย่างไร เพื่อให้การอธิบายนั้นเป็นการให้เหตุผลที่ถูกต้องจริง ๆ

สรุปภาพรวม

กล่าวอย่างย่อ บทบาทของตรรกศาสตร์ในหนังสือเล่มนี้มีอย่างน้อยสามระดับ

1. ช่วยให้เราเข้าใจโครงสร้างของประโยคทางคณิตศาสตร์
2. ช่วยให้เราแปลข้อความภาษาไทยให้เป็นรูปสัญลักษณ์ที่ชัดเจนขึ้น
3. ช่วยกำหนดรูปแบบการให้เหตุผลและการพิสูจน์

ในหัวข้อถัด ๆ ไป เราจะค่อย ๆ ใช้ตรรกศาสตร์ในระดับที่ลึกขึ้น ตั้งแต่การพิสูจน์ข้อความเกี่ยวกับตัวเชื่อมพื้นฐาน ไปจนถึงข้อความที่มีตัวแปรและตัวบ่งปริมาณ ซึ่งจะเป็นเครื่องมือสำคัญของคณิตศาสตร์ดีสครีตตลอดทั้งรายวิชา

1.4 ตัวบ่งปริมาณ บริบท และตรรกศาสตร์อันดับหนึ่ง

หลายข้อความในคณิตศาสตร์ไม่ได้พูดถึงวัตถุเพียงตัวเดียว แต่พูดถึง ``ทุกตัว" หรือ ``มีบางตัว" เช่น

- ``สำหรับทุกจำนวนเต็ม n , ถ้า n เป็นจำนวนคู่แล้ว n^2 เป็นจำนวนคู่"
- ``มีจำนวนเฉพาะที่เป็นจำนวนคู่"

ข้อความแบบนี้ต้องใช้ ตัวบ่งปริมาณ (quantifier)

ตัวบ่งปริมาณสากล

$$\forall x \in U, P(x)$$

แปลว่า ``สำหรับทุก x ในโดเมน U , $P(x)$ เป็นจริง"

ตัวบ่งปริมาณการมีอยู่

$$\exists x \in U, P(x)$$

แปลว่า ``มี x บางตัวในโดเมน U ที่ทำให้ $P(x)$ เป็นจริง"

ตรรกะที่แนวคิดเรื่อง บริบท หรือ โดเมน มีความสำคัญมาก เพราะข้อความเดียวกันอาจเปลี่ยนค่าความจริงได้ ถ้าเปลี่ยนโดเมน เช่น

$$\forall x, x^2 \geq x$$

ถ้าไม่ระบุ x ว่าจะอยู่ในอะไร ข้อความนี้จะกำกวมทันที แต่ถ้าระบุ $x \in \mathbb{N}$ ความหมายจะชัดเจนมาก

ในเชิงกว้าง ตรรกศาสตร์ที่ใช้ตัวแปร predicate และตัวบ่งปริมาณเช่นนี้ คือสิ่งที่เรียกว่า ตรรกศาสตร์อันดับหนึ่ง (first-order logic) ในหนังสือเล่มนี้ เราไม่ได้ตั้งใจพัฒนาวิชาตรรกศาสตร์อันดับหนึ่งแบบเชิงทฤษฎีเต็มรูปแบบ แต่เราจะใช้มุมมองของมันอยู่ตลอด กล่าวคือ

- ต้องรู้ว่าเรากำลังพูดถึงวัตถุในโดเมนใด
- ต้องรู้ว่า predicate แต่ละตัวหมายถึงอะไร
- ต้องอ่านให้ออกว่าข้อความถูกครอบด้วย $\forall$ หรือ $\exists$
- ต้องระวังว่าลำดับของตัวบ่งปริมาณมีผลต่อความหมาย

ตัวอย่างที่สำคัญมากคือ

$$\forall x \exists y P(x, y) \quad \text{กับ} \quad \exists y \forall x P(x, y).$$

สองข้อความนี้แทบไม่เคยแปลว่าอย่างเดียวกัน ข้อความแรกหมายถึง ``สำหรับทุก x เราหา y ที่ขึ้นกับ x ได้" แต่

ข้อความหลังหมายถึง “มี y ตัวเดียวที่ใช้ได้กับทุก x ” ซึ่งเป็นข้ออ้างที่แข็งแกร่งกว่ามาก

1.5 ความสัมพันธ์และฟังก์ชัน: การอธิบายการเชื่อมโยงระหว่างสิ่งของ

หลังจากรู้จักเซตและตรรกศาสตร์แล้ว ขั้นตอนต่อไปคือการอธิบายว่า วัตถุจากกลุ่มหนึ่งเชื่อมโยงกับวัตถุจากอีกกลุ่มอย่างไร นี่คือบทบาทของ ความสัมพันธ์ (relation) และ ฟังก์ชัน (function)

1.5.1 ความสัมพันธ์

ถ้า A และ B เป็นเซต ความสัมพันธ์จาก A ไป B คือกฎหรือเงื่อนไขบางอย่าง ที่บอกว่าองค์ประกอบใดของ A เชื่อมโยงกับองค์ประกอบใดของ B

ในเชิงเซตอย่างเป็นทางการ ความสัมพันธ์คือเซตย่อยของ $A \times B$ แต่ในขั้นนี้ สิ่งสำคัญกว่ารูปนิยามคือภาพความคิด: มันเป็นเครื่องมือสำหรับบอกว่า “อะไรสัมพันธ์กับอะไร”

ตัวอย่างเช่น

- “ $x < y$ ” เป็นความสัมพันธ์บนจำนวน
- “นักศึกษาคนนี้ลงทะเบียนเรียนรายวิชานี้”
- “จุดยอดสองจุดนี้มีเส้นเชื่อมกัน” ในกราฟ

ดังนั้นความสัมพันธ์จึงสำคัญมากในคณิตศาสตร์ไม่ต่อเนื่อง เพราะหัวข้อจำนวนมากในรายวิชานี้ แท้จริงแล้วคือการศึกษาคุณสมบัติของความสัมพันธ์ในรูปแบบต่าง ๆ

1.5.2 ฟังก์ชัน

ฟังก์ชันเป็นความสัมพันธ์ชนิดพิเศษ ที่แต่ละ input มี output ที่กำหนดแน่นอนเพียงค่าเดียว

ถ้า $f : A \rightarrow B$ เราจะเข้าใจว่า ทุก $a \in A$ ต้องถูกส่งไปยังสมาชิกหนึ่งของ B อย่างแน่นอน เช่น

$$f(n) = n^2$$

หรือ

$$\text{tail}(A)$$

ที่ส่งลิสต์ A ไปเป็นลิสต์ที่ตัดสมาชิกตัวแรกออก

ฟังก์ชันมีความสำคัญมากในวิชาคอมพิวเตอร์ เพราะมันเป็นแบบจำลองพื้นฐานของแนวคิด “รับ input แล้วให้ output” ไม่ว่าจะเป็นนิพจน์ทางคณิตศาสตร์ โปรแกรมย่อย อัลกอริทึม หรือขั้นตอนคำนวณใด ๆ ก็ตาม

ในบทนี้ เราจะยังไม่ลงลึกในคุณสมบัติของ relation และ function มากนัก เพียงแต่อยากให้เห็นว่า เมื่อเราพูดถึงเรื่อง relation หรือ function เรากำลังใช้ภาษาเพื่ออธิบาย “โครงสร้างของการเชื่อมโยง” ไม่ใช่เพียงการแทนค่าหรือคำนวณตัวเลข

1.6 การอ่านข้อความคณิตศาสตร์ให้เป็นโครงสร้าง

หนึ่งในทักษะสำคัญที่สุดก่อนเริ่มพิสูจน์จริง คือการอ่านข้อความให้แตกว่า มันมีโครงสร้างแบบไหน เพราะวิธีพิสูจน์มักถูกกำหนดโดยรูปของข้อความเอง

ข้อความต่อไปนี้เป็นตัวอย่างที่พบบ่อยมาก

กรณีที่ 1: ข้อความแบบ $\forall x \in U, P(x)$

ถ้าต้องการพิสูจน์ข้อความลักษณะนี้ เรามักเริ่มโดย

ให้ x เป็นสมาชิกใด ๆ ใน U แล้วพิสูจน์ว่า $P(x)$ เป็นจริง

หัวใจคือคำว่า “ใด ๆ” เพราะเราไม่ได้เลือกตัวพิเศษ แต่เลือกตัวแทนทั่วไปของโดเมน

กรณีที่ 2: ข้อความแบบ $\forall x \in U, P(x) \rightarrow Q(x)$

นี่เป็นรูปที่สำคัญที่สุดรูปหนึ่งในหนังสือเล่มนี้ โดยปกติเราจะเริ่มว่า

ให้ $x \in U$ เป็นสมาชิกใด ๆ และสมมติว่า $P(x)$ เป็นจริง ต้องการพิสูจน์ว่า $Q(x)$ เป็นจริง

สังเกตว่าเราไม่ได้เริ่มพิสูจน์ $Q(x)$ ทันที แต่ต้อง “รับเงื่อนไข” $P(x)$ เข้ามาก่อน เพราะประโยคมีรูป “ถ้า...แล้ว...”

กรณีที่ 3: ข้อความแบบ $\exists x \in U, P(x)$

ถ้าต้องการพิสูจน์ข้อความแบบมีอยู่ เราต้องหาสิ่งที่เรียกว่า **พยาน (witness)** นั่นคือให้ตัวอย่างเฉพาะสักตัว แล้วตรวจว่ามันทำให้ $P(x)$ เป็นจริงจริง

กรณีที่ 4: การหักล้างข้อความ

ถ้าต้องการหักล้างข้อความแบบสากล เช่น

$$\forall x \in U, P(x),$$

สิ่งที่เพียงพอคือหาสมาชิกตัวเดียวที่ทำให้ $P(x)$ ล้มเหลว นั่นคือหาพยานของ

$$\exists x \in U, \neg P(x).$$

สิ่งนี้เรียกว่า **counterexample** และเป็นเครื่องมือสำคัญมากในรายวิชานี้

กรณีที่ 5: การพิสูจน์ว่าเซตสองเซตเท่ากัน

ถ้าต้องการพิสูจน์ว่า $A = B$ วิธีมาตรฐานคือแยกเป็นสองส่วน:

$$A \subseteq B \quad \text{และ} \quad B \subseteq A.$$

แล้วแต่ละส่วนก็ใช้โครงพิสูจน์ของเซตย่อยตามนิยามอีกที

ดังนั้นก่อนลงมือพิสูจน์ทุกครั้ง คำถามแรกที่ควรถามตัวเองคือ

ข้อความนี้มีตัวบ่งปริมาณอะไรบ้าง มีตัวเชื่อมหลักเป็นอะไร และฉันควรเริ่มจากสมมติอะไรได้บ้าง

1.7 การให้เหตุผลทางคณิตศาสตร์และการพิสูจน์

เมื่อเริ่มเรียนคณิตศาสตร์ในระดับสูงขึ้น นักศึกษาหลายคนมักพบว่าบรรยากาศของวิชาเปลี่ยนไปพอสมควร ในระดับก่อนหน้า เราอาจคุ้นกับการคำนวณให้ได้คำตอบสุดท้าย โจทย์จำนวนมากจบลงเมื่อแทนค่าสูตรถูกหรือคิดเลขไม่ผิด แต่ในคณิตศาสตร์แบบที่ใช้ในรายวิชา discrete mathematics นั้น คำถามสำคัญมักไม่ใช่เพียงว่า “คำตอบคืออะไร” หากเป็นว่า “เรารู้ได้อย่างไรว่าคำตอบนั้นถูกต้อง”

ตรงนี้เองที่ทำให้ *การพิสูจน์* กลายเป็นหัวใจของวิชา คณิตศาสตร์ไม่ต่างจากการสร้างคำอธิบายที่น่าเชื่อถือที่สุดแบบหนึ่ง เราไม่ได้พอใจกับการเห็นตัวอย่างหลาย ๆ ตัวแล้วเดาว่าน่าจะเป็นจริง และไม่ได้หยุดเพียงเพราะคำตอบที่คำนวณออกมาดูสมเหตุสมผล แต่เราต้องการเหตุผลที่ยืนยันได้ว่า ภายใต้เงื่อนไขที่กำหนดไว้ ข้อความนั้นเป็นจริงเสมอ ไม่ใช่จริงแค่บางกรณีที่เราลองดูแล้วบังเอิญไม่พลาด

เพราะฉะนั้น การพิสูจน์จึงไม่ใช่พิธีกรรมทางภาษา และไม่ใช่การเขียนข้อความยาว ๆ ให้ดูซับซ้อน แก่นของมันคือ

การเริ่มจากสิ่งที่ยอมรับหรือทราบอยู่แล้ว เช่น นิยาม สมบัติพื้นฐาน หรือสมมติฐานของโจทย์ แล้วค่อย ๆ เชื่อมโยงไปยังสิ่งที่ต้องการสรุป ทุกบรรทัดในพิสูจน์ที่ดีจึงควรตอบได้เสมอว่า ``บรรทัดนี้ตามมาจากระไร" และ ``ทำไมจึงตามมาได้"

การพิสูจน์คือการเล่าเรื่องจากสิ่งที่รู้ไปยังสิ่งที่อยากได้

ถ้าจะอธิบายอย่างไม่เป็นทางการ การพิสูจน์มีลักษณะคล้ายการเล่าเรื่องชนิดหนึ่ง แต่เป็นการเล่าเรื่องที่มีระเบียบเคร่งครัดกว่าปกติ ผู้เขียนต้องบอกให้ชัดว่า ตอนนี้เรารู้อะไรอยู่ กำลังสมมติอะไร กำลังจะใช้ข้อเท็จจริงใด และสุดท้ายต้องการไปถึงข้อความใด

ในแง่นี้ การพิสูจน์ไม่ใช่การกระโดดจากต้นไปปลาย แต่เป็นการพาผู้อ่านเดินทีละก้าว เหมือนเรากำลังสร้างสะพานจากฝั่งของ ``สมมติฐาน" ไปยังฝั่งของ ``ข้อสรุป" ถ้ามีช่วงใดของสะพานที่ขาดหายไป แม้ผู้เขียนจะรู้ในใจว่าควรข้ามไปได้ ผู้อ่านก็ยังถือว่าพิสูจน์นั้นไม่สมบูรณ์

มุมมองแบบนี้สำคัญมากสำหรับนักศึกษาสายคอมพิวเตอร์ด้วย เพราะในหลายสถานการณ์ การเขียนพิสูจน์มีลักษณะคล้ายการเขียนโปรแกรมที่ดี กล่าวคือไม่ใช่เพียงได้ผลลัพธ์สุดท้าย แต่ต้องมีลำดับเหตุผลที่ตรวจสอบย้อนกลับได้ รู้ว่าข้อมูลขึ้นใดถูกใช้ตรงไหน และถ้าเปลี่ยนสมมติฐานบางข้อ ก็ต้องรู้ว่าผลสรุปส่วนใดจะพังตามไปด้วย

ก่อนพิสูจน์ ต้องอ่านโจทย์ให้ออกก่อนว่าโจทย์กำลังพูดแบบไหน

ในทางปฏิบัติ ปัญหาสำคัญของผู้เริ่มต้นมักไม่ได้อยู่ที่ ``ไม่รู้วิธีพิสูจน์เลย" แต่อยู่ที่ยังอ่านโครงสร้างของข้อความไม่ออก ทั้งที่หลายครั้ง รูปแบบของประโยคเป็นตัวบอกแนวทางของพิสูจน์อยู่แล้ว

ตัวอย่างเช่น ถ้าโจทย์อยู่ในรูป

$$\forall x \in U, P(x)$$

สิ่งที่โจทย์ต้องการคือการอธิบายว่า ไม่ว่าเราจะหยิบสมาชิกตัวใดจากเอกภพสัมพัทธ์ U มา ข้อความ $P(x)$ จะจริงเสมอ ดังนั้นธรรมชาติของพิสูจน์จึงมักเริ่มจาก ``ให้ x เป็นสมาชิกใด ๆ ของ U " แล้วพิสูจน์ $P(x)$

ถ้าโจทย์อยู่ในรูป

$$\exists x \in U, P(x)$$

โจทย์ไม่ได้ขอให้พิสูจน์กับทุกตัว แต่ขอเพียงให้แสดงว่า มีอย่างน้อยหนึ่งตัวที่ทำให้เงื่อนไขเป็นจริง ในกรณีนี้ การพิสูจน์มักเป็นการเลือกหรือสร้าง *พยาน* ขึ้นมา แล้วตรวจสอบว่าพยานนั้นทำให้ $P(x)$ จริงจริง ๆ

และถ้าโจทย์อยู่ในรูป

$$P \rightarrow Q$$

เราควรอ่านมันเป็น ``เมื่อสมมติ P แล้ว ต้องไปให้ถึง Q " นี่เป็นเหตุผลว่าทำไมข้อพิสูจน์แบบ if-then จึงมักเริ่ม

จากการสมมติส่วนหน้า ไม่ใช่เริ่มจากส่วนที่ยากพิสูจน์

กล่าวอีกแบบหนึ่งคือ การพิสูจน์ที่ดีเริ่มตั้งแต่ก่อนเขียนบรรทัดแรก เพราะเราต้องอ่านโจทย์ให้รู้ก่อนว่า ข้อความนั้นมีโครงสร้างตรรกะแบบใด อะไรคือสมมติฐาน อะไรคือข้อสรุป และคำสำคัญของโจทย์กำลังบังคับรูปของการให้เหตุผลไว้แบบไหน

วิธีพิสูจน์ไม่ได้มีแบบเดียว

เมื่อพูดถึงการพิสูจน์ นักศึกษามักเผลอคิดว่ามี “วิธีมาตรฐาน” เพียงแบบเดียว คือเริ่มเขียนอะไรบางอย่างจากสมมติฐาน แล้วหวังว่าจะไปถึงข้อสรุปได้เอง แต่ในความจริง ข้อความต่างชนิดกันมักเหมาะกับวิธีพิสูจน์ต่างกัน

บางข้อความพิสูจน์แบบตรงได้เลย กล่าวคือเริ่มจากสมมติฐานแล้วใช้คำนิยามหรือข้อเท็จจริงที่รู้จัก เดินไปข้างหน้าทีละขั้นจนถึงข้อสรุป วิธีนี้เรียกว่า *direct proof* และมักเป็นจุดเริ่มต้นที่ดีที่สุดเมื่อข้อความไม่ซับซ้อนเกินไป

บางข้อความพิสูจน์ตรงแล้วอึดอัด แต่ถ้าเปลี่ยนมาพิสูจน์ข้อความที่สมมูลกันในรูปกลับ งานจะง่ายขึ้นมาก ตัวอย่างคลาสสิกคือการพิสูจน์

$$P \rightarrow Q$$

ผ่านข้อความสมมูล

$$\neg Q \rightarrow \neg P.$$

วิธีนี้คือ *contrapositive proof* ซึ่งไม่ได้เป็น “ลูกเล่น” แต่เป็นการอาศัยความสมมูลทางตรรกะมาเปลี่ยนโจทย์ให้พิสูจน์ง่ายขึ้น

นอกจากนี้ยังมี *proof by contradiction* ซึ่งเริ่มจากการสมมติว่าข้อความที่ยากพิสูจน์นั้นไม่จริง แล้วให้เหตุผลต่อจนเกิดความขัดแย้ง ความขัดแย้งนี้อาจขัดกับนิยาม ขัดกับสมบัติพื้นฐาน หรือขัดกับสิ่งที่สมมติไว้ก่อนหน้านี้เอง เมื่อถึงจุดนั้น เราจึงสรุปได้ว่าสมมติฐานที่ตั้งขึ้นมาเพื่อแย้งต้องผิด ดังนั้นข้อความเดิมจึงเป็นจริง

ในภายหลัง เรายังจะได้เจอวิธีพิสูจน์อีกหลายแบบ เช่น การพิสูจน์โดยแยกกรณี การพิสูจน์ด้วยอุปนัย การพิสูจน์เกี่ยวกับความถูกต้องของอัลกอริทึม หรือการพิสูจน์ว่ามีหรือไม่มีวัตถุบางชนิดอยู่จริง แต่ไม่ว่าวิธีจะต่างกันเพียงใด สิ่งที่เหมาะสมเสมอคือ ทุกวิธีล้วนต้องอาศัยตรรกะที่ถูกต้องและการจัดลำดับเหตุผลอย่างระมัดระวัง

การพิสูจน์ไม่ใช่แค่เรื่องของความจริง แต่เป็นเรื่องของการสื่อสาร

มีข้อพิสูจน์จำนวนไม่น้อยที่ “คนเขียนรู้ว่าตัวเองหมายถึงอะไร” แต่ผู้อ่านกลับตามไม่ทัน สาเหตุไม่ได้มาจากคณิตศาสตร์ยากเกินไปเสมอไป แต่อาจมาจากการเขียนที่ไม่บอกบริบทให้พอ

ตัวอย่างเช่น ถ้าผู้เขียนเริ่มต้นด้วยการใช้ตัวแปร n โดยไม่บอกว่า n เป็นจำนวนเต็มหรือจำนวนจริง ผู้อ่านก็อาจไม่รู้ว่ากำลังทำงานอยู่ในโดเมนไหน หรือถ้าผู้เขียนเขียนว่า “ดังนั้นชัดเจนว่าได้ข้อสรุป” โดยไม่อธิบายว่าชัดเจน

จากอะไร พิสูจน์นั้นก็ยังอ่อนเกินไปในเชิงการสื่อสาร

ดังนั้น การเขียนพิสูจน์ที่ดีจึงไม่ใช่เพียง ``ตรรกะถูก" แต่ต้อง ``เล่าเรื่องให้ผู้อ่านตามทัน" ด้วย ควรบอกให้ชัดว่าเราเริ่มจากสมมติฐานใด ใช้คำนิยามตรงไหน กำลังพิสูจน์ทิศทางไหน และข้อสรุปที่ได้ในแต่ละช่วงกำลังพาเราเข้าใกล้เป้าหมายอย่างไร

ในหนังสือเล่มนี้ เราจะให้ความสำคัญกับการพิสูจน์ในฐานะการสื่อสารเชิงเหตุผล ไม่ใช่ในฐานะงานเขียนที่ต้องประดับคำให้ดูยาก ถ้าข้อความสั้นแต่ลำดับตรรกะครบถ้วน ก็ถือว่าดี แต่ถ้าข้อความยาวมากแต่กระโดดข้ามเหตุผลสำคัญไป ก็ยังไมถือว่าเป็นพิสูจน์ที่สมบูรณ์

สิ่งที่ผู้เริ่มต้นมักพลาด

เมื่อเริ่มฝึกเขียนพิสูจน์ ความผิดพลาดที่พบบ่อยมีอยู่ไม่กี่แบบ แต่เกิดซ้ำบ่อยมาก

อย่างแรกคือการใช้ตัวอย่างแทนการพิสูจน์ เช่น ลองแทนค่าหลาย ๆ ค่าแล้วพบว่าข้อความจริงทุกครั้ง จึงสรุปว่ามันจริงเสมอ วิธีคิดนี้อาจช่วยให้ ``เดา" คำตอบได้ แต่ยังไม่ใช่วิธีการพิสูจน์ เพราะยังไม่ครอบคลุมทุกกรณี

อย่างที่สองคือการเริ่มจากสิ่งที่ต้องการพิสูจน์ แล้วค่อยจัดรูปย้อนกลับไปหาสมมติฐาน วิธีนี้บางครั้งใช้เป็นการสำรวจแนวทางได้ แต่ถ้านำมาเขียนเป็นพิสูจน์โดยตรงโดยไม่ระวัง ก็จะกลายเป็นการสมมติสิ่งที่อยากได้อยู่แล้ว

อย่างที่สามคือการสับสนระหว่าง ``พิสูจน์ข้อความ if--then" กับ ``พิสูจน์ข้อความ if and only if" ข้อความแรกต้องพิสูจน์เพียงทิศเดียว แต่ข้อความหลังต้องพิสูจน์สองทิศ ถ้าลืมนัดนี้ พิสูจน์จะดูเหมือนเสร็จทั้งที่จริงยังขาดไปครึ่งหนึ่ง

อย่างที่สี่คือการปฏิเสธข้อความที่มีตัวบ่งปริมาณผิด เช่น ปฏิเสธ

$$\forall x, P(x)$$

แล้วเขียนเป็น

$$\forall x, \neg P(x)$$

ทั้งที่ที่ถูกต้องควรเป็น

$$\exists x, \neg P(x).$$

ความผิดพลาดแบบนี้มีผลโดยตรงต่อการพิสูจน์แบบ contradiction และการหาตัวอย่างค้าน

เป้าหมายของส่วนนี้

ในหัวข้อนี้ เราไม่ได้ต้องการให้ผู้อ่านจำชื่อเทคนิคพิสูจน์ให้ได้มากที่สุด แต่ต้องการให้เริ่มเห็นภาพว่า การพิสูจน์แต่ละแบบเกิดจากโครงสร้างของข้อความที่ต่างกัน และการเลือกวิธีพิสูจน์ที่เหมาะสม มักเริ่มจากการอ่านประโยคให้แตกก่อนเสมอ

เมื่ออ่านจบบทนี้ ผู้อ่านควรเริ่มตอบคำถามต่อไปนี้ได้ดีขึ้น:

- โจทย์กำลังให้สมมติฐานอะไร และอยากให้สรุปอะไร
- ข้อความนี้เป็นข้อความแบบ $\forall$, $\exists$, หรือ if--then
- ควรเริ่มข้อพิสูจน์จากการเลือกสมาชิกทั่วไป การหาพยาน หรือการสมมติส่วนหน้า
- เมื่อใดควรใช้ direct proof, contrapositive, contradiction หรือการแยกกรณี
- จะเขียนอย่างไรให้ผู้อ่านเห็นที่มาที่ไปของเหตุผลอย่างชัดเจน

Key Takeaway

- การพิสูจน์คือการเชื่อมสมมติฐานไปยังข้อสรุปผ่านตรรกะที่ถูกต้อง
- ก่อนเลือกวิธีพิสูจน์ ต้องอ่านโครงสร้างของข้อความให้ออกก่อน
- ข้อพิสูจน์ที่ดีต้องไม่เพียงถูกต้อง แต่ต้องสื่อสารลำดับเหตุผลให้ผู้อ่านตามได้

จากตรงนี้ไป เราจะค่อย ๆ ศึกษาเครื่องมือของการพิสูจน์อย่างเป็นระบบมากขึ้น เริ่มจากองค์ประกอบย่อยของประโยคเชิงตรรกะ ต่อด้วยข้อความที่มีตัวบ่งปริมาณ ข้อความแบบเงื่อนไขผลสรุป และเทคนิคพิสูจน์รูปแบบต่าง ๆ เพื่อให้ผู้อ่านไม่เพียงอ่านพิสูจน์ของคนอื่นรู้เรื่อง แต่สามารถออกแบบข้อพิสูจน์ของตนเองได้ด้วย

1.8 บทบาทของบทนี้ต่อบทถัดไป

เมื่ออ่านถึงตรงนี้ ผู้อ่านยังไม่จำเป็นต้องรู้สึกรู้ว่า ตนเองต้องเชี่ยวชาญทุกเรื่องในบทนี้ทันที บทนี้มีหน้าที่คล้าย “คู่มือภาษา” ที่ให้เราเริ่มอ่านประโยคคณิตศาสตร์ออก และเริ่มเห็นว่าการพิสูจน์ทำงานอย่างไรในระดับโครงสร้าง

จากนี้ไป หลายแนวคิดจะกลับมาปรากฏอีกครั้งในบทที่เฉพาะทางมากขึ้น เช่น

- เรื่อง **logic, reasoning, and proof** จะถูกใช้จริงใน almost every chapter
- เรื่อง **induction** จะมีบทบาทเฉพาะของมันเอง
- เรื่อง **recursive definitions** และ **recurrence relations** จะใช้ทั้งภาษาเรื่องฟังก์ชัน นิยาม และการพิสูจน์
- เรื่อง **relations** และ **functions** จะกลับมามีบทบาทมากขึ้นในบริบทของโครงสร้างไม่ต่อเนื่องและอัลกอริทึม

ดังนั้นถ้าจะสรุปบทนี้ในประโยคเดียว ผู้เขียนอยากให้จำไว้ว่า

คณิตศาสตร์ในหนังสือเล่มนี้ไม่ใช่ศิลปะของการแทนค่าสูตร แต่เป็นศิลปะของการนิยามให้ชัดเจน
อ่านโครงสร้างให้ออก และให้เหตุผลอย่างตรวจสอบได้

และนั่นคือพื้นฐานทั้งหมดที่เราต้องการ ก่อนจะเริ่มบทเรียนจริงที่เน้นการพิสูจน์ในสัปดาห์ถัดไป

Final Draft
Handout version
Phaphontee Yamchote

Final Draft
Handout version
Phaphontee Yamchote

ภาค II

พื้นฐานการพิสูจน์และการให้เหตุผลทางคณิตศาสตร์

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 2

ตรรกศาสตร์ประพจน์และโครงสร้างของการพิสูจน์

คณิตศาสตร์ในบริบทของวิทยาการคอมพิวเตอร์ไม่ได้ทำหน้าที่เป็นเพียงเครื่องมือสำหรับคำนวณ แต่ทำหน้าที่เป็น *ภาษา* สำหรับอธิบายโครงสร้าง ความสัมพันธ์ เงื่อนไข และความถูกต้องของสิ่งที่เราสนใจด้วย เมื่อเราพูดถึงอัลกอริทึม เราไม่ได้ต้องการเพียงคำตอบว่าโปรแกรม *ดูเหมือน* ทำงานได้ในตัวอย่างที่ลอง แต่เราต้องการอธิบายอย่างชัดเจนว่า *เพราะเหตุใด* มันจึงควรทำงานได้ถูกต้องในทุกกรณีที่อยู่ภายใต้เงื่อนไขที่กำหนด

ในแง่นี้ คณิตศาสตร์ทำหน้าที่คล้ายภาษาสำหรับเขียน *specification* ของความคิด เรานิยามคำศัพท์ให้ชัด เราระบุเงื่อนไขให้ครบ เราแยกสิ่งนี้ออกจากเหตุผลที่ใช้รองรับ และเมื่อจำเป็น เราเขียนการพิสูจน์เพื่อแสดงให้เห็นว่าแต่ละบรรทัดของการให้เหตุผลนั้นตามมาจากอะไร กระบวนการทั้งหมดนี้เป็นหัวใจของการให้เหตุผลเชิงคณิตศาสตร์ และเป็นทักษะพื้นฐานของการเรียนคอมพิวเตอร์ในระดับที่จริงจัง

2.1 ประพจน์และประโยคเปิด

2.1.1 ประพจน์

Definition 2.1.1. ประพจน์ (proposition) คือประโยคที่มีค่าความจริงแน่นอนว่าเป็น *จริง* หรือ *เท็จ* แต่ไม่เป็นทั้งสองอย่างพร้อมกัน

แนวคิดสำคัญของนิยามนี้อยู่ที่คำว่า “มีค่าความจริงแน่นอน” กล่าวคือ เมื่อประโยคหนึ่งเป็นประพจน์ เราต้องสามารถมองมันได้ว่าในท้ายที่สุดแล้ว มันมีสถานะเพียงสองแบบเท่านั้น คือจริงหรือเท็จ ไม่ใช่ประโยคกำกวม ไม่ใช่คำสั่ง และไม่ใช่ประโยคที่ยังขาดข้อมูลสำคัญบางอย่าง

Example 2.1.2. ประโยคต่อไปนี้นี้เป็นประพจน์

- $2 + 3 = 5$

2. ``กรุงเทพฯ เป็นเมืองหลวงของประเทศไทย"
3. ``เลข 7 เป็นจำนวนเฉพาะ"

Example 2.1.3. ประโยคต่อไปนี้ *ไม่ใช่* ประพจน์

1. ``กรุณาปิดประตู" (เป็นคำสั่ง)
2. `` $x^2 - 3x + 2 = 0$ " (ยังไม่ทราบว่า x คืออะไร)
3. `` n เป็นจำนวนเฉพาะ" (ยังไม่ทราบว่า n คืออะไร)

ประเด็นที่ควรสังเกตก็คือ ประโยคบางชนิด *ดูเหมือน* จะเป็นข้อความทางคณิตศาสตร์ แต่ยังไม่ใช่ประพจน์ในทันที เพราะมันยังมีตัวแปรที่ไม่ได้ระบุความหมายอย่างสมบูรณ์

2.1.2 Predicate หรือประโยคเปิด

Definition 2.1.4. **Predicate** หรือ **ประโยคเปิด** (open sentence) คือประโยคที่มีตัวแปรอยู่ในนั้น และจะกลายเป็นประพจน์ได้ก็ต่อเมื่อเรา

1. กำหนดค่าของตัวแปรนั้นลงไป หรือ
2. ใส่ตัวบ่งปริมาณ เช่น ``สำหรับทุก" หรือ ``มีอย่างน้อยหนึ่ง"

ตัวอย่างเช่น ถ้าเราพิจารณา predicate

$$P(n) : ``n \text{ เป็นจำนวนเฉพาะ}"$$

ประโยคนี้อยู่ไม่ใช่ประพจน์ เพราะยังไม่รู้ว่า n คือจำนวนใด แต่ถ้าเราแทนค่า $n = 4$ เราจะได้ประโยค

$$P(4) : ``4 \text{ เป็นจำนวนเฉพาะ}"$$

ซึ่งตอนนี้เป็นประพจน์แล้ว และเป็นประพจน์เท็จ

อีกวิธีหนึ่งคือใส่ตัวบ่งปริมาณ เช่น

$$\forall n \in \mathbb{N}, P(n) \quad \text{หรือ} \quad \exists n \in \mathbb{N}, P(n)$$

ข้อความเหล่านี้เป็นประพจน์ เพราะมีการระบุชัดแล้วว่าเรากำลังพูดถึงสมาชิกทุกตัว หรือบางตัวของเซตที่กำหนด

2.1.3 จากประโยคเปิดสู่ประพจน์

การเปลี่ยน predicate ให้เป็นประพจน์มีสองแนวทางพื้นฐาน

1. แทนค่าตัวแปร

เช่น จาก " n เป็นจำนวนเฉพาะ" ไปเป็น " 13 เป็นจำนวนเฉพาะ"

2. ใส่ตัวบ่งปริมาณพร้อมระบุโดเมน

เช่น " $\forall n \in \mathbb{Z}$ " หรือ " $\exists n \in \mathbb{N}$ บางตัว"

หากสิ่งที่เรากำลังพิจารณามีอยู่เพียงไม่กี่ตัว เราอาจนิยามตัวบ่งปริมาณได้จากการเชื่อมประโยคหลายประโยคเข้าด้วยกัน เช่นข้อความว่า

" $\forall$ ทุกจำนวนใน $\{2, 3, 4\}$ เป็นจำนวนเฉพาะ"

สามารถมองอย่างไม่เป็นทางการได้ว่าเทียบเท่ากับ

" 2 เป็นจำนวนเฉพาะ และ 3 เป็นจำนวนเฉพาะ และ 4 เป็นจำนวนเฉพาะ"

ส่วนข้อความว่า

" $\exists$ มีจำนวนใน $\{2, 3, 4\}$ เป็นจำนวนเฉพาะ"

ก็เทียบเท่ากับ

" 2 เป็นจำนวนเฉพาะ หรือ 3 เป็นจำนวนเฉพาะ หรือ 4 เป็นจำนวนเฉพาะ"

วิธีคิดแบบนี้ช่วยให้เห็นหน้าที่ของ " $\forall$ " และ " $\exists$ " แต่เมื่อเราต้องพูดถึงเซตที่มีสมาชิกอนันต์ตัว เราจำเป็นต้องใช้สัญลักษณ์ของตัวบ่งปริมาณอย่างจริงจังมากขึ้น ซึ่งจะเป็นหัวข้อสำคัญในบทถัดไป

2.2 เอกภพสัมพัทธ์และข้อตกลงพื้นฐาน

ข้อตกลงที่ใช้ตลอดเล่มช่วงต้น

$$\mathbb{N} = \{0, 1, 2, \dots\}$$

กล่าวคือ ในเอกสารชุดนี้เรานับ 0 เป็นจำนวนนับ

ข้อตกลงเรื่องโดเมนหรือเอกภพสัมพัทธ์เป็นเรื่องเล็กที่มักถูกมองข้าม แต่จริง ๆ แล้วเป็นส่วนสำคัญของการสื่อสารทางคณิตศาสตร์ เพราะข้อความเดียวกันอาจให้ค่าความจริงต่างกันได้ หากเปลี่ยนกลุ่มของวัตถุที่เราากำลังพูดถึง ตัวอย่างเช่น ข้อความว่า

$$\exists x, x^2 = 2$$

ยังไม่สมบูรณ์พอ เพราะเราไม่รู้ว่ x ถูกเลือกมาจากไหน ถ้า $x \in \mathbb{R}$ ข้อความนี้เป็นจริง แต่ถ้า $x \in \mathbb{Z}$ ข้อความนี้

เป็นเท็จ

ดังนั้น ทุกครั้งที่เราเขียนประโยคในรูป ``สำหรับทุก  $x$ '' หรือ ``มี x บางตัว'' เราควรถามตัวเองทันทีว่า x มาจากไหน

ประโยคทางคณิตศาสตร์ที่ดีไม่เพียงต้อง ``จริง'' แต่ต้อง ``ชัด'' ด้วย หลายครั้งความผิดพลาดไม่ได้เกิดจากการคำนวณผิด แต่เกิดจากการไม่ระบุโดเมนให้ชัดตั้งแต่ต้น

2.3 การพิสูจน์คืออะไร

Definition 2.3.1. การพิสูจน์ในมุมมองของรายวิชานี้ คือชุดของบรรทัดข้อความที่เรียงต่อกันอย่างมีเหตุผล โดยแต่ละบรรทัดต้องมีสิ่งรองรับว่า ตามมาจากนิยาม ข้อเท็จจริงที่พิสูจน์แล้ว หรือการอ้างตรรกะที่ถูกต้อง และผู้อ่านต้องสามารถตรวจสอบย้อนกลับได้ว่า แต่ละบรรทัดมาจากอะไร

นิยามนี้สะท้อนมุมมองสำคัญข้อหนึ่ง: การพิสูจน์ไม่ใช่การเขียนให้ดูยาก และไม่ใช่ว่าการเล่าเรื่องยาว ๆ โดยหวังว่าผู้อ่านจะ ``รู้สึกเข้าใจ'' แต่เป็นการจัดลำดับของเหตุผลให้ตรวจสอบได้

ในระดับเริ่มต้น เราจึงควรแยกให้ออกระหว่างสองบทบาทต่อไปนี้

1. สิ่งที่ยืนยัน (claim)
คือประโยคที่เราต้องการบอกว่าจริง
2. เหตุผลหรือฐานรองรับ (reason/justification)
คือคำอธิบายว่าทำไมเราจึงมีสิทธิ์พูดประโยคนั้น

ตัวอย่างเช่น ถ้าเราพูดว่า

`` n เป็นจำนวนคู่''

ประโยคนี้นี้เป็น claim แต่ถ้าจะเขียนมันลงไปในการพิสูจน์ เราต้องตอบให้ได้ด้วยว่า เรารู้ได้อย่างไร มาจากสมมติฐานหรือไม่ มาจากนิยามหรือไม่ หรือเพิ่งพิสูจน์ได้จากบรรทัดก่อนหน้านี้

2.3.1 ทำไมการผ่าน test cases ไม่ใช่การพิสูจน์

ในงานคอมพิวเตอร์ ผู้เรียนจำนวนมากคุ้นกับการตรวจสอบโปรแกรมด้วยการทดลองรัน ซึ่งเป็นสิ่งจำเป็นในทางปฏิบัติ แต่การผ่าน test cases ไม่ได้เท่ากับการพิสูจน์ความถูกต้อง

เหตุผลก็คือ test cases เป็นเพียงการตรวจสอบบางกรณี ในขณะที่ข้ออ้างจำนวนมากในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ มีลักษณะเป็นข้อความแบบ ``สำหรับทุกอินพุตที่เป็นไปตามเงื่อนไข'' ซึ่งเป็นข้อความที่ครอบคลุมกรณีอนันต์จำนวนหรืออย่างน้อยก็จำนวนมหาศาลเกินกว่าจะทดสอบครบได้

ดังนั้นการพิสูจน์จึงเข้ามาทำหน้าที่ตอบคำถามที่การทดลองตอบไม่ได้: ไม่ใช่แค่ว่า *ลองแล้วผ่าน* แต่เป็นว่า *เพราะอะไรจึงต้องผ่านเสมอ*

2.4 นิยามพื้นฐานที่ใช้ฝึกพิสูจน์

เพื่อเริ่มฝึกการพิสูจน์อย่างเป็นรูปธรรม เราจะใช้นิยามของจำนวนคู่และจำนวนคี่ ซึ่งเป็นตัวอย่างที่เรียบง่ายแต่ทรงพลังมากในการฝึก “แก่นิยาม”

Definition 2.4.1. ให้ $n \in \mathbb{Z}$

n เป็นจำนวนคู่ $\iff \exists k \in \mathbb{Z}$ ที่ทำให้ $n = 2k$,

n เป็นจำนวนคี่ $\iff \exists k \in \mathbb{Z}$ ที่ทำให้ $n = 2k + 1$.

จุดสำคัญของนิยามนี้ไม่ได้อยู่ที่การท่องจำรูป $2k$ หรือ $2k + 1$ แต่อยู่ที่การมองเห็นว่า คำว่า “เป็นจำนวนคู่” และ “เป็นจำนวนคี่” จริง ๆ แล้วเป็นข้อความเชิงการมีอยู่ ($\exists$) ซึ่งบอกว่า มี จำนวนเต็มตัวหนึ่งทำให้เขียน n อยู่ในรูปที่กำหนดได้

นี่เป็นเหตุผลว่าทำไมเวลาพิสูจน์เรื่องจำนวนคู่หรือคี่ เรามักเริ่มจากประโยคประเภท “ตามนิยาม จะมี $k \in \mathbb{Z}$ ที่ทำให้...” เพราะเรากำลังเปิดใช้นิยามโดยตรง

เมื่อข้อความหนึ่งอยู่ในรูป “มีบางสิ่งที่ทำให้...” เรามักต้องแสดง *พยาน (witness)* ว่าสิ่งนั้นคืออะไร การตั้งชื่อพยานให้ชัด เช่น k, m, t ช่วยให้พิสูจน์อ่านง่ายขึ้นมาก

2.5 โครงสร้างมาตรฐานของการพิสูจน์แบบอิมพลีเคชัน

ข้อความในรูป

$$P \rightarrow Q$$

อ่านว่า “ถ้า P แล้ว Q ” มักพิสูจน์ด้วยรูปแบบมาตรฐานดังนี้

แพตเทิร์นพื้นฐาน

เพื่อพิสูจน์ ``ถ้า  $P$  แล้ว  $Q$ ``

1. ระบุเอกภพหรือวัตถุที่กำลังพิจารณาให้ชัด
2. สมมติว่า P เป็นจริง
3. ใช้นิยามหรือข้อเท็จจริงที่เกี่ยวข้องผลักดันไปข้างหน้า
4. สรุปว่า Q เป็นจริง

โครงสร้างนี้สำคัญมาก เพราะมันเป็นต้นแบบของพิสูจน์จำนวนมากในวิชาคณิตศาสตร์ โดยเฉพาะข้อความที่มีทั้งตัวบ่งปริมาณและอิมพลิเคชัน เช่น

$$\forall n \in \mathbb{Z}, P(n) \rightarrow Q(n).$$

ในกรณีนี้ เรามักเริ่มด้วยการเลือก $n \in \mathbb{Z}$ แบบใด ๆ แล้วสมมติว่า $P(n)$ จริง ก่อนจะพาเหตุผลไปสู่ $Q(n)$

2.5.1 ตัวอย่างต้นแบบ: ถ้าจำนวนคี่แล้วยกกำลังสองยังคงเป็นคี่

Theorem 2.5.1. สำหรับทุกจำนวนเต็ม n ถ้า n เป็นจำนวนคี่ แล้ว n^2 เป็นจำนวนคี่

Proof idea. ใช้นิยามของจำนวนคี่: ถ้า n คี่ แปลว่ามี $k \in \mathbb{Z}$ บางตัวที่ทำให้ $n = 2k + 1$ จากนั้นคำนวณ n^2 และจัดรูปให้อยู่ในรูป $2(\cdots) + 1$ เพื่อกลับไปใช้นิยามของจำนวนคี่อีกครั้ง

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคี่ ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k + 1.$$

ดังนั้น

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

ให้

$$m = 2k^2 + 2k.$$

เนื่องจาก $k \in \mathbb{Z}$ จึงได้ว่า $m \in \mathbb{Z}$ และจากสมการข้างต้นมีว่า

$$n^2 = 2m + 1.$$

จึงสรุปได้ตามนิยามว่า n^2 เป็นจำนวนคี่

□

2.5.2 อ่านพิสูจน์อย่างนักเขียนพิสูจน์

พิสูจน์ข้างต้นควรถูกอ่านอย่างมีโครงสร้าง ไม่ใช่อ่านเพียงตามบรรทัด

1. “ให้ $n \in \mathbb{Z}$ ” คือการระบุเอกภพสัมพัทธ์
2. “สมมติว่า n เป็นจำนวนคี่” คือการเริ่มต้นพิสูจน์อิมพลิเคชัน
3. “ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z}$...” คือการกระดานิช
4. การตั้ง $m = 2k^2 + 2k$ คือการเลือกพยานของข้อความเชิงการมีอยู่
5. บรรทัดสุดท้ายคือการย้อนกลับไปใช้นิยามเดิมเพื่อปิดพิสูจน์

การอ่านพิสูจน์ด้วยสายตาแบบนี้จะช่วยให้เห็นว่า พิสูจน์ไม่ได้เป็นการคำนวณลอย ๆ แต่เป็นการประกอบโครงสร้างตรรกะทีละขั้น

2.5.3 อีกหนึ่งตัวอย่างสั้น: ถ้าจำนวนคู่แล้วกำลังสองยังคงเป็นคู่

Theorem 2.5.2. สำหรับทุกจำนวนเต็ม n ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

Proof idea. ถ้า n เป็นจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้ $n = 2k$ เมื่อนำไปยกกำลังสองจะได้ $n^2 = 4k^2$ ซึ่งเขียนเป็น $2(\dots)$ ได้

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคู่ ตามนิยามของจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k.$$

ดังนั้น

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2).$$

ให้ $m = 2k^2$ ซึ่งเป็นจำนวนเต็ม จะได้ว่า $n^2 = 2m$ จึงสรุปว่า n^2 เป็นจำนวนคู่ตามนิยาม □

2.6 ตัวอย่างค้านและการปฏิเสธข้อความแบบ “สำหรับทุก”

นอกจากการพิสูจน์ว่าอะไรจริงแล้ว อีกทักษะหนึ่งที่สำคัญไม่แพ้กันคือการชี้ว่าอะไรไม่จริง สำหรับข้อความในรูปแบบ

$$\forall x, P(x)$$

การปฏิเสธไม่ได้ต้องการการอธิบายยาว แต่ต้องการเพียงการหาตัวอย่างสักตัวที่ทำให้ $P(x)$ ล้มเหลว

Definition 2.6.1. ตัวอย่างค้าน (counterexample) คือค่าหรือตัวอย่างเฉพาะที่ทำให้ข้อความแบบ ``สำหรับทุก" เป็นเท็จ

แนวคิดนี้ง่ายแต่ทรงพลังมาก: หากมีคนอ้างว่า

$$\forall p \in \mathbb{Z}, (p \text{ เป็นจำนวนเฉพาะ}) \rightarrow (p \text{ เป็นจำนวนคี่}),$$

เราไม่ต้องรีบพิสูจน์หรือโต้เถียงด้วยคำพูดยาว ๆ แต่ควรถามทันทีว่า มีตัวอย่างสักตัวใหม่ที่เป็นจำนวนเฉพาะแต่ไม่เป็นจำนวนคี่

คำตอบคือ $p = 2$ เพราะ 2 เป็นจำนวนเฉพาะ และ 2 ไม่เป็นจำนวนคี่ ดังนั้นข้อความข้างต้นเป็นเท็จ

สำหรับข้อความสากล ($\forall$) บางครั้งวิธีที่เร็วและชัดที่สุดไม่ใช่การคิดว่าจะพิสูจน์อย่างไร แต่คือการถามกลับว่า ``มีตัวอย่างค้านไหม" นิยมนิยมใช้บ่อยมากทั้งในคณิตศาสตร์และการออกแบบอัลกอริทึม

2.7 การเขียนพิสูจน์ให้เหมือนหนังสือ มากกว่าเหมือนแบบเติมคำ

ในระยะเริ่มต้น ผู้เรียนมักคุ้นกับโครงร่างพิสูจน์แบบเติมช่องว่าง ซึ่งมีประโยชน์มากสำหรับฝึกจับแพทเทิร์น แต่เมื่อเราจะเขียนเป็นหนังสือหรือเอกสารอ้างอิง เป้าหมายควรเปลี่ยนจาก ``เติมให้ครบ" เป็น ``เล่าให้ชัด"

การเขียนที่ดีจึงควรมีคุณลักษณะดังนี้

1. ระบุว่าเรากำลังพิสูจน์ข้อความอะไร
2. บอกแนวคิดสั้น ๆ ว่าจะใช้นิยามหรือวิธีใด
3. เดินพิสูจน์ทีละบรรทัดอย่างมีที่มา
4. จบบทพิสูจน์ในจุดที่ข้อสรุปตามมาอย่างชัดเจน

รูปแบบ

Claim $\rightarrow$ Proof idea $\rightarrow$ Proof $\rightarrow$ QED

จึงเป็นรูปแบบที่ดีสำหรับการฝึกในช่วงต้น เพราะช่วยแยกบทบาทของแต่ละส่วนออกจากกันอย่างชัดเจน และส่งเสริมให้อ่านตรวจสอบโครงสร้างเหตุผลได้ง่าย

2.8 ตัวเชื่อมตรรกะและสูตรตรรกะ

ทั้งหมดทั้งมวลที่กล่าวมา เราเริ่มจากแนวคิดพื้นฐานว่า คณิตศาสตร์เป็นภาษาสำหรับเขียนข้ออ้างให้ชัดเจนและตรวจสอบได้ เราได้เห็นแล้วว่า การพิสูจน์ไม่ได้เป็นเพียงการคำนวณหรือการลองตัวอย่างหลาย ๆ ค่า แต่เป็นการ

เรียงเหตุผลที่ละบรรทัด โดยแต่ละบรรทัดต้องมีที่มารองรับอย่างชัดเจน

บทนี้จะขยับจากระดับของ ``ประพจน์เดี่ยว`` ไปสู่ระดับของ ``รูปประโยค`` กล่าวคือ เราจะพิจารณาว่าถ้าเรานำประพจน์มาประกอบกันด้วยคำว่า ``ไม่``, ``และ``, ``หรือ``, ``ถ้า...แล้ว...``, และ ``ก็ต่อเมื่อ`` แล้วรูปของประโยคที่ได้จะบอกอะไรเกี่ยวกับวิธีพิสูจน์ที่เหมาะสม

แนวคิดสำคัญของบทนี้คือ

รูปของข้อความ $\Rightarrow$ รูปแบบของการพิสูจน์

เมื่อเห็นข้อความในรูป $P \rightarrow Q$ เราควรถามทันทีว่าควรพิสูจน์ตรงดีหรือไม่ หรือควรเปลี่ยนไปพิสูจน์แบบ contrapositive หรือ contradiction เมื่อเห็นข้อความในรูป $P \leftrightarrow Q$ เราควรรู้ทันทีว่าต้องพิสูจน์สองทิศ และเมื่อเห็นข้อความที่มี ``หรือ`` เราควรแยกให้ออกว่าเรากำลังสร้าง ประโยค $\vee$ หรือกำลังใช้ ประโยค $\vee$ ที่มีอยู่แล้วเพื่อสรุปผล

Definition 2.8.1. ให้ P, Q เป็นประพจน์ เราสามารถสร้างประพจน์ใหม่ได้จากตัวเชื่อมตรรกะดังต่อไปนี้

$$\neg P, \quad P \wedge Q, \quad P \vee Q, \quad P \rightarrow Q, \quad P \leftrightarrow Q.$$

ตัวเชื่อมเหล่านี้มีความหมายพื้นฐานดังนี้

- $\neg P$ อ่านว่า ``ไม่เป็นจริงที่  $P$ `` หรือ ``ไม่  $P$ ``
- $P \wedge Q$ อ่านว่า `` $P$  และ  $Q$ ``
- $P \vee Q$ อ่านว่า `` $P$  หรือ  $Q$ `` โดยในวิชานี้หมายถึง ``หรืออย่างน้อยหนึ่งอย่าง`` และ *อาจจริงทั้งคู่ได้*
- $P \rightarrow Q$ อ่านว่า ``ถ้า  $P$  แล้ว  $Q$ ``
- $P \leftrightarrow Q$ อ่านว่า `` $P$  ก็ต่อเมื่อ  $Q$ ``

ค่าความจริงของประพจน์ที่ประกอบด้วยตัวเชื่อมเหล่านี้สามารถอธิบายได้ด้วยตารางค่าความจริง แต่ในรายวิชานี้เป้าหมายหลักไม่ใช่การท่องตารางให้ได้ เป้าหมายคือการเข้าใจ *ความหมาย* ของแต่ละตัวเชื่อมให้ดีพอจนสามารถเลือกวิธีพิสูจน์ที่เหมาะสมได้

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

แม้ตารางนี้จะดูเป็นเรื่องของ ``ค่าความจริง`` แต่ในทางปฏิบัติ สิ่งที่เราสนใจกว่าคือความหมายเชิงโครงสร้างของมัน โดยเฉพาะคอลัมน์ของ $P \rightarrow Q$ และ $P \leftrightarrow Q$ เพราะสองตัวเชื่อมนี้สัมพันธ์กับการพิสูจน์โดยตรง

2.9 อิมพลีเคชัน: ความหมายของข้อความรูป $P \rightarrow Q$

2.9.1 อิมพลีเคชันในฐานะคำสัญญา

ประโยค $P \rightarrow Q$ ควรถูกอ่านว่า

“ทุกครั้งที่ P จริง จะต้องทำให้ Q จริงด้วย”

ในมุมมองของวิทยาการคอมพิวเตอร์ เราอาจมอง P เป็น *precondition* และ Q เป็น *postcondition* กล่าวคือ ถ้าเงื่อนไขตั้งต้น P เกิดขึ้น ระบบจะต้องรับผิดชอบทำให้ผลลัพธ์ Q เกิดขึ้นตามมา

ดังนั้น เมื่อเราเห็นข้อความในรูป $P \rightarrow Q$ สิ่งที่ต้องจับให้ได้ไม่ใช่เพียงคำอ่าน แต่คือคำถามว่า

“ข้อความนี้จะพึงได้ในกรณีใด?”

2.9.2 อิมพลีเคชันเป็นเท็จได้เมื่อใด

จากตารางค่าความจริง เราเห็นว่า $P \rightarrow Q$ เป็นเท็จได้เพียงกรณีเดียว คือ

P จริง แต่ Q เท็จ

นี่เป็นข้อสังเกตที่สำคัญมาก เพราะมันบอกเราทันทีว่า ถ้าต้องการ **หักล้าง** ข้อความในรูป $P \rightarrow Q$ เราต้องหากรณีที่ทำให้ “ส่วนหน้า” เป็นจริง แต่ “ส่วนหลัง” เป็นเท็จ

หลักคิดสำหรับการหักล้างอิมพลีเคชัน

ถ้าต้องการหักล้างข้อความ $P \rightarrow Q$ ให้มองหาตัวอย่างที่ทำให้

P จริง และ Q เท็จ

เพียงตัวอย่างเดียวก็พอที่จะทำให้ข้อความทั้งประโยคเป็นเท็จ

Example 2.9.1. พิจารณาข้อความ

$$\forall n \in \mathbb{Z}, (n^2 \text{ เป็นจำนวนคู่}) \rightarrow (n \text{ หารด้วย 4 ลงตัว})$$

ข้อความนี้เป็นเท็จ เพราะเราเลือก $n = 2$ ได้

เมื่อ $n = 2$ จะมี $n^2 = 4$ ซึ่งเป็นจำนวนคู่ ดังนั้นส่วนหน้าของอิมพลีเคชันเป็นจริง แต่ 2 ไม่หารด้วย 4 ลงตัว ดังนั้นส่วนหลังเป็นเท็จ จึงเป็นตัวอย่างค้านของข้อความนี้

2.9.3 ความจริงแบบว่าง

ข้อที่ผู้เรียนมักรู้สึกขัดใจในช่วงแรกคือ ถ้า P เป็นเท็จ แล้ว $P \rightarrow Q$ จะถือว่าเป็นจริง ไม่ว่า Q จะเป็นจริงหรือเท็จก็ตาม

แนวคิดนี้เรียกว่า **vacuous truth** หรือความจริงแบบว่าง

เหตุผลคือ อิมพลีเคชันสัญญาเฉพาะกรณีที่ P เกิดขึ้นเท่านั้น ถ้า P ไม่เกิดขึ้นเลย ก็ไม่มีกรณีใดที่สัญญาถูกละเมิด

Example 2.9.2. ประโยค

“ถ้า 1 เป็นจำนวนคู่ แล้ว $2 + 2 = 5$ ”

เป็นจริงในเชิงตรรกะ ไม่ใช่เพราะ $2 + 2 = 5$ จริง แต่เพราะส่วนหน้า “1 เป็นจำนวนคู่” เป็นเท็จ

2.9.4 ถ้อยคำที่มักแปลผิด

ข้อความรูป $P \rightarrow Q$ มักถูกเขียนในภาษาธรรมชาติได้หลายแบบ และผู้เรียนมักสับสนกับคำว่า *if*, *only if*, *sufficient*, และ *necessary*

ถ้อยคำที่เทียบเท่ากับ $P \rightarrow Q$

ข้อความต่อไปนี้ล้วนมีความหมายเท่ากับ $P \rightarrow Q$

- “ถ้า P แล้ว Q ”
- “ P only if Q ”
- “ Q if P ”
- “ P เป็นเงื่อนไขเพียงพอสำหรับ Q ”
- “ Q เป็นเงื่อนไขจำเป็นสำหรับ P ”

วิธีจำที่มีประโยชน์มากคือ

“ P only if Q ” หมายความว่า ถ้าไม่มี Q ก็ห้ามมี P

นั่นคือ Q เป็นสิ่งที่ P ต้องพึ่งพา จึงเป็น *necessary condition* ของ P

2.10 จากระบบนิรนัยสู่รูปแบบการพิสูจน์

ในบทก่อนหน้า เราเห็นแล้วว่า การพิสูจน์ข้อความในรูป “ถ้า P แล้ว Q ” มักเริ่มด้วยการสมมติว่า P เป็นจริง แล้วค่อยผลักดันเหตุผลไปจนถึง Q ตอนนี้เราจะเขียนหลักคิดนี้ให้ชัดเจนขึ้นในเชิงตรรกะ

โครงสร้างมาตรฐานของการพิสูจน์แบบ direct proof

เมื่อต้องการพิสูจน์ข้อความ $P \rightarrow Q$

1. ระบุวัตถุที่กำลังพิจารณาให้ชัด
2. สมมติว่า P เป็นจริง
3. ใช้นิยาม ข้อเท็จจริง หรือทฤษฎีบทที่เกี่ยวข้อง
4. สรุปว่า Q เป็นจริง

Theorem 2.10.1. สำหรับทุกจำนวนเต็ม n ถ้า n หารด้วย 4 ลงตัว แล้ว n เป็นจำนวนคู่

Proof idea. ใช้ความหมายของ “หารด้วย 4 ลงตัว” แล้วจัดรูปให้เห็นว่า n อยู่ในรูป $2(\dots)$

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n หารด้วย 4 ลงตัว ตามนิยาม จะมีจำนวนเต็ม k ที่ทำให้

$$n = 4k.$$

ดังนั้น

$$n = 4k = 2(2k).$$

เนื่องจาก $2k \in \mathbb{Z}$ จึงได้ว่า n อยู่ในรูป $2m$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n เป็นจำนวนคู่ □

ทฤษฎีบทนี้เป็นตัวอย่างสำคัญ เพราะมันแสดงรูปแบบพื้นฐานที่สุดของการพิสูจน์อิมพลีเคชัน: เริ่มจากสมมติฐานเปิดนิยาม แล้วจัดรูปไปสู่ข้อสรุป

2.11 สมมูลตรรกะที่มีประโยชน์ต่อการพิสูจน์

บางครั้งการพิสูจน์ $P \rightarrow Q$ ตรง ๆ อาจไม่สะดวก ในกรณีเช่นนี้ เรามักเปลี่ยนรูปประโยคให้สมมูลกับประโยคเดิม แต่พิสูจน์ง่ายกว่า

2.11.1 Contrapositive

สมมูลที่สำคัญที่สุดอันหนึ่งคือ

$$P \rightarrow Q \quad \equiv \quad \neg Q \rightarrow \neg P.$$

ข้อความด้านขวาเรียกว่า **contrapositive** ของข้อความด้านซ้าย

ความสำคัญของ contrapositive ไม่ได้อยู่ที่การเล่นสัญลักษณ์ แต่อยู่ที่ข้อเท็จจริงว่า บางครั้งการพิสูจน์ ``ถ้า  $P$  แล้ว  $Q$ `` ตรง ๆ นั้นยาก แต่การพิสูจน์ ``ถ้าไม่  $Q$  แล้วไม่  $P$ `` กลับง่ายกว่ามาก

ข้อควรจำ

สำหรับข้อความ $P \rightarrow Q$

- **contrapositive** คือ $\neg Q \rightarrow \neg P$
- **converse** คือ $Q \rightarrow P$

ข้อความเดิมสมมูลกับ contrapositive แต่ไม่จำเป็นต้องสมมูลกับ converse

Theorem 2.11.1. สำหรับทุกจำนวนเต็ม n ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่

Proof idea. พิสูจน์แบบ contrapositive แทน กล่าวคือพิสูจน์ว่า ถ้า n ไม่เป็นจำนวนคู่ หรือกล่าวอีกแบบคือ n เป็นจำนวนคี่ แล้ว n^2 ไม่เป็นจำนวนคู่ หรือก็คือ n^2 เป็นจำนวนคี่

Proof. ให้ $n \in \mathbb{Z}$ และสมมติว่า n เป็นจำนวนคี่ ตามนิยามของจำนวนคี่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k + 1.$$

ดังนั้น

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

จึงได้ว่า n^2 อยู่ในรูป $2m + 1$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n^2 เป็นจำนวนคี่

เราได้พิสูจน์แล้วว่า ถ้า n คี่ แล้ว n^2 คี่ ซึ่งเป็น contrapositive ของข้อความที่ต้องการ ดังนั้นสรุปได้ว่า ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่ □

2.11.2 De Morgan

เมื่อใช้ contrapositive หรือ contradiction เรามักต้องปฏิเสธข้อความที่ประกอบด้วย ``และ" หรือ ``หรือ" จึงควรรู้สมมูลของ De Morgan ต่อไปนี้

$$\neg(A \wedge B) \equiv (\neg A \vee \neg B)$$

$$\neg(A \vee B) \equiv (\neg A \wedge \neg B)$$

สมมูลคู่นี้ใช้บ่อยมาก เพราะมันบอกเราว่า การปฏิเสธประโยคประกอบไม่ได้เพียงแค่ใส่เครื่องหมาย $\neg$ ไว้ข้างหน้า แต่ต้องเปลี่ยนตัวเชื่อมและปฏิเสธองค์ประกอบภายในด้วย

Example 2.11.2. นิเสธของข้อความ “ a เป็นจำนวนคู่ และ b เป็นจำนวนคู่” คือ “ a ไม่เป็นจำนวนคู่ หรือ b ไม่เป็นจำนวนคู่”

นิเสธของข้อความ “ $a \geq 0$ หรือ $b < 0$ ” คือ “ $a < 0$ และ $b \geq 0$ ”

2.11.3 อิมพลีเคชันในรูปดิสจังก์ชัน

อีกสมมูลหนึ่งที่ใช้กันได้บ่อยคือ

$$P \rightarrow Q \equiv \neg P \vee Q.$$

สมมูลนี้ช่วยอธิบายว่าทำไมอิมพลีเคชันจึงเป็นจริงเมื่อ P เท็จ และยังช่วยในการแปลงรูปข้อความให้มองเห็นโครงสร้างของการพิสูจน์ชัดเจน โดยเฉพาะเวลาที่ต้องพิสูจน์ข้อความในรูปดิสจังก์ชัน

2.12 การพิสูจน์แบบ contradiction

นอกจากการพิสูจน์ตรงและการพิสูจน์แบบ contrapositive แล้ว อีกวิธีสำคัญคือ **proof by contradiction**

แนวคิดพื้นฐานมีสองแบบที่เกี่ยวข้องกัน

1. ถ้าต้องการพิสูจน์ประโยคเดียว P เราอาจสมมติว่า $\neg P$ แล้วพาไปสู่ความขัดแย้ง
2. ถ้าต้องการพิสูจน์ $P \rightarrow Q$ เราอาจสมมติว่า P จริงและ Q เท็จพร้อมกัน

เชิงตรรกะแล้ว

$$P \rightarrow Q \equiv \neg(P \wedge \neg Q)$$

ดังนั้น ถ้าเราพิสูจน์ได้ว่า “กรณี P จริงแต่ Q เท็จ” เป็นไปไม่ได้ เราก็พิสูจน์ $P \rightarrow Q$ ได้

Theorem 2.12.1. ไม่มีจำนวนเต็มใดที่เป็นทั้งจำนวนคู่และจำนวนคี่พร้อมกัน

Proof idea. สมมติว่ามีจำนวนเต็มดังกล่าวอยู่จริง แล้วใช้นิยามของจำนวนคู่และจำนวนคี่พร้อมกัน จะได้สมการที่เป็นไปไม่ได้

Proof. สมมติขัดแย้งว่า มีจำนวนเต็ม n ที่เป็นทั้งจำนวนคู่และจำนวนคี่พร้อมกัน

เพราะ n เป็นจำนวนคู่ จะมี $a \in \mathbb{Z}$ ที่ทำให้

$$n = 2a.$$

และเพราะ n เป็นจำนวนคี่ จะมี $b \in \mathbb{Z}$ ที่ทำให้

$$n = 2b + 1.$$

ดังนั้น

$$2a = 2b + 1.$$

จัดรูปได้ว่า

$$2(a - b) = 1.$$

แต่จำนวนทางซ้ายเป็นจำนวนคู่ ส่วนจำนวนทางขวาเป็นจำนวนคี่ ซึ่งเป็นไปไม่ได้ เกิดความขัดแย้ง

ดังนั้นสมมติฐานที่ว่า n เป็นจำนวนคี่เป็นทั้งคู่และคี่พร้อมกันจึงเป็นเท็จ สรุปว่าไม่มีจำนวนเต็มดังกล่าว $\square$

ตัวอย่างนี้แสดงให้เห็นว่า การพิสูจน์แบบ contradiction ไม่ได้แปลว่า เขียนคำว่า "ขัดแย้ง" ลงไปแล้วจบ แต่ต้องแสดงให้เห็นอย่างชัดเจนว่าความขัดแย้งเกิดขึ้นตรงไหน

2.13 การใช้อิมพลีเคชันที่พิสูจน์แล้ว: Modus Ponens

เมื่อเราพิสูจน์ข้อความอิมพลีเคชันได้แล้ว เรามักต้องนำมันไปใช้ต่อในพิสูจน์อื่น กฎการอ้างเหตุผลพื้นฐานที่สุดคือ **Modus Ponens**

Modus Ponens	
ถ้าเรารู้ว่า	$P \rightarrow Q$ และ P
แล้วเราสรุปได้ว่า	$Q.$

ในภาษาเรียบง่าย กฎนี้บอกว่า ถ้ามีกฎว่าทุกครั้งที่ P จริงจะได้ Q จริง และตอนนี้เรารู้แล้วว่า P จริง เราก็มีสิทธิ์สรุปว่า Q จริง

Example 2.13.1. สมมติว่าเราได้พิสูจน์มาแล้วว่า

$$(\forall n \in \mathbb{Z})(n \text{ เป็นจำนวนคู่} \rightarrow n^2 \text{ เป็นจำนวนคู่}).$$

เราต้องการพิสูจน์ว่า

$$(\forall x \in \mathbb{Z})(x \text{ เป็นจำนวนคู่} \rightarrow x^2 + 1 \text{ เป็นจำนวนคี่}).$$

ให้ $x \in \mathbb{Z}$ และสมมติว่า x เป็นจำนวนคู่ จากทฤษฎีบทที่พิสูจน์มาแล้ว สรุปได้ว่า x^2 เป็นจำนวนคู่ ดังนั้นจะมี $k \in \mathbb{Z}$ ที่ทำให้ $x^2 = 2k$ จึงได้ว่า

$$x^2 + 1 = 2k + 1$$

ซึ่งเป็นจำนวนคี่

ตัวอย่างนี้สะท้อนภาพสำคัญของคณิตศาสตร์ทั้งวิชา: เราไม่ได้พิสูจน์แต่ละข้อความแยกขาดจากกัน แต่สะสมข้อเท็จจริงไว้เป็นเครื่องมือสำหรับพิสูจน์ข้อความใหม่ต่อไป

2.14 สรุปส่งท้ายอิมพลีเคชัน

ถ้าเป้าหมายอยู่ในรูป

$$A \rightarrow B,$$

แล้วการพิสูจน์ตรง (*direct proof*) เริ่มจากการสมมติว่า A เป็นจริง และพาเหตุผลไปจนได้ B

โครงพื้นฐานของ *direct proof*

เมื่อจะพิสูจน์ $A \rightarrow B$

1. สมมติว่า A เป็นจริง
2. ใช้นิยาม ข้อเท็จจริง หรือผลลัพธ์ที่พิสูจน์แล้ว
3. สรุปว่า B เป็นจริง

ถ้าจะพิสูจน์แบบ *contrapositive* เราจะสมมติ A แต่จะสมมติ $\neg B$ แล้วพยายามสรุป $\neg A$ เพราะ

$$A \rightarrow B \quad \text{สมมูลกับ} \quad \neg B \rightarrow \neg A.$$

ถ้าจะพิสูจน์แบบ *contradiction* เราจะสมมติกรณีที่ทำให้ $A \rightarrow B$ ล้มเหลว นั่นคือ

$$A \wedge \neg B,$$

แล้วผลักเหตุผลไปจนเกิดสิ่งที่เป็นไปไม่ได้

ข้อควรจำ

เวลาพิสูจน์ข้อความรูป $A \rightarrow B$

- direct: เริ่มจาก A
- contrapositive: เริ่มจาก $\neg B$
- contradiction: เริ่มจาก A และ $\neg B$

ดังนั้น ถ้าเริ่มพิสูจน์ผิดจุดตั้งแต่ต้น แม้บรรทัดถัด ๆ ไปจะดูสมเหตุสมผล พิสูจน์นั้นก็ยิ่งผิดโครงสร้างอยู่ดี

2.15 ดิสจังก์ชันและการพิสูจน์แบบแยกกรณี

2.15.1 ความหมายของ $P \vee Q$

ในวิชานี้ $P \vee Q$ หมายถึง “ P หรือ Q หรือทั้งคู่” ดังนั้นมันเป็นจริงเมื่อมีอย่างน้อยหนึ่งประโยคในสองประโยคนี้นี้เป็นจริง

เมื่อเจอดิสจังก์ชัน เรามักพบอยู่ในสองสถานการณ์หลัก

1. เราต้องการพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง
2. เราารู้แล้วว่า $P \vee Q$ เป็นจริง และต้องการใช้มันเพื่อสรุปผลบางอย่าง

สองสถานการณ์นี้หน้าตาคล้ายกัน แต่ใช้เหตุผลคนละแบบ

2.15.2 การพิสูจน์ให้ได้ว่า $P \vee Q$ เป็นจริง

ถ้าต้องการพิสูจน์ $P \vee Q$ เรามีหลายแนวทาง เช่น

- พิสูจน์ P ให้ได้โดยตรง
- หรือพิสูจน์ Q ให้ได้โดยตรง
- หรือแปลงรูปข้อความเดิมให้อยู่ในรูปที่พิสูจน์ง่ายกว่า

Theorem 2.15.1. ให้ $a, b \in \mathbb{Z}$. ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

Proof idea. พิสูจน์แบบ contrapositive จะง่ายกว่า เพราะ contrapositive ของข้อความนี้คือ ถ้า a ไม่เป็นจำนวนคู่ และ b ไม่เป็นจำนวนคู่ แล้ว ab ไม่เป็นจำนวนคู่ ซึ่งเท่ากับการพิสูจน์ว่า ถ้า a, b เป็นจำนวนคี่ทั้งคู่ แล้ว ab เป็นจำนวนคี่

Proof. เราจะพิสูจน์ contrapositive

ให้ $a, b \in \mathbb{Z}$ และสมมติว่า a ไม่เป็นจำนวนคู่ และ b ไม่เป็นจำนวนคู่ สำหรับจำนวนเต็ม ข้อความ “ไม่เป็นจำนวนคู่” เท่ากับ “เป็นจำนวนคี่” ดังนั้นจะมี $r, s \in \mathbb{Z}$ ที่ทำให้

$$a = 2r + 1, \quad b = 2s + 1.$$

คูณกันได้

$$ab = (2r + 1)(2s + 1) = 4rs + 2r + 2s + 1 = 2(2rs + r + s) + 1.$$

จึงได้ว่า ab เป็นจำนวนคี่ นั่นคือ ab ไม่เป็นจำนวนคู่

ดังนั้น contrapositive เป็นจริง จึงสรุปได้ว่า ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ $\square$

2.15.3 การใช้ $P \vee Q$: proof by cases

ถ้าเรารู้ว่า $P \vee Q$ เป็นจริง เรา ยังเลือกไม่ได้ทันที ว่า P จริงหรือ Q จริง สิ่งที่เราารู้มีเพียงว่า อย่างน้อยหนึ่งกรณีต้องเกิดขึ้น

ดังนั้นถ้าต้องการสรุปผลลัพธ์เดียวกัน R เราต้องพิสูจน์ว่า

$$P \rightarrow R \quad \text{และ} \quad Q \rightarrow R.$$

นี่คือหลักของ **proof by cases** หรือ $\vee$ -elimination

Proof by Cases

ถ้ารู้ว่า $P \vee Q$ เป็นจริง และเราพิสูจน์ได้ว่า

$$P \rightarrow R \quad \text{และ} \quad Q \rightarrow R$$

แล้วสรุปได้ว่า R จริง

Theorem 2.15.2. ให้ $a, b \in \mathbb{Z}$. ถ้า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ แล้ว ab เป็นจำนวนคู่

Proof idea. แยกเป็นสองกรณีตามดิซจังก์ชันที่กำหนดมา

Proof. ให้ $a, b \in \mathbb{Z}$ และสมมติว่า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

เราพิจารณาเป็นสองกรณี

กรณีที่ 1: a เป็นจำนวนคู่

จะมี $k \in \mathbb{Z}$ ที่ทำให้ $a = 2k$ ดังนั้น

$$ab = (2k)b = 2(kb).$$

เนื่องจาก $kb \in \mathbb{Z}$ จึงได้ว่า ab เป็นจำนวนคู่

กรณีที่ 2: b เป็นจำนวนคู่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้ $b = 2\ell$ ดังนั้น

$$ab = a(2\ell) = 2(a\ell).$$

เนื่องจาก $a\ell \in \mathbb{Z}$ จึงได้ว่า ab เป็นจำนวนคู่

ในทุกกรณี เราสรุปได้ว่า ab เป็นจำนวนคู่ ดังนั้นถ้า a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่ แล้ว ab เป็นจำนวนคู่ $\square$

ข้อผิดพลาดที่พบบ่อย

เมื่อมี $P \vee Q$ อยู่ในมือ เรา ห้าม เลือกเอาเองว่า ``นั่นให้ P จริง" เพราะข้อมูลที่มิได้บอกว่า P จริง
แน่ ๆ มันบอกเพียงว่าต้องมีอย่างน้อยหนึ่งใน P, Q ที่จริง ดังนั้นถ้าจะสรุปผลจาก $P \vee Q$ ให้ถูกต้อง ต้อง
ครอบคลุมทุกกรณีที่เป็นไปได้

2.16 ไบคอนดิชันนัลและการพิสูจน์สองทิศ

ข้อความ $P \leftrightarrow Q$ อ่านว่า `` P ก็ต่อเมื่อ Q '' ซึ่งหมายความว่า P และ Q จริงเทียบเท่ากัน กล่าวคือ แต่ละ
ข้อความเป็นทั้งเงื่อนไขจำเป็นและเงื่อนไขเพียงพอของอีกข้อความหนึ่ง

เชิงตรรกะ เรามีสมมูล

$$P \leftrightarrow Q \quad \equiv \quad (P \rightarrow Q) \wedge (Q \rightarrow P).$$

ดังนั้น เมื่อต้องการพิสูจน์ข้อความแบบ iff สิ่งขั้นต่ำที่ต้องทำคือพิสูจน์ให้ครบสองทิศ

Theorem 2.16.1. สำหรับทุกจำนวนเต็ม n ,

$$n \text{ หารด้วย 4 ลงตัว} \leftrightarrow (n \text{ เป็นจำนวนคู่ และ } n/2 \text{ เป็นจำนวนคู่})$$

Proof idea. พิสูจน์ทีละทิศ ทิศแรกเปิดนิยามของการหารด้วย 4 ลงตัว ส่วนทิศกลับใช้ข้อมูลว่า $n/2$ เป็นจำนวน
คู่ แล้วคูณกลับด้วย 2

Proof. เราจะแยกพิสูจน์สองทิศ

($\rightarrow$) สมมติว่า n หารด้วย 4 ลงตัว ดังนั้นมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 4k.$$

จึงได้ว่า

$$n = 2(2k),$$

ดังนั้น n เป็นจำนวนคู่

อีกทั้ง

$$n/2 = 2k,$$

ซึ่งเป็นจำนวนคู่เช่นกัน จึงได้ว่า n เป็นจำนวนคู่ และ $n/2$ เป็นจำนวนคู่

($\leftarrow$) สมมติว่า n เป็นจำนวนคู่ และ $n/2$ เป็นจำนวนคู่ จากที่ $n/2$ เป็นจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n/2 = 2k.$$

คูณทั้งสองข้างด้วย 2 จะได้

$$n = 4k.$$

ดังนั้น n หารด้วย 4 ลงตัว

เมื่อพิสูจน์ครบทั้งสองทิศแล้ว สรุปได้ว่า

$$n \text{ หารด้วย 4 ลงตัว} \leftrightarrow (n \text{ เป็นจำนวนคู่ และ } n/2 \text{ เป็นจำนวนคู่})$$

เป็นจริงสำหรับทุกจำนวนเต็ม n

□

2.17 ข้อผิดพลาดที่พบบ่อยในการเขียนพิสูจน์

เมื่อเริ่มใช้ตรรกศาสตร์เป็นเครื่องมือวางโครงพิสูจน์ ผู้เรียนมักผิดพลาดในรูปแบบเดิม ๆ ไม่ก็แบบ การรู้จักข้อผิดพลาดเหล่านี้จะช่วยให้ตรวจงานของตนเองได้ดีขึ้นมาก

2.17.1 พิสูจน์ converse แทนที่จะพิสูจน์ข้อความเดิม

ถ้าต้องการพิสูจน์ $P \rightarrow Q$ แต่กลับเริ่มจากสมมติว่า Q จริง แล้วสรุปว่า P จริง สิ่งที่กำลังพิสูจน์อยู่คือ $Q \rightarrow P$ ซึ่งเป็น converse ไม่ใช่ข้อความเดิม

ตัวอย่างเช่น ถ้าต้องการพิสูจน์

ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

แต่กลับเริ่มจากสมมติว่า n^2 เป็นจำนวนคู่ นั้นไม่ใช่การพิสูจน์ตรงของข้อความนี้

2.17.2 พิสูจน์ iff เพียงทิศเดียว

ถ้าต้องการพิสูจน์ $P \leftrightarrow Q$ แต่พิสูจน์ได้แค่ $P \rightarrow Q$ ก็ยังถือว่าไม่จบ เพราะยังเหลืออีกทิศหนึ่งที่ต้องแสดง

2.17.3 ใช้ proof by cases แบบไม่ครบทุกกรณี

ถ้ามีเพียง $P \vee Q$ เราไม่อาจเลือกข้างเองว่ากรณี P เกิดขึ้น เพราะข้อมูลยังไม่พอ สิ่งที่ต้องทำคือแสดงให้ครบว่าไม่ว่าจะเป็นกรณี P หรือกรณี Q ผลลัพธ์ที่ต้องการก็ยังคงตามมา

2.17.4 พิสูจน์แบบ contradiction โดยสมมติผิดตัว

ถ้าต้องการพิสูจน์ $P \rightarrow Q$ ด้วย contradiction สิ่งที่เราควรสมมติคือ $P \wedge \neg Q$ ไม่ใช่สมมติ $\neg Q$ อย่างลอย ๆ และไม่ใช่สมมติสิ่งเดียวกับที่ต้องการพิสูจน์แล้วประกาศว่าขัดแย้งทันที

ความขัดแย้งต้องเป็นสิ่งที่เกิดขึ้นจากเหตุผล ไม่ใช่เพียงคำบอกว่า “ขัดแย้ง”

2.18 สรุป

ใจความของบทนี้สามารถย่อได้ดังนี้

1. ตัวเชื่อมตรรกะไม่ได้มีไว้เพื่อเขียนตารางค่าความจริงเท่านั้น แต่มันบอกเราด้วยว่าพิสูจน์อย่างไร
2. ข้อความ $P \rightarrow Q$ เป็นเท็จได้เพียงกรณีเดียว คือ P จริงแต่ Q เท็จ
3. ถ้าพิสูจน์ $P \rightarrow Q$ ตรง ๆ ยาก เราอาจเปลี่ยนไปพิสูจน์ contrapositive $\neg Q \rightarrow \neg P$
4. การพิสูจน์แบบ contradiction คือการแสดงว่ากรณีตรงข้ามนำไปสู่สิ่งที่เป็นไปไม่ได้

5. ดิสจังก์ชัน $P \vee Q$ ต้องแยกให้ออกว่าเรากำลังสร้างมัน หรือกำลังใช้มัน
6. ข้อความ $P \leftrightarrow Q$ ต้องพิสูจน์สองทิศเสมอ
7. การพิสูจน์ที่ดีต้องไม่เพียงถูก แต่ต้องชัดด้วยว่าแต่ละบรรทัดมาจากอะไร

มุมมองที่ควรติดตัวไปต่อ

เมื่อเห็นข้อความทางคณิตศาสตร์ อย่าเพิ่งรีบคำนวณ ให้ถามก่อนว่า “รูปของข้อความนี้คืออะไร” เพราะบ่อยครั้ง วิธีพิสูจน์ที่เหมาะสมถูกซ่อนอยู่ในรูปของประโยคนั้นเอง

แบบฝึกหัดท้ายบท

Exercise 2.18.1. พิจารณาว่าข้อความต่อไปนี้ประพจน์หรือไม่ ถ้าเป็น ให้ระบุว่าเป็นจริงหรือเท็จ

1. $5^2 = 25$
2. " $x + 1 = 3$ "
3. "กรุณาส่งงานตรงเวลา"
4. "มีจำนวนเต็มที่ยกกำลังสองแล้วได้ 9"

Exercise 2.18.2. อธิบายว่าข้อความ

$$\forall x, x^2 \geq 0$$

ยังไม่สมบูรณ์พออย่างไรหากไม่ระบุโดเมนของ x และจงเขียนใหม่ให้สมบูรณ์อย่างน้อยสองแบบ

Exercise 2.18.3. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า a และ b เป็นจำนวนคู่ แล้ว $a + b$ เป็นจำนวนคู่

Exercise 2.18.4. หาตัวอย่างค้านของข้อความต่อไปนี้

$$\forall n \in \mathbb{Z}, (n \text{ เป็นจำนวนคี่}) \rightarrow (n \text{ หารด้วย 3 ลงตัว})$$

และอธิบายสั้น ๆ ว่าทำไมตัวอย่างนั้นจึงใช้ได้

Exercise 2.18.5. พิจารณาว่าข้อความต่อไปนี้จริงหรือเท็จ และถ้าเป็นเท็จให้หาตัวอย่างค้าน

1. $\forall n \in \mathbb{Z}, (n \text{ หารด้วย 6 ลงตัว}) \rightarrow (n \text{ เป็นจำนวนคู่})$
2. $\forall n \in \mathbb{Z}, (n \text{ เป็นจำนวนคู่}) \rightarrow (n \text{ หารด้วย 4 ลงตัว})$
3. $\forall p \in \mathbb{Z}, (p \text{ เป็นจำนวนเฉพาะ}) \rightarrow (p \text{ เป็นจำนวนคี่})$

Exercise 2.18.6. เขียนข้อความต่อไปนี้เป็นสัญลักษณ์ โดยให้ P, Q เป็นประพจน์

1. " P only if Q "
2. " Q เป็นเงื่อนไขจำเป็นสำหรับ P "
3. " P เป็นเงื่อนไขเพียงพอสำหรับ Q "
4. " P ก็ต่อเมื่อ Q "

Exercise 2.18.7. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า $a + b$ เป็นจำนวนคี่ แล้วอย่างน้อยหนึ่งใน a, b เป็นจำนวนคี่

Exercise 2.18.8. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม a, b ถ้า a และ b เป็นจำนวนคี่ทั้งคู่ แล้ว ab เป็นจำนวนคี่ แล้วใช้ผลลัพธ์นี้เพื่อสรุปอีกครั้งว่า ถ้า ab เป็นจำนวนคู่ แล้ว a เป็นจำนวนคู่ หรือ b เป็นจำนวนคู่

Exercise 2.18.9. พิสูจน์ว่า สำหรับทุกจำนวนเต็ม n ,

$$n^2 \text{ เป็นจำนวนคู่} \vee n^2 \text{ เป็นจำนวนคี่}$$

โดยใช้การแยกกรณีจากข้อเท็จจริงที่ว่า จำนวนเต็มทุกตัวเป็นจำนวนคู่หรือจำนวนคี่

Exercise 2.18.10. อธิบายว่าทำไมการพิสูจน์ข้อความ $P \leftrightarrow Q$ ด้วยการพิสูจน์เพียงทิศเดียวจึงยังไม่เพียงพอ พร้อมยกตัวอย่างข้อความจริงสักหนึ่งข้อความในรูป iff แล้วเขียนโครงพิสูจน์สองทิศให้ครบ

Exercise 2.18.11. ให้โดเมนเป็น “โปรแกรมที่รับอินพุตจำนวนเต็ม n ” และให้ $P(n)$ = “โปรแกรมคืนค่า 1”, $Q(n)$ = “ n เป็นจำนวนคี่”

(ก) เขียนประโยค “โปรแกรมคืนค่า 1 iff อินพุตเป็นจำนวนคี่” เป็นสัญลักษณ์

(ข) เขียน “อินพุตเป็นคี่เป็นเงื่อนไขเพียงพอสำหรับการคืนค่า 1” เป็นสัญลักษณ์

Exercise 2.18.12. พิสูจน์ว่า “สำหรับทุก $a, b \in \mathbb{Z}$, ถ้า a เป็นจำนวนคู่ แล้ว $a + b$ เป็นจำนวนคี่ หรือ b เป็นจำนวนคู่”

1. ด้วยวิธี direct proof
2. ด้วยวิธี contrapositive
3. ด้วยวิธี contradiction

Exercise 2.18.13. เพราะเหตุใดที่การจะพิสูจน์ข้อความ $p \rightarrow q$ ด้วยวิธี contradiction จึงไม่ขึ้นอยู่กับทิศทางของข้อความไม่ว่าจะตั้งข้อความแบบ direct หรือแบบ contrapositive

Exercise 2.18.14. จงยกตัวอย่างได้แก่ให้แต่ละข้อความต่อไปนี้ และเขียนอธิบาย 1 ประโยคว่าทำไมใช้ได้

- (a) $\forall n \in \mathbb{Z}, (n \text{ เป็นจำนวนคี่}) \rightarrow (n \text{ หารด้วย 3 ลงตัว})$
- (b) $\forall p \in \mathbb{Z}, (p \text{ เป็นจำนวนเฉพาะ}) \rightarrow (p \text{ เป็นจำนวนคี่})$

Exercise 2.18.15. ปฏิเสธข้อความต่อไปนี้โดยหาตัวอย่างค้าน:

$$\forall n \in \mathbb{Z}, ((n \text{ หารด้วย 2 ลงตัว}) \vee (n \text{ หารด้วย 3 ลงตัว})) \rightarrow (n \text{ หารด้วย 6 ลงตัว})$$

โดยนิยามของการหารลงตัวคือ

Definition 2.18.16. สำหรับจำนวนเต็ม a, b ใด ๆ จะกล่าวว่า a หารด้วย b ลงตัว ก็ต่อเมื่อ มีจำนวนเต็ม k ที่ทำให้ $a = bk$ และเราจะเรียก a ว่าตัวตั้ง เรียก b ว่าตัวหาร และเรียก k ว่าผลหารจากการหารลงตัวนี้

บทที่ 3

ตัวบ่งปริมาณและแพตเทิร์นพื้นฐานของการพิสูจน์

บทก่อนหน้านี้เราศึกษาตรรกศาสตร์ประพจน์ในฐานะ โครง ของการพิสูจน์ ใจความสำคัญคือ เมื่อเรามองเห็นรูปของข้อความ เรามักมองเห็นวิธีพิสูจน์ที่เหมาะสมตามไปด้วย ข้อความในรูป ``ถ้า ... แล้ว ...`` ชี้ให้เราเริ่มจากสมมติส่วนหน้า ข้อความในรูป ``ก็ต่อเมื่อ`` บอกเราว่าต้องเดินเหตุผลให้ครบสองทิศ และข้อความที่มี ``หรือ`` บังคับให้เราะวังว่าคำว่า ``หรือ`` กำลังปรากฏอยู่ในฐานะ ข้อมูลที่มีอยู่แล้ว หรือเป็น ข้อสรุปที่เราต้องการสร้าง

ในบทนี้เราจะเดินต่อจากจุดนั้นไปสู่ประโยคที่มีตัวบ่งปริมาณ ได้แก่ ``สำหรับทุก`` และ ``มีอยู่`` ซึ่งเป็นภาษามาตรฐานในการกล่าวอ้างเกี่ยวกับวัตถุจำนวนมาก โดยเฉพาะข้อความที่พบอยู่เสมอในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ เช่น

$$\forall n \in \mathbb{Z}, P(n), \quad \exists x \in U, Q(x), \quad \forall x \in U \exists y \in V, R(x, y).$$

เมื่อมีตัวบ่งปริมาณเข้ามา การพิสูจน์จะต้องระวังมากขึ้นอีกหนึ่งระดับ: เราไม่เพียงต้องดูว่าโครงของข้อความเป็นอิมพลีเคชันหรือไม่ แต่ต้องดูด้วยว่าเรากำลังพูดถึงวัตถุ *ทุกตัว* หรือเพียง *บางตัว* และหากเป็น ``บางตัว`` เราต้องแสดงพยานอย่างไร หากเป็น ``ทุกตัว`` เราต้องเลือกสมาชิกแบบใดจึงจะถูกต้อง

3.1 ตัวบ่งปริมาณและเอกภพสัมพัทธ์

เมื่อเราเขียนข้อความที่มีตัวแปร เรายังไม่ได้ประพจน์ที่มีค่าความจริงแน่นอนเสมอไป หลายครั้งเราจะได้เพียง *predicate* หรือประโยคเปิด ซึ่งจะกลายเป็นประพจน์ก็ต่อเมื่อกำหนดตัวแปรให้ชัด หรือครอบมั้นด้วยตัวบ่งปริมาณ

Definition 3.1.1. ให้ U เป็นเอกภพสัมพัทธ์

1. ตัวบ่งปริมาณสากล (universal quantifier)

$$\forall x \in U, P(x)$$

หมายความว่า $P(x)$ เป็นจริงสำหรับทุก x ใน U

2. ตัวบ่งปริมาณการมี (existential quantifier)

$$\exists x \in U, P(x)$$

หมายความว่าอย่างน้อยหนึ่ง x ใน U ที่ทำให้ $P(x)$ เป็นจริง

สิ่งที่ห้ามมองข้ามคือ *เอกภพสัมพัทธ์* หรือโดเมน เพราะข้อความเดียวกันอาจเปลี่ยนค่าความจริงได้ทันทีเมื่อเปลี่ยนโดเมน

Example 3.1.2. ข้อความ

$$\exists x, x^2 = 2$$

ยังไม่สมบูรณ์พอถ้าไม่บอกว่า x มาจากไหน

ถ้า $x \in \mathbb{R}$ ข้อความนี้เป็นจริง แต่ถ้า $x \in \mathbb{Z}$ ข้อความนี้เป็นเท็จ

ดังนั้นการเขียนว่า “สำหรับทุก x ” โดยไม่บอกขอบเขตของ x มักทำให้ข้อความกำกวมโดยไม่จำเป็น โดยเฉพาะในบทพิสูจน์ที่ต้องตรวจสอบละเอียดที่ละบรรทัด

หลักการเขียนที่ควรยึด

เมื่อเขียนข้อความที่มีตัวแปร ให้พยายามระบุโดเมนให้ชัดเจนเสมอ เช่น

$$\forall n \in \mathbb{Z}, \quad \exists x \in \mathbb{R}, \quad \forall user \in U.$$

3.2 เมื่อเป้าหมายเป็น $\forall$: เลือกสมาชิกแบบ arbitrary

การพิสูจน์ข้อความสากลมีรูปแบบที่ค่อนข้างตายตัว

แพตเทิร์นของการพิสูจน์ $\forall$

เมื่อต้องการพิสูจน์

$$\forall x \in U, P(x),$$

ให้เริ่มโดยเขียนว่า

$$\text{``ให้ } x \in U \text{ เป็นใด ๆ"}$$

จากนั้นพิสูจน์ว่า $P(x)$ เป็นจริง โดยใช้เพียงข้อเท็จจริงที่เป็นจริงสำหรับสมาชิกทั่วไปของ U

คำว่า ``เป็นใด ๆ" สำคัญมาก เพราะมันเตือนเราว่า x ต้องเป็นสมาชิก *ทั่วไป* ไม่ใช่สมาชิกพิเศษที่เราเลือกเพราะสะดวกต่อการพิสูจน์

Example 3.2.1. ถ้าต้องการพิสูจน์

$$\forall n \in \mathbb{Z}, n^2 + n \text{ เป็นจำนวนคู่}$$

เราไม่อาจเริ่มด้วย ``ให้ $n = 0$ " แล้วพิสูจน์ว่าจริงสำหรับ 0 เพราะนั่นพิสูจน์เพียงกรณีเดียว ไม่ใช่ทุกกรณี

Theorem 3.2.2. สำหรับทุกจำนวนเต็ม n , ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

Proof idea. ข้อความนี้เป็นอิมพลีเคชันภายในตัวบ่งปริมาณสากล ดังนั้นให้ $n \in \mathbb{Z}$ เป็นใด ๆ แล้วสมมติส่วนหน้าของอิมพลีเคชัน จากนั้นแคะนิยามของจำนวนคู่

Proof. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ และสมมติว่า n เป็นจำนวนคู่ ตามนิยามของจำนวนคู่ จะมี $k \in \mathbb{Z}$ ที่ทำให้

$$n = 2k.$$

ดังนั้น

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2).$$

เนื่องจาก $2k^2 \in \mathbb{Z}$ จึงได้ว่า n^2 อยู่ในรูป $2m$ สำหรับบาง $m \in \mathbb{Z}$ ดังนั้น n^2 เป็นจำนวนคู่ □

3.3 เมื่อเป้าหมายเป็น $\exists$: ระบุพยาน

การพิสูจน์ข้อความเชิงการมีมีแนวคิดต่างออกไป เราจะไม่เริ่มจากการเลือกสมาชิกทั่วไป แต่ต้อง *เสนอพยาน* ว่าใครคือวัตถุที่ทำให้เงื่อนไขเป็นจริง

แพทเทิร์นของการพิสูจน์ $\exists$

เมื่อต้องการพิสูจน์

$$\exists x \in U, P(x),$$

ให้เลือกหรือกำหนดสมาชิกตัวหนึ่ง $w \in U$ แล้วตรวจสอบว่า $P(w)$ เป็นจริง

คำว่า *witness* หรือพยาน ไม่ได้หมายถึงวัตถุที่สืบลับซับซ้อน แต่มักเป็นเพียงค่าที่เราเสนอออกมาอย่างชัดเจนว่า “ตัวนี้แหละใช้ได้”

Example 3.3.1. ข้อความ

$$\exists n \in \mathbb{Z}, n^2 = 4$$

พิสูจน์ได้โดยเลือกพยาน $n = 2$ เพราะ $2 \in \mathbb{Z}$ และ $2^2 = 4$

Example 3.3.2. ข้อความ

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z}, z + m = m$$

พิสูจน์ได้โดยเลือกพยาน $z = 0$ แล้วตรวจว่า $0 + m = m$ จริงสำหรับทุก $m \in \mathbb{Z}$

ในการเขียนพิสูจน์แบบมีอยู่ เรามักไม่จำเป็นต้องเขียนกระบวนการค้นหาพยานอย่างละเอียด สิ่งสำคัญกว่าคือ เมื่อเลือกพยานแล้ว ต้องตรวจสอบให้ชัดว่าพยานนั้นทำงานจริง

3.4 หลายตัวบ่งปริมาณ: ลำดับมีผล

เมื่อข้อความมีตัวบ่งปริมาณมากกว่าหนึ่งตัว ลำดับของมันมีผลโดยตรงต่อความหมาย

$$\forall x \exists y R(x, y) \quad \text{ต่างจาก} \quad \exists y \forall x R(x, y)$$

ประโยคสองประโยคนี้นดูคล้ายกันมาก แต่จริง ๆ แล้วต่างกันอย่างมีนัยสำคัญ

3.4.1 แนวคิดเชิงภาษา

ข้อความ

$$\forall x \exists y R(x, y)$$

แปลว่า

“สำหรับทุก x เราสามารถหาค่า y ที่เหมาะกับ x ตัวนั้นได้”

ส่วนข้อความ

$$\exists y \forall x R(x, y)$$

แปลว่า

“มี y ตัวเดียวสักตัวหนึ่งที่ใช้ได้พร้อมกันสำหรับทุก x ”

ประโยคแบบแรกอนุญาตให้ y เปลี่ยนไปตาม x แต่ประโยคแบบหลังไม่อนุญาต เพราะ y ถูกเลือกก่อนแล้วต้องใช้ตัวเดิมกับทุก x

ข้อแตกต่างที่ต้องเห็นให้ชัด

ในประโยค

$$\forall x \exists y P(x, y),$$

พยาน y อาจขึ้นกับ x

แต่ในประโยค

$$\exists y \forall x P(x, y),$$

พยาน y ต้องเป็น ค่าคงที่ตัวเดียว ที่ใช้ได้กับทุก x

3.4.2 ตัวอย่างเชิงจำนวน

พิจารณาข้อความสองประโยคต่อไปนี้ใน $\mathbb{Z}$

$$\forall x \in \mathbb{Z} \exists y \in \mathbb{Z} : y > x$$

และ

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : y > x.$$

ข้อความแรกเป็นจริง เพราะเมื่อให้ $x \in \mathbb{Z}$ มาใด ๆ เราเลือก

$$y = x + 1$$

ได้เสมอ และแน่นอนว่า $y > x$

แต่ข้อความที่สองเป็นเท็จ เพราะไม่มีจำนวนเต็มตัวใดที่มากกว่าจำนวนเต็มทุกตัวพร้อมกัน ไม่ว่าคุณจะเสนอ y ตัวไหน เราก็เลือก $x = y + 1$ เพื่อให้ $y > x$ ล้มเหลวได้ทันที

3.4.3 ตัวอย่างเชิงระบบคอมพิวเตอร์

ให้ U เป็นเซตผู้ใช้งานทั้งหมด และ R เป็นเซตทรัพยากรทั้งหมดในระบบ โดย $allowed(user, resource)$ หมายความว่า ผู้ใช้งานคนนั้นเข้าถึงทรัพยากรนั้นได้

ประโยค

$$\forall user \in U \exists resource \in R, allowed(user, resource)$$

หมายความว่า

“ผู้ใช้งานทุกคนมีทรัพยากรอย่างน้อยหนึ่งอย่างที่เข้าถึงได้”

แต่ประโยค

$$\exists resource \in R \forall user \in U, allowed(user, resource)$$

หมายความว่า

“มีทรัพยากรชิ้นหนึ่งที่ผู้ใช้งานทุกคนเข้าถึงได้”

ประโยคหลังเข้มกว่าอย่างชัดเจน เพราะต้องการทรัพยากร ร่วม ตัวเดียว ไม่ใช่ทรัพยากรคนละตัวกัน

3.5 การปฏิเสธตัวบ่งปริมาณ

การหาตัวอย่างค้านหรือการพิสูจน์โดยวิธีแย้ง มักบังคับให้เราปฏิเสธข้อความที่มีตัวบ่งปริมาณ ดังนั้นสมมูลต่อไปนี้จึงสำคัญมาก

กฎปฏิเสธตัวบ่งปริมาณ

$$\neg(\forall x \in U, P(x)) \equiv \exists x \in U, \neg P(x),$$

$$\neg(\exists x \in U, P(x)) \equiv \forall x \in U, \neg P(x).$$

หลักจำคือ

“ปฏิเสธ $\forall$ แล้วกลายเป็น $\exists$, ปฏิเสธ $\exists$ แล้วกลายเป็น $\forall$ ”

พร้อมกับต้องผลักเครื่องหมายปฏิเสธเข้าไปที่เงื่อนไขภายในด้วย

Example 3.5.1. นิเสธของข้อความ

$$\forall x \in U, P(x)$$

คือ

$$\exists x \in U, \neg P(x),$$

ซึ่งแปลว่า “มีอย่างน้อยหนึ่งตัวที่ไม่ทำให้ P จริง”

ไม่ใช่

$$\forall x \in U, \neg P(x),$$

ซึ่งเป็นข้อความที่แรงกว่ามาก เพราะแปลว่า “ไม่มีตัวไหนเลยที่ทำให้ P จริง”

Example 3.5.2. ถ้าเราปฏิเสธข้อความ

$$\forall n \in \mathbb{Z}, \exists k \in \mathbb{Z} : n < k$$

เราจะได้

$$\exists n \in \mathbb{Z}, \forall k \in \mathbb{Z} : n \geq k.$$

กล่าวคือ “มีจำนวนเต็มตัวหนึ่งซึ่งมากกว่าหรือเท่ากับจำนวนเต็มทุกตัว” ซึ่งใน $\mathbb{Z}$ เป็นไปไม่ได้

3.6 ตัวอย่างทฤษฎีบทผสม $\forall \exists$

ข้อความที่มีทั้ง $\forall$ และ $\exists$ มักต้องใช้สองแพตเทิร์นพร้อมกัน: เริ่มจากเลือกตัวทั่วไปสำหรับ $\forall$ แล้วจึงสร้างพยานสำหรับ $\exists$

Theorem 3.6.1. สำหรับทุกจำนวนเต็ม n จะมีจำนวนเต็ม k ที่ทำให้

$$n(n+1) = 2k.$$

Proof idea. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ แล้วแยกกรณีว่า n เป็นจำนวนคู่หรือจำนวนคี่ ในแต่ละกรณีจะสามารถจัดรูป $n(n+1)$ ให้อยู่ในรูป $2k$ ได้

Proof. ให้ $n \in \mathbb{Z}$ เป็นใด ๆ เราพิจารณาเป็นสองกรณี

กรณีที่ 1: n เป็นจำนวนคู่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้

$$n = 2\ell.$$

ดังนั้น

$$n(n+1) = (2\ell)(2\ell+1) = 2(\ell(2\ell+1)).$$

ให้

$$k = \ell(2\ell+1).$$

เนื่องจาก $\ell \in \mathbb{Z}$ จึงได้ว่า $k \in \mathbb{Z}$ และมี

$$n(n+1) = 2k.$$

กรณีที่ 2: n เป็นจำนวนคี่

จะมี $\ell \in \mathbb{Z}$ ที่ทำให้

$$n = 2\ell + 1.$$

ดังนั้น

$$n(n+1) = (2\ell+1)(2\ell+2) = 2((2\ell+1)(\ell+1)).$$

ให้

$$k = (2\ell+1)(\ell+1).$$

แล้ว $k \in \mathbb{Z}$ และ

$$n(n+1) = 2k.$$

ในทุกกรณี เราหาจำนวนเต็ม k ได้ที่ทำให้ $n(n+1) = 2k$ ดังนั้น

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n(n+1) = 2k.$$

เป็นจริง

□

จุดที่พลาดบ่อย

เวลาเห็นข้อความ

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n(n+1) = 2k$$

ห้ามเลือก n แบบเฉพาะเจาะจง เช่น $n = 0$ แล้วสรุปทันทีว่า “จริงสำหรับทุก n ” เพราะสิ่งที่ต้องพิสูจน์คือ ทุก n ไม่ใช่เพียงกรณีตัวอย่าง

3.7 ตัวอย่างทฤษฎีบทแบบ $\exists\forall$

Theorem 3.7.1. มีจำนวนเต็ม z ตัวหนึ่งที่ทำให้สำหรับทุกจำนวนเต็ม m ,

$$z + m = m.$$

Proof idea. เดาพยานก่อนว่าควรเป็นใคร แล้วจึงตรวจว่าพยานนั้นใช้ได้กับทุก m

Proof. เลือก

$$z = 0.$$

ชัดเจนว่า $z \in \mathbb{Z}$

ต่อไปให้ $m \in \mathbb{Z}$ เป็นใด ๆ จะได้ว่า

$$z + m = 0 + m = m.$$

ดังนั้น $z = 0$ ใช้ได้กับทุก $m \in \mathbb{Z}$ จึงสรุปว่า

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z} : z + m = m.$$

เป็นจริง □

ตัวอย่างนี้เรียบง่ายมาก แต่มีประโยชน์เชิงโครงสร้างสูง เพราะมันบังคับให้เห็นชัดว่าในประโยคแบบ $\exists\forall$ เราต้องเลือกพยาน *เพียงครั้งเดียว* แล้วใช้พยานตัวเดิมกับทุกค่าในภายหลัง

3.8 การหักล้างข้อความที่มีตัวบ่งปริมาณ

การหักล้างข้อความที่มีตัวบ่งปริมาณ มักเริ่มจากการปฏิเสธมันอย่างถูกต้องก่อน แล้วจึงมองหาว่าจะให้ตัวอย่างค้านหรือเหตุผลแบบใด

Theorem 3.8.1. ข้อความ

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x < y$$

เป็นเท็จ

Proof idea. ถ้าข้อความนี้จริง จะมีจำนวนเต็มตัวหนึ่งที่ยิ่งกว่าจำนวนเต็มทุกตัว แต่ไม่ว่าคุณจะเสนอ y ตัวไหน เราสามารถเลือก $x = y + 1$ เพื่อทำให้เงื่อนไข $x < y$ ล้มเหลวได้

Proof. สมมติว่า

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x < y$$

เป็นจริง ดังนั้นจะมีจำนวนเต็ม y ตัวหนึ่งที่ทำให้

$$x < y \quad \text{สำหรับทุก } x \in \mathbb{Z}.$$

แต่เมื่อ $y \in \mathbb{Z}$ เราสามารถเลือก

$$x = y + 1$$

ซึ่งเป็นจำนวนเต็มเช่นกัน แล้วจะได้ว่า

$$y + 1 < y,$$

ซึ่งเป็นไปไม่ได้

เกิดข้อขัดแย้ง จึงสรุปได้ว่าข้อความเดิมเป็นเท็จ $\square$

ในอีกมุมหนึ่ง เราอาจหักล้างได้แบบตรงไปตรงมาว่า ``สำหรับทุก $y \in \mathbb{Z}$ ถ้าเสนอ y มา เราเลือก $x = y + 1$ ได้" ซึ่งเป็นรูปแบบการคิดที่มีประโยชน์มากเมื่อเจอข้อความเชิงการมีที่ดูน่าสงสัย

3.9 ตัวอย่างเสริมเกี่ยวกับการแปลความหมาย

ก่อนจะพิสูจน์ข้อความที่มี quantifier การแปลเป็นประโยคภาษาไทยให้ชัดมักช่วยได้มาก

Example 3.9.1. ให้ $R(x, t)$ แปลว่า ``โปรแกรมบนอินพุต x จบภายในเวลาไม่เกิน t " โดย $t \in \mathbb{N}$

ข้อความ

$$\forall x \exists t \in \mathbb{N}, R(x, t)$$

แปลว่า

``สำหรับทุกอินพุต มีเวลาบางค่าที่ทำให้โปรแกรมจบบนอินพุตนั้น"

ส่วนข้อความ

$$\exists t \in \mathbb{N} \forall x, R(x, t)$$

แปลว่า

``มีเวลาค่าหนึ่งที่ใช้ได้กับทุกอินพุต"

ข้อความหลังแรงกว่ามาก เพราะต้องการ bound เดียวที่ครอบคลุมทุกอินพุต

ตัวอย่างเช่นนี้มีความสำคัญมากในวิทยาการคอมพิวเตอร์ เพราะมันสัมพันธ์โดยตรงกับการแยกความต่างระหว่าง ``เวลาอาจเปลี่ยนไปตามอินพุต" กับ ``มี worst-case bound เดียวที่ใช้กับทุกอินพุต"

3.10 สรุปแพตเทิร์นของบทนี้

เราสามารถย่อใจความของบทนี้ให้อยู่ในแพตเทิร์นหลักไม่กี่ข้อได้ดังนี้

Pattern Summary

1. ถ้าเป้าหมายเป็น $A \rightarrow B$: เริ่มจาก A
2. ถ้าเป้าหมายเป็น $A \leftrightarrow B$: พิสูจน์สองทิศ
3. ถ้ามี $P \vee Q$ เป็นข้อมูลและต้องการ R : ทำ proof by cases
4. ถ้าเป้าหมายเป็น $\forall x \in U, P(x)$: ให้ $x \in U$ เป็นใด ๆ
5. ถ้าเป้าหมายเป็น $\exists x \in U, P(x)$: ระบุพยานแล้วตรวจให้ชัด
6. ระวังลำดับตัวบ่งปริมาณ เพราะ

$$\forall x \exists y R(x, y) \neq \exists y \forall x R(x, y)$$

7. ปฏิเสธ quantifier ให้ถูก:

$$\neg \forall = \exists \neg, \quad \neg \exists = \forall \neg$$

3.11 ข้อผิดพลาดที่พบบ่อยก่อนเริ่มพิสูจน์จริง

ก่อนปิดบทนี้ ผู้เขียนอยากชวนให้ผู้อ่านมองอีกด้านหนึ่งของการเรียนพิสูจน์: บางครั้งเราพัฒนาทักษะการพิสูจน์ได้เร็วขึ้น ไม่ใช่เพียงจากการอ่านตัวอย่างที่ถูกต้อง แต่จากการมองให้ออกว่า พิสูจน์แบบใดที่ดูเหมือนถูก แต่จริง ๆ แล้วผิด เพราะข้อผิดพลาดจำนวนมากในการเขียนพิสูจน์ ไม่ได้เกิดจากการคำนวณพลาด แต่เกิดจากการอ่านโครงสร้างของประโยคไม่ชัด สมมติผิดจุด หรือข้ามขั้นที่ควรอธิบายให้ตรวจสอบได้

ในห้องเรียนจริง ผู้สอนมักพบว่าความผิดพลาดบางแบบเกิดซ้ำแทบทุกภาคการศึกษา โดยเฉพาะในช่วงแรกที่นักศึกษา กำลังเปลี่ยนจากการทำโจทย์แบบ ``หาคำตอบ" มาเป็นการเขียนคำอธิบายว่า ``ทำไมคำตอบนั้นจึงถูกต้อง" การรู้ข้อผิดพลาดเหล่านี้ล่วงหน้าจึงมีประโยชน์มาก เพราะช่วยให้เราตรวจงานของตนเองได้ดีขึ้น และยังช่วยให้เวลามองพิสูจน์ของผู้อื่น เรารู้ว่าต้องระวังจุดใดเป็นพิเศษ

1. ใช้ตัวอย่างแทนการพิสูจน์

นี่เป็นความผิดพลาดที่พบบ่อยที่สุดอย่างหนึ่ง โดยเฉพาะเมื่อข้อความที่ต้องการพิสูจน์อยู่ในรูป ``สำหรับ

ทุก" นักศึกษาหลายคนลองแทนค่าไม่ก่ค่า เห็นว่าผลออกมาถูกต้อง แล้วจึงสรุปว่าข้อความนั้นเป็นจริงเสมอ

ปัญหาคือ การลองตัวอย่างมีประโยชน์เพียงในฐานะเครื่องมือช่วย คาดเดา หรือ มองหา *pattern* แต่ยังไม่ใช้การพิสูจน์ เพราะข้อความเชิงสากลต้องการเหตุผลที่ครอบคลุมทุกกรณีในโดเมน ไม่ใช่เพียงบางกรณีที่เราเลือกมาลอง

ตัวอย่างเช่น ถ้ามีผู้เขียนว่า

“เพราะ $2 + 4 = 6$ และ $4 + 6 = 10$ ซึ่งเป็นจำนวนคู่ ดังนั้นผลบวกของจำนวนคู่สองจำนวนเป็นจำนวนคู่”

ข้อความนี้ยังไม่พอเป็นพิสูจน์ เพราะมันแสดงเพียงว่า บางคู่ ให้ผลลัพธ์ตามที่ต้องการ แต่ยังไม่ได้อธิบายว่าเหตุใด จำนวนคู่ทุกคู่ จึงต้องมีสมบัตินี้

สิ่งที่ควรทำแทนคือ กลับไปใช้นิยามของคำว่า “จำนวนคู่” แล้วพิสูจน์ในกรณีทั่วไป เช่น ให้ $a = 2m$ และ $b = 2n$ สำหรับจำนวนเต็มบางตัว m, n จากนั้นจึงแสดงว่า

$$a + b = 2(m + n),$$

ซึ่งอยู่ในรูปของจำนวนคู่ตามนิยาม

สิ่งที่ควรสังเกตคือ ตัวอย่างมีหน้าที่ช่วยให้เราเดาทิศทาง แต่พิสูจน์มีหน้าที่รับประกันความจริงทั่วไป สองอย่างนี้จึงเกี่ยวข้องกัน แต่แทนกันไม่ได้

2. ไม่ระบุดomenของตัวแปร

ในคณิตศาสตร์ ประโยคเดียวกันอาจมีค่าความจริงต่างกัน ถ้าเปลี่ยนโดเมนของตัวแปร ดังนั้นการเขียนข้อความโดยไม่บอกว่า ตัวแปรกำลังวิ่งอยู่ในเซตใด จึงทำให้ข้อความกำกวมทันที

ตัวอย่างเช่น ประโยค

$$\forall x, x^2 \geq x$$

ถ้าไม่ระบุว่า x เป็นสมาชิกของเซตใด ผู้อ่านย่อมไม่รู้ว่าควรตีความอย่างไร ถ้า $x \in \mathbb{N}$ ข้อความนี้เป็นจริง แต่ถ้า $x \in \mathbb{R}$ กลับไม่จริงเสมอไป เช่นเมื่อ $x = \frac{1}{2}$ จะได้

$$\left(\frac{1}{2}\right)^2 = \frac{1}{4} < \frac{1}{2}.$$

ข้อผิดพลาดแบบนี้มีผลไม่ใช่แค่กับค่าความจริงของข้อความ แต่ยังมีผลต่อรูปพิสูจน์ด้วย เพราะนิยามและสมบัติที่เราอนุญาตให้ใช้ ขึ้นอยู่กับโดเมนโดยตรง หากบอกเพียงว่า “ให้ n เป็นจำนวน” ผู้อ่านจะไม่รู้ว่าเราหมายถึงจำนวนเต็ม จำนวนจริง หรือจำนวนนับ

ดังนั้นนิสัยที่ควรสร้างตั้งแต่ต้นคือ เมื่อเริ่มพิสูจน์ควรถามตัวเองเสมอว่า “ตัวแปรนี้อยู่ในเอกภพอะไร” และถ้ายังไม่ได้ระบุให้ชัด ควรระบุก่อนใช้มันในข้ออ้างถัดไป

3. สลับลำดับของตัวบ่งปริมาณ

ผู้เริ่มต้นมักรู้แล้วว่า $\forall$ แปลว่า “สำหรับทุก” และ $\exists$ แปลว่า “มีอยู่” แต่ยังมองไม่เห็นชัดว่าลำดับของตัว

บ่งปริมาณส่งผลต่อความหมายอย่างมาก
ตัวอย่างคลาสสิกคือ

$$\forall x \exists y P(x, y) \quad \text{กับ} \quad \exists y \forall x P(x, y).$$

ข้อความแรกหมายความว่า สำหรับแต่ละ x เราอาจเลือก y คนละตัวกันก็ได้ แต่ข้อความหลังหมายความว่า มี y ตัวเดียวที่ใช้ได้กับทุก x ซึ่งเป็นข้ออ้างที่แรงกว่ามาก
ในภาษาธรรมชาติ ความแตกต่างนี้อาจไม่สะดุดตาในตอนแรก เช่น

“ทุกคนมีเพื่อนสักคน”

กับ

“มีคนคนหนึ่งที่เป็นเพื่อนของทุกคน”

สองประโยคนี้ไม่เหมือนกันเลย แต่ผู้เรียนมักสับสนกันโดยไม่รู้ตัว
ข้อผิดพลาดนี้สำคัญมาก เพราะมันไม่ได้ทำให้พิสูจน์ “ไม่สวย” แต่ทำให้เราพยายามพิสูจน์คนละข้อความ
กับที่โจทย์ถาม ดังนั้นเมื่อเห็นข้อความที่มีตัวบ่งปริมาณมากกว่าหนึ่งตัว ควรอ่านซ้ำ ๆ และถามให้ชัดว่า
พยานที่เลือกได้ “ขึ้นกับอะไร” และ “ต้องใช้ตัวเดียวกับทุกกรณีหรือไม่”

4. สรุปย้อนจาก $P \rightarrow Q$ เป็น $Q \rightarrow P$

นี่เป็นข้อผิดพลาดเชิงตรรกะที่คลาสสิกมาก การที่เราพิสูจน์ได้ว่า

$$P \rightarrow Q$$

ไม่ได้แปลว่าเราจะสรุปกลับได้ทันทีว่า

$$Q \rightarrow P.$$

สองข้อความนี้เป็นคนละข้อความ และโดยทั่วไปไม่มีเหตุผลใดรับประกันว่าถ้าทิศแรกจริง อีกทิศหนึ่งจะ
ต้องจริงตามไปด้วย
ตัวอย่างเช่น

ถ้า n เป็นจำนวนคู่ แล้ว n^2 เป็นจำนวนคู่

เป็นข้อความจริง แต่เราไม่ควรเขียนต่อทันทีว่า

ดังนั้นถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่

โดยไม่มีพิสูจน์เพิ่ม แม้ว่าข้อความหลังจะจริงเช่นกัน แต่ต้องพิสูจน์อีกครั้ง อาจพิสูจน์โดยตรงหรือพิสูจน์
ผ่าน contrapositive ก็ได้

จุดที่นักศึกษามักพลาดคือ เห็นว่าประโยคสองอัน “เกี่ยวข้องกันมาก” จึงเผลอคิดว่าอันหนึ่งตามจากอีก
อันอัตโนมัติ แต่ในทางตรรกศาสตร์ ความรู้สึกว่า “มันน่าจะกลับกันได้” ไม่เพียงพอ เราต้องพิสูจน์แต่ละ
ทิศอย่างเป็นอิสระ เว้นแต่เราจะมีเหตุผลชัดเจนว่าทั้งสองทิศสมมูลกันจริง

5. เริ่มพิสูจน์จากสิ่งที่ต้องการพิสูจน์

เมื่อเจอโจทย์ยาก ผู้เรียนจำนวนมากมักลองเขียนจากข้อสรุปที่ต้องการก่อน แล้วค่อยจัดรูปย้อนกลับไปหาสมมติฐาน ในฐานะ *การสำรวจแนวคิด* วิธีนี้ใช้ได้ เพราะมันช่วยให้มองเห็นว่าต้องใช้เครื่องมือใด แต่ถ้านำมาเขียนเป็นพิสูจน์โดยตรงโดยไม่ระวัง จะเกิดปัญหาร้ายแรงทันที: เรากำลังใช้สิ่งที่ยังไม่ได้พิสูจน์เป็นจุดเริ่มต้นของพิสูจน์เสียเอง

ตัวอย่างเช่น ถ้าโจทย์ต้องการพิสูจน์ว่า “ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่” การเขียนว่า

“สมมติว่า n เป็นจำนวนคู่ จึงได้ว่า ...”

ย่อมไม่ตอบโจทย์ เพราะเราไปเริ่มจากข้อความที่อยากสรุป แทนที่จะเริ่มจากสมมติฐานจริงของโจทย์ สิ่งที่เราควรทำคือ ข้อพิสูจน์ต้องเดินจากสิ่งที่ *ได้รับอนุญาตให้ใช้* ไปสู่สิ่งที่ *ต้องการสรุป* ไม่ใช่เดินจากสิ่งที่อยากได้แล้วหวังว่าจะย้อนกลับมาหาสิ่งที่รู้อยู่แล้วได้เสมอ

แน่นอนว่าในงานจริง การคิดย้อนจากเป้าหมายเป็นเทคนิคที่มีประโยชน์มาก แต่เมื่อถึงเวลาจะเขียนพิสูจน์ เราต้องจัดเรียงใหม่ให้ผู้อ่านเห็นลำดับเหตุผลที่ถูกต้อง กล่าวคือ ใช้การคิดย้อนเพื่อหาไอเดียได้ แต่ใช้การเดินไปข้างหน้าเพื่อสื่อสารพิสูจน์

6. ละชั้นสำคัญโดยคิดว่า “เห็นอยู่แล้ว”

ข้อผิดพลาดนี้มักเกิดหลังจากผู้เรียนเริ่มคุ้นกับนิยามบางอย่างแล้ว เช่นพอเห็นว่า $a \mid b$ ก็รีบสรุปผลหลายบรรทัดต่อทันที โดยไม่เขียนว่ากำลังใช้นิยามไหน หรือพอเห็นว่า n เป็นจำนวนคู่ ก็เขียนต่อไปหลายบรรทัดโดยไม่เคยบอกเลยว่า กำลังใช้ข้อเท็จจริงว่า $n = 2k$ สำหรับจำนวนเต็มบางตัว k

ปัญหาไม่ได้อยู่ที่ผู้เขียน “ไม่เข้าใจ” แต่คือพิสูจน์นั้นตรวจสอบไม่ได้ เพราะผู้อ่านไม่รู้ว่ามีบรรทัดใหม่เกิดจากอะไร ในรายวิชานี้ เราไม่ได้ต้องการเพียงประโยคที่ดูคล้ายคณิตศาสตร์ แต่ต้องการบรรทัดที่มีสถานะชัดเจนว่า เป็นสมมติฐาน เป็นผลจากนิยาม เป็นผลจากข้อเท็จจริงก่อนหน้านี้ หรือเป็นการสรุปจากรูปแบบตรรกะใด

กล่าวอีกแบบหนึ่งคือ ในการพิสูจน์ เราไม่เพียงต้องมี *claim* แต่ต้องมี *reason* ด้วย ถ้าบรรทัดหนึ่งดูเหมือนถูก แต่ตอบไม่ได้ว่ามาจากไหน บรรทัดนั้นก็ยังไม่แข็งแรงพอในฐานะส่วนหนึ่งของพิสูจน์

แน่นอนว่าเราไม่จำเป็นต้องเขียนทุกอย่างละเอียดจนยาวเกินเหตุ แต่ควรเขียนให้พอที่ผู้อ่านซึ่งรู้พื้นฐานระดับเดียวกันจะตรวจตามได้ หลักง่าย ๆ คือ ถ้าการข้ามขั้นนั้นทำให้ผู้อ่านต้อง “เดา” เองว่าเรากำลังใช้อะไร ก็มักเป็นสัญญาณว่าควรเติมคำอธิบายอีกเล็กน้อย

จากข้อผิดพลาดทั้งหมดนี้ ผู้อ่านอาจเห็นได้ว่าปัญหาส่วนใหญ่ไม่ได้อยู่ที่ “ไม่เก่งคำนวณ” แต่เกิดจากการไม่แยกให้ออกว่า อะไรคือสมมติฐาน อะไรคือข้อสรุป ตัวแปรกำลังวิ่งอยู่ในโดเมนใด และบรรทัดหนึ่งตามมาจากรบรทัดก่อนหน้านี้ได้อย่างไร นี่จึงเป็นเหตุผลว่าทำไมในช่วงต้นของรายวิชา เราจึงให้ความสำคัญมากกับการอ่านโครงสร้างของข้อความ การแกะนิยาม การชี้บรรทัดแรกที่ผิดกฎ และการหาตัวอย่างค้านเมื่อสงสัยว่าข้อความอาจไม่จริง

หลักตรวจตนเองอย่างรวดเร็วก่อนส่งพิสูจน์ ก่อนส่งงานหรือก่อนสรุปว่าพิสูจน์ของตนเองเสร็จแล้ว ผู้เขียนแนะนำให้หยุดถามตัวเองสั้น ๆ ดังนี้:

- ฉันกำลังพิสูจน์ข้อความรูปอะไร: $\forall, \exists$, หรือ $P \rightarrow Q$?

- ฉันทระบุดีเมนของตัวแปรครบหรือยัง?
- แต่ละบรรทัดมีเหตุผลรองรับหรือยัง หรือมีบางจุดที่ฉันเพียง ``รู้สึกว่ใช่"?
- ฉันทเผลอใช้ตัวอย่างแทนการพิสูจน์หรือไม่?
- ถ้าข้อความนี้อาจไม่จริง ฉันทลองหาตัวอย่างค้านแล้วหรือยัง?

นิสัยเล็ก ๆ เหล่านี้อาจดูธรรมดา แต่มีผลมากต่อคุณภาพของการเขียนพิสูจน์ เพราะช่วยให้เราไม่เพียงเขียนได้ถูกขึ้น แต่ยังคิดได้เป็นระบบขึ้นด้วย และนั่นคือเป้าหมายสำคัญของบทปูพื้นบทนี้ก่อนจะเข้าสู่เทคนิคการพิสูจน์อย่างจริงจังในบทถัดไป

แบบฝึกหัดท้ายบท

Exercise 3.11.1. จงอธิบายความแตกต่างของข้อความต่อไปนี้เป็นภาษาไทยอย่างชัดเจน

$$\forall \text{student} \in U \exists \text{problem} \in H, \text{solved}(\text{student}, \text{problem})$$

และ

$$\exists \text{problem} \in H \forall \text{student} \in U, \text{solved}(\text{student}, \text{problem}).$$

Exercise 3.11.2. จงเขียนปฏิเสธของข้อความต่อไปนี้ให้ถูกต้อง

$$\forall n \in \mathbb{Z} \exists k \in \mathbb{Z} : n < k.$$

Exercise 3.11.3. จงพิสูจน์ว่

$$\forall n \in \mathbb{Z}, \exists m \in \mathbb{Z} : m > n.$$

Exercise 3.11.4. จงพิสูจน์ว่

$$\exists z \in \mathbb{Z} \forall m \in \mathbb{Z} : m + z = m.$$

Exercise 3.11.5. จงหักล้างข้อความต่อไปนี้

$$\exists y \in \mathbb{Z} \forall x \in \mathbb{Z} : x \leq y.$$

Exercise 3.11.6. จงพิสูจน์ว่

$$\forall n \in \mathbb{Z}, \exists k \in \mathbb{Z} : n^2 + n = 2k.$$

โดยเขียนในรูป

Claim $\rightarrow$ Proof idea $\rightarrow$ Proof $\rightarrow$ QED

ให้ครบ

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 4

โครงสร้างดีสคริต: เซต ความสัมพันธ์ และฟังก์ชัน

ในบทที่ผ่านมา เราใช้เวลาไปกับการฝึกอ่านและเขียนข้อพิสูจน์ เรียนรู้วิธีแยกสมมติฐานออกจากข้อสรุป แยก claim ออกจาก reason และฝึกมองว่าโครงสร้างของประโยคเป็นตัวกำหนดแนวทางของการพิสูจน์อย่างไร เมื่อเครื่องมือเชิงเหตุผลเริ่มพร้อมแล้ว คำถามถัดไปจึงไม่ใช่เพียงว่า “จะพิสูจน์อย่างไร” แต่คือ “เรากำลังพิสูจน์เรื่องของวัตถุชนิดใดอยู่”

บทนี้จึงมีบทบาทต่างจากบท proof methods เล็กน้อย เพราะเราจะหันมาศึกษา โครงสร้างพื้นฐาน ที่ปรากฏซ้ำแล้วซ้ำอีกในคณิตศาสตร์ดีสคริต ได้แก่ เซต ความสัมพันธ์ และ ฟังก์ชัน สามสิ่งนี้อาจดูเป็นหัวข้อพื้นฐาน แต่แท้จริงแล้วเป็นภาษาหลักของคณิตศาสตร์แทบทุกส่วน เมื่อเราพูดว่าอะไร “อยู่ในกลุ่มเดียวกัน” เมื่อเราพูดว่าองค์ประกอบสองตัว “เกี่ยวข้องกัน” หรือเมื่อเราพูดว่าวัตถุหนึ่ง “ถูกส่งไป” เป็นอีกวัตถุหนึ่งอย่างเป็นระบบ เรากำลังใช้แนวคิดของเซต ความสัมพันธ์ และฟังก์ชันอยู่เสมอ

สิ่งที่ควรเข้าใจตั้งแต่ต้นคือ บทนี้ไม่ใช่บทที่เน้นการคำนวณมากที่สุดในรายวิชา หลายส่วนของมันอาจยังไม่แสดงภาพการประยุกต์ที่เด่นชัดเท่าบททฤษฎีจำนวน ทฤษฎีกราฟ หรือบทที่เกี่ยวกับการวิเคราะห์อัลกอริทึม แต่บทนี้มีความสำคัญในอีกความหมายหนึ่ง: มันฝึกให้เราคิดเชิงโครงสร้าง กล่าวคือฝึกมองว่าวัตถุทางคณิตศาสตร์ไม่ได้โดดเดี่ยวที่ละชิ้น แต่มีรูปแบบของการรวมกลุ่ม การจัดระเบียบ และการเชื่อมโยงกันอยู่เบื้องหลัง

ยิ่งไปกว่านั้น เซต ความสัมพันธ์ และฟังก์ชันยังเป็นตัวอย่งที่ดีมากของการใช้ทักษะพิสูจน์ที่เราเพิ่งเรียนมา เพราะนิยามต่าง ๆ ในบทนี้มักอยู่ในรูปของข้อความเชิงตรรกะโดยตรง เช่น นิยามของเซตย่อย นิยามของความสัมพันธ์สมมูล หรือเงื่อนไขของฟังก์ชันหนึ่งต่อหนึ่งและทั่วถึง ดังนั้นบทนี้จึงเป็นทั้งบทเนื้อหาและบทฝึก reasoning ไปพร้อมกัน

4.1 เซต: ภาษาของการเป็นสมาชิกและการรวมกลุ่ม

แนวคิดเรื่องเซตเกิดขึ้นจากความต้องการง่าย ๆ แต่ทรงพลังมาก: เราต้องการภาษาที่ใช้พูดถึง “กลุ่มของสิ่งที่น่าสนใจ” อย่างเป็นระบบ ถ้าเราพูดว่า “นักศึกษาที่ลงทะเบียนวิชานี้” “จำนวนคู่” “จุดยอดของกราฟ” หรือ “สตริงที่ยอมรับได้โดยอัตโนมัติ” เรากำลังพูดถึงกลุ่มของวัตถุบางอย่างทั้งสิ้น ภาษาของเซตจึงช่วยให้เราจัดรูปความคิดเหล่านี้ให้รัดกุมขึ้น

ถ้า a เป็นสมาชิกของเซต A เราจะเขียนว่า

$$a \in A$$

และถ้า a ไม่เป็นสมาชิกของ A เราจะเขียนว่า

$$a \notin A.$$

สัญลักษณ์ง่าย ๆ นี้มีบทบาทสำคัญมาก เพราะมันทำให้เราพูดเรื่อง “การอยู่ในกลุ่ม” ได้อย่างตรงไปตรงมา และเมื่อเราต้องการอธิบายเซตด้วยคุณสมบัติของสมาชิก เรามักใช้การเขียนแบบกำหนดเงื่อนไขว่า

$$\{x \in \mathcal{U} \mid x \text{ มีสมบัติที่ต้องการ}\},$$

โดย $\mathcal{U}$ คือเอกภพสัมพัทธ์หรือโดเมนที่เรากำลังสนใจอยู่ในขณะนั้น

จุดนี้ควรระวังเล็กน้อยว่า ภาษาของเซตแม้จะดูเป็นเพียงเรื่องของสัญลักษณ์ แต่แท้จริงแล้วมันผูกกับตรรกศาสตร์ทันที เพราะเมื่อเราบอกว่า

$$A = \{x \in \mathcal{U} \mid P(x)\},$$

เรากำลังใช้ predicate $P(x)$ เพื่อคัดสมาชิกจากเอกภพสัมพัทธ์ ดังนั้นการเขียนเซตให้ดีจึงไม่ใช่เพียงเขียนวงเล็บปีกกาให้ถูก แต่ต้องเขียนเงื่อนไขให้ชัดว่ากำลังเลือกสมาชิกจากที่ใด และด้วยเหตุผลอะไร

4.1.1 ความเท่ากันของเซต

ในคณิตศาสตร์ เราไม่ได้ถือว่าเซตสองเซตเท่ากันเพราะเขียนหน้าตาคล้ายกัน หรือเรียงสมาชิกในลำดับเดียวกัน แต่ถือว่าเท่ากันเมื่อมีสมาชิกเหมือนกันทุกตัว นี่คือหลักคิดที่สำคัญมาก เพราะมันบอกว่าตัวตนของเซตขึ้นอยู่กับ “ใครเป็นสมาชิก” ไม่ใช่ขึ้นอยู่กับวิธีบรรยาย

ดังนั้นเราจึงใช้นิยามต่อไปนี้:

$$A = B \iff (\forall x)(x \in A \leftrightarrow x \in B).$$

นิยามนี้ดูสั้น แต่จริง ๆ แล้วบอกวิธีพิสูจน์ความเท่ากันของเซตไว้อย่างชัดเจน: หากต้องการพิสูจน์ว่า $A = B$ เราต้องแสดงว่าการเป็นสมาชิกของ A และ B ให้ผลเหมือนกันสำหรับทุก x ในทางปฏิบัติ วิธีที่นิยมที่สุดคือแยกพิสูจน์สองทางว่า

$$A \subseteq B \quad \text{และ} \quad B \subseteq A,$$

ซึ่งจะอธิบายในหัวข้อถัดไป

4.1.2 เซตย่อย

แนวคิดเรื่องเซตย่อยทำให้เราพูดได้ว่า “ทุกสมาชิกของกลุ่มหนึ่ง อยู่ในอีกกลุ่มหนึ่งด้วย” ซึ่งเป็นรูปแบบที่พบตลอดในคณิตศาสตร์

นิยาม ให้ A และ B เป็นเซต เรากล่าวว่า A เป็นเซตย่อยของ B และเขียนว่า $A \subseteq B$ ถ้า

$$(\forall x)(x \in A \rightarrow x \in B).$$

สิ่งที่ควรสังเกตคือ นิยามนี้ไม่ใช่เพียงนิยามของเรื่องเซต แต่เป็นนิยามที่ฝังโครงสร้างตรรกศาสตร์ไว้ชัดเจน: มีทั้งตัวบ่งปริมาณ $\forall$ และตัวเชื่อม $\rightarrow$ ดังนั้นเมื่อเราจะพิสูจน์ว่า $A \subseteq B$ รูปพิสูจน์ที่เหมาะสมจึงมักเป็นแบบเดียวกับการพิสูจน์ข้อความรูป “สำหรับทุก x ถ้า $x \in A$ แล้ว $x \in B$ ” นั่นคือ ให้ x เป็นสมาชิกใด ๆ และสมมติว่า $x \in A$ แล้วจึงพิสูจน์ว่า $x \in B$

ตัวอย่าง ถ้า E เป็นเซตของจำนวนคู่ทั้งหมด และ Z เป็นเซตของจำนวนเต็มทั้งหมด เราย่อมมี

$$E \subseteq Z$$

เพราะทุกจำนวนคู่เป็นจำนวนเต็มด้วย

ตัวอย่างนี้ดูธรรมดา แต่สำคัญเพราะมันแสดงให้เห็นว่า เซตย่อยคือภาษาที่ทำให้ข้อความภาษาไทยอย่าง “จำนวนคู่ทุกจำนวนเป็นจำนวนเต็ม” ถูกเขียนใหม่ในรูปที่กระชับและพร้อมสำหรับการพิสูจน์มากขึ้น

Claim ถ้า $A \subseteq B$ และ $B \subseteq C$ แล้ว $A \subseteq C$

Idea ถ้าสมาชิกทุกตัวของ A อยู่ใน B และสมาชิกทุกตัวของ B อยู่ใน C สมาชิกของ A ก็ย่อมอยู่ใน C

บทพิสูจน์. ให้ x เป็นสมาชิกใด ๆ และสมมติว่า $x \in A$ เนื่องจาก $A \subseteq B$ จึงได้ว่า $x \in B$ และเนื่องจาก

$B \subseteq C$ จึงได้ว่า $x \in C$ ดังนั้นสำหรับทุก x ถ้า $x \in A$ แล้ว $x \in C$ จึงสรุปได้ว่า $A \subseteq C$ $\square$

ตัวอย่างนี้สำคัญเพราะเป็นตัวอย่างพื้นฐานของการ “แคะนิยามแล้วใช้ต่อ” ซึ่งเป็นนิสัยสำคัญของการพิสูจน์ในวิชา

4.1.3 ข้อควรระวังเรื่องการนิยามเซต

หากมองอย่างไม่เป็นทางการ เราอาจอยากพูดว่าเซตคือ “กลุ่มของสิ่งของใด ๆ” แต่ประโยคนี้อันตรายกว่าที่คิด เพราะถ้าเปิดให้สร้างเซตได้อย่างอิสระจากทุกคุณสมบัติที่จินตนาการได้ ระบบจะเกิดความขัดแย้งในตัวเอง ตัวอย่างคลาสสิกคือ *Russell's paradox*

ลองพิจารณาเซต

$$R = \{A \mid A \notin A\},$$

กล่าวคือ R เป็น “เซตของทุกเซตที่ไม่เป็นสมาชิกของตนเอง” คำถามคือ $R \in R$ หรือไม่

ถ้าสมมติว่า $R \in R$ ตามนิยามของ R จะต้องได้ว่า $R \notin R$ แต่ถ้าสมมติว่า $R \notin R$ ตามนิยามเดียวกันกลับต้องได้ว่า $R \in R$ ไม่ว่าจะตอบอย่างไรก็นำไปสู่ความขัดแย้ง

บทเรียนจากตัวอย่างนี้คือ เราไม่ควรเปิดให้สร้างเซตจาก “ทุกคุณสมบัติ” อย่างไร้ข้อจำกัด ในทางรากฐานของคณิตศาสตร์ เราจึงใช้ระบบสัจพจน์ของทฤษฎีเซต ซึ่งกำหนดอย่างระมัดระวังว่า อะไรบ้างนับว่าเป็นเซต และเราสร้างเซตใหม่จากเซตเดิมได้ด้วยวิธีใดบ้าง

ในหนังสือเล่มนี้ เราไม่ได้ตั้งใจพัฒนาทฤษฎีเซตเชิงสัจพจน์อย่างเต็มรูปแบบ แต่การยก *Russell's paradox* ขึ้นมามีประโยชน์มาก เพราะช่วยให้ผู้อ่านเห็นว่า นิยามทางคณิตศาสตร์ไม่ใช่เรื่องของความสะดวกอย่างเดียว แต่เป็นเรื่องของความสอดคล้องเชิงตรรกะด้วย

4.1.4 เซตว่าง

เซตที่พื้นฐานที่สุดคือ เซตว่าง เขียนแทนด้วย $\emptyset$ ซึ่งเป็นเซตที่ไม่มีสมาชิกเลย กล่าวคือสำหรับทุก x ข้อความ $x \in \emptyset$ เป็นเท็จเสมอ

แม้แนวคิดนี้จะเรียบง่ายมาก แต่มีบทบาทสำคัญอย่างยิ่ง เพราะเป็นเหมือนวัตถุดิบตั้งต้นของการสร้างโครงสร้างหลายอย่างในคณิตศาสตร์

Claim ถ้า E เป็นเซตที่ไม่มีสมาชิกเลย แล้ว $E = \emptyset$

Idea เราต้องการแสดงว่าเซตที่ “ไม่มีอะไรอยู่ข้างในเลย” มีได้เพียงแบบเดียว

บทพิสูจน์. เพื่อพิสูจน์ว่า $E = \emptyset$ พอเพียงที่จะพิสูจน์ว่า $E \subseteq \emptyset$ และ $\emptyset \subseteq E$

ก่อนอื่นพิสูจน์ว่า $E \subseteq \emptyset$ ให้ x เป็นสมาชิกใด ๆ และสมมติว่า $x \in E$ แต่ข้อนี้เป็นไปไม่ได้ เพราะ E ไม่มีสมาชิกเลย ดังนั้นข้อความ “ถ้า $x \in E$ แล้ว $x \in \emptyset$ ” จึงเป็นจริงสำหรับทุก x จึงได้ว่า $E \subseteq \emptyset$

ต่อไปพิสูจน์ว่า $\emptyset \subseteq E$ ให้ x เป็นสมาชิกใด ๆ และสมมติว่า $x \in \emptyset$ ซึ่งเป็นไปไม่ได้เช่นกัน ดังนั้นข้อความ “ถ้า $x \in \emptyset$ แล้ว $x \in E$ ” เป็นจริงสำหรับทุก x จึงได้ว่า $\emptyset \subseteq E$

เมื่อได้ทั้งสองทิศทาง จึงสรุปว่า $E = \emptyset$ □

พิสูจน์นี้เป็นตัวอย่างที่ดีของการทำงานกับข้อความที่ส่วนหน้าของอิมพลีเคชันไม่อาจเกิดขึ้นได้ ซึ่งเป็นสถานการณ์ที่นักศึกษามักรู้สึกไม่คุ้นในตอนแรก

Claim เซตว่างเป็นเซตย่อยของทุกเซต

บทพิสูจน์. ให้ A เป็นเซตใด ๆ เราจะพิสูจน์ว่า $\emptyset \subseteq A$

ให้ x เป็นสมาชิกใด ๆ และสมมติว่า $x \in \emptyset$ ซึ่งเป็นไปไม่ได้ ดังนั้นข้อความ “ถ้า $x \in \emptyset$ แล้ว $x \in A$ ” เป็นจริงสำหรับทุก x จึงได้ว่า $\emptyset \subseteq A$ □

ข้อความนี้เป็นอีกตัวอย่างที่ดีของการพิสูจน์จากนิยามโดยตรง และยังเป็นจุดที่ช่วยฝึกให้ผู้เรียนแยกให้ออกว่า บางครั้ง “ไม่มีกรณีให้ตรวจ” ก็เพียงพอที่จะทำให้ข้อความเชิงสากลเป็นจริงได้

4.1.5 เซตกำลัง

เมื่อเรามีเซตหนึ่งเซต เรามักไม่ได้สนใจเพียงสมาชิกของมัน แต่สนใจด้วยว่า “มีเซตย่อยอะไรได้บ้าง” แนวคิดนี้นำไปสู่เซตกำลัง

นิยาม ให้ A เป็นเซต เซตกำลังของ A เขียนแทนด้วย $\mathcal{P}(A)$ คือเซตของเซตย่อยทั้งหมดของ A :

$$\mathcal{P}(A) = \{X \mid X \subseteq A\}.$$

ตัวอย่างเช่น ถ้า $A = \{1, 2, 3\}$ แล้ว

$$\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}.$$

สิ่งที่ควรสังเกตคือ สมาชิกของ $\mathcal{P}(A)$ ไม่ใช่ตัวเลข 1, 2, 3 แต่เป็น เซตย่อย ของ A นี่เป็นจุดที่ผู้เริ่มต้นมักพลาด เพราะผลอสับสระหว่าง

$$1 \in A \quad \text{กับ} \quad \{1\} \in \mathcal{P}(A).$$

สองประโยคนี้นั้นละชนิดกัน ประโยคแรกพูดถึงสมาชิกของ A ส่วนประโยคหลังพูดถึงเซตย่อยของ A

ถ้า A เป็นเซตจำกัดที่มีสมาชิก n ตัว เราจะได้ว่า

$$|\mathcal{P}(A)| = 2^n.$$

เหตุผลเชิงความคิดที่ดีที่สุดข้อหนึ่งคือ สำหรับสมาชิกแต่ละตัวของ A เรามีทางเลือกสองแบบ: จะเลือกเข้าเซตย่อยหรือไม่เลือก เมื่อมีสมาชิก n ตัว จึงมีรูปแบบการเลือกทั้งหมด 2^n แบบ

วิธีคิดนี้สำคัญมากในวิชาทฤษฎีสารสนเทศ เพราะมันเชื่อมการมองเซตย่อยเข้ากับการนับ และยังสะท้อนแนวคิดแบบ bitstring ได้อย่างเป็นธรรมชาติด้วย

4.1.6 การดำเนินการของเซต

เซตไม่ได้มีไว้เพียงเก็บสมาชิก แต่ยังมีการดำเนินการมาตรฐานที่ใช้สร้างเซตใหม่จากเซตเดิม ซึ่งสอดคล้องกับคำเชื่อมทางตรรกศาสตร์อย่างใกล้ชิด

ให้ A และ B เป็นเซตในเอกภพสัมพัทธ์ $\mathcal{U}$

นิยาม

$$A \cup B = \{x \in \mathcal{U} \mid x \in A \vee x \in B\},$$

$$A \cap B = \{x \in \mathcal{U} \mid x \in A \wedge x \in B\},$$

$$A^c = \{x \in \mathcal{U} \mid x \notin A\}.$$

จากนิยามจะเห็นได้ทันทีว่า ยูเนียนสัมพันธ์กับ "หรือ" อินเตอร์เซกชันสัมพันธ์กับ "และ" และ คอมพลีเมนต์สัมพันธ์กับ "ไม่" นี่คือการจุดหนึ่งทีที่แสดงว่าเซตกับตรรกศาสตร์ไม่ได้แยกจากกันจริง ๆ แต่เป็นภาษาสองชั้นที่ช่วยกันอธิบายโครงสร้างเดียวกัน

Claim ถ้า $A \subseteq B$ แล้ว $A \cup B = B$ และ $A \cap B = A$

Idea เมื่อทุกสมาชิกของ A อยู่ใน B อยู่แล้ว การเอา A มารวมกับ B จะไม่เพิ่มอะไรใหม่ และการตัดกันจะเหลือเพียง A

บทพิสูจน์. พิสูจน์ก่อนว่า $A \cup B = B$

ให้ x เป็นสมาชิกใด ๆ แล้ว

$$x \in A \cup B \iff (x \in A \vee x \in B).$$

แต่เพราะ $A \subseteq B$ ถ้า $x \in A$ ก็ต้องมี $x \in B$ ดังนั้นเงื่อนไขข้างขวาสอดคล้องกับเพียง $x \in B$ จึงได้ว่า

$$x \in A \cup B \iff x \in B.$$

สรุปว่า $A \cup B = B$

ต่อไปพิสูจน์ว่า $A \cap B = A$

ให้ x เป็นสมาชิกใด ๆ แล้ว

$$x \in A \cap B \iff (x \in A \wedge x \in B).$$

ถ้า $x \in A$ แล้วเพราะ $A \subseteq B$ จึงได้ $x \in B$ ดังนั้นเงื่อนไขข้างขวาสอดคล้องกับเพียง $x \in A$ จึงได้ว่า

$$x \in A \cap B \iff x \in A.$$

สรุปว่า $A \cap B = A$

□

Claim (De Morgan)

$$(A \cap B)^c = A^c \cup B^c.$$

บทพิสูจน์. ให้ x เป็นสมาชิกใด ๆ จะได้ว่า

$$x \in (A \cap B)^c \iff x \notin A \cap B \iff \neg(x \in A \wedge x \in B).$$

จากกฎของ De Morgan ในตรรกศาสตร์ เงื่อนไขนี้สมมูลกับ

$$(x \notin A) \vee (x \notin B),$$

ซึ่งก็คือ

$$x \in A^c \cup B^c.$$

ดังนั้นสำหรับทุก x

$$x \in (A \cap B)^c \iff x \in A^c \cup B^c,$$

จึงสรุปว่า

$$(A \cap B)^c = A^c \cup B^c.$$

□

พิสูจน์นี้เป็นตัวอย่างสำคัญมาก เพราะแสดงให้เห็นอย่างเป็นรูปธรรมว่า กฎทางตรรกศาสตร์สามารถถูกแปลเป็นกฎของเซตได้อย่างไร

4.1.7 ผลคูณคาร์ทีเซียน

ก่อนเข้าสู่เรื่องความสัมพันธ์และฟังก์ชัน เราต้องมีเครื่องมือสำหรับพูดถึง “คู่” ของสมาชิกก่อน ถ้า A และ B เป็นเซต ผลคูณคาร์ทีเซียนของ A และ B เขียนว่า $A \times B$ คือนิยามของเซตของคู่อันดับทั้งหมด

$$A \times B = \{(a, b) \mid a \in A, b \in B\}.$$

จุดที่ต้องระวังคือ คู่อันดับ (a, b) มีลำดับ ดังนั้นโดยทั่วไป

$$(a, b) \neq (b, a).$$

สิ่งนี้ทำให้ $A \times B$ เหมาะมากสำหรับใช้บันทึก “การเชื่อมโยงแบบมีทิศทาง” ซึ่งจะกลายเป็นหัวใจของทั้งความสัมพันธ์และฟังก์ชันในลำดับถัดไป

4.2 ความสัมพันธ์: การอธิบายว่าอะไรเกี่ยวข้องกับอะไร

เมื่อมีวัตถุอยู่ในเซตแล้ว คำถามถัดไปที่เกิดขึ้นตามธรรมชาติคือ วัตถุเหล่านี้เชื่อมโยงกันอย่างไร คำตอบอย่างเป็นระบบของคำถามนี้คือแนวคิดเรื่อง **ความสัมพันธ์**

ถ้า A และ B เป็นเซต ความสัมพันธ์จาก A ไป B คือเซตย่อยของ $A \times B$ กล่าวคือ

$$R \subseteq A \times B.$$

เมื่อ $(a, b) \in R$ เรามักเขียนย่อว่า

$$a R b$$

เพื่อสื่อว่า a มีความสัมพันธ์ R กับ b

นิยามนี้ดูเรียบง่ายมาก แต่ทรงพลังอย่างยิ่ง เพราะทำให้แนวคิดที่หลากหลายถูกเขียนในรูปเดียวกันได้ เช่น ``น้อยกว่า" ``หารลงตัว" ``เป็นเพื่อนกับ" ``เป็นวิชาบังคับก่อนของ" หรือ ``มีเส้นเชื่อมถึงกันในกราฟ"

4.2.1 สมบัติสำคัญของความสัมพันธ์

เมื่อความสัมพันธ์อยู่บนเซตเดียวกัน คือ $R \subseteq A \times A$ เรามักสนใจสมบัติบางอย่างเป็นพิเศษ

นิยาม ให้ R เป็นความสัมพันธ์บนเซต A

- R เป็น สะท้อนกลับ (reflexive) ถ้า $(\forall x \in A)(x R x)$
- R เป็น สมมาตร (symmetric) ถ้า $(\forall x, y \in A)(x R y \rightarrow y R x)$
- R เป็น ปฏิสมมาตร (antisymmetric) ถ้า $(\forall x, y \in A)((x R y \wedge y R x) \rightarrow x = y)$
- R เป็น ถ่ายทอด (transitive) ถ้า $(\forall x, y, z \in A)((x R y \wedge y R z) \rightarrow x R z)$

สิ่งที่ควรสังเกตคือ นิยามเหล่านี้ไม่ใช่เรื่องที่ต้องท่องจำเพียงอย่างเดียว แต่เป็นนิยามที่ควรอ่านให้เห็นความหมายเชิงโครงสร้าง

ตัวอย่างเช่น reflexive แปลว่าแต่ละตัวสัมพันธ์กับตัวเอง symmetric แปลว่าทิศทางกลับได้ antisymmetric แปลว่าถ้ากลับไปกลับมาได้ทั้งสองทาง ก็ต้องเป็นตัวเดียวกัน ส่วน transitive แปลว่าใช้ความสัมพันธ์สามารถส่งต่อได้

ตัวอย่าง บนเซตจำนวนเต็ม $\mathbb{Z}$ ความสัมพันธ์ $=$ มีทั้ง reflexive, symmetric, antisymmetric และ transitive

ส่วนความสัมพันธ์ $\leq$ มี reflexive, antisymmetric และ transitive แต่ไม่ symmetric

ส่วนความสัมพันธ์ ``เป็นเพื่อนกับ" มักถูกมองว่า symmetric แต่ไม่จำเป็นต้อง transitive

การฝึกจำแนกตัวอย่างเช่นนี้สำคัญมาก เพราะช่วยให้ผู้เรียนเริ่มเห็นว่า นิยามแต่ละตัวกำลังจับ ``พฤติกรรม" แบบใดของความสัมพันธ์

4.2.2 ความสัมพันธ์สมมูล

ความสัมพันธ์ที่สำคัญมากชนิดหนึ่งคือความสัมพันธ์สมมูล เพราะมันทำหน้าที่แบ่งเซตออกเป็นกลุ่มของสิ่งที่ ``ถือว่าเหมือนกันในมุมที่เราสนใจ"

นิยาม ความสัมพันธ์ R บนเซต A เรียกว่า **ความสัมพันธ์สมมูล** (equivalence relation) ถ้ามีสมบัติครบทั้งสามข้อคือ reflexive, symmetric, และ transitive

ตัวอย่างสำคัญคือความสัมพันธ์ ``มีเศษเท่ากันเมื่อหารด้วย n " บนเซตจำนวนเต็ม เช่นบน $\mathbb{Z}$ นิยามว่า

$$a \sim b \iff a \equiv b \pmod{n}.$$

ความสัมพันธ์นี้เป็น equivalence relation เพราะจำนวนเต็มที่ให้เศษเท่ากันเมื่อหารด้วย n ควรถูกมองว่าอยู่ในชั้นเดียวกัน

เมื่อมี equivalence relation เราสามารถนิยาม **ชั้นสมมูล** (equivalence class) ของ $a \in A$ ได้ว่า

$$[a] = \{x \in A \mid xRa\}.$$

สิ่งที่ควรสังเกตคือ ชั้นสมมูลไม่ได้เลือกสมาชิกเพราะมันเท่ากับ a ตรง ๆ แต่เลือกสมาชิกที่ ``สมมูล" กับ a ภายใต้ความสัมพันธ์ที่กำหนด

แนวคิดนี้มีความสำคัญมาก เพราะในหลายบริบทของคณิตศาสตร์ เราไม่สนใจวัตถุแต่ละชิ้นอย่างละเอียดทุกประการ แต่สนใจเพียงว่ามันอยู่ในชั้นประเภทเดียวกันหรือไม่ เช่นจำนวนเต็ม modulo n หรือโครงสร้างที่เท่ากันในบางแง่มุมแต่ไม่จำเป็นต้องเหมือนกันทุกจุด

Claim ถ้า R เป็น equivalence relation บน A แล้วชั้นสมมูลสองชั้นใด ๆ จะเหมือนกันทุกประการหรือไม่ก็ทับกันเลย

Idea ถ้าชั้นสมมูลสองชั้นมีสมาชิกตัวหนึ่งร่วมกัน สมบัติ transitive และ symmetric จะบังคับให้สมาชิกทั้งหมดของสองชั้นนั้นตรงกัน

บทพิสูจน์. ให้ $a, b \in A$ และสมมติว่า $[a] \cap [b] \neq \emptyset$ ดังนั้นมีสมาชิก c บางตัวที่ $c \in [a]$ และ $c \in [b]$

จาก $c \in [a]$ ได้ว่า cRa และจาก $c \in [b]$ ได้ว่า cRb

เราจะแสดงว่า $[a] \subseteq [b]$ ให้ $x \in [a]$ แล้ว xRa จาก cRa และสมมาตรของ R ได้ว่า aRc ดังนั้นจาก xRa

และ aRc ใช้ transitive ได้ xRc อีกทั้งจาก cRb ใช้ transitive อีกครั้งได้ xRb จึงได้ว่า $x \in [b]$ สรุปว่า $[a] \subseteq [b]$

ด้วยเหตุผลแบบเดียวกันสลับบทบาทของ a และ b จะได้ว่า $[b] \subseteq [a]$ ดังนั้น $[a] = [b]$

จึงสรุปได้ว่า ถ้าชั้นสมมูลสองชั้นมีสมาชิกซ้ำกันแม้เพียงตัวเดียว สองชั้นนั้นต้องเท่ากันทั้งชั้น □

นี่คือเหตุผลที่ equivalence relation ถูกมองว่าเป็นเครื่องมือสำหรับ “แบ่งเซตออกเป็นชั้น”

4.2.3 ความสัมพันธ์อันดับบางส่วน

ความสัมพันธ์สำคัญอีกชนิดหนึ่งคือ **partial order** ซึ่งใช้จัดวางวัตถุในลักษณะ “เปรียบเทียบได้บางคู่ แต่ไม่จำเป็นต้องทุกคู่”

นิยาม ความสัมพันธ์ R บนเซต A เรียกว่า **partial order** ถ้ามีสมบัติ reflexive, antisymmetric และ transitive

ตัวอย่างคลาสสิกคือความสัมพันธ์ $\subseteq$ บนเซตกำลัง $\mathcal{P}(A)$ เพราะทุกเซตเป็นเซตย่อยของตัวเอง ถ้า $X \subseteq Y$ และ $Y \subseteq X$ ก็ต้องได้ $X = Y$ และถ้า $X \subseteq Y$ กับ $Y \subseteq Z$ ก็ได้ $X \subseteq Z$

ตัวอย่างนี้สำคัญมาก เพราะช่วยให้เราเห็นว่า partial order ไม่จำเป็นต้องเป็น “น้อยกว่า” ในเชิงตัวเลขเสมอไป แต่มันคือความสัมพันธ์ที่สะท้อนลำดับเชิงโครงสร้างบางอย่าง เช่น “อยู่ข้างใน” “เป็นวิชาบังคับก่อน” หรือ “ละเอียดกว่า”

4.3 ฟังก์ชัน: ความสัมพันธ์ที่กำหนดผลลัพธ์อย่างแน่นอน

ฟังก์ชันเป็นความสัมพันธ์ชนิดพิเศษ และเป็นหนึ่งในแนวคิดที่สำคัญที่สุดในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ในทางไม่เป็นทางการ ฟังก์ชันคือกฎที่รับ input แล้วให้ output อย่างแน่นอน แต่ในทางคณิตศาสตร์ เราต้องระบุให้ชัดกว่าเดิมว่า “อย่างแน่นอน” นั้นหมายความว่าอะไร

นิยาม ให้ A และ B เป็นเซต ฟังก์ชัน f จาก A ไป B เขียนว่า

$$f : A \rightarrow B$$

คือความสัมพันธ์ $f \subseteq A \times B$ ที่มีคุณสมบัติสองข้อพร้อมกัน:

1. สำหรับทุก $a \in A$ มี $b \in B$ อย่างน้อยหนึ่งตัวที่ $(a, b) \in f$
2. ถ้า $(a, b_1) \in f$ และ $(a, b_2) \in f$ แล้ว $b_1 = b_2$

ข้อแรกบอกว่าทุก input ต้องมี output ข้อที่สองบอกว่า input เดียวกันให้ output ได้เพียงตัวเดียว ดังนั้นฟังก์ชันจึงไม่ใช่เพียง relation ธรรมดา แต่เป็น relation ที่มีทั้ง *existence* และ *uniqueness*

เมื่อ $(a, b) \in f$ เรามักเขียนว่า

$$f(a) = b.$$

สิ่งที่นักศึกษาควรพลาตคือการสับสนระหว่าง **codomain** กับ **range** ถ้า $f : A \rightarrow B$ แล้ว A คือโดเมน B คือโคโดเมน ส่วนเรนจ์หรือ image ของ f คือเซตของค่าที่ถูกส่งไปถึงจริง:

$$\text{Im}(f) = \{f(a) \mid a \in A\}.$$

ดังนั้นโดยทั่วไป

$$\text{Im}(f) \subseteq B,$$

และไม่จำเป็นต้องเท่ากับ B

4.3.1 ฟังก์ชันหนึ่งต่อหนึ่ง ทัวถึง และพหุภาพ

คุณสมบัติของฟังก์ชันที่สำคัญมากมีสามแบบ

นิยาม ให้ $f : A \rightarrow B$

- f เป็น หนึ่งต่อหนึ่ง (injective) ถ้า

$$f(a_1) = f(a_2) \rightarrow a_1 = a_2$$

สำหรับทุก $a_1, a_2 \in A$

- f เป็น ทัวถึง (surjective) ถ้า

$$(\forall b \in B)(\exists a \in A) f(a) = b$$

- f เป็น พหุภาพ (bijective) ถ้าเป็นทั้ง injective และ surjective

ในเชิง intuition, injective หมายถึง input ต่างกันไม่ชนกันที่ output เดียวกัน surjective หมายถึงทุกสมาชิกของโคโดเมนถูก hit โดย input อย่างน้อยหนึ่งตัว ส่วน bijective หมายถึงจับคู่กันได้พอดีแบบไม่มีเหลือและไม่มี

ชนกัน

ตัวอย่าง ฟังก์ชัน $f : \mathbb{Z} \rightarrow \mathbb{Z}$ ที่นิยามโดย $f(n) = n + 1$ เป็น bijective แต่ฟังก์ชัน $g : \mathbb{Z} \rightarrow \mathbb{Z}$ ที่นิยามโดย $g(n) = n^2$ ไม่ injective เพราะ $g(1) = g(-1)$ และไม่ surjective เพราะไม่มีจำนวนเต็มใดที่ยกกำลังสองแล้วได้ -1

ตัวอย่างเช่นนี้สำคัญเพราะช่วยฝึกให้อ่านนิยามเชิงปริมาณได้จริง โดยเฉพาะ surjective ซึ่งมักถูกเข้าใจผิดง่ายที่สุด เพราะต้องระวังว่า $\forall b \in B$ อยู่ข้างหน้า $\exists a \in A$

4.3.2 ฟังก์ชันประกอบและฟังก์ชันผกผัน

ถ้า $f : A \rightarrow B$ และ $g : B \rightarrow C$ เรานิยาม ฟังก์ชันประกอบ $g \circ f : A \rightarrow C$ โดย

$$(g \circ f)(a) = g(f(a)).$$

แนวคิดนี้ตรงไปตรงมาแต่สำคัญมาก เพราะสะท้อนการทำงานแบบเป็นขั้นตอน: ส่ง a ผ่าน f ก่อน แล้วนำผลลัพธ์ไปผ่าน g ต่อ

สิ่งที่ควรระวังคือ ลำดับมีความสำคัญ โดยทั่วไป

$$g \circ f \neq f \circ g$$

แม้ว่าทั้งสองข้างจะนิยามได้ก็ตาม

ถ้า $f : A \rightarrow B$ เป็น bijection จะมีฟังก์ชันผกผัน

$$f^{-1} : B \rightarrow A$$

ซึ่งทำให้

$$f^{-1}(f(a)) = a \quad \text{และ} \quad f(f^{-1}(b)) = b.$$

เงื่อนไขการเป็น bijection จึงไม่ใช่เพียงสมบัติสวยงาม แต่เป็นเงื่อนไขที่รับประกันว่า ข้อมูลไม่สูญหายและไม่ชนกัน จนสามารถย้อนกลับได้จริง

Claim ถ้า $f : A \rightarrow B$ และ $g : B \rightarrow C$ เป็น injective ทั้งคู่ แล้ว $g \circ f$ เป็น injective

บทพิสูจน์. สมมติว่า

$$(g \circ f)(a_1) = (g \circ f)(a_2).$$

ดังนั้น

$$g(f(a_1)) = g(f(a_2)).$$

เพราะ g เป็น injective จึงได้ว่า

$$f(a_1) = f(a_2).$$

และเพราะ f เป็น injective จึงสรุปว่า

$$a_1 = a_2.$$

ดังนั้น $g \circ f$ เป็น injective

□

พิสูจน์นี้สั้นแต่มีคุณค่ามาก เพราะเป็นตัวอย่างของการใช้ “นิยามล้วน ๆ” โดยแทบไม่ต้องคำนวณอะไรเลย

4.4 จากความสัมพันธ์สู่การนับ: ขนาดของเซต

เมื่อเราเริ่มมองเซตอย่างจริงจัง คำถามตามธรรมชาติอีกข้อหนึ่งคือ “แล้วเซตสองเซตมีขนาดเท่ากันได้อย่างไร” สำหรับเซตจำกัด เราอาจนับสมาชิกตรง ๆ ได้ แต่เมื่อมองในมุมมองที่กว้างขึ้น วิธีคิดที่ทรงพลังกว่าคือการเปรียบเทียบผ่านการจับคู่

นิยาม เราเรียกเซต A และ B ว่า **สมมูลกันเชิงการนับ** หรือ **มีคาร์ดินัลเท่ากัน** ถ้ามี bijection จาก A ไป B

นิยามนี้สำคัญมาก เพราะมันเปลี่ยนคำถามเรื่อง “จำนวนเท่ากันไหม” ให้กลายเป็นคำถามเรื่อง “จับคู่กันได้พอดีไหม” ซึ่งเป็นมุมมองเชิงโครงสร้างมากกว่า

สำหรับเซตจำกัด นิยามนี้สอดคล้องกับสัญชาตญาณเดิมของเรา เช่นเซต $\{a, b, c\}$ และ $\{1, 2, 3\}$ มีขนาดเท่ากัน เพราะจับคู่กันแบบหนึ่งต่อหนึ่งและทั่วถึงได้

แต่เมื่อไปถึงเซตอนันต์ นิยามนี้จะเผยภาพที่น่าสนใจมากขึ้น และเป็นจุดเริ่มต้นของทฤษฎีคาร์ดินัล

4.4.1 ทฤษฎีบทของแคนเทอร์

หนึ่งในผลลัพธ์ที่สำคัญและงดงามที่สุดเกี่ยวกับเซตคือ **เซตกำลังของเซตใด ๆ มีขนาดใหญ่มากกว่าเซตเดิมเสมอ** ข้อความนี้เป็นจริงแม้ในกรณีที่เซตเดิมเป็นเซตอนันต์

Claim (Cantor's Theorem) ไม่มีฟังก์ชันทั่วถึงจากเซต A ไปยัง $\mathcal{P}(A)$

หรือกล่าวอีกแบบหนึ่งคือ

$$|A| < |\mathcal{P}(A)|.$$

Idea สมมติว่ามีฟังก์ชันทั่วถึง $f : A \rightarrow \mathcal{P}(A)$ แล้วสร้างเซตหนึ่งขึ้นมาโดย "แยง" กับค่าของ f ทุกตัว เพื่อบังคับให้เซตนั้นไม่อาจเป็นค่าของ $f(a)$ สำหรับ a ใด ๆ ได้เลย

บทพิสูจน์. สมมติย้อนแย้งว่ามีฟังก์ชันทั่วถึง

$$f : A \rightarrow \mathcal{P}(A).$$

นิยามเซต

$$D = \{a \in A \mid a \notin f(a)\}.$$

เพราะ $D \subseteq A$ จึงมี $D \in \mathcal{P}(A)$

เนื่องจาก f เป็น surjective ต้องมีสมาชิก $d \in A$ ที่ทำให้

$$f(d) = D.$$

ตอนนี้ถามว่า $d \in D$ หรือไม่

ถ้า $d \in D$ จากนิยามของ D จะได้ว่า $d \notin f(d)$ แต่ $f(d) = D$ จึงได้ว่า $d \notin D$ เกิดความขัดแย้ง

ในทางกลับกัน ถ้า $d \notin D$ จากนิยามของ D จะได้ว่า $d \in f(d)$ และเพราะ $f(d) = D$ จึงได้ว่า $d \in D$ เกิดความขัดแย้งอีกเช่นกัน

ดังนั้นสมมติฐานที่ว่าไม่มีฟังก์ชันทั่วถึงจาก A ไปยัง $\mathcal{P}(A)$ ต้องเป็นเท็จ สรุปว่าไม่มีฟังก์ชันทั่วถึงเช่นนั้น $\square$

ทฤษฎีบทนี้ควรถูกมองว่าเป็นมากกว่าผลลัพธ์เฉพาะทางของทฤษฎีเซต เพราะมันแสดงให้เห็นพลังของการนิยามวัตถุใหม่อย่างแยบยล แล้วใช้เหตุผลเชิงขัดแย้งเพื่อพิสูจน์ว่า สิ่งที่คุณเหมือน ๆ น่าจะจับคู่ได้หมด" แท้จริงแล้วจับคู่ไม่ได้ และวิธีพิสูจน์แบบ diagonal นี้จะกลับมาปรากฏอีกในหลายบริบทของวิทยาการคอมพิวเตอร์เช่นกัน

4.5 สรุปภาพรวมของบท

เมื่อมองย้อนกลับไป เราจะเห็นว่าบทนี้ไม่ได้เป็นเพียงการรวบรวมนิยามพื้นฐาน แต่เป็นการวางรากของภาษาที่ใช้ตลอดทั้งเล่ม

เซตทำให้เราพูดถึงการเป็นสมาชิก การรวมกลุ่ม และการสร้างวัตถุใหม่จากวัตถุเดิมได้อย่างเป็นระบบ ความสัมพันธ์ทำให้เราพูดถึงการเชื่อมโยงระหว่างวัตถุ และเปิดทางไปสู่แนวคิดอย่าง *equivalence relation* และ *partial order* ส่วนฟังก์ชันทำให้เราพูดถึงการจับคู่ *input* กับ *output* อย่างแน่นอน ซึ่งเป็นแกนกลางของทั้งคณิตศาสตร์เชิงนามธรรมและการมองโปรแกรมเชิงคำนวณ

ที่สำคัญไม่แพ้กันคือ บทนี้แสดงให้เห็นอีกครั้งว่า คณิตศาสตร์ดิสครีตไม่ได้เดินด้วยการคำนวณเพียงอย่างเดียว แต่เดินด้วยนิยาม โครงสร้าง และการพิสูจน์ นิยามของเซตย่อยบอกวิธีพิสูจน์เรื่องเซตย่อย นิยามของ *equivalence relation* บอกสิ่งที่ต้องตรวจเพื่อพิสูจน์ความสัมพันธ์สมมูล นิยามของ *injective* และ *surjective* บอกทันทีว่าพิสูจน์ต้องเริ่มจากสมมติฐานใดและควรจบที่ข้อสรุปใด

ดังนั้นถ้าจะสรุปบทนี้ในประโยคเดียว ผู้เขียนอยากให้จำไว้ว่า

เซต ความสัมพันธ์ และฟังก์ชัน ไม่ใช่เพียงรายการหัวข้อพื้นฐานของวิชาดิสครีต แต่เป็นภาษาหลักที่ทำให้เราสามารถนิยามโครงสร้างต่าง ๆ และให้เหตุผลเกี่ยวกับโครงสร้างเหล่านั้นได้อย่างรัดกุม

ในบทถัดไป แนวคิดเหล่านี้จะไม่หายไปไหน แต่จะถูกนำกลับมาใช้อีกครั้งในบริบทที่ลึกซึ้ง ไม่ว่าจะเป็นการนับ การวิเคราะห์อัลกอริทึม โครงสร้างกราฟ หรือการให้เหตุผลกับวัตถุที่นิยามแบบเวียนเกิดในภายหลัง

ภาค III

การแก้ปัญหาเชิงกระบวนการและการเวียนเกิด

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 5

แนวคิดพื้นฐานของการเวียนเกิด

หัวข้อนี้ต่างจากบทก่อนหน้าอยู่พอสมควร เพราะเป้าหมายหลักไม่ใช่การพิสูจน์ข้อความเชิงตรรกะโดยตรง แต่เป็นการฝึก *ออกแบบวิธีคิด* สำหรับปัญหาที่มีโครงสร้างซ้ำ กล่าวอย่างง่ายที่สุด การคิดแบบเวียนเกิดคือการพยายามตอบคำถามต่อไปนี้เสมอว่า

ถ้าฉันแก้ปัญหานี้ได้แล้ว ฉันจะเอาคำตอบนั้นมาประกอบเป็นคำตอบของปัญหาลูกข่ายได้อย่างไร

ในมุมมองของการเขียนโปรแกรม ความคิดนี้มักปรากฏในรูปของฟังก์ชันที่เรียกใช้ตัวเอง แต่แก่นแท้ของ *recursion* ไม่ได้อยู่ที่การ “เรียกตัวเอง” แต่อยู่ที่การนิยามปัญหาลูกข่ายด้วยปัญหา *ชนิดเดิม* แต่มีขนาดเล็กลง พร้อมกับมีกรณีฐานที่เล็กจนตอบได้ทันทีโดยไม่ต้องย้อนลงไปอีก

5.1 Recursion

การแก้ปัญหาทาง *computational* บนโลกนี้มี 2 แนวคิดหลัก

1. **Iterative programming:** ใช้ *loop* ที่ผู้พัฒนาตั้ง *logic* การแก้ปัญหาในแต่ละรอบไว้แบบชัดเจนแล้ว -- อ่านง่าย(สำหรับคนส่วนใหญ่) วิธีการแก้ปัญหาเหมือนพาจับมือทำ ภาษาโปรแกรมส่วนใหญ่ออกแบบมาเพื่อเขียนแบบนี้อยู่แล้ว แต่การได้วิธีการแก้บางโจทย์ค่อนข้างยากและไม่ตรงไปตรงมา และตรวจสอบความถูกต้องทางทฤษฎีได้ลำบาก
2. **Recursive programming:** ฟังก์ชันเรียกใช้ตัวเองเพื่อเอาคำตอบจากปัญหาลูกข่ายที่เล็กกว่า -- บางคนบอกว่าอ่านยาก(แต่สำหรับผม *code* คลีนกว่า) วิธีการได้มาของผลลัพธ์เหมือนต้องอาศัยความรู้เก่าเลยไม่รู้ว่าจะได้มายังไง ภาษาแบบ *functional programming* ถูกพัฒนาเพื่อเขียนแบบนี้ (เช่น *Haskell*, *Scala*) แต่ภาษาที่ *iterative* ก็ทำได้ แต่ว่าการคิดวิธีการแก้โจทย์ค่อนข้างชัดเจนและเป็นธรรมชาติกับวิธีคิดมาก

กว่า และที่สำคัญ ตรวจสอบความถูกต้องของการทำงานของโปรแกรมด้วยการพิสูจน์ทางคณิตศาสตร์ได้ (อาจารย์คิดว่าวิธีคิดของมนุษย์ใกล้เคียงการแก้ปัญหาแบบ recursive มากกว่า iterative เพราะปกติมนุษย์มักใช้ประสบการณ์ในการลองแก้ปัญหาที่ง่ายกว่าก่อนแล้วค่อยพัฒนาผลลัพธ์ปัญหาง่ายให้เป็นผลลัพธ์ของปัญหาที่ยากขึ้น)

การเวียนเกิด (recursion) คือรูปแบบการแก้ปัญหา (หรือการคำนวณ) ปัญหาหนึ่ง ๆ ที่อาศัยผลลัพธ์จากการแก้ปัญหาเดียวกันแต่ input เล็กกว่านำมาประกอบขึ้นเป็นผลลัพธ์ของปัญหาที่กำลังแก้ ซึ่งในทางการเขียนโปรแกรมนั้น input ที่เล็กกว่ามักจะเป็นส่วนย่อยของ input ที่อยากแก้ปัญหา ตัวอย่างเช่น

- อยากแก้ปัญหาที่รับ array $A[1 \dots n]$ เป็น input โดยใช้ผลลัพธ์จากโจทย์เดียวกันแต่แก้กับ $A[1 \dots (n - 1)]$ (กล่าวคือลองดูกับแค่ตัวแรกจนถึงตัวรองสุดท้ายก่อนว่าได้ผลลัพธ์เป็นอะไรแล้วค่อยนำคนสุดท้ายมาพิจารณาพร้อมผลลัพธ์ที่ได้มา)
- อยากคำนวณค่าของโจทย์หนึ่งที่ได้รับ n เข้าไป โดยใช้ผลลัพธ์จากโจทย์เดียวกันที่คำนวณกับ $n - 1$ และ $n - 2$ มาแล้ว

5.1.1 Recursion ในฐานะรูปแบบของการแก้ปัญหา

Definition 5.1.1. การเวียนเกิด (recursion) คือวิธีนิยามหรือแก้ปัญหาปัญหาหนึ่งโดยอาศัยผลลัพธ์ของปัญหาเดียวกันในกรณีที่เล็กกว่า ร่วมกับ **กรณีฐาน (base case)** ที่ตอบได้โดยตรง

นิยามนี้มีองค์ประกอบสำคัญอยู่สามส่วน

1. **ปัญหามีสเกลเล็กกว่า**
ต้องมีความหมายชัดเจนว่า "เล็กกว่า" อย่างไร เช่น ใช้ข้อมูลน้อยลงหนึ่งตัว ใช้จำนวนเต็มที่เล็กลงหนึ่งหน่วย หรือใช้โครงสร้างย่อยของวัตถุเดิม
2. **กฎสำหรับประกอบคำตอบ**
เมื่อรู้คำตอบของกรณีเล็กกว่าแล้ว ต้องอธิบายให้ได้ว่าจะประกอบเป็นคำตอบของกรณีปัจจุบันอย่างไร
3. **กรณีฐาน**
ถ้าไม่มีกรณีฐาน การนิยามแบบ recursion จะไม่หยุด และจึงไม่ให้ความหมายที่ใช้งานได้จริง

สิ่งที่ต้องมีเสมอ

ถ้าจะนิยามสิ่งใดด้วย recursion ให้ตรวจสอบอย่างนี้เสมอ:

1. ปัญหาเล็กลงจริงหรือไม่
2. จากคำตอบเดิมประกอบคำตอบใหม่ได้จริงหรือไม่
3. มีกัณฑ์หรือยัง กล่าวคือมี base case หรือยัง

5.1.2 เปรียบเทียบกับการทำแบบวนซ้ำ

ในทางการเขียนโปรแกรม เรามักพบสองแนวคิดหลักในการแก้ปัญหาเชิงคำนวณ

1. การทำแบบวนซ้ำ (iteration):

ผู้เขียนกำหนดลำดับการทำงานที่ลูปผ่าน loop อย่างชัดเจน

2. การทำแบบเวียนเกิด (recursion):

ผู้เขียนนิยามคำตอบของปัญหาปัจจุบันจากคำตอบของปัญหาย่อยชนิดเดิม

ทั้งสองวิธีสามารถให้ผลลัพธ์เดียวกันได้ในหลายโจทย์ แต่ให้ "มุมมอง" ต่อปัญหาต่างกัน iteration เน้นลำดับคำสั่ง ส่วน recursion เน้นโครงสร้างของปัญหา

ในรายวิชานี้ เราให้ความสำคัญกับ recursion เพราะมันช่วยบังคับให้เราเห็นว่า ปัญหาหนึ่งแตกออกเป็นปัญหาย่อยกว่าอย่างไร และในหลายกรณี ความคิดแบบนี้เป็นธรรมชาติอย่างมากสำหรับทั้งคณิตศาสตร์และวิทยาการคอมพิวเตอร์

5.2 ตัวอย่างอุปกรณ์: การนิยามลำดับแบบเวียนเกิด

ก่อนจะไปสู่ปัญหาประเภทอัลกอริทึม เราจะเริ่มจากตัวอย่างที่คุ้นเคยอย่างลำดับเชิงตัวเลข เพราะตัวอย่างเหล่านี้ทำให้เห็นโครงสร้างของ recursion ได้ชัดที่สุด

5.2.1 ลำดับเลขคณิต

ลำดับเลขคณิตคือ ลำดับที่ผลต่างระหว่างพจน์ติดกันมีค่าคงที่เสมอ ถ้าเราใช้ $a(n)$ แทนพจน์ที่ n และให้ d เป็นผลต่างร่วม เราจะเขียนนิยามแบบ recursion ได้ว่า

$$a(0) = a_0, \quad a(n) = a(n-1) + d \quad \text{สำหรับทุก } n \geq 1.$$

ประโยคนี้ควรถูกอ่านว่า พจน์ที่ n ไม่ได้เกิดขึ้นอย่างลอย ๆ แต่ได้มาจากพจน์ก่อนหน้า แล้วบวกด้วยค่าคงที่เดิม d

Example 5.2.1. ถ้า

$$a(0) = 5, \quad d = 3$$

แล้วลำดับนี้เริ่มเป็น

$$5, 8, 11, 14, 17, \dots$$

เพราะ

$$a(1) = a(0) + 3 = 8, \quad a(2) = a(1) + 3 = 11, \quad a(3) = a(2) + 3 = 14.$$

ดังนั้น

$$a(7) = 26.$$

จุดสำคัญของตัวอย่างนี้ไม่ได้อยู่ที่การหาคำตอบ 26 แต่อยู่ที่การเห็นว่าการนิยามแบบ recursion มีลักษณะเป็น

“รู้ค่าหนึ่งค่าเริ่มต้น แล้วเดินไปข้างหน้าทีละขั้น”

5.2.2 ลำดับฟีโบนัคชี

อีกตัวอย่างมาตรฐานหนึ่งคือลำดับฟีโบนัคชี ซึ่งแต่ละพจน์ได้มาจากผลบวกของสองพจน์ก่อนหน้า

$$Fib(0) = 1, \quad Fib(1) = 1,$$

และสำหรับทุก $n \geq 2$,

$$Fib(n) = Fib(n-1) + Fib(n-2).$$

ในที่นี้เราต้องมีกรณีฐาน สองค่า เพราะความสัมพันธ์เวียนเกิดอ้างถึงสองพจน์ก่อนหน้า ดังนั้นถ้ามีเพียง $Fib(0)$ ค่าเดียว เรายังเริ่มคำนวณต่อไม่ได้

Example 5.2.2. จากนิยามข้างต้นจะได้ว่า

$$Fib(2) = Fib(1) + Fib(0) = 1 + 1 = 2,$$

$$Fib(3) = Fib(2) + Fib(1) = 2 + 1 = 3,$$

$$Fib(4) = Fib(3) + Fib(2) = 3 + 2 = 5,$$

$$Fib(5) = Fib(4) + Fib(3) = 5 + 3 = 8.$$

ข้อสังเกต

จากตัวอย่างลำดับเลขคณิตและฟีโบนัคชี เราเริ่มเห็นโครงซ้ำของ recursion แล้ว:

1. มีค่าฐาน
2. มีสูตรที่บอกว่าจะสร้างค่าถัดไปจากค่าก่อนหน้าอย่างไร
3. ยิ่งสูตรอ้างอิงย้อนหลังหลายขั้น ก็ยิ่งต้องมีฐานมากพอ

5.3 Recursion ในฐานะขั้นตอนวิธี: Tower of Hanoi

ตัวอย่างต่อไปนี้สำคัญมาก เพราะแสดงให้เห็น recursion ไม่ใช่แค่การนิยามลำดับ แต่เป็นการออกแบบ *วิธีทำงาน* อย่างเป็นขั้นตอน

5.3.1 กติกาของปัญหา

เกม Tower of Hanoi มีเสาสามต้น ซึ่งเราจะเรียกว่า A, B, C และมีแผ่นดิสก์หลายแผ่นวางซ้อนกันอยู่บนเสาต้นหนึ่ง โดยแผ่นเล็กกว่าจะอยู่บนแผ่นใหญ่กว่าเสมอ เราต้องการย้ายกองแผ่นทั้งหมดจากเสาดั้งเดิมต้นไปยังเสาปลายทาง ภายใต้กติกาสองข้อคือ

1. ย้ายได้ครั้งละหนึ่งแผ่นเท่านั้น
2. ห้ามวางแผ่นใหญ่บนแผ่นเล็ก

5.3.2 เริ่มจากกรณี 4 แผ่น

สมมติว่าต้องการย้ายดิสก์ 4 แผ่นจากเสา A ไปเสา C โดยใช้เสา B เป็นตัวช่วย ถ้าเรามองเฉพาะแผ่นล่างสุด ซึ่งเป็นแผ่นใหญ่ที่สุด จะพบว่ามันย้ายได้เพียงครั้งเดียว และจะย้ายได้ก็ต่อเมื่อแผ่นเล็กอีก 3 แผ่นด้านบนถูกย้ายออกไปหมดแล้ว

นี่คือจุดที่ recursion โผล่ขึ้นมาอย่างเป็นธรรมชาติ: ถ้าเรามี “ผู้ช่วย” ที่สามารถย้ายกอง 3 แผ่นได้สำเร็จเสมอ เราก็สามารถใช้ผู้ช่วยคนนั้นเป็นส่วนหนึ่งของแผนสำหรับ 4 แผ่นได้ทันที

โครงของวิธีแก้จึงเป็นดังนี้

1. ย้ายแผ่น 3 แผ่นบนสุดจาก A ไป B
2. ย้ายแผ่นใหญ่สุดจาก A ไป C
3. ย้ายแผ่น 3 แผ่นจาก B ไป C

สิ่งที่น่าสนใจคือ ขั้นตอนที่ 1 และ 3 เป็น “ปัญหายาชนิดเดิม” เพียงแต่ขนาดเล็กลงจาก 4 แผ่นเหลือ 3 แผ่นเท่านั้น

5.3.3 รูปแบบทั่วไปสำหรับ n แผ่น

จากตัวอย่าง 4 แผ่น เราเห็นรูปแบบทั่วไปทันที ที่ถ้าต้องการย้าย n แผ่นจากเสาดั้งเดิมต้นไปยังเสาปลายทาง โดยใช้อีกเสาหนึ่งเป็นตัวช่วย เราต้องทำ 3 ขั้นตอน

ขั้นตอนวิธีแบบ recursion ของ Tower of Hanoi

เพื่อย้าย n แผ่นจากเสา X ไปเสา Y โดยใช้เสา Z เป็นตัวช่วย:

1. ย้าย $n - 1$ แผ่นบนสุดจาก X ไป Z
2. ย้ายแผ่นใหญ่สุดจาก X ไป Y
3. ย้าย $n - 1$ แผ่นจาก Z ไป Y

ส่วนกรณีฐานคือ $n = 1$ เพราะถ้ามีเพียงแผ่นเดียว เราเพียงย้ายมันตรงจากต้นทางไปปลายทางได้ทันที

5.3.4 จำนวนครั้งที่ต้องย้าย

ให้ $T(n)$ แทนจำนวนครั้งน้อยที่สุดที่ต้องใช้ในการย้าย Tower of Hanoi n แผ่น จากขั้นตอนวิธีข้างต้น เราจะได้ว่า

$$T(1) = 1,$$

และสำหรับทุก $n \geq 2$,

$$T(n) = T(n-1) + 1 + T(n-1) = 2T(n-1) + 1.$$

นี่เป็นความสัมพันธ์เวียนเกิดที่ได้จากการวิเคราะห์ปัญหาโดยตรง: ย้าย $n - 1$ แผ่นครั้งหนึ่ง, ย้ายแผ่นใหญ่สุดอีกหนึ่งครั้ง, แล้วจึงย้าย $n - 1$ แผ่นอีกครั้ง

Example 5.3.1. จากสมการ

$$T(1) = 1, \quad T(n) = 2T(n-1) + 1$$

จะได้ว่า

$$T(2) = 2T(1) + 1 = 3,$$

$$T(3) = 2T(2) + 1 = 7,$$

$$T(4) = 2T(3) + 1 = 15.$$

ในบทนี้ เราย้ำเน้นที่การ *ตั้งสมการเวียนเกิด* จากโครงสร้างของปัญหาเป็นหลัก ส่วนการหาสูตรปิด เช่น

$$T(n) = 2^n - 1$$

และการพิสูจน์ว่าสูตรดังกล่าวถูกต้อง จะเป็นหัวข้อที่สัมพันธ์กับบทอุปนัยต่อไป

5.4 อีกตัวอย่างหนึ่ง: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น

ปัญหานี้เป็นตัวอย่างที่ดีมากของการคิดว่า “เมื่อเพิ่มวัตถุใหม่เข้ามาอีกหนึ่งชิ้น มันสร้างผลกระทบเพิ่มขึ้นเท่าไร”

Example 5.4.1. ให้ $A(n)$ แทนจำนวนชิ้นมากที่สุดที่สามารถเกิดขึ้นได้เมื่อเราใช้เส้นตรง n เส้นแบ่งกระดาษ จงหาความสัมพันธ์เวียนเกิดของ $A(n)$

ถ้ายังไม่วางเส้นเลย กระดาษทั้งแผ่นเป็นเพียงชิ้นเดียว ดังนั้น

$$A(0) = 1.$$

ทีนี้สมมติว่าเราได้วางเส้นไว้แล้ว $n - 1$ เส้น และแบ่งกระดาษได้มากที่สุดเท่าที่จะทำได้ คำถามคือ เมื่อเติมเส้นที่ n ลงไปอีกหนึ่งเส้น จะเพิ่มจำนวนชิ้นได้มากที่สุดกี่ชิ้น

เพื่อให้ได้มากที่สุด เส้นที่ n ควรตัดกับเส้นเดิมทุกเส้น และไม่ควรผ่านจุดตัดเดิมใด ๆ ดังนั้นบนเส้นใหม่จะมีจุดตัด $n - 1$ จุด ซึ่งแบ่งเส้นใหม่ออกเป็น n ท่อนย่อย

แต่ละท่อนย่อยอยู่ในชิ้นเดิมชิ้นหนึ่งของกระดาษ และเมื่อเราใช้ท่อนย่อยนั้นตัดผ่านชิ้นเดิม มันจะเพิ่มจำนวนชิ้นขึ้นอีกหนึ่งชิ้น ดังนั้นเส้นที่ n จะเพิ่มจำนวนชิ้นได้มากที่สุด n ชิ้น

สรุปว่า

$$A(n) = A(n - 1) + n \quad \text{สำหรับทุก } n \geq 1.$$

Example 5.4.2. จาก

$$A(0) = 1, \quad A(n) = A(n - 1) + n$$

จะได้ว่า

$$A(1) = 2, \quad A(2) = 4, \quad A(3) = 7, \quad A(4) = 11.$$

จุดเด่นของปัญหานี้คือ recursion ไม่ได้มาจากการ “เรียกตัวเอง” แบบ Tower of Hanoi แต่มาจากการถามว่าการเปลี่ยนจากกรณี $n - 1$ ไปเป็นกรณี n สร้างของใหม่เพิ่มขึ้นเท่าไร

5.5 การนับจำนวนวิธีด้วยการแยกกรณี: จำนวนเซตย่อย

ให้ $S(n)$ แทนจำนวนเซตย่อยทั้งหมดของเซต

$$\{1, 2, 3, \dots, n\}.$$

เราต้องการหาความสัมพันธ์เวียนเกิดของ $S(n)$

เริ่มจากกรณีฐานก่อน ถ้า $n = 0$ เรากำลังพูดถึงเซตว่าง ซึ่งมีเซตย่อยเพียงเซตเดียว คือเซตว่างเอง ดังนั้น

$$S(0) = 1.$$

ทีนี้ให้ $n \geq 1$ พิจารณาเซตย่อยใด ๆ ของ $\{1, 2, \dots, n\}$ เซตย่อยนั้นจะอยู่ในหนึ่งในสองกรณี และไม่มีกรณีอื่นนอกจากนั้น

1. ไม่มีสมาชิก n
2. มีสมาชิก n

ถ้าไม่มีก็เทียบได้พอดีกับเซตย่อยของ $\{1, 2, \dots, n-1\}$

ถ้ามี n อยู่ด้วย เมื่อเอา n ออก จะเหลือเซตย่อยหนึ่งของ $\{1, 2, \dots, n-1\}$

ดังนั้นจำนวนเซตย่อยในสองกรณีนี้เท่ากันทั้งคู่ คือเท่ากับ $S(n-1)$ จึงได้ว่า

$$S(n) = S(n-1) + S(n-1) = 2S(n-1) \quad \text{สำหรับทุก } n \geq 1.$$

Example 5.5.1. จาก

$$S(0) = 1, \quad S(n) = 2S(n-1)$$

จะได้ว่า

$$S(1) = 2, \quad S(2) = 4, \quad S(3) = 8, \quad S(4) = 16.$$

นี่เป็นอีกตัวอย่างหนึ่งที่มีความสำคัญมาก เพราะรูปแบบการนับแบบ “แยกเป็นกรณีที่มีวัตถุชิ้นสุดท้าย” กับ “ไม่มีวัตถุชิ้นสุดท้าย” จะกลับมาปรากฏซ้ำอีกหลายครั้งในคณิตศาสตร์ดิสครีต

5.6 สรุปภาพรวมของการตั้งความสัมพันธ์เวียนเกิด

จากตัวอย่างทั้งหมดในบทนี้ เราจะเห็นว่าแม้โจทย์จะหน้าตาแตกต่างกันมาก แต่รูปแบบการคิดมีแกนร่วมกันอยู่

1. ระบุปริมาณที่ต้องการหา
เช่น $a(n)$, $Fib(n)$, $T(n)$, $A(n)$, $S(n)$
2. ถามว่ากรณีขนาด n สัมพันธ์กับกรณีขนาดเล็กกว่าอย่างไร
บางครั้งคือ $n-1$, บางครั้งคือ $n-2$, และบางครั้งต้องใช้มากกว่าหนึ่งกรณี

3. ระบุกรณีฐาน

เพื่อให้การนิยามเริ่มต้นได้จริง

4. ตรวจสอบว่าความสัมพันธ์มีเหตุผลเชิงโครงสร้าง

กล่าวคือ สูตรไม่ได้เดามาลอย ๆ แต่สะท้อนการประกอบคำตอบจากปัญหาย่อยจริง

Pattern Summary

การตั้ง recursion ที่ดีมักตอบคำถามสี่ข้อให้ได้:

1. เรากำลังหาค่าอะไร
2. ค่าของขนาด n สร้างจากขนาดเล็กลงอย่างไร
3. ต้องมี base case อะไรบ้าง
4. เหตุผลของสูตรนี้มาจากโครงสร้างของปัญหาใด

5.7 ข้อควรระวัง

แม้แนวคิดของ recursion จะดูตรงไปตรงมาในตอนแรก คือ “นิยามปัญหาปัจจุบันจากปัญหาที่เล็กกว่า” แต่เมื่อเริ่มลงมือเขียนจริง ผู้เรียนมักพลาดอยู่ไม่กี่แบบเดิมซ้ำ ๆ และความผิดพลาดเหล่านี้สำคัญกว่าที่เห็น เพราะถ้าพลาดตั้งแต่ระดับนิยาม สิ่งตามมาทั้งหมด ไม่ว่าจะเป็นการคำนวณ การเขียนโปรแกรม หรือการพิสูจน์ความถูกต้องของ recursion นั้น ก็จะไม่มั่นคงไปด้วย

สิ่งที่ควรจำไว้เสมอคือ การตั้ง recursion ไม่ใช่เพียงการเขียนสมการที่มีตัวมันเองอยู่ทางขวา แต่ต้องเป็นการนิยามที่ให้ความหมายได้จริง กล่าวคือ ต้องเริ่มคำนวณได้ ต้องหยุดได้ และต้องอธิบายได้ว่าทำไมสูตรนั้นจึงสะท้อนโครงสร้างของปัญหาจริง หัวข้อนี้จึงไม่ได้มีไว้เพียงเตือนว่า “อย่าพลาด” แต่มีไว้ช่วยให้เราทราบว่าตรวจสอบ recursion ของตนเองอย่างไรตั้งแต่ก่อนใช้งานจริง

5.7.1 มี recursive step แต่ไม่มี base case

นี่เป็นข้อผิดพลาดพื้นฐานที่สุด ถ้ามีเพียงสูตรเช่น

$$F(n) = F(n-1) + 3$$

แต่ไม่บอกว่าเริ่มต้นที่ค่าใด สูตรนี้ยังไม่สามารถคำนวณอะไรได้จริง เพราะเมื่อถามหาค่า $F(5)$ เราต้องย้อนกลับไปที่ $F(4)$ แล้วต้องหา $F(3)$ $F(2)$ $F(1)$ และสุดท้ายก็ยังไม่รู้ว่าจะหยุดที่ตรงไหน

กล่าวอีกแบบหนึ่งคือ recursive step เพียงอย่างเดียวบอกได้แค่ว่า ถ้าเรารู้คำตอบของกรณีก่อนหน้าแล้ว จะสร้างคำตอบของกรณีปัจจุบันอย่างไร แต่มันยังไม่ได้บอกว่า เรารู้คำตอบเริ่มต้นมาจากที่ไหน ดังนั้น recursion ที่

ไม่มี base case จึงเหมือนบันไดที่บอกวิธีขึ้นขั้นถัดไปได้ แต่ไม่มีขั้นแรกให้ยืน

Example 5.7.1. ถ้ากำหนดเพียง

$$F(n) = F(n - 1) + 3$$

เรายังไม่อาจหาค่า $F(1)$ ได้ แต่ถ้ากำหนดเพิ่มว่า

$$F(0) = 2,$$

แล้ว recursion นี้จึงเริ่มทำงานได้จริง เพราะจะได้

$$F(1) = 5, \quad F(2) = 8, \quad F(3) = 11, \dots$$

สิ่งที่ควรถามตัวเองทุกครั้งคือ

ถ้าฉันไล่ recursion ย้อนลงไปเรื่อย ๆ จะมีจุดที่หยุดได้จริงหรือไม่

5.7.2 มี base case แต่ยังไม่เพียงพอ

บางครั้งผู้เรียนรู้แล้วว่าต้องมี base case แต่ยังไม่พอสำหรับสูตรที่ตั้งไว้ ข้อผิดพลาดนี้พบบ่อยมากในลำดับที่อ้างถึงหลายพจน์ก่อนหน้านี้

ตัวอย่างเช่น ถ้านิยามว่า

$$G(n) = G(n - 1) + G(n - 2) \quad \text{สำหรับทุก } n \geq 2$$

แล้วให้เพียงค่าเดียว เช่น

$$G(0) = 1$$

ก็ยังไม่พอ เพราะเมื่อจะหาค่า $G(2)$ เราต้องใช้ทั้ง $G(1)$ และ $G(0)$ แต่ค่า $G(1)$ ยังไม่ถูกกำหนด

หลักคิดง่าย ๆ คือ ถ้า recursive step อ้างย้อนหลังหลายตำแหน่ง base case ก็ต้องมากพอที่จะ ``รองรับการเริ่มต้น" ของสูตรนั้นด้วย

Example 5.7.2. ลำดับฟีโบนัคชีต้องมีฐานสองค่า เช่น

$$Fib(0) = 1, \quad Fib(1) = 1,$$

เพราะ recursive step ใช้ทั้ง $Fib(n - 1)$ และ $Fib(n - 2)$

ดังนั้นเวลาเห็น recursion ที่อ้างถึง $n - 1, n - 2, \dots, n - k$ ควรตรวจทันทีว่า เรามีค่าฐานเพียงพอจนเริ่มคำนวณพจน์แรกที่สุดที่สูตรนี้ต้องการหรือยัง

5.7.3 ปัญหาไม่เล็กลงจริง

แก่นของ recursion คือการลดปัญหาไปสู่กรณีที่เล็กกว่า ดังนั้นถ้านิยามยังย้อนกลับไปหาปัญหาขนาดเดิม หรือขนาดไม่ได้ลดลงอย่างแน่นอน recursion นั้นก็ไม่มีเหตุผลที่จะหยุด

ตัวอย่างเช่น ถ้าเขียนว่า

$$H(n) = H(n) + 1$$

หรือเขียนว่า

$$T(n) = T(n + 1) - 2$$

จะเห็นว่าปัญหาไม่ได้เล็กลงเลย กรณีแรกวนอยู่กับตัวเอง ส่วนกรณีหลังยังพาไปสู่ input ที่ใหญ่ขึ้น

บางครั้งความผิดพลาดนี้อาจไม่ชัดเจนขนาดนั้น เช่นในโจทย์เกี่ยวกับโครงสร้างข้อมูล ผู้เรียนอาจเขียนว่า ``แก้ปัญหาลิสต์ปัจจุบันโดยเรียกฟังก์ชันเดิมกับลิสต์เดิมอีกครั้ง" ซึ่งในภาษาธรรมชาติฟังดูเหมือนทำงาน แต่ในเชิง recursion แล้วไม่มีอะไรลดลงเลย

สิ่งที่ควรถามคือ

มาตรวัดความเล็กลงของปัญหาคืออะไร

สำหรับบางโจทย์ มาตรวัดคือจำนวนเต็ม n สำหรับบางโจทย์คือจำนวนสมาชิกในลิสต์ สำหรับบางโจทย์คือความสูงของต้นไม้ แต่ไม่ว่าจะใช้มาตรวัดแบบใด ต้องลดลงอย่างชัดเจนทุกครั้งที่เรียกซ้ำ

5.7.4 ลดขนาดลงแล้ว แต่ไม่รับประกันว่าจะไปถึงฐาน

ความผิดพลาดอีกแบบหนึ่งที่ละเอียดขึ้นคือ ผู้เรียนเขียนสูตรที่ ``ดูเหมือน" ลดขนาดลง แต่ไม่สามารถรับประกันได้ว่าท้ายที่สุดจะลงถึง base case จริง

ตัวอย่างเช่น ถ้ากำหนดว่า

$$F(n) = F(n - 2) + 1 \quad \text{สำหรับทุก } n \geq 1$$

และให้ฐานเพียง

$$F(0) = 0$$

นิยามนี้อาจใช้ได้กับจำนวนคู่บางค่า แต่ถ้าถามหา $F(5)$ เราจะได้

$$F(5) \rightarrow F(3) \rightarrow F(1) \rightarrow F(-1),$$

ซึ่งหลุดออกจากโดเมนที่ตั้งใจไว้

ปัญหาจึงไม่ใช่แค่ “เล็กลง” แต่ต้องเล็กลงในลักษณะที่ *เดินไปถึงฐานได้สำหรับทุก input ในโดเมนที่ประกาศไว้* นี่เป็นเหตุผลว่าทำไมเวลาเขียน recursion เราต้องระบุช่วงของ n และระบุฐานให้สอดคล้องกันด้วย

5.7.5 ไม่ระบุโดเมนหรือเงื่อนไขของ input ให้ชัด

recursion ที่ดีต้องบอกให้ชัดว่ากำลังนิยามบน input ประเภทใด เช่น จำนวนนับที่รวมศูนย์หรือไม่ จำนวนเต็มบวก ลิสต์ที่ไม่ว่าง หรือต้นไม้ไบนารีชนิดใด

ถ้าไม่ระบุโดเมนให้ชัด สูตรเดียวกันอาจกลายเป็นข้อความกำกวมทันที เช่น

$$n! = n(n-1)!$$

ประโยคนี้อาจใช้กับ n ใดบ้าง ถ้า n เป็นจำนวนเต็มลบ เราจะไม่สามารถไล่ลงไปถึงฐานแบบเดียวกันได้ ดังนั้นการนิยามแฟกทอเรียลจึงต้องระบุว่า กำลังทำงานบน $\mathbb{N}$ และมีฐานเช่น $0! = 1$

ในโจทย์เกี่ยวกับโครงสร้างข้อมูลก็เช่นเดียวกัน ถ้าเขียนนิยามของ $\text{head}(L)$ หรือ $\text{tail}(L)$ โดยไม่บอกว่า L ต้องเป็นลิสต์ไม่ว่าง นิยามนั้นก็ยังไม่สมบูรณ์

ดังนั้นก่อนตั้ง recursion ควรถามก่อนเสมอว่า

input ที่อนุญาตคืออะไร และ recursion นี้ถูกนิยามสำหรับทุกกรณีในโดเมนนั้นหรือไม่

5.7.6 สูตรไม่ครอบคลุมทุกกรณีที่ควรเกิดขึ้น

บาง recursion มีฐานแล้ว มีการลดขนาดแล้ว แต่ยังไม่ครอบคลุมทุกกรณีที่อาจเกิดขึ้นในปัญหา

ตัวอย่างเช่น ถ้าเราต้องการนิยามฟังก์ชันบนลิสต์ แล้วเขียนเพียงว่า

ถ้า L ไม่ว่าง ให้ตอบจาก $\text{tail}(L)$

แต่ไม่เคยบอกว่าถ้า L ว่างจะอย่างไร นิยามนั้นก็ยังไม่สมบูรณ์ หรือถ้าปัญหามีได้หลายโครงสร้างย่อย แต่เราให้ recursive rule ไว้เพียงบางแบบ ก็จะทำให้เกิด input บางตัวที่ไม่รู้จะใช้กฎข้อใด

ในเชิงการเขียนโปรแกรม นี่เทียบได้กับการเขียนฟังก์ชันที่บางกรณีไม่มีทางออก หรือไม่มี branch ที่รองรับ แต่ในเชิงคณิตศาสตร์ มันหมายถึงนิยามยังไม่ครบ

ดังนั้น recursion ที่ดีไม่เพียงต้องมีกรณีฐาน แต่ต้องมี การแบ่งกรณีที่ครอบคลุมทุก input ที่เป็นไปได้ ด้วย

5.7.7 สูตรไม่สะท้อนโครงสร้างจริงของปัญหา

บางครั้งผู้เรียนเห็นคำตอบสองสามค่าตัวแรก แล้วเดาสมการขึ้นมาได้ เช่นเห็นลำดับ

$$1, 3, 5, 7, \dots$$

แล้วเขียนว่า

$$a(n) = a(n-1) + 2$$

ซึ่งในกรณีนี้อาจถูกต้อง แต่ปัญหาคือผู้เรียนยังไม่ได้อธิบายว่า ทำไมคำตอบของกรณี n จึงควรได้มาจากกรณี $n-1$ บวก 2

ในรายวิชานี้ เราไม่ได้สนใจ recursion ในฐานะการเดารูปแบบของตัวเลขเพียงอย่างเดียว แต่สนใจ recursion ที่มาจาก เหตุผลของปัญหา เช่น ใน Tower of Hanoi สมการ

$$T(n) = 2T(n-1) + 1$$

ไม่ได้ถูกเดาจากค่าของ $T(1), T(2), T(3)$ เท่านั้น แต่เกิดจากการวิเคราะห์ขั้นตอนการย้ายดิสก์จริง ๆ ว่าต้องทำสามช่วง คือย้าย $n-1$ แผ่นออกก่อน ย้ายแผ่นใหญ่สุดหนึ่งครั้ง แล้วจึงย้าย $n-1$ แผ่นกลับเข้าไปใหม่

ความต่างระหว่าง “เดาได้” กับ “อธิบายได้” สำคัญมาก เพราะ recursion ที่เดาได้อาจบังเอิญตรงกับสองสามค่าต้น แต่ใช้ไม่ได้กับทั้งปัญหา ในขณะที่ recursion ที่มาจากโครงสร้างจริง สามารถนำไปพิสูจน์ต่อได้และเชื่อถือได้มากกว่า

5.7.8 สืบสนระหว่าง recursion กับสูตรปิด

ผู้เรียนบางคนเมื่อเห็นลำดับหรือปัญหาแบบเวียนเกิด จะรีบพยายามหาสูตรปิดทันที ทั้งที่โจทย์ในช่วงต้นอาจยังต้องการเพียงการตั้ง recursion ให้ถูกต้อง

ตัวอย่างเช่น จาก

$$A(0) = 1, \quad A(n) = A(n-1) + n$$

ผู้เรียนบางคนอาจรีบเดาว่า

$$A(n) = 1 + \frac{n(n+1)}{2}$$

ซึ่งอาจเป็นสูตรที่ถูก แต่ถ้าข้ามขั้น reasoning ที่นำไปสู่ recursion เราจะพลาดหัวใจของบทนี้ไป เพราะ recursion ไม่ได้มีไว้เพียงเพื่อ ``หาคำตอบให้เร็ว" แต่มันเป็นวิธีจัดโครงสร้างความคิดของปัญหา

ดังนั้นในช่วงแรก เป้าหมายหลักไม่ใช่การหาสูตรปิดเสมอไป แต่คือการตอบให้ได้ว่า

คำตอบของกรณีปัจจุบันประกอบจากกรณีเล็กกว่าอย่างไร

5.7.9 คิดว่าเขียน recursion ได้แล้ว แปลว่าถูกต้องแล้ว

นี่เป็นข้อควรระวังในเชิงทัศนคติ ผู้เรียนจำนวนไม่น้อยรู้สึกว้าว ถ้าเขียน base case และ recursive step ครบแล้ว งานก็จบ แต่ในความจริง เรายังมีคำถามสำคัญอีกอย่างคือ *recursion นี้ถูกต้องหรือไม่*

กล่าวคือ สูตรที่เขียนขึ้นมานั้นให้คำตอบตรงกับปัญหาจริงทุกกรณีหรือไม่ และทำไมเราจึงมั่นใจเช่นนั้น คำถามนี้จะนำเราไปสู่บทถัดไปที่ว่าด้วยการพิสูจน์ความถูกต้องของนิยามและอัลกอริทึมแบบเวียนเกิด ซึ่งมักใช้การให้เหตุผลแบบอุปนัยเข้ามาช่วย

ดังนั้นการตั้ง recursion ให้ได้ เป็นเพียงครั้งแรกของงาน อีกครั้งหนึ่งคือการพิสูจน์ว่า recursion นั้นสมเหตุสมผลจริง

Checklist ก่อนยอมรับ recursion ใด ๆ

ก่อนจะสรุปว่า recursion ที่ตั้งไว้ใช้ได้แล้ว ควรถามตัวเองอย่างน้อยหกข้อดังนี้

1. มี base case ครบและเพียงพอหรือยัง
2. input ทุกตัวในโดเมนจะไล่ลงไปถึง base case ได้จริงหรือไม่
3. recursive call ทำให้ปัญหาเล็กลงอย่างชัดเจนหรือไม่
4. นิยามครอบคลุมทุกกรณีที่เป็นไปได้หรือยัง
5. สูตรนี้มาจากโครงสร้างของปัญหาจริงหรือเพียงเดาจาก pattern
6. ถ้าจะต้องพิสูจน์ความถูกต้องของสูตรนี้ต่อ ฉันพออธิบายเหตุผลของมันได้หรือไม่

ถ้าผ่านคำถามเหล่านี้ได้ recursion ที่ได้มักจะไม่เพียง ``เขียนสวย" แต่ยังเป็น recursion ที่พร้อมใช้งาน พร้อมวิเคราะห์ และพร้อมพิสูจน์ในขั้นถัดไปของรายวิชา

แบบฝึกหัดท้ายบท

Exercise 5.7.3. กำหนดลำดับโดย

$$b(0) = 2, \quad b(n) = b(n-1) + 5 \text{ สำหรับทุก } n \geq 1.$$

จงหาค่า $b(1), b(2), b(3), b(6)$

Exercise 5.7.4. กำหนดลำดับโดย

$$G(0) = 2, \quad G(1) = 3, \quad G(n) = G(n-1) + G(n-2) \text{ สำหรับทุก } n \geq 2.$$

จงหาค่า $G(2), G(3), G(4), G(5)$

Exercise 5.7.5. ให้ $T(n)$ แทนจำนวนครั้งน้อยที่สุดในการย้าย Tower of Hanoi n แผ่น จงอธิบายด้วยภาษาของตนเองว่าทำไม

$$T(n) = 2T(n-1) + 1$$

จึงเป็นสมการที่สมเหตุสมผล

Exercise 5.7.6. ให้ $A(n)$ แทนจำนวนชิ้นมากที่สุดที่ได้จากการใช้เส้นตรง n เส้นแบ่งระนาบ
จงหาค่า $A(1), A(2), A(3), A(4), A(5)$ เมื่อกำหนดว่า

$$A(0) = 1, \quad A(n) = A(n-1) + n.$$

Exercise 5.7.7. ให้ $S(n)$ แทนจำนวนเซตย่อยของ $\{1, 2, \dots, n\}$ จงอธิบายว่าทำไมเซตย่อยทุกเซตจึงถูกแบ่งได้เป็นสองชนิด คือ เซตที่มี n และเซตที่ไม่มี n แล้วใช้แนวคิดนี้เพื่อเสนอ recursion ของ $S(n)$

Exercise 5.7.8. ทุกคนทราบดีว่านิยามของการคำนวณ factorial คือ $n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$ หรือถ้ากล่าวเป็นภาษาชาวบ้านคือ "ผลคูณของจำนวนนับตั้งแต่ 1 จนถึง n " เช่นตอนเป็นนักเรียนมัธยมถ้าอยากคำนวณ $5!$ เราก็จะคำนวณ $1 \times 2 \times 3 \times 4 \times 5$ ได้ผลลัพธ์เป็น 120

จงเขียนนิยามของการคำนวณ factorial ในรูปแบบ recursion โดยสมมติให้สัญลักษณ์ $Fac(n)$ แทนค่า factorial ของ n พร้อมทั้งระบุผลลัพธ์ขั้นฐานเมื่อ $n = 0$

นอกจากนี้ จงเขียน code (Pseudocode หรือ syntax ภาษาที่ถนัดก็ได้) ที่แสดงการคำนวณ factorial แบบ iteration และ recursion

Exercise 5.7.9. นักศึกษากำลังพิจารณาโจทย์ปัญหาการหาว่าจำนวนเต็มที่กำหนดมาให้มีอยู่ใน array ที่กำหนดมาให้หรือไม่ ถ้ามีให้คืนค่า index แรกสุดที่เจอ แต่ถ้าไม่มีให้คืน -1 ตัวอย่างเช่น ให้ $A = [1, -5, 0, 3, -5]$ และถ้าค้นหา -5 จะต้องได้ค่า return ออกมาเป็น 1 หรือถ้าค้นหา 7 จะต้องได้ค่า return ออกมาเป็น -1

แน่นอนว่าวิธีการหนึ่งที่นักศึกษาทุกคนต้องเคยทำมาแล้วคือการวนลูปตาม index แต่ในคราวนี้ให้นักศึกษาอธิบายวิธีการแก้ปัญหานี้ด้วยแนวคิด recursion โดยสมมติว่านักศึกษากำลังออกแบบวิธีคิดของฟังก์ชัน FindFirst(arr,x) โดยที่ arr เป็น array และ x คือสมาชิกที่อยากค้นหาใน arr

Exercise 5.7.10. ในบริบทเดียวกันกับข้อ 2 แต่คราวนี้อยากให้ทำ FindLast(arr,x) กล่าวคือค้นหา index สุดท้ายที่ x ปรากฏใน arr เช่น FindLast([1, -5, 0, 3, -5], -5) ต้องได้ 4

(หมายเหตุ: กันคนหัวหมอ - ห้าม reverse แล้วใช้ FindFirst)

Exercise 5.7.11. สมมตินักศึกษาอยากนิยามเซตของ bit strings (ตัวอักษรที่สร้างมาจากเลข 0 หรือ 1) ที่มีคุณสมบัติว่าห้ามมีเลข 0 อยู่ติดกันเกิน 2 ตัว กล่าวคืออยากได้เซต

$$A = \{0, 1, 00, 01, 10, 11, 001, 010, 011, 100, 101, 110, 111, 0010, \dots\}$$

กล่าวคือสมาชิกที่หายไปจากเซตที่เขียนไว้คือ 000, 0000, 0001 เป็นต้น

แต่แน่นอนการจะเขียนโปรแกรมให้คอมพิวเตอร์ช่วยสร้าง bit strings นี้ด้วย for loop อาจจะต้องมีการควบคุม flow ที่มีเงื่อนไขจุกจิก (แต่ยังทำได้จริงอยู่) คราวนี้เลยอยากออกแบบวิธีสร้างแบบ recursion โดยสมมติว่าจะแบ่งการนิยามออกเป็นตามความยาว $A = A_1 \cup A_2 \cup A_3 \cup \dots$ โดยที่ A_n คือเซตของ bit strings ความยาว n ที่สอดคล้องเงื่อนไขที่เรากำลังสนใจ กล่าวคือ

$$A_1 = \{0, 1\}, A_2 = \{00, 01, 10, 11\}, A_3 = \{001, 010, 011, 100, 101, 110, 111\}$$

$$A_4 = \{0010, 0011, 0100, 0101, 0110, 0111, 1001, 1010, 1001, 1100, 1101, 1110, 1111\}$$

จงนิยาม A_n โดย recursion (คำใบ้: ต้องใช้ A_{n-2} มาช่วยนิยาม และนิยาม A_1 และ A_2 เป็นพื้นฐาน)

บทที่ 6

Mathematical Induction

ในบทก่อนหน้า เราใช้แนวคิดแบบเวียนเกิดเพื่อ นิยาม ลำดับ นิยามเซตบางชนิด และอธิบายวิธีคิดที่สร้างคำตอบจากปัญหาที่ย่อยลง แต่เมื่อได้สูตรหรือได้วัตถุที่นิยามแบบ recursion มาแล้ว ยังมีคำถามสำคัญอีกข้อหนึ่งเสมอคือ

เราจะพิสูจน์ได้อย่างไรว่าสิ่งที่นิยามไว้นั้นมีคุณสมบัติที่ต้องการจริงสำหรับทุกขนาด?

ตัวอย่างเช่น ถ้าเราคาดเดาว่า

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

หรือถ้านิยามลำดับด้วยความสัมพันธ์เวียนเกิดแล้วคาดเดาสูตรปิดบางอย่างขึ้นมา เราจะไม่พอใจเพียงการลองแทนค่า $n = 1, 2, 3, 4$ แล้วเห็นว่า “น่าจะจริง” เพราะในคณิตศาสตร์ เราต้องการเหตุผลที่ครอบคลุมทุกค่าที่เป็นไปได้ ไม่ใช่เพียงตัวอย่างไม่กี่ค่า

เครื่องมือพื้นฐานที่สุดสำหรับคำถามลักษณะนี้คือ **อุปนัยทางคณิตศาสตร์** ซึ่งเหมาะสมอย่างยิ่งกับข้อความที่เรียงตามขั้นของจำนวนนับ เช่นข้อความตระกูล

$$P(0), P(1), P(2), \dots$$

โดยที่ $P(n)$ เป็นประโยคที่ขึ้นกับดัชนี n

ภาพเปรียบเทียบที่ใช้กันบ่อยคือแถวโดมิโน: ถ้าเราทำให้ตัวแรกเริ่มล้มได้ และถ้าทุกครั้งที่ตัวที่ k ล้มแล้วตัวที่ $k+1$ จะล้มตามเสมอ เราก็สรุปได้ว่าโดมิโนทุกตัวในแถวจะล้มทั้งหมด ในบริบทของคณิตศาสตร์ สิ่งนี้ “ล้ม” ไม่ใช่แผ่นโดมิโน แต่คือความจริงของข้อความ $P(n)$

6.1 Ordinary Induction

6.1.1 หลักการพื้นฐาน

Theorem 6.1.1 (Principle of Mathematical Induction). ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนนับตั้งแต่ค่าเริ่มต้น n_0 เป็นต้นไป ถ้าเราพิสูจน์ได้ว่า

1. **ขั้นฐาน** (base step): $P(n_0)$ เป็นจริง
2. **ขั้นอุปนัย** (inductive step): สำหรับทุก $k \geq n_0$, ถ้า $P(k)$ เป็นจริง แล้ว $P(k + 1)$ เป็นจริง

จะสรุปได้ว่า

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

แก่นของหลักการนี้อยู่ตรงความคิดว่า เราไม่จำเป็นต้องพิสูจน์ทีละค่า $n = 0, 1, 2, 3, \dots$ แบบไม่สิ้นสุด แต่เราพิสูจน์เพียงสองอย่างแทน: เริ่มได้ถูกต้องหนึ่งครั้ง และเดินต่อจากขั้นหนึ่งไปขั้นถัดไปได้เสมอ เมื่อสองอย่างนี้ครบ จึงสรุปได้ว่าทุกขั้นเป็นจริงทั้งหมด

ในทางปฏิบัติ เรามักเขียนการพิสูจน์แบบอุปนัยในโครงมาตรฐานดังนี้

โครงสร้างมาตรฐานของการพิสูจน์แบบอุปนัย

ให้ $P(n)$ แทนข้อความที่ต้องการพิสูจน์

1. **กำหนดข้อความ** ให้ชัดว่า $P(n)$ คืออะไร
2. **ขั้นฐาน** พิสูจน์ว่า $P(n_0)$ เป็นจริง
3. **ขั้นอุปนัย** ให้ $k \geq n_0$ เป็นค่าทั่วไป และสมมติว่า $P(k)$ เป็นจริง
4. ใช้สมมติฐานขั้นอุปนัยนั้นพิสูจน์ว่า $P(k + 1)$ เป็นจริง
5. สรุปด้วยหลักอุปนัยว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

สมมติฐานที่ว่า $P(k)$ เป็นจริง เรียกว่า **สมมติฐานขั้นอุปนัย** (induction hypothesis)

6.1.2 สิ่งที่ $P(n)$ เป็น และสิ่งที่มันไม่ใช่

จุดที่นักศึกษามักสับสนตั้งแต่ต้นคือ ไม่เห็นความต่างระหว่าง $P(n)$, $P(k)$, และ $P(k + 1)$

- $P(n)$ คือ รูปแบบของข้อความ ที่ยังเปิดตัวแปร n ไว้
- $P(k)$ คือข้อความเดียวกัน แต่แทน n ด้วยค่าทั่วไป k แล้ว
- $P(k + 1)$ คือข้อความที่อยากพิสูจน์ในขั้นถัดไป

ดังนั้น เมื่อเข้าสู่ขั้นอุปนัย เราจะไม่สามารถพูดว่า “จาก $P(k)$ แทน k เป็น $k + 1$ จึงได้ $P(k + 1)$ ” เพราะนั่น

ไม่ใช่เหตุผล แต่เป็นเพียงการเปลี่ยนสัญลักษณ์ สิ่งที่ต้องทำจริง ๆ คือ ใช้เนื้อหาของ $P(k)$ เป็นข้อมูลตั้งต้น แล้วเดินเหตุผลไปถึง $P(k + 1)$

ข้อควรจำ

อุปนัยไม่ใช่การ “เดาแพตเทิร์นแล้วเปลี่ยน k เป็น $k + 1$ ” แต่อุปนัยคือการพิสูจน์ว่า

$$\text{ถ้าข้อความจริงที่ระดับ } k \implies \text{ข้อความจริงที่ระดับ } k + 1$$

โดยมีเหตุผลรองรับจริง

6.1.3 ข้อผิดพลาดที่พบบ่อย

ก่อนจะไปดูตัวอย่าง ควรแยกข้อผิดพลาดที่พบบ่อยที่สุดออกมาก่อน เพราะในการพิสูจน์แบบอุปนัย ความผิดพลาดจำนวนมากไม่ได้เกิดจากการจัดรูปพลาตเพียงอย่างเดียว แต่เกิดจากการเข้าใจ “โครงของอุปนัย” ไม่ชัดเจน กล่าวคือยังไม่แยกให้ออกว่า อะไรคือขั้นฐาน อะไรคือสมมติฐานขั้นอุปนัย และอะไรคือสิ่งที่ต้องพิสูจน์ในขั้นถัดไป

สิ่งที่น่าระวังคือ บทพิสูจน์อุปนัยที่ผิด หลายครั้งดูคล้ายบทพิสูจน์ที่ถูกมาก เพียงแต่มีจุดเล็ก ๆ ที่หลุดไป เช่น เริ่มฐานผิดค่า สมมติเกินสิ่งที่ได้รับอนุญาต หรือใช้สมมติฐานขั้นอุปนัยในจังหวะที่ยังไม่มีสิทธิ์ใช้ ดังนั้นก่อนฝึกทำโจทย์จำนวนมาก ควรทำความเข้าใจกับข้อผิดพลาดเหล่านี้ให้ชัดเจนเสียก่อน

1. เริ่มขั้นฐานผิดค่า

โจทย์บางข้อเริ่มที่ $n \geq 1$ บางข้อเริ่มที่ $n \geq 0$ บางข้ออาจเริ่มที่ $n \geq 2$ หรือเริ่มจากค่าที่ใหญ่กว่านั้น อีก ถ้าเลือกฐานผิดตั้งแต่แรก บทพิสูจน์ทั้งหมดอาจพังทันที แม้ว่าขั้นอุปนัยจะเขียนได้ถูกต้องก็ตาม ตัวอย่างเช่น ถ้าข้อความถูกอ้างว่าเป็นจริงสำหรับทุก $n \geq 1$ แต่เราไปตรวจเพียง $n = 0$ นั้นยังไม่เพียงพอ เพราะสิ่งที่ต้องการคือการทำให้ “แถวโดมิโน” เริ่มล้มที่จุดเริ่มต้นจริงของโจทย์ ไม่ใช่ที่ค่าซึ่งอยู่นอกโดเมนที่กำหนด

ยิ่งกว่านั้น บางโจทย์มี recursive pattern ที่ก้าวทีละมากกว่าหนึ่งขั้น หรือใช้ค่าก่อนหน้ามากกว่าหนึ่งค่า ในกรณีเช่นนี้ อาจต้องตรวจฐานหลายค่า ไม่ใช่เพียงค่าเดียว ดังนั้นก่อนเริ่มพิสูจน์ คำถามแรกที่ควรถามคือ

ข้อความนี้เริ่มมีความหมายตั้งแต่ค่าใด และจำเป็นต้องตรวจฐานกี่ค่าเพื่อให้ขั้นอุปนัยเริ่มทำงานได้จริง

2. สมมติฐานขั้นอุปนัยแรงเกินจำเป็น

ถ้ากำลังใช้อุปนัยธรรมดา ควรสมมติเพียง $P(k)$ ไม่ใช่แอบสมมติ

$$P(0), P(1), \dots, P(k)$$

ทั้งหมด เพราะนั่นไม่ใช่ ordinary induction แล้ว แต่เป็น strong induction

จุดนี้สำคัญมาก เพราะในเชิงความคิด ผู้เรียนมักรู้สึกว่ “สมมติให้เยอะหน่อยน่าจะง่ายกว่า” แต่ในการพิสูจน์ เราต้องระวังว่าเรากำลังใช้อะไรเป็นเครื่องมืออยู่ ถ้าจะใช้อุปนัยธรรมดา ก็ต้องซื้อสัตย์กับรูปของมัน กล่าวคือพิสูจน์จาก $P(k)$ ไป $P(k + 1)$ ให้ได้จริง

แน่นอนว่าในหลายโจทย์ strong induction เป็นวิธีที่เหมาะสมกว่า แต่ถ้าจะใช้ ก็ต้องประกาศให้ชัดว่าเรากำลังใช้มัน ไม่ใช่เขียนพิสูจน์แบบ ordinary induction แต่แอบหยิบสมมติฐานของ strong induction มาใช้โดยไม่บอก

3. สมมติฐานขั้นอุปนัยอ่อนเกินไปสำหรับโจทย์

ข้อผิดพลาดก่อนหน้าเป็นกรณี “แรงเกินไป” แต่อีกแบบหนึ่งที่พบบ่อยไม่แพ้กันคือ “อ่อนเกินไป” กล่าวคือโจทย์บางข้อแท้จริงต้องใช้ strong induction หรืออย่างน้อยต้องมีหลายฐาน แต่ผู้เรียนพยายามใช้อุปนัยธรรมดาเพียงเพราะคุ้นมือกับรูปแบบนั้น

ตัวอย่างเช่น ถ้าข้อความที่ระดับ $k + 1$ ต้องอาศัยทั้ง $P(k)$ และ $P(k - 1)$ การสมมติเพียง $P(k)$ อาจไม่พอ หรือถ้าเป็นโจทย์เกี่ยวกับโครงสร้างที่แตกออกเป็นหลายปัญหาย่อย สมมติฐานขั้นอุปนัยแบบธรรมดาอาจไม่ครอบคลุมสิ่งที่ต้องใช้จริง

ดังนั้นเวลาพิสูจน์ไม่ไปต่อ ไม่ควรรีบสรุปว่า “จัดรูปไม่เป็น” แต่อาจต้องย้อนกลับมาตรวจว่า เราเลือกชนิดของอุปนัยเหมาะกับโครงสร้างของโจทย์แล้วหรือยัง

4. เริ่มจากสิ่งที่อยากพิสูจน์แล้วจัดรูปย้อนกลับ

นี่เป็นข้อผิดพลาดที่พบได้บ่อยมาก โดยเฉพาะเมื่อผู้เรียนมองว่าการพิสูจน์อุปนัยเป็นเพียงการจัดรูปสมการ

ในขั้นอุปนัย สิ่งที่เราารู้คือ $P(k)$ เป็นจริง และสิ่งที่ต้องการพิสูจน์คือ $P(k + 1)$ เป็นจริง ดังนั้นวิธีเขียนที่ดีควรเริ่มจากสิ่งที่รู้ว่าเป็นจริง เช่นสมมติฐานขั้นอุปนัย นิยาม หรือข้อเท็จจริงที่พิสูจน์แล้ว แล้วค่อยเดินไปหาสิ่งที่ต้องการ

ถ้าเริ่มจากด้านที่อยากพิสูจน์ทันที แล้วค่อยจัดรูปย้อนกลับไปหาสิ่งที่รู้อยู่แล้ว งานเขียนนั้นอาจมีประโยชน์ในฐานะ การหาไอเดีย แต่ยังไม่ใช่ข้อพิสูจน์ที่สมบูรณ์โดยอัตโนมัติ เว้นแต่จะอธิบายให้ชัดว่าการจัดรูปแต่ละขั้นเป็นสมมูลสองทาง หรือจะเขียนใหม่ให้อยู่ในทิศทางจากสิ่งที่รู้ไปหาสิ่งที่ต้องการ

กล่าวอีกแบบหนึ่งคือ การคิดย้อนใช้ช่วยค้นทางได้ แต่การเขียนพิสูจน์ควรพาผู้อ่านเดินไปข้างหน้า

5. สิมว่าค่า k เป็นค่าทั่วไป

ในขั้นอุปนัย k ไม่ใช่ค่าคงที่เฉพาะตัว และไม่ใช่ตัวเลขที่เราเลือกมาแบบกรณีพิเศษ แต่มันเป็นตัวแทนของทุกค่าที่อยู่ในโดเมนที่โจทย์กำหนด

ดังนั้นเมื่อเขียนว่า

ให้ $k \geq n_0$ และสมมติว่า $P(k)$ เป็นจริง

เรากำลังพูดถึง “ค่าใด ๆ” ที่อยู่ในช่วงนั้น ไม่ใช่แค่ค่าหนึ่งค่าที่หยิบมาเฉพาะกิจ

ความสำคัญของจุดนี้คือ ข้อสรุป $P(k + 1)$ ที่เราได้ ต้องเป็นผลที่ใช้ได้กับทุก k เช่นกัน ถ้าระหว่างพิสูจน์เราเผลอใช้สมบัติที่จริงเฉพาะบางค่า เช่นหารด้วย $k - 1$ โดยไม่ได้ตรวจว่า $k \neq 1$ หรือใช้ข้ออ้างที่ใช้ได้เฉพาะเมื่อ k เป็นจำนวนคู่ พิสูจน์นั้นก็จะไม่ครอบคลุมทุกกรณีตามทีอุปนัยต้องการ

6. ไม่เขียนให้ชัดว่า $P(n)$ คืออะไร

ผู้เริ่มต้นจำนวนมากรีบลงมือพิสูจน์ทันที โดยไม่กำหนดให้ชัดเจนก่อนว่า ข้อความ $P(n)$ ที่กำลังจะใช้อุปนัยนั้นคืออะไร ผลคือระหว่างพิสูจน์เกิดความสับสนระหว่าง $P(n)$, $P(k)$, และ $P(k + 1)$ จนบางครั้งใช้สมมติฐานผิดตัวหรือเขียนเป้าหมายผิดรูปไปเลย

นิสัยที่ติดมากคือ ก่อนเริ่มพิสูจน์ ควรประกาศให้ชัดเจนว่า

$P(n)$: ข้อความที่ต้องการพิสูจน์สำหรับดัชนี n

เมื่อทำเช่นนี้แล้ว ขั้นตอน ขั้นตอน และเป้าหมายของขั้นตอนจะชัดเจนขึ้นทันที
ข้อนี้ดูเหมือนเป็นเพียงเรื่องรูปแบบการเขียน แต่จริง ๆ แล้วช่วยลดความผิดพลาดเชิงตรรกะได้มาก เพราะทำให้เราเห็นชัดว่า สมมติฐานคือข้อความใดแน่ และสิ่งที่ต้องพิสูจน์ในขั้นถัดไปมีหน้าตาอย่างไรแน่

7. ใช้สมมติฐานขั้นตอนอย่างไม่ถูกต้อง

บางครั้งผู้เรียนรู้ว่าต้องใช้สมมติฐานขั้นตอน แต่ใช้ไม่ถูกต้องตำแหน่ง เช่น เขียนว่า

$$1 + 2 + \cdots + k + (k + 1) = \frac{(k + 1)(k + 2)}{2}$$

แล้วบอกว่า “จากสมมติฐานขั้นตอน” ทั้งที่สมมติฐานขั้นตอนให้ข้อมูลเพียง

$$1 + 2 + \cdots + k = \frac{k(k + 1)}{2}$$

ไม่ได้ให้สูตรของทั้ง $k + 1$ มาเลย

การใช้สมมติฐานที่ถูกต้องคือ เราต้องเริ่มจากนิพจน์ที่มีส่วน

$$1 + 2 + \cdots + k$$

ซ่อนอยู่จริง แล้วจึงแทนส่วนนั้นด้วยสิ่งที่สมมติฐานบอก จากนั้นค่อยจัดรูปต่อ

ดังนั้นทุกครั้งที่เราเขียนว่า “โดยสมมติฐานขั้นตอน” ควรถามตัวเองว่า

สิ่งที่ฉันกำลังแทนหรือกำลังสรุปอยู่นี้ ตรงกับเนื้อหาของ $P(k)$ จริงหรือไม่

8. พิสูจน์จาก $P(k)$ ไปได้เพียงกรณีพิเศษของ $P(k + 1)$

ข้อผิดพลาดนี้เกิดเมื่อข้อความ $P(n)$ มีตัวแปรอื่นร่วมอยู่ด้วย เช่น มีทั้ง n และ x หรือมีข้อความเชิงปริมาณอย่าง “สำหรับทุก x ” ผู้เรียนอาจเผลอพิสูจน์เพียงสำหรับค่าบางตัวของตัวแปรประกอบ แต่เข้าใจไปเองว่าได้ $P(k + 1)$ ครบแล้ว

ตัวอย่างเช่น ถ้า $P(n)$ เป็นข้อความว่า “สำหรับทุก x บางสมบัติเป็นจริง” ในขั้นตอน เราต้องพิสูจน์ข้อความระดับ $k + 1$ ให้ครบในความหมายของ “สำหรับทุก x ” ไม่ใช่เลือก x มาเพียงค่าหนึ่งโดยไม่มีเหตุผล

ดังนั้นเมื่อเจอหมีตัวแปรมากกว่าหนึ่งตัว ต้องระวังให้มากกว่า เราใช้อุปนัยกับตัวแปรใด และยังมีตัวแปรใดที่ต้องจัดการด้วยเหตุผลแบบ “ทั่วไป” ไม่ใช่แบบเฉพาะค่า

9. สรุปโดยไม่ปิดวงจรของอุปนัยให้ครบ

ผู้เรียนบางคนตรวจขั้นฐานแล้ว ทำขั้นอุปนัยได้แล้ว แต่จบงานเขียนทันทีโดยไม่กล่าวสรุป แม้ในเชิงสาระอาจมองออกว่าต้องการจบอย่างไร แต่ในเชิงตรรกะ ควรเขียนให้ครบว่า เมื่อขั้นฐานเป็นจริง และขั้นอุปนัยเป็นจริง จึงสรุปด้วยหลักอุปนัยได้ว่า $P(n)$ เป็นจริงสำหรับทุก n ในโดเมนที่กำหนด การสรุปนี้อาจดูเป็นเพียงประโยคสั้น ๆ แต่มีความสำคัญเพราะเป็นจุดที่บอกชัดว่า เราไม่ได้เพียงตรวจสอบสองส่วนแยกกัน แต่ได้นำสองส่วนนี้มาประกอบเป็นบทพิสูจน์แบบอุปนัยที่สมบูรณ์แล้ว

10. ตรวจสอบขั้นฐานผ่านเพียงเพราะ “น่าจะจริง”

ในโจทย์ที่สูตรดูคุ้นตา ผู้เรียนมักรีบผ่านขั้นฐานเร็วเกินไป เช่นเขียนว่า “ตรวจได้ง่าย” หรือ “เห็นชัดว่าเป็นจริง” โดยไม่แทนค่าจริง แม้หลายครั้งขั้นฐานจะง่ายจริง แต่การเขียนให้ชัดยังสำคัญมาก เพราะขั้นฐานไม่ใช่พิธีการ มันคือจุดเริ่มต้นทั้งหมดของอุปนัย ถ้าฐานไม่จริง ขั้นอุปนัยที่ดีเพียงใดก็ช่วยไม่ได้ โดยเฉพาะในโจทย์ที่สูตรซับซ้อน หรือมีหลายกรณีฐาน การตรวจอย่างละเอียดในช่วงต้นมักช่วยกันความผิดพลาดภายหลังได้มาก

สรุปอย่างย่อ ข้อผิดพลาดของการพิสูจน์อุปนัยส่วนใหญ่ไม่ได้อยู่ที่ “คำนวณไม่เก่ง” แต่อยู่ที่การอ่านโครงสร้างของพิสูจน์ไม่ชัด อุปนัยที่ดีต้องตอบคำถามให้ครบเสมอว่า

ข้อความ $P(n)$ คืออะไร
เริ่มจริงที่ค่าใด
ในขั้นอุปนัยสมมติอะไรได้บ้าง
และจากสิ่งที่สมมติอยู่นั้น จะเดินไปถึง $P(k+1)$ ได้อย่างไร

หากตรวจสอบจุดนี้ทุกครั้งก่อนส่งพิสูจน์ ความผิดพลาดส่วนใหญ่ในบทนี้จะลดลงอย่างมาก และจะทำให้การอ่านตัวอย่างถัดไปมีความหมายมากขึ้น เพราะเราจะไม่เพียงเห็นว่า “เขาทำอะไร” แต่จะเห็นด้วยว่า “ทำไมต้องทำแบบนี้”

6.2 ตัวอย่างแรก: ผลบวกของจำนวนเต็มตัวแรก

ตัวอย่างแรกของบทนี้ควรเป็นสูตรที่เรียบง่ายพอให้เห็นโครงสร้างของอุปนัยชัด โดยยังไม่ถูกรบกวนด้วยการจัดรูปที่ซับซ้อนเกินไป สูตรมาตรฐานที่สุดคือ

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Theorem 6.2.1. สำหรับทุก $n \geq 0$,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2},$$

โดยตกลงว่าเมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่างและมีค่าเป็น 0

Proof idea. กำหนด

$$P(n) : 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

ขั้นฐานตรวจสอบตรง ๆ ที่ $n = 0$ ส่วนขั้นอุปนัยใช้สมมติฐานสำหรับผลบวกถึง k แล้วเติม $(k+1)$ เข้าไปอีกหนึ่งพจน์

Proof. ให้

$$P(n) : 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

ขั้นฐาน: เมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่าง จึงมีค่าเป็น 0 และด้านขวาเป็น

$$\frac{0(0+1)}{2} = 0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า $P(k)$ เป็นจริง นั่นคือ

$$1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2}.$$

เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง กล่าวคือ

$$1 + 2 + 3 + \cdots + k + (k+1) = \frac{(k+1)(k+2)}{2}.$$

เริ่มจากด้านซ้ายของข้อความที่ต้องการพิสูจน์:

$$1 + 2 + 3 + \cdots + k + (k+1).$$

ใช้สมมติฐานขั้นอุปนัยแทนผลบวกส่วนแรก จะได้

$$\frac{k(k+1)}{2} + (k+1).$$

จัดรูปต่อเป็น

$$\frac{k(k+1)}{2} + \frac{2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

ซึ่งตรงกับด้านขวาของ $P(k+1)$ ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วยหลักอุปนัยว่า

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

เป็นจริงสำหรับทุก $n \geq 0$

□

6.2.1 สิ่งที่ต้องอย่างนี้สอนเรา

ตัวอย่างนี้สำคัญไม่ใช่เพราะสูตรใหม่ แต่เพราะมันแสดงรูปแบบการทำงานของ induction อย่างชัดเจน: เรามีข้อความที่ระดับ k แล้วใช้มันเป็น “สะพาน” พาเราไปยังข้อความที่ระดับ $k+1$

กล่าวอีกแบบหนึ่ง อุปนัยไม่ได้พิสูจน์ทุกค่าพร้อมกัน แต่มันพิสูจน์ว่า “ถ้าจริงถึงตรงนี้ ก็จริงต่อไปได้อีกหนึ่งก้าว” ซึ่งเพียงพอเมื่อจับคู่กับขั้นฐาน

6.2.2 ผลบวกกำลังสอง

หลังจากเห็นสูตรผลบวกของจำนวนเต็มตัวแรกแล้ว ตัวอย่างถัดไปที่เป็นธรรมชาติคือผลบวกกำลังสอง ซึ่งยังอยู่ในโลกของ ordinary induction แต่ต้องจัดรูปมากขึ้นอีกเล็กน้อย จึงช่วยให้เห็นชัดว่า induction ไม่ได้พึ่งเพียงการท่องแพตเทิร์น แต่ต้องอาศัยการจัดรูปอย่างมีเป้าหมายด้วย

Theorem 6.2.2. สำหรับทุก $n \geq 0$,

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

Proof idea. กำหนด

$$P(n) : 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

ขั้นฐานตรวจได้ตรง ๆ ส่วนขั้นอุปนัยใช้สมมติฐานสำหรับผลบวกถึง k แล้วบวก $(k+1)^2$ เข้าไปอีกหนึ่งพจน์ ก่อนจัดรูปให้กลายเป็นสูตรของ $P(k+1)$

Proof. ให้

$$P(n) : 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

ขั้นฐาน: เมื่อ $n = 0$ ด้านซ้ายเป็นผลบวกว่างจึงมีค่าเป็น 0 และด้านขวาเป็น

$$\frac{0(0+1)(2 \cdot 0 + 1)}{6} = 0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$1^2 + 2^2 + \cdots + k^2 = \frac{k(k+1)(2k+1)}{6}.$$

ต้องการพิสูจน์ว่า

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}.$$

เริ่มจากด้านซ้ายของข้อความที่ต้องการพิสูจน์:

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2.$$

ใช้สมมติฐานขั้นอุปนัย จะได้

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2.$$

ดึง $(k+1)$ ออกเป็นตัวร่วม:

$$(k+1) \left(\frac{k(2k+1)}{6} + k+1 \right).$$

เขียน $k+1$ ให้อยู่ในส่วน 6:

$$(k+1) \left(\frac{k(2k+1) + 6(k+1)}{6} \right).$$

จัดรูปในวงเล็บ:

$$(k+1) \left(\frac{2k^2 + k + 6k + 6}{6} \right) = (k+1) \left(\frac{2k^2 + 7k + 6}{6} \right).$$

แยกตัวประกอบได้เป็น

$$2k^2 + 7k + 6 = (2k+3)(k+2).$$

จึงได้ว่า

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วยหลักอุปนัยว่า

$$1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

สำหรับทุก $n \geq 0$

□

6.3 อีกสองตัวอย่างของ ordinary induction

หลังจากเห็นตัวอย่างแรกแล้ว เราควรดูโจทย์อีกสองแบบที่มีลักษณะแตกต่างกัน: แบบหนึ่งเป็นผลบวกที่มีแพตเทิร์นเฉพาะ อีกแบบหนึ่งเป็นสมการ เพื่อให้เห็นว่า induction ไม่ได้ใช้เฉพาะกับสมการผลบวกเท่านั้น

6.3.1 ผลบวกของจำนวนคี่

Theorem 6.3.1. สำหรับทุก $n \geq 0$,

$$1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

Proof idea. กำหนด

$$P(n) : 1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

ประเด็นสำคัญคือ เมื่อเพิ่มจำนวนคี่ตัวถัดไป ผลรวมจะขยับจากกำลังสองหนึ่งไปยังกำลังสองถัดไปพอดี

Proof. ให้

$$P(n) : 1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$1 = (0+1)^2.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$1 + 3 + 5 + \cdots + (2k+1) = (k+1)^2.$$

ต้องการพิสูจน์ว่า

$$1 + 3 + 5 + \cdots + (2k+1) + (2k+3) = (k+2)^2.$$

ใช้สมมติฐานขั้นอุปนัยกับด้านซ้าย จะได้

$$(k+1)^2 + (2k+3) = k^2 + 2k + 1 + 2k + 3 = k^2 + 4k + 4 = (k+2)^2.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปได้ว่า

$$1 + 3 + 5 + \cdots + (2n+1) = (n+1)^2$$

สำหรับทุก $n \geq 0$

□

6.3.2 ตัวอย่างอสมการ

Theorem 6.3.2. สำหรับทุก $n \geq 1$,

$$3^n > 2^n.$$

Proof idea. ในกรณีของอสมการ เรามักต้องระวังเรื่องการคูณหรือการบวกว่ารักษาทิศของอสมการหรือไม่ ในที่นี้ประเด็นง่ายมาก เพราะเราคูณด้วยจำนวนบวกเท่านั้น

Proof. ให้

$$P(n) : 3^n > 2^n.$$

ขั้นฐาน: เมื่อ $n = 1$,

$$3^1 = 3 > 2 = 2^1.$$

ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$3^k > 2^k.$$

ต้องการพิสูจน์ว่า

$$3^{k+1} > 2^{k+1}.$$

จากสมมติฐานขั้นอุปนัย เมื่อคูณทั้งสองข้างด้วย 3 จะได้

$$3^{k+1} > 3 \cdot 2^k.$$

และเพราะ $3 > 2$ จึงมี

$$3 \cdot 2^k > 2 \cdot 2^k = 2^{k+1}.$$

ดังนั้น

$$3^{k+1} > 2^{k+1}.$$

จึงได้ว่า $P(k+1)$ เป็นจริง

สรุปว่า

$$3^n > 2^n$$

สำหรับทุก $n \geq 1$

□

6.4 Induction กับสูตรปิดของลำดับที่นิยามแบบเวียนเกิด

ถึงตรงนี้ ภาพสำคัญเริ่มปรากฏชัด: *recursion* ใช้บอกเราว่าวัตถุหรือค่าถัดไปเกิดจากอะไร ส่วน *induction* ใช้พิสูจน์ว่าคำอธิบายแบบปิดที่เราคาดเดาไว้นั้นถูกต้องจริง

ตรงนี้เป็นจุดเชื่อมธรรมชาติระหว่างบท *recursion* กับบท *induction* เพราะเมื่อเรานิยามลำดับด้วยความสัมพันธ์เวียนเกิดแล้ว สิ่งที่เราอยากจะทำต่อคือหาสูตรที่ไม่ต้องย้อนกลับหลายครั้ง แล้วใช้ *induction* รับรองว่าสูตรนั้นถูกต้อง

6.4.1 ลำดับเลขคณิต

Theorem 6.4.1. ให้ลำดับ $a(n)$ นิยามโดย

$$a(0) = s, \quad a(n) = a(n-1) + d \quad \text{สำหรับทุก } n \geq 1.$$

แล้วสำหรับทุก $n \geq 0$,

$$a(n) = s + nd.$$

Proof idea. นี่เป็นตัวอย่างมาตรฐานของความสัมพันธ์ระหว่าง *recursion* กับ *induction*: นิยามเวียนเกิดบอกว่า “แต่ละขั้นเพิ่ม d ” ส่วนสูตรปิดบอกว่า “หลังเดินมา n ขั้น จะเพิ่มรวม nd ”

Proof. ให้

$$P(n) : a(n) = s + nd.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$a(0) = s = s + 0d.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$a(k) = s + kd.$$

จากนิยามเวียนเกิด

$$a(k+1) = a(k) + d.$$

แทนสมมติฐานขั้นอุปนัย จะได้

$$a(k+1) = s + kd + d = s + (k+1)d.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า

$$a(n) = s + nd$$

สำหรับทุก $n \geq 0$

□

6.4.2 ลำดับเรขาคณิต

ตัวอย่างลำดับเลขคณิตแสดงให้เห็นว่า recursion แบบ ``บวกค่าคงที่ทีละขั้น" นำไปสู่สูตรปิดเชิงเส้นใน n ตัวอย่างถัดไปคือกรณีที่แต่ละขั้นไม่ได้บวกค่าคงที่ แต่คูณด้วยค่าคงที่แทน ซึ่งนำไปสู่สูตรปิดในรูปเลขยกกำลัง

Theorem 6.4.2. ให้ลำดับ $g(n)$ นิยามโดย

$$g(0) = s, \quad g(n) = g(n-1)r \quad \text{สำหรับทุก } n \geq 1.$$

แล้วสำหรับทุก $n \geq 0$,

$$g(n) = sr^n.$$

Proof. ให้

$$P(n) : g(n) = sr^n.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$g(0) = s = sr^0.$$

ดังนั้น $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$g(k) = sr^k.$$

จากนิยามเวียนเกิด

$$g(k+1) = g(k)r.$$

แทนสมมติฐานขั้นอุปนัย จะได้

$$g(k+1) = sr^k \cdot r = sr^{k+1}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า

$$g(n) = sr^n$$

สำหรับทุก $n \geq 0$ □

สองตัวอย่างนี้ชี้ให้เห็น pattern ที่สำคัญมาก: นิยามแบบ recursion มักทำหน้าที่เป็น “กฎอุปเดตจากขั้นก่อนหน้า” ส่วน induction ใช้พิสูจน์ว่า “กฎอุปเดตนั้นสะสมออกมาเป็นสูตรทั่วไปที่เราคาดเดาได้”

6.5 อีกตัวอย่างหนึ่งที่มีโครงสร้างมากขึ้น: การปูบอร์ดด้วย L-tromino

ตัวอย่างก่อนหน้านี้ส่วนใหญ่เป็นสูตรผลบวก อสมการ และสูตรปิดของลำดับ ซึ่งล้วนเป็นข้อความที่มีหน้าตาเชิงพีชคณิตค่อนข้างชัด แต่ induction ไม่ได้มีประโยชน์เฉพาะกับสูตรเชิงตัวเลขเท่านั้น มันยังใช้กับปัญหาเชิงโครงสร้างได้ด้วย และตัวอย่างคลาสสิกที่สุดตัวอย่างหนึ่งคือการปูบอร์ดด้วยบล็อกรูปตัว L

Theorem 6.5.1. ให้ $n \geq 1$ พิจารณาบอร์ดขนาด $2^n \times 2^n$ ที่มีช่องหายไปหนึ่งช่อง แล้วจะสามารถปูช่องที่เหลือทั้งหมดได้พอดีด้วยบล็อกรูปตัว L ซึ่งแต่ละบล็อกปิดได้ 3 ช่อง

Proof idea. กรณี 2×2 ที่หายไปหนึ่งช่องปูได้ทันทีด้วยบล็อกตัว L หนึ่งชิ้น สำหรับบอร์ดขนาดใหญ่ขึ้น ให้แบ่งบอร์ดออกเป็น 4 ควอดแรนต์เท่า ๆ กัน ช่องที่หายไปจริงจะอยู่ในหนึ่งควอดแรนต์ จากนั้นวางบล็อกตัว L หนึ่งชิ้นตรงกลาง เพื่อให้ควอดแรนต์อีกสามส่วน “มีช่องหายไป” ส่วนละหนึ่งช่องเช่นกัน แล้วใช้สมมติฐานขั้นอุปนัยกับทั้งสี่ควอดแรนต์

Proof. ให้

$P(n)$: บอร์ดขนาด $2^n \times 2^n$ ที่มีช่องหายไปหนึ่งช่อง สามารถปูที่เหลือด้วยบล็อกตัว L ได้พอดี.

ขั้นฐาน: เมื่อ $n = 1$ บอร์ดมีขนาด 2×2 ถ้าหายไปหนึ่งช่อง จะเหลือ 3 ช่องพอดี ซึ่งปิดได้ด้วยบล็อกตัว L หนึ่งชิ้น ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า $P(k)$ เป็นจริง เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาบอร์ดขนาด $2^{k+1} \times 2^{k+1}$ ที่มีช่องหายไปหนึ่งช่อง แบ่งบอร์ดนี้ออกเป็น 4 ควอดแรนต์ แต่ละควอดแรนต์มีขนาด $2^k \times 2^k$

ช่องที่หายไปเดิมจะอยู่ในหนึ่งควอดแรนต์แน่นอน ตอนนี้พิจารณาช่อง 4 ช่องที่อยู่ติดจุดกึ่งกลางของบอร์ด ซึ่งแต่ละช่องอยู่คนละควอดแรนต์ วางบล็อกตัว L หนึ่งชิ้นปิดช่องกึ่งกลาง 3 ช่องที่อยู่ในสามควอดแรนต์ ซึ่งเดิมไม่มีช่องหายไป

ผลคือ:

- ควอดแรนต์ที่มีช่องหายไปเดิมอยู่แล้ว ยังคงเป็นบอร์ด $2^k \times 2^k$ ที่มีช่องหายไปหนึ่งช่อง
- อีกสามควอดแรนต์ ก็กลายเป็นบอร์ด $2^k \times 2^k$ ที่มีช่องหายไปหนึ่งช่องเช่นกัน

ดังนั้นจากสมมติฐานขั้นอุปนัย เราสามารถปูแต่ละควอดแรนต์ที่เหลือได้พอดีด้วยบล็อกตัว L เมื่อรวมกับบล็อกตรงกลางที่วางไว้ก่อนหน้านี้ จึงปูทั้งบอร์ดได้พอดี

ดังนั้น $P(k + 1)$ เป็นจริง และสรุปด้วยอุปนัยว่า ข้อความเป็นจริงสำหรับทุก $n \geq 1$ □

6.6 จาก ordinary induction สู่ structural induction

ordinary induction ทำงานได้ดีเมื่อวัตถุหรือข้อความถูกจัดเรียงตามเลข n ที่ละขั้น แต่ในหลายกรณี สิ่งที่เราสนใจไม่ได้ถูกอธิบายว่า “เป็นกรณีที่ n ” แต่อธิบายว่า “ถูกสร้างขึ้นจากกฎบางอย่าง”

ตัวอย่างเช่น เราอาจนิยามเซต R ว่า

1. มีสมาชิกตั้งต้นบางตัวอยู่ใน R
2. ถ้านำสมาชิกเดิมใน R มาประกอบตามกฎที่กำหนด จะได้สมาชิกใหม่ที่ยังอยู่ใน R

ถ้าเราต้องการพิสูจน์ว่า “สมาชิกทุกตัวของ R มีคุณสมบัติบางอย่าง” วิธีที่เป็นธรรมชาติที่สุดคือพิสูจน์ตาม วิธีที่สมาชิกเหล่านั้นถูกสร้างขึ้น นั่นคือพิสูจน์ว่าฐานมีคุณสมบัติ และพิสูจน์ว่าถ้าของเดิมมีคุณสมบัติแล้ว ของใหม่ที่สร้างจากมันก็ยังมีคุณสมบัตินั้น

แนวคิดนี้เรียกว่า **structural induction**

6.7 Structural Induction

6.7.1 หลักทั่วไป

โครงทั่วไปของ structural induction

สมมติว่าเซต R ถูกนิยามแบบเวียนเกิดด้วย

1. กฎฐาน ที่ระบุสมาชิกตั้งต้นของ R
2. กฎสร้าง ที่บอกว่าถ้ามีสมาชิกบางตัวอยู่ใน R แล้ว จะสร้างสมาชิกใหม่ตัวใดให้อยู่ใน R ได้อีก

ถ้าต้องการพิสูจน์ว่า "สมาชิกทุกตัวของ R มีคุณสมบัติ Q " ให้ทำสองอย่าง:

1. พิสูจน์ว่าสมาชิกฐานทุกตัวมีคุณสมบัติ Q
2. พิสูจน์ว่าถ้าสมาชิกเดิมที่ใช้ในกฎสร้างมีคุณสมบัติ Q แล้วสมาชิกใหม่ที่สร้างขึ้นก็มีคุณสมบัติ Q ด้วย

หัวใจของมันคือ: แทนที่จะพิสูจน์ตามเลข n , เราพิสูจน์ตาม ประวัติการสร้างวัตถุ

6.7.2 การพิสูจน์ความเท่ากันของเซต $R = A$

ในโจทย์หลายข้อ เรามีเซตสองชุดดังนี้

- R : เซตที่นิยามแบบ recursion
- A : เซตที่นิยามด้วยคุณสมบัติที่เราเชื่อว่า R ควรเท่ากับมัน

ถ้าต้องการพิสูจน์ว่า

$$R = A$$

มักแยกเป็นสองส่วน:

แพตเทิร์นของการพิสูจน์ $R = A$

1. พิสูจน์ว่า $R \subseteq A$
กล่าวคือ ทุกสิ่งที่สร้างได้จากกฎของ R ล้วนมีคุณสมบัติของสมาชิกใน A
2. พิสูจน์ว่า $A \subseteq R$
กล่าวคือ ทุกสิ่งที่ควรอยู่ใน A สามารถสร้างได้จริงจากกฎของ R

โดยทั่วไป ทิศ $R \subseteq A$ มักใช้ structural induction เพราะเรากำลังวิ่งตามกฎการสร้างของ R ส่วนทิศ $A \subseteq R$ มักใช้ ordinary induction หรือวิธีอื่นที่เหมาะสมกับรูปของ A

6.7.3 ตัวอย่าง: เซตของกำลังของ 2

Theorem 6.7.1. ให้เซต R นิยามโดย

1. $2 \in R$
2. ถ้า $x \in R$ แล้ว $2x \in R$

แล้ว

$$R = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทศ $R \subseteq A$ แปลว่า “ทุกสิ่งทีสร้างจากกฎนี้เป็นกำลังของ 2” จึงเหมาะกับ structural induction ส่วนทศ $A \subseteq R$ แปลว่า “กำลังของ 2 ทุกตัวสร้างได้จริง” จึงใช้อุปนัยธรรมดาบนเลขชี้กำลัง n

Proof. ให้

$$A = \{2^n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction ตามกฎของ R

ขั้นฐาน:

$$2 = 2^1 \in A.$$

ขั้นอุปนัย: ให้ $x \in R$ และสมมติว่า $x \in A$ ดังนั้นมี $m \geq 1$ ที่ทำให้

$$x = 2^m.$$

จากกฎสร้างของ R จะได้ว่า $2x \in R$ และ

$$2x = 2^{m+1} \in A.$$

จึงได้ว่า $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 2^n \in R.$$

ขั้นฐาน:

$$2^1 = 2 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า $2^k \in R$ จากกฎของ R จะได้ว่า

$$2^{k+1} = 2 \cdot 2^k \in R.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมสองทิศเข้าด้วยกัน จึงได้

$$R = A.$$

□

6.7.4 ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 จากกฎ $x \mapsto x + 5$

ตัวอย่างก่อนหน้าแสดงให้เห็นว่า เมื่อเซต R ถูกนิยามด้วยสมาชิกฐานและกฎสร้างแบบง่าย เรามักพิสูจน์ว่า $R = A$ โดยแยกเป็นสองทิศ ตัวอย่างถัดไปมีรูปแบบคล้ายกัน แต่เปลี่ยนจากการคูณด้วย 2 เป็นการบวกด้วย 5

Theorem 6.7.2. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $x \in R$ แล้ว $x + 5 \in R$

แล้ว

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทิศ $R \subseteq A$ ใช้ structural induction ตามกฎการสร้างของ R ส่วนทิศ $A \subseteq R$ ใช้อุปนัยธรรมดาบน n

Proof. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction

ขั้นฐาน:

$$5 = 5 \cdot 1 \in A.$$

ขั้นอุปนัย: ให้ $x \in R$ และสมมติว่า $x \in A$ ดังนั้นมี $m \geq 1$ ที่ทำให้

$$x = 5m.$$

จากกฎสร้างของ R จะได้ว่า

$$x + 5 \in R.$$

และ

$$x + 5 = 5m + 5 = 5(m + 1) \in A.$$

ดังนั้น $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 5n \in R.$$

ขั้นฐาน:

$$5 \cdot 1 = 5 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$5k \in R.$$

จากกฎของ R จะได้ว่า

$$5k + 5 = 5(k + 1) \in R.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมสองทิศ จะได้ว่า

$$R = A.$$

□

6.7.5 ตัวอย่างเพิ่ม: เซตของพหุคูณของ 5 ที่ปิดภายใต้การบวก

ตัวอย่างนี้สำคัญเพราะช่วยขยายภาพของ structural induction ออกไปอีกขั้น: กฎสร้างไม่จำเป็นต้องรับสมาชิกเดิมเพียงตัวเดียวแล้วสร้างสมาชิกใหม่ แต่อาจรับสมาชิกเดิมหลายตัวพร้อมกัน แล้วประกอบเป็นสมาชิกใหม่ตัวหนึ่งก็ได้

นั่นคือในขั้นอุปนัย เราอาจต้องสมมติคุณสมบัติของวัตถุเดิมมากกว่าหนึ่งตัวพร้อมกัน แล้วพิสูจน์ว่าของใหม่ที่สร้าง

จากวัตถุเหล่านั้นยังคงมีคุณสมบัติเดิมอยู่

Theorem 6.7.3. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $a \in R$ และ $b \in R$ แล้ว $a + b \in R$

แล้ว

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Proof idea. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

ทศ $R \subseteq A$ ใช้ structural induction ตามกฎของ R

สิ่งที่ต่างจากตัวอย่างก่อนคือ ในกฎสร้างข้อที่สอง เราไม่ได้เริ่มจากสมาชิกเดิมเพียงตัวเดียว แต่เริ่มจากสมาชิกสองตัว $a, b \in R$ ดังนั้นในขั้นอุปนัยเราจะสมมติว่า a และ b ต่างก็เป็นพหุคูณของ 5 แล้วตรวจว่า $a + b$ ก็ยังเป็นพหุคูณของ 5

ส่วนทศ $A \subseteq R$ คราวนี้ยิ่งง่ายขึ้น เพราะเมื่อมี $5 \in R$ แล้ว เราสามารถสร้าง

$$10 = 5 + 5, \quad 15 = 10 + 5, \quad 20 = 15 + 5, \dots$$

ได้ต่อไปเรื่อย ๆ จึงยังใช้อุปนัยธรรมดาบน n ได้เหมือนเดิม

Proof. ให้

$$A = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

พิสูจน์ว่า $R \subseteq A$: ใช้ structural induction ตามกฎของ R

ขั้นฐาน:

$$5 = 5 \cdot 1 \in A.$$

ขั้นอุปนัย: ให้ $a, b \in R$ และสมมติว่า

$$a \in A, \quad b \in A.$$

ดังนั้นมีจำนวนเต็มบวก m, n ที่ทำให้

$$a = 5m, \quad b = 5n.$$

จากกฎสร้างของ R จะได้ว่า

$$a + b \in R.$$

และ

$$a + b = 5m + 5n = 5(m + n) \in A.$$

ดังนั้นสมาชิกใหม่ที่ได้จากการบวกสมาชิกเดิมสองตัว ก็ยังอยู่ใน A จึงสรุปได้ว่า $R \subseteq A$

พิสูจน์ว่า $A \subseteq R$: ใช้อุปนัยธรรมดาบน $n \geq 1$

ให้

$$P(n) : 5n \in R.$$

ขั้นฐาน:

$$5 \cdot 1 = 5 \in R.$$

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$5k \in R.$$

เพราะ $5 \in R$ จากขั้นฐาน และกฎของ R บอกว่าถ้า $a, b \in R$ แล้ว $a + b \in R$ จึงได้ว่า

$$5k + 5 = 5(k + 1) \in R.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า $A \subseteq R$ เมื่อรวมกับ $R \subseteq A$ จะได้ว่า

$$R = A.$$

□

6.7.6 ตัวอย่าง: คุณสมบัติของสมาชิกทุกตัวในเซตที่สร้างแบบ recursion

structural induction ไม่ได้ใช้เฉพาะเวลาจะพิสูจน์ความเท่ากันของเซต แต่ยังใช้พิสูจน์ “คุณสมบัติร่วมของสมาชิกทุกตัวที่สร้างได้” ด้วย

Theorem 6.7.4. ให้เซต R นิยามโดย

1. $(0, 0) \in R$

2. ถ้า $(x, y) \in R$ แล้ว

$$(x, y + 1), (x + 1, y + 1), (x + 2, y + 1) \in R$$

แล้วสำหรับทุก $(x, y) \in R$ จะมี

$$x \leq 2y.$$

Proof idea. สิ่งที่ต้องพิสูจน์ไม่ได้เกี่ยวกับ “สมาชิกตัวที่ n ” เลย แต่เกี่ยวกับคู่ลำดับทุกตัวที่สร้างได้จากกฎ ดังนั้น structural induction จึงเป็นภาษาที่ตรงกับโจทย์มากที่สุด

Proof. ให้

$$Q(x, y) : x \leq 2y.$$

เราจะพิสูจน์ว่า สมาชิกทุกตัวของ R มีคุณสมบัติ Q

ขั้นฐาน: สมาชิกฐานคือ $(0, 0)$ และ

$$0 \leq 2 \cdot 0.$$

ดังนั้น $Q(0, 0)$ เป็นจริง

ขั้นอุปนัย: ให้ $(u, v) \in R$ และสมมติว่า

$$u \leq 2v.$$

เราต้องพิสูจน์ว่าสมาชิกใหม่ทั้งสามแบบยังสอดคล้องกับสมการนี้

สำหรับ $(u, v + 1)$,

$$u \leq 2v \leq 2v + 2 = 2(v + 1).$$

จึงได้ว่า

$$u \leq 2(v + 1).$$

สำหรับ $(u + 1, v + 1)$, จาก $u \leq 2v$ จะได้

$$u + 1 \leq 2v + 1 \leq 2v + 2 = 2(v + 1).$$

สำหรับ $(u + 2, v + 1)$, จาก $u \leq 2v$ จะได้

$$u + 2 \leq 2v + 2 = 2(v + 1).$$

ดังนั้นสมาชิกใหม่ทุกแบบยังมีคุณสมบัติ $x \leq 2y$ สรุปด้วย structural induction ได้ว่า ทุก $(x, y) \in R$ มี

คุณสมบัติ

$$x \leq 2y.$$

□

6.7.7 ตัวอย่างกับ bit string

Theorem 6.7.5. ให้เซตของ bit string R นิยามโดย

1. string ว่าง ϵ อยู่ใน R
2. ถ้า $x \in R$ แล้ว $0x1$ และ $1x0$ อยู่ใน R
3. ถ้า $x, y \in R$ แล้ว $xy \in R$

แล้วทุก string ใน R มีจำนวนเลข 0 เท่ากับจำนวนเลข 1

Proof idea. ข้อนี้นำให้เห็น structural induction ในรูปที่เป็นธรรมชาติที่สุด: ฐานคือ string ว่าง กฎสร้างแต่ละข้อบอกชัดว่าจะสร้าง string ใหม่จาก string เดิมอย่างไร ดังนั้นเราก็พิสูจน์ตามกฎทั้งสามข้อทีละข้อ

Proof. ให้ $\#_0(w)$ และ $\#_1(w)$ แทนจำนวนเลข 0 และ 1 ใน string w และให้

$$Q(w) : \#_0(w) = \#_1(w).$$

ขั้นฐาน: สำหรับ string ว่าง ϵ ,

$$\#_0(\epsilon) = 0 = \#_1(\epsilon).$$

ดังนั้น $Q(\epsilon)$ เป็นจริง

ขั้นอุปนัยจากกฎข้อ (2): ให้ $x \in R$ และสมมติว่า

$$\#_0(x) = \#_1(x).$$

พิจารณา $0x1$ จะมี

$$\#_0(0x1) = \#_0(x) + 1, \quad \#_1(0x1) = \#_1(x) + 1.$$

จึงได้ว่า

$$\#_0(0x1) = \#_1(0x1).$$

ในการทำงานเดียวกัน

$$\#_0(1x0) = \#_0(x) + 1, \quad \#_1(1x0) = \#_1(x) + 1,$$

จึงได้ว่า

$$\#_0(1x0) = \#_1(1x0).$$

ขั้นอุปนัยจากกฎข้อ (3): ให้ $x, y \in R$ และสมมติว่า

$$\#_0(x) = \#_1(x), \quad \#_0(y) = \#_1(y).$$

สำหรับ string ต่อกัน xy จะมี

$$\#_0(xy) = \#_0(x) + \#_0(y), \quad \#_1(xy) = \#_1(x) + \#_1(y).$$

ดังนั้น

$$\#_0(xy) = \#_1(xy).$$

สรุปด้วย structural induction ว่า ทุก string ใน R มีจำนวนเลข 0 เท่ากับจำนวนเลข 1

□

6.8 จาก structural induction สู่ strong induction

ตอนนี้เรามีภาพอยู่สองแบบแล้ว: ordinary induction ใช้กับข้อความที่เดินจาก n ไป $n + 1$, ส่วน structural induction ใช้กับวัตถุที่สร้างตามกฎ

แต่ยังมีอีกสถานการณ์หนึ่งที่พบบ่อยมาก: ข้อความที่ระดับ $k + 1$ อาจไม่ได้อาศัยเพียงระดับ k อย่างเดียว แต่อาศัยหลายขั้นก่อนหน้า หรืออาศัยการแตกปัญหาออกเป็นหลายส่วนย่อยที่ล้วนเล็กกว่าเดิม

ในสถานการณ์เช่นนี้ การสมมติเพียง $P(k)$ อาจไม่พอ เราจึงต้องใช้ **strong induction**

6.9 Strong Induction

6.9.1 หลักการ

Theorem 6.9.1 (Principle of Strong Induction). ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนนับตั้งแต่ค่าเริ่มต้น n_0 เป็นต้นไป ถ้า

1. ตรวจสอบฐานที่จำเป็นครบ
2. สำหรับทุก $k \geq n_0$, ถ้าสมมติว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แล้วพิสูจน์ได้ว่า $P(k + 1)$ เป็นจริง

จะสรุปได้ว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

ความแตกต่างจาก ordinary induction จึงไม่ใช่การเปลี่ยนข้อสรุป แต่เป็นการเปลี่ยน “สิ่งที่ได้รับอนุญาตให้สมมติในขั้นอุปนัย”

เปรียบเทียบ ordinary induction กับ strong induction

ordinary induction:

$$\boxed{P(k)} \Rightarrow P(k + 1)$$

strong induction:

$$\boxed{P(n_0) \wedge P(n_0 + 1) \wedge \dots \wedge P(k)} \Rightarrow P(k + 1)$$

6.9.2 เมื่อไรควรใช้ strong induction

โดยคร่าวที่สุด strong induction เหมาะกับโจทย์ที่มีลักษณะอย่างน้อยหนึ่งข้อในต่อไปนี้

1. ขึ้นใหม่ขึ้นกับหลายชั้นก่อนหน้า เช่น recurrence ที่อ้างถึง $n - 1$ และ $n - 2$
2. ปัญหาถูกแบ่งเป็นหลายปัญหาย่อยที่มีขนาดเล็กกว่าเดิม
3. เรารู้เพียงว่าขั้นใหม่จะย้อนกลับไปใช้ “บางชั้นก่อนหน้า” แต่ยังไม่แน่ว่าเป็นขั้นใด

6.9.3 ตัวอย่าง: recurrence ที่ย้อนสองชั้น

Theorem 6.9.2. ให้ลำดับ (a_n) นิยามโดย

$$a_0 = 1, \quad a_1 = 2, \quad a_n = 5a_{n-1} - 6a_{n-2} \quad (n \geq 2).$$

แล้วสำหรับทุก $n \in \mathbb{N}$,

$$a_n = 2^n.$$

Proof idea. เพราะนิยามของ a_n ใช้ทั้ง a_{n-1} และ a_{n-2} บทพิสูจน์จึงต้องมีสิทธิ์เรียกใช้ผลของหลายขั้นก่อนหน้า ซึ่ง strong induction เหมาะโดยตรง

Proof. ให้

$$P(n) : a_n = 2^n.$$

ขั้นฐาน: เมื่อ $n = 0$,

$$a_0 = 1 = 2^0.$$

เมื่อ $n = 1$,

$$a_1 = 2 = 2^1.$$

ดังนั้น $P(0)$ และ $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

จากสมมติฐาน จะได้โดยเฉพาะว่า

$$a_k = 2^k, \quad a_{k-1} = 2^{k-1}.$$

ดังนั้น

$$a_{k+1} = 5a_k - 6a_{k-1} = 5 \cdot 2^k - 6 \cdot 2^{k-1} = 5 \cdot 2^k - 3 \cdot 2^k = 2 \cdot 2^k = 2^{k+1}.$$

ดังนั้น $P(k+1)$ เป็นจริง

สรุปด้วย strong induction ว่า

$$a_n = 2^n$$

สำหรับทุก $n \in \mathbb{N}$

□

6.9.4 ตัวอย่าง: หักช็อกโกแลตบาร์

Theorem 6.9.3. ถ้ามีช็อกโกแลตบาร์ขนาด $n \times 1$ และต้องการหักให้เหลือเป็นชิ้น 1×1 ทั้งหมด จะต้องใช้การหักพอดี $n - 1$ ครั้งเสมอ

Proof idea. จำนวนครั้งที่ต้องหักไม่ได้ขึ้นกับกรณี $n - 1$ เพียงกรณีเดียว เพราะการหักครั้งแรกอาจแบ่งบาร์ออกเป็นท่อนย่อยขนาด u_1 และ u_2 ซึ่งต่างก็เล็กกว่า n และต้องเอาผลของทั้งสองท่อนมารวมกัน

Proof. ให้

$P(n)$: ซ็อกโกแลตบาร์ขนาด $n \times 1$ ต้องหัก $n - 1$ ครั้งเพื่อแยกเป็นชิ้น 1×1 ทั้งหมด.

ขั้นฐาน: เมื่อ $n = 1$ บาร์เป็นชิ้น 1×1 อยู่แล้ว จึงไม่ต้องหักเลย และ

$$0 = 1 - 1.$$

ดังนั้น $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(1), P(2), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

พิจารณาบาร์ขนาด $(k + 1) \times 1$ การหักครั้งแรกจะแบ่งมันออกเป็นสองท่อน ขนาด $u_1 \times 1$ และ $u_2 \times 1$ โดยที่

$$u_1, u_2 \geq 1, \quad u_1 + u_2 = k + 1.$$

เพราะ $u_1, u_2 \leq k$ จึงใช้สมมติฐานขั้นอุปนัยได้กับทั้งสองท่อน

ดังนั้นท่อนแรกต้องหักอีก $u_1 - 1$ ครั้ง ท่อนที่สองต้องหักอีก $u_2 - 1$ ครั้ง รวมกับการหักครั้งแรกอีก 1 ครั้ง จำนวนครั้งที่หักทั้งหมดจึงเป็น

$$1 + (u_1 - 1) + (u_2 - 1) = u_1 + u_2 - 1 = (k + 1) - 1 = k.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า บาร์ขนาด $n \times 1$ ต้องหัก $n - 1$ ครั้งเสมอ □

6.9.5 ตัวอย่าง: แพ็กเหรียญขนาด 4 และ 5

Theorem 6.9.4. สำหรับทุกจำนวนเต็ม $n \geq 12$ จะมีจำนวนเต็มไม่ลบ a, b ที่ทำให้

$$n = 4a + 5b.$$

Proof idea. โจทย์นี้เป็นตัวอย่างที่ดีของกรณีที่มีการสร้าง $k + 1$ ไม่ได้ย้อนไปที่ k โดยตรงเสมอ แต่ย้อนกลับไปยัง $k - 3$ แทน strong induction จึงสะดวกกว่า induction แบบพื้นฐาน

Proof. ให้

$$P(n) : \exists a, b \in \mathbb{N} \cup \{0\} \text{ ที่ทำให้ } n = 4a + 5b.$$

ขั้นฐาน:

$$12 = 4 + 4 + 4, \quad 13 = 4 + 4 + 5, \quad 14 = 4 + 5 + 5, \quad 15 = 5 + 5 + 5.$$

ดังนั้น $P(12), P(13), P(14), P(15)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 15$ และสมมติว่า

$$P(12), P(13), \dots, P(k)$$

เป็นจริงทั้งหมด ต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

เพราะ $k \geq 15$ จึงมี

$$k + 1 - 4 = k - 3 \geq 12.$$

จากสมมติฐานขั้นอุปนัย $P(k - 3)$ เป็นจริง จึงมี $a, b \geq 0$ ที่ทำให้

$$k - 3 = 4a + 5b.$$

บวก 4 ทั้งสองข้าง จะได้

$$k + 1 = 4(a + 1) + 5b.$$

ดังนั้น $P(k + 1)$ เป็นจริง

สรุปว่า สำหรับทุก $n \geq 12$, สามารถเขียน n ให้อยู่ในรูป $4a + 5b$ ได้

□

6.9.6 ตัวอย่างเพิ่ม: การแยกตัวประกอบเป็นจำนวนเฉพาะ

ตัวอย่างก่อนหน้าของ strong induction ทำให้เห็นกรณีที่ขึ้นใหม่อาศัยหลายขั้นก่อนหน้า หรืออาศัยค่าที่ไม่ใช่เพียง k โดยตรง ตัวอย่างคลาสสิกอีกข้อหนึ่งคือทฤษฎีบทพื้นฐานว่า จำนวนเต็มทุกตัวที่มากกว่า 1 สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้

จุดเด่นของตัวอย่างนี้คือ เมื่อ n ไม่เป็นจำนวนเฉพาะ เราจะเขียน $n = ab$ ได้โดยที่ทั้ง a และ b เล็กกว่า n แล้วใช้สมมติฐานขั้นอุปนัยกับทั้งสองตัวพร้อมกัน

Theorem 6.9.5. สำหรับทุกจำนวนเต็ม $n \geq 2$, สามารถเขียน n ให้อยู่ในรูปผลคูณของจำนวนเฉพาะได้

Proof idea. ถ้า n เป็นจำนวนเฉพาะ ก็จบในทันทีเพราะ n เองคือผลคูณที่มีเพียงหนึ่งตัวประกอบ แต่ถ้า n ไม่เป็นจำนวนเฉพาะ จะมี

$$n = ab$$

โดยที่ $2 \leq a < n$ และ $2 \leq b < n$ จากนั้นใช้ strong induction กับทั้ง a และ b

Proof. ให้

$P(n)$: จำนวนเต็ม $n \geq 2$ สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้.

ขั้นฐาน: เมื่อ $n = 2$, จำนวน 2 เป็นจำนวนเฉพาะ จึงเขียนเป็นผลคูณของจำนวนเฉพาะได้แล้ว ดังนั้น $P(2)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 2$ และสมมติว่า

$$P(2), P(3), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณา $k+1$

ถ้า $k+1$ เป็นจำนวนเฉพาะ ก็จบในทันที เพราะ $k+1$ เองเป็นผลคูณของจำนวนเฉพาะหนึ่งตัว

ถ้า $k+1$ ไม่เป็นจำนวนเฉพาะ จะมีจำนวนเต็ม a, b ที่ทำให้

$$k+1 = ab, \quad 2 \leq a < k+1, \quad 2 \leq b < k+1.$$

ดังนั้น $a \leq k$ และ $b \leq k$ จึงใช้สมมติฐานขั้นอุปนัยได้กับทั้ง a และ b

จากสมมติฐาน a เขียนเป็นผลคูณของจำนวนเฉพาะได้ และ b ก็เขียนเป็นผลคูณของจำนวนเฉพาะได้เช่นกัน ดังนั้นเมื่อคูณสองผลคูณนี้เข้าด้วยกัน จะได้ว่า

$$k+1 = ab$$

เขียนเป็นผลคูณของจำนวนเฉพาะได้

ดังนั้น $P(k+1)$ เป็นจริง สรุปด้วย strong induction ว่า ทุกจำนวนเต็ม $n \geq 2$ สามารถเขียนเป็นผลคูณของจำนวนเฉพาะได้ □

6.10 สรุปความสัมพันธ์ของสามแนวคิด

ถึงตรงนี้ เราไม่ได้มีเพียง “อุปนัยแบบหนึ่ง” แต่มีมุมมองอยู่สามแบบที่สัมพันธ์กัน

สามรูปแบบของ induction

1. ordinary induction

ใช้เมื่อข้อความเดินจากระดับ n ไป $n + 1$ โดยตรง

2. structural induction

ใช้เมื่อวัตถุถูกนิยามแบบ recursion และเราต้องการพิสูจน์ตามวิธีที่วัตถุนั้นถูกสร้างขึ้น

3. strong induction

ใช้เมื่อขั้นใหม่ต้องอาศัยหลายขั้นก่อนหน้า หรืออาศัยปัญหาย่อยที่เล็กกว่าหลายกรณีพร้อมกัน

ถ้ามองย้อนกลับไปจะเห็นว่า สามแนวคิดนี้ไม่ได้แยกขาดจากกัน แต่เป็นการขยายวิธีคิดเดียวกันให้เหมาะกับบริบทที่ต่างกัน:

ordinary induction $\rightarrow$ structural induction $\rightarrow$ strong induction

ordinary induction เป็นจุดเริ่มต้น structural induction คือการปรับอุปนัยให้สอดคล้องกับ “โครงสร้างการสร้างวัตถุ” และ strong induction คือการขยายสิ่งที่เราสมมติได้ในขั้นอุปนัย เพื่อรับมือกับปัญหาที่ซับซ้อนเกินไป อาศัยเพียงขั้นเดียวก่อนหน้า

6.11 ความสมมูลกันของ ordinary and strong induction

เมื่อเรียน strong induction เป็นครั้งแรก ผู้เรียนจำนวนมากมักรู้สึกว่า strong induction “แรงกว่า” เพราะในขั้นอุปนัย เราได้รับอนุญาตให้สมมติว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แทนที่จะสมมติเพียง

$$P(k)$$

เหมือนใน ordinary induction

ความรู้สึกนี้มีเหตุผลในเชิงการใช้งาน: เวลาลงมือพิสูจน์จริง strong induction มักดูสะดวกกว่า โดยเฉพาะเมื่อข้อความที่ระดับ $k + 1$ ต้องอาศัยหลายขั้นก่อนหน้า หรืออาศัยขั้นก่อนหน้าบางขั้นที่ยังไม่รู้แน่ชัดว่าเป็นขั้นใด

อย่างไรก็ตาม ในเชิงหลักการแล้ว **ordinary induction** และ **strong induction** สมมูลกัน กล่าวคือ ถ้ายอมรับหลักอุปนัยแบบหนึ่ง เราสามารถพิสูจน์อีกแบบหนึ่งได้ ดังนั้น strong induction ไม่ได้เป็น “หลักใหม่ที่ทรงพลังกว่าเดิม” แต่เป็นเพียงอีกรูปแบบหนึ่งของแนวคิดเดียวกัน

Theorem 6.11.1. บนจำนวนนับ หลักอุปนัยธรรมดา (ordinary induction) และหลักอุปนัยแบบเข้ม (strong induction) สมมูลกัน

เราจะพิสูจน์สมมูลนี้เป็นสองทิศ

6.11.1 Strong induction $\Rightarrow$ Ordinary induction

ทิศนี้ค่อนข้างตรงไปตรงมา เพราะถ้าเราใช้งาน strong induction ได้ เราก็ย่อมใช้งาน ordinary induction ได้ทันที

เหตุผลคือ ใน ordinary induction ขั้นตอนต้องการเพียงว่า

$$P(k) \Rightarrow P(k+1).$$

แต่ใน strong induction เราสมมติได้แรงกว่านั้น คือ

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด ดังนั้นโดยเฉพาะ เราก็รู้ว่า $P(k)$ เป็นจริงด้วย จึงใช้เหตุผลแบบ ordinary induction ต่อได้ทันที

บทพิสูจน์. สมมติว่าเรายอมรับ strong induction และต้องการพิสูจน์ ordinary induction

ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนเต็ม $n \geq n_0$ และสมมติว่า

1. $P(n_0)$ เป็นจริง
2. สำหรับทุก $k \geq n_0$, ถ้า $P(k)$ เป็นจริง แล้ว $P(k+1)$ เป็นจริง

เราต้องการสรุปว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

เพื่อใช้ strong induction ให้ $k \geq n_0$ และสมมติว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด จากสมมติฐานนี้ เราได้โดยเฉพาะว่า $P(k)$ เป็นจริง ดังนั้นจากเงื่อนไขข้อ (2) จะได้ว่า $P(k+1)$ เป็นจริง

จึงตรวจสอบขั้นอุปนัยในรูปของ strong induction ได้ครบ และเมื่อมีขั้นฐาน $P(n_0)$ อยู่แล้ว สรุปได้ด้วย strong induction ว่า

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

ดังนั้น strong induction ให้ ordinary induction ได้

□

□

6.11.2 Ordinary induction $\Rightarrow$ Strong induction

ทฤษฎีนี้เป็นทฤษฎีที่สำคัญกว่า เพราะมันแสดงให้เห็นว่า strong induction แม้ดูเหมือนจะอนุญาตให้สมมติได้มากกว่า แต่จริง ๆ แล้วสามารถสร้างขึ้นจาก ordinary induction ได้

แนวคิดหลักคือ: ถ้า strong induction ต้องการให้เราว่า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

ทั้งหมดเป็นจริงพร้อมกัน เราก็สร้างข้อความใหม่ขึ้นมาหนึ่งข้อความ ให้มันรวบสิ่งเหล่านี้ไว้เป็นก้อนเดียว

ความคิดหลักของบทพิสูจน์

แทนที่จะพิสูจน์ $P(n)$ ตรง ๆ ให้กำหนดข้อความใหม่

$$Q(n) : P(n_0) \wedge P(n_0 + 1) \wedge \dots \wedge P(n).$$

ถ้าพิสูจน์ $Q(n)$ ได้ด้วย ordinary induction ก็ย่อมได้ว่า $P(n)$ จริงสำหรับทุก $n \geq n_0$

นี่คือเทคนิคที่สำคัญมาก: ordinary induction พิสูจน์ทีละขั้นก็จริง แต่เราสามารถทำให้ “หนึ่งขั้น” บรรจุข้อมูลสะสมของทุกขั้นก่อนหน้านี้ไว้ด้วยกันได้

บทพิสูจน์. สมมติว่าเรายอมรับ ordinary induction และต้องการพิสูจน์ strong induction

ให้ $P(n)$ เป็นประโยคที่นิยามบนจำนวนเต็ม $n \geq n_0$ และสมมติว่า

1. $P(n_0)$ เป็นจริง
2. สำหรับทุก $k \geq n_0$, ถ้า

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด แล้ว $P(k + 1)$ เป็นจริง

เราต้องการสรุปว่า $P(n)$ เป็นจริงสำหรับทุก $n \geq n_0$

กำหนดข้อความใหม่

$$Q(n) : P(n_0) \wedge P(n_0 + 1) \wedge \cdots \wedge P(n).$$

เราจะพิสูจน์ว่า $Q(n)$ เป็นจริงสำหรับทุก $n \geq n_0$ ด้วย ordinary induction

ขั้นฐาน: เมื่อ $n = n_0$, ข้อความ $Q(n_0)$ ก็คือเพียง

$$P(n_0),$$

ซึ่งเป็นจริงตามสมมติฐานข้อ (1)

ขั้นอุปนัย: ให้ $k \geq n_0$ และสมมติว่า $Q(k)$ เป็นจริง นั่นคือ

$$P(n_0), P(n_0 + 1), \dots, P(k)$$

เป็นจริงทั้งหมด

จากสมมติฐานข้อ (2) จึงได้ว่า

$$P(k + 1)$$

เป็นจริงด้วย

ดังนั้นเราสามารถรวมข้อความเดิมทั้งหมดกับ $P(k + 1)$ ได้เป็น

$$P(n_0) \wedge P(n_0 + 1) \wedge \cdots \wedge P(k) \wedge P(k + 1),$$

ซึ่งก็คือ $Q(k + 1)$

จึงได้ว่า $Q(k) \Rightarrow Q(k + 1)$

สรุปด้วย ordinary induction ว่า

$$Q(n)$$

เป็นจริงสำหรับทุก $n \geq n_0$

แต่เมื่อ $Q(n)$ เป็นจริง ย่อมได้โดยเฉพาะว่า $P(n)$ เป็นจริง ดังนั้น

$$P(n) \text{ เป็นจริงสำหรับทุก } n \geq n_0.$$

จึงสรุปได้ว่า ordinary induction ให้ strong induction ได้



6.11.3 แล้วเหตุใดเรายังแยก ordinary กับ strong induction ออกจากกัน?

เมื่อรู้แล้วว่าสองหลักนี้สมมูลกัน อาจเกิดคำถามตามมาว่า แล้วทำไมหนังสือเรียนจึงยังแยก ordinary induction กับ strong induction ออกจากกันอยู่

คำตอบคือ แม้มันจะสมมูลกันในเชิงตรรกะ แต่มัน *ไม่เท่ากัน* ในเชิงการใช้งาน

ordinary induction เหมาะมากเมื่อโครงสร้างของปัญหาชัดเจนว่า ขั้นที่ $k + 1$ สร้างจากขั้นที่ k โดยตรง เช่น สูตรผลบวก ลำดับเลขคณิต หรือลำดับเรขาคณิต

ส่วน strong induction เหมาะกว่าเมื่อขั้นใหม่อาศัยหลายขั้นก่อนหน้า หรือเมื่อปัญหาถูกแตกออกเป็นหลายปัญหาย่อยที่เล็กกว่า เช่น การแยกตัวประกอบเป็นจำนวนเฉพาะ หรือการให้เหตุผลเกี่ยวกับการแบ่งปัญหา

ดังนั้น strong induction จึงไม่ได้ “พิสูจน์ได้มากกว่า” แต่หลายครั้งมัน เขียนพิสูจน์ได้ตรงธรรมชาติของปัญหามากกว่า

ข้อสรุปสำคัญ

ordinary induction และ strong induction สมมูลกันในเชิงหลักการ แต่ strong induction มัก สะดวกกว่าในเชิงการใช้งาน เมื่อข้อความที่ระดับใหม่ต้องอาศัยหลายกรณีก่อนหน้า

6.11.4 ความสัมพันธ์กับ structural induction

ควรระวังว่า ข้อสรุปเรื่อง “ordinary และ strong induction สมมูลกัน” พูดถึงเฉพาะหลักอุปนัยบนจำนวนนับเป็นหลัก

ส่วน structural induction เป็นการนำความคิดแบบอุปนัย ไปเขียนในภาษาที่สอดคล้องกับ *กฎการสร้างของวัตถุ* ดังนั้นบทบาทของมันจึงต่างออกไปเล็กน้อย: มันไม่ใช่เพียงการเลือกว่า “จะสมมติหนึ่งขั้นก่อนหน้า หรือทุกขั้นก่อนหน้า” แต่เป็นการเลือกพิสูจน์ตามโครงสร้างการสร้างวัตถุโดยตรง

อย่างไรก็ตาม ในเชิงลึก structural induction ก็ยังอยู่ในครอบครัวเดียวกับแนวคิดอุปนัยทั้งหมด: เราพิสูจน์ฐาน แล้วพิสูจน์ว่ากฎการสร้างไม่ทำให้คุณสมบัติที่ต้องการสูญหายไป

6.12 บทสรุปของ chapter นี้

ถ้าจะสรุปบทนี้เป็นประโยคเดียว ใจความสำคัญคือ:

recursion บอกว่าสิ่งหนึ่งถูกสร้างขึ้นอย่างไร และ **induction** บอกว่าเราจะพิสูจน์คุณสมบัติของสิ่งนั้นอย่างไร

สำหรับลำดับ เราเห็นว่า recursion ใช้นิยามค่าถัดไป แล้ว ordinary induction ใช้พิสูจน์สูตรทั่วไป สำหรับเซตหรือ bit string ที่สร้างจากกฎ เราใช้ structural induction เดินตามกฎการสร้าง และเมื่อขั้นถัดไปขึ้นกับหลายขั้นก่อนหน้า เราใช้ strong induction แทน

จุดนี้เองที่ทำให้บทอุปนัยเป็นสะพานสำคัญไปยังบทถัดไป: เมื่อเราเริ่มศึกษาความถูกต้องของอัลกอริทึมแบบ recursion โดยเฉพาะอัลกอริทึมที่แตกปัญหาเป็นปัญหาย่อย เครื่องมือหลักที่กลับมาอีกครั้งก็คือ induction โดยเฉพาะ strong induction เพียงแต่คราวนี้สิ่งที่เราพิสูจน์ไม่ใช่สูตรของลำดับ แต่เป็นความถูกต้องของ algorithm

แบบฝึกหัดท้ายบท

Exercise 6.12.1. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 0$,

$$2 + 4 + 6 + \cdots + 2n = n(n + 1).$$

Exercise 6.12.2. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 1$,

$$2^n \geq n + 1.$$

Exercise 6.12.3. จงพิสูจน์ด้วย ordinary induction ว่า สำหรับทุก $n \geq 0$,

$$1 + 2 + 2^2 + \cdots + 2^n = 2^{n+1} - 1.$$

Exercise 6.12.4. ให้ลำดับ $b(n)$ นิยามโดย

$$b(0) = 4, \quad b(n) = b(n - 1) + 7 \quad (n \geq 1).$$

จงคาดเดาสูตรปิดของ $b(n)$ และพิสูจน์ด้วย induction

Exercise 6.12.5. ให้ลำดับ $h(n)$ นิยามโดย

$$h(0) = 5, \quad h(n) = 3h(n - 1) \quad (n \geq 1).$$

จงคาดเดาสูตรปิดของ $h(n)$ และพิสูจน์ด้วย induction

Exercise 6.12.6. ให้เซต R นิยามโดย

1. $5 \in R$
2. ถ้า $x \in R$ แล้ว $x + 5 \in R$

จงพิสูจน์ว่า

$$R = \{5n \mid n \in \mathbb{N}, n \geq 1\}.$$

Exercise 6.12.7. ให้เซต R นิยามโดย

1. $\epsilon \in R$
2. ถ้า $x \in R$ แล้ว $0x1$ และ $1x0 \in R$
3. ถ้า $x, y \in R$ แล้ว $xy \in R$

จงพิสูจน์ว่า string ทุกตัวใน R มีความยาวเป็นจำนวนคู่

Exercise 6.12.8. ให้ลำดับ (b_n) นิยามโดย

$$b_0 = 3, \quad b_1 = 6, \quad b_n = 4b_{n-1} - 4b_{n-2} \quad (n \geq 2).$$

จงคาดเดาสสูตรของ b_n และพิสูจน์ด้วย strong induction

Exercise 6.12.9. จงพิสูจน์ด้วย strong induction ว่า สำหรับทุกจำนวนเต็ม $n \geq 2$, สามารถเขียน n ให้อยู่ในรูปผลคูณของจำนวนเฉพาะได้

Exercise 6.12.10. กำหนดให้ R เป็นเซตที่นิยามแบบเวียนเกิดดังนี้

- $1 \in R$
- สำหรับ $n \in \mathbb{N}$ ใด ๆ ถ้า $n \in R$ แล้ว $5n, n^2 \in R$

จงพิสูจน์ว่า ทุก ๆ สมาชิก $x \in R$ จะมี $q \in \mathbb{N}$ ที่ทำให้ $x = 4q + 1$ (กล่าวคือทุกสมาชิกใน R คือพวกจำนวนนับที่หารด้วย 4 แล้วเหลือเศษ 1)

Exercise 6.12.11. นิยามฟังก์ชันการ reverse สายอักขระ (string) ดังนี้

- $\text{reverse}(\epsilon) = \epsilon$ โดยที่ ϵ คือตัวสายอักขระว่าง (ความยาว 0)
- สำหรับตัวอักขระ a และสายอักขระ w จะได้ว่า $\text{reverse}(wa) = a \cdot \text{reverse}(w)$ โดย $\cdot$ หมายถึงการต่อข้อความ (concatenation)

ตัวอย่างเช่น $\text{reverse}(abcd) = dcba$ หรือ $\text{reverse}(a) = a$ หรือถ้ามองในระบับการเวียนเกิด จะได้ว่า

$$\text{reverse}(abcd) = d \cdot \text{reverse}(abc) = dc \cdot \text{reverse}(ab) = dcb \cdot \text{reverse}(a) = dcba$$

จงพิสูจน์ว่า $\text{reverse}(xy) = \text{reverse}(y) \cdot \text{reverse}(x)$ สำหรับสายอักขระ x และ y ใด ๆ

Exercise 6.12.12. ในโลกคอมพิวเตอร์ เราต้องใช้เลขฐานสองเป็นตัวแทนของจำนวนนับเพื่อให้คอมพิวเตอร์คำนวณได้ เช่น $6 + 10 = 110_2 + 1010_2 = 10000_2 = 16$ และนิยามของเลขฐานสองคือ

$$(x_n x_{n-1} \dots x_2 x_1 x_0)_2 = x_n \times 2^n + x_{n-1} \times 2^{n-1} + \dots + x_2 \times 2^2 + x_1 \times 2^1 + x_0 \times 2^0$$

$$\text{เช่น } 110_2 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 2^2 + 2^1 \text{ หรือ } 1010_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 2^3 + 2^1$$

ซึ่งคำถามที่ควรถามเป็นอันดับแรกก็คือ "เราสามารถแสดงผลจำนวนนับเป็นเลขฐานสองได้เสมอหรือไม่" (เพราะไม่เช่นนั้น จะกลายเป็นว่าคอมพิวเตอร์จะไม่สามารถคำนวณบางจำนวนไม่ได้) ให้นักศึกษาดังข้อคำถามดังกล่าวเป็นข้อความทางคณิตศาสตร์ที่จะต้องพิสูจน์ พร้อมทั้งพิสูจน์ข้อความดังกล่าว

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 7

Recursive Correctness Proof

บทนี้แยกออกมาเพื่อศึกษาคำถามที่ต่างจากบทอุปนัยโดยตรง ในบทอุปนัย เรามักพิสูจน์ว่า *ข้อความ* หนึ่งเป็นจริงสำหรับทุกจำนวนเต็มบวก หรือพิสูจน์ว่าคุณสมบัติบางอย่างเป็นจริงสำหรับวัตถุที่นิยามแบบเวียนเกิด แต่ในบทนี้สิ่งที่เราต้องการพิสูจน์ไม่ใช่เพียงข้อความคณิตศาสตร์ แต่เป็น *ความถูกต้องของอัลกอริทึม* โดยเฉพาะอัลกอริทึมที่เรียกตัวเอง

เมื่อเราออกแบบ *recursive algorithm* เรามักเริ่มจากการลดปัญหาใหญ่ให้กลายเป็นปัญหาย่อยที่เล็กลง แล้วใช้คำตอบของปัญหาย่อยเหล่านั้นมาประกอบเป็นคำตอบของปัญหาเดิม แนวคิดนี้ทำให้เราได้ *algorithm* ที่ดูสมเหตุสมผล แต่ในทางคณิตศาสตร์ เรายังต้องตอบอีกคำถามหนึ่งให้ชัดว่า

ทำไมอัลกอริทึมนี้จึงให้คำตอบที่ถูกต้องจริงสำหรับทุก *input* ในโดเมนที่กำหนด?

การตอบคำถามนี้มักประกอบด้วย 3 ส่วน:

1. ระบุ **specification** ให้ชัดว่า *output* ที่ถูกต้องต้องมีคุณสมบัติอะไร
2. พิสูจน์ว่า *algorithm* หยุดทำงาน (*termination*)
3. พิสูจน์ว่าเมื่อมันหยุดแล้ว *output* ที่ได้ ตรงตาม *spec* (*correctness*)

ตัวอย่างเปิดบทนี้จะเป็น *Merge Sort* ซึ่งเป็น *recursive algorithm* แบบ *divide-and-conquer* เพราะตัวอย่างนี้แสดงโครงสร้างของ *recursive correctness proof* ได้ชัดเจนมาก: เราต้องพิสูจน์ทั้งการเรียกตัวเองในส่วนย่อย และต้องพิสูจน์ด้วยว่าขั้น *combine* ไม่ทำลายความถูกต้องของคำตอบย่อย

7.1 Specification, Termination, and Correctness

ก่อนเริ่มพิสูจน์อัลกอริทึมใด ๆ เราควรแยกสามคำนี้ออกจากกันให้ชัดเจน

Definition 7.1.1 (Specification). spec ของอัลกอริทึมคือคำอธิบายว่า เมื่อป้อน input ที่อยู่ในโดเมนที่อนุญาต เข้าไปแล้ว output ที่ถือว่าถูกต้องต้องมีคุณสมบัติอะไร

Definition 7.1.2 (Termination). อัลกอริทึม terminate ถ้ามันหยุดทำงานหลังจากทำไปจำนวนขั้นจำกัด ไม่เกิดการเรียกตัวเองต่อไปเรื่อย ๆ อย่างไม่มีที่สิ้นสุด

Definition 7.1.3 (Correctness). อัลกอริทึม correct ถ้าสำหรับทุก input ในโดเมนที่กำหนด เมื่ออัลกอริทึม หยุดทำงานแล้ว output ที่ได้ตรงตาม spec

ข้อควรระวัง

การรู้ว่า base case ถูกต้อง และสมมติว่า recursive call ถูกต้อง ยัง *ไม่พอ* ที่จะสรุปว่าอัลกอริทึมทั้งตัวถูกต้อง

เรายังต้องตรวจสอบอย่างน้อยสองจุด:

1. ปัญหาย่อยที่เรียกต่อมีขนาดเล็กลงจริงหรือไม่
2. ขั้นที่นำคำตอบย่อยมาประกอบกลับเป็นคำตอบใหญ่ ยังรักษาความถูกต้องไว้หรือไม่

7.2 Divide-and-Conquer: Merge Sort

7.2.1 แนวคิด Divide--Conquer--Combine

Merge Sort เป็นตัวอย่างมาตรฐานของอัลกอริทึมแบบ divide-and-conquer โดยโครงสร้างมี 3 ขั้นตอน:

1. **Divide:** แบ่งลิสต์เดิมออกเป็นลิสต์ย่อย 2 ส่วนที่เล็กกว่าเดิม
2. **Conquer:** sort แต่ละส่วนด้วยการเรียกตัวเอง
3. **Combine:** รวมลิสต์ย่อยที่ sort แล้วให้เป็นลิสต์เดียวที่ sort แล้ว

ในขั้น combine ไม่ใช่การต่อปลายเฉย ๆ แต่เป็นการรวมสองลิสต์ที่ sorted แล้วเข้าด้วยกันอย่างระมัดระวังผ่าน procedure ที่ชื่อว่า Merge

ดังนั้นถ้าจะพิสูจน์ความถูกต้องของ Merge Sort อย่างเป็นระบบ เราควรแยกเป็นสองขั้น:

1. พิสูจน์ว่า Merge ทำงานถูกต้อง
2. ใช้ผลนั้นไปพิสูจน์ว่า MergeSort ทำงานถูกต้อง

7.2.2 นิยาม sorted และสเปกของ MergeSort

ให้ลิสต์

$$A = (a_1, a_2, \dots, a_n)$$

เราเรียกว่า A เป็นลิสต์ที่ **sorted** แบบไม่ลดลง ถ้า

$$a_1 \leq a_2 \leq \dots \leq a_n.$$

เพื่อให้การพิสูจน์ correctness ครบถ้วน เราไม่ควรระบุเพียงว่า output ``เรียงแล้ว" เพราะ sorting algorithm อาจให้ลิสต์ที่เรียงแล้วแต่สมาชิกไม่เหมือนเดิมก็ได้ ดังนั้น spec ที่ถูกต้องต้องมีทั้งเรื่อง *ความเรียง* และ *การคงสมาชิกเดิมไว้*

ให้ $\#_x(A)$ แทนจำนวนครั้งที่ค่า x ปรากฏในลิสต์ A

Spec ของ MergeSort

สำหรับทุกลิสต์ A , ผลลัพธ์ $\text{MergeSort}(A)$ ต้องมีคุณสมบัติ 2 ข้อพร้อมกัน:

1. $\text{MergeSort}(A)$ เป็นลิสต์ที่ sorted
2. สำหรับทุกค่า x , มี

$$\#_x(\text{MergeSort}(A)) = \#_x(A)$$

กล่าวคือ ผลลัพธ์เป็นเพียงการ ``เรียงใหม่" ของสมาชิกเดิม ไม่ทำสมาชิกหาย ไม่เพิ่มสมาชิกใหม่ และไม่เปลี่ยนจำนวนซ้ำของสมาชิก

จากข้อที่สอง โดยเฉพาะจะได้ว่า

$$|\text{MergeSort}(A)| = |A|.$$

7.2.3 Pseudocode ของ MergeSort และ Merge

Pseudocode: MergeSort และ Merge

MergeSort(A):

if $|A| \leq 1$ then return A

แบ่ง A เป็น L, R โดย $|L| = \lfloor |A|/2 \rfloor$ และ $|R| = \lceil |A|/2 \rceil$

return Merge(MergeSort(L), MergeSort(R))

Merge(L, R): (สมมติว่า L และ R sorted แล้ว)

if $L = []$ then return R

if $R = []$ then return L

if $\text{first}(L) \leq \text{first}(R)$ then

return $[\text{first}(L)] \parallel \text{Merge}(\text{tail}(L), R)$

else

return $[\text{first}(R)] \parallel \text{Merge}(L, \text{tail}(R))$

สัญลักษณ์ $\parallel$ หมายถึงการนำสมาชิกรวมตัวหนึ่งไปต่อหน้าลิสต์ และ $\text{tail}(L)$ หมายถึงลิสต์ที่เหลือหลังตัดสมาชิกรวมตัวแรกของ L ออก

7.3 พิสูจน์ความถูกต้องของ Merge

ขั้น combine ของ Merge Sort คือ Merge ดังนั้นเราต้องเริ่มจากตรงนี้ก่อน

Theorem 7.3.1 (Correctness of Merge). ถ้า L และ R เป็นลิสต์ที่ sorted แล้ว และให้

$$M = \text{Merge}(L, R),$$

จะได้ว่า

1. M เป็นลิสต์ที่ sorted
2. สำหรับทุกค่า x ,

$$\#_x(M) = \#_x(L) + \#_x(R)$$

ดังนั้นโดยเฉพาะ

$$|M| = |L| + |R|.$$

Proof idea. พิสูจน์ด้วย strong induction บนขนาดปัญหา

$$n = |L| + |R|.$$

ใจความสำคัญคือ สมาชิกตัวแรกที่ Merge ปล่อยออกมาจะต้องเป็นตัวที่เล็กที่สุดในบรรดาหัวของทั้งสองลิสต์ จากนั้นปัญหาที่เหลือจะมีขนาดลดลง 1 จึงใช้สมมติฐานขั้นอุปนัยกับส่วนที่เหลือได้

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ sorted L, R ที่ทำให้ $|L| + |R| = n$, ลิสต์ $Merge(L, R)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับการรวมสมาชิกของ L และ R แบบนับซ้ำ

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$

ถ้า $|L| + |R| = 0$ จะต้องเป็น

$$L = [], \quad R = [].$$

ดังนั้น

$$Merge(L, R) = [].$$

ลิสต์ว่างย่อม sorted และมีสมาชิกครบถ้วนตามต้องการ จึงได้ว่า $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาลิสต์ sorted L, R ที่มี

$$|L| + |R| = k + 1.$$

ถ้า $L = []$ หรือ $R = []$ ผลลัพธ์ของ Merge คืออีกลิสต์หนึ่งที่ซึ่งยังคง sorted และสมาชิกก็ครบถ้วนชัดเจน ดังนั้นกรณีนี้ไม่มีปัญหา

จึงเหลือกรณีที่ทั้ง L และ R ไม่ว่าง เขียน

$$L = (\ell_1, \ell_2, \dots, \ell_s), \quad R = (r_1, r_2, \dots, r_t).$$

เราพิจารณาเป็นสองกรณี

กรณีที่ 1: $\ell_1 \leq r_1$

จาก algorithm จะได้

$$\text{Merge}(L, R) = [\ell_1] \parallel \text{Merge}(L', R),$$

โดยที่ $L' = (\ell_2, \dots, \ell_s)$

สังเกตว่า

$$|L'| + |R| = (|L| - 1) + |R| = k.$$

จึงใช้สมมติฐานขั้นอุปนัยได้ ดังนั้น $\text{Merge}(L', R)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ L' กับ R รวมกันแบบนับซ้ำ

เราต้องตรวจสอบว่าเมื่อเอา ℓ_1 มาต่อหน้าลิสต์นี้ ผลลัพธ์ยัง sorted อยู่หรือไม่

เพราะ L sorted จึงสมาชิกทุกตัวใน L' ไม่เล็กกว่า ℓ_1 และเพราะ R sorted กับ $\ell_1 \leq r_1$ สมาชิกทุกตัวใน R ก็ไม่เล็กกว่า ℓ_1 เช่นกัน ดังนั้นสมาชิกทุกตัวใน $\text{Merge}(L', R)$ ไม่เล็กกว่า ℓ_1 จึงได้ว่า

$$[\ell_1] \parallel \text{Merge}(L', R)$$

ยัง sorted

ด้านสมาชิกแบบนับซ้ำ ลิสต์ผลลัพธ์มีสมาชิกคือ ℓ_1 บวกกับสมาชิกทั้งหมดของ L' และ R ซึ่งเท่ากับสมาชิกทั้งหมดของ L และ R รวมกันพอดี

กรณีที่ 2: $r_1 < \ell_1$

พิสูจน์แบบสมมาตร:

$$\text{Merge}(L, R) = [r_1] \parallel \text{Merge}(L, R'),$$

โดยที่ $R' = (r_2, \dots, r_t)$ และจากสมมติฐานขั้นอุปนัย ลิสต์ $\text{Merge}(L, R')$ sorted และมีสมาชิกครบ เมื่อรวมกับข้อเท็จจริงว่าสมาชิกทุกตัวใน L และ R' ไม่เล็กกว่า r_1 จึงได้ว่าผลลัพธ์สุดท้าย sorted และมีสมาชิกครบเช่นกัน

ดังนั้น $P(k + 1)$ เป็นจริง สรุปด้วย strong induction ได้ว่า Merge ถูกต้อง □

7.4 พิสูจน์ความถูกต้องของ MergeSort

เมื่อเรารู้แล้วว่าขั้น combine ถูกต้อง จึงสามารถพิสูจน์ correctness ของ MergeSort ทั้งตัวได้

Theorem 7.4.1 (Correctness of MergeSort). สำหรับทุกลิสต์ A , ลิสต์ $\text{MergeSort}(A)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ A แบบนับซ้ำ

Proof idea. พิสูจน์ด้วย strong induction บนความยาวลิสต์

$$n = |A|.$$

ถ้าลิสต์มีความยาว 0 หรือ 1 อัลกอริทึมคืนลิสต์เดิมทันที ส่วนกรณีที่ลิสต์ยาวกว่านั้น อัลกอริทึมจะแบ่งลิสต์เดิมออกเป็นสองลิสต์ที่สั้นกว่าเดิม เรียกตัวเองกับแต่ละลิสต์ย่อย แล้วใช้ความถูกต้องของ Merge เพื่อรวมผลลัพธ์กลับมา

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ A ที่มีความยาว n , ลิสต์ $\text{MergeSort}(A)$ เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ A แบบนับซ้ำ

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$ และ $n = 1$

ถ้า $|A| = 0$ หรือ $|A| = 1$, algorithm คืนค่า A ทันที ซึ่งย่อม sorted อยู่แล้ว และแน่นอนว่ามีสมาชิกเท่ากับ A เอง ดังนั้น $P(0)$ และ $P(1)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 1$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k+1)$ เป็นจริง

พิจารณาลิสต์ A ที่มี

$$|A| = k + 1.$$

เพราะ $k + 1 > 1$, algorithm จะไม่หยุดที่กรณีฐาน แต่จะแบ่ง A เป็นลิสต์ย่อย L และ R โดยมี

$$|L| = \lfloor (k+1)/2 \rfloor, \quad |R| = \lceil (k+1)/2 \rceil.$$

ดังนั้นทั้ง L และ R มีขนาดไม่เกิน k

จากสมมติฐานขั้นอุปนัย

$$\text{MergeSort}(L)$$

เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ L แบบนับซ้ำ และ

$$\text{MergeSort}(R)$$

เป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับ R แบบนับซ้ำ

ตอนนี้ algorithm คืนค่า

$$\text{Merge}(\text{MergeSort}(L), \text{MergeSort}(R)).$$

แต่จากทฤษฎีบทความถูกต้องของ Merge เมื่อเอาลิสต์ sorted สองลิสต์มา merge กัน ผลลัพธ์จะยังเป็นลิสต์ที่ sorted และมีสมาชิกเท่ากับสมาชิกของลิสต์ทั้งสองรวมกันแบบนับซ้ำ

ดังนั้นผลลัพธ์สุดท้ายของ $\text{MergeSort}(A)$ sorted และมีสมาชิกเท่ากับ L กับ R รวมกันแบบนับซ้ำ ซึ่งก็คือสมาชิกของ A เดิมทั้งหมด

จึงได้ว่า $P(k + 1)$ เป็นจริง สรุปด้วย strong induction ว่า MergeSort ถูกต้อง $\square$

7.5 Recursive Binary Search

หลังจากเห็นตัวอย่างของ recursive correctness proof ใน MergeSort แล้ว เราจะดูตัวอย่างอีกแบบหนึ่งที่โครงของ recursive call ไม่ได้เกิดจากการแบ่ง input ออกเป็นสองส่วนแล้วรวมคำตอบกลับ แต่เกิดจากการเลือกเพียงครั้งเดียว ของปัญหาเดิมเพื่อเรียกต่อ นั่นคือ Binary Search

7.5.1 นิยามและสเปก

สมมติว่า

$$A[1], A[2], \dots, A[n]$$

เป็นลิสต์ที่ sorted แบบไม่ลดลง กล่าวคือ

$$A[1] \leq A[2] \leq \dots \leq A[n].$$

เราจะเขียน $A[\ell..r]$ เพื่อแทนช่วงดัชนีตั้งแต่ ℓ ถึง r และยอมให้กรณี $\ell > r$ เป็นช่วงว่าง

นิยามฟังก์ชัน

$RBSearch(A, x, \ell, r)$

เพื่อค้นหา x ภายในช่วง $A[\ell..r]$

Spec ของ Binary Search

กำหนด input เป็นลิสต์ sorted A , ค่าที่ต้องการหา x , และดัชนี ℓ, r ที่ทำให้

$$1 \leq \ell \leq r + 1 \leq |A| + 1.$$

ให้

$$out = RBSearch(A, x, \ell, r).$$

ผลลัพธ์ที่ถูกต้องต้องเป็นอย่างใดอย่างหนึ่ง:

1. $out = 0$ และไม่มีดัชนี i ในช่วง $\ell \leq i \leq r$ ที่ทำให้ $A[i] = x$
2. $out \neq 0$ และ $A[out] = x$ โดยมี $\ell \leq out \leq r$

สังเกตว่า spec นี้ครอบคลุมทั้งกรณี ``เจอ" และ ``ไม่เจอ" ดังนั้นเวลาพิสูจน์ correctness induction hypothesis ก็ต้องถูกเขียนให้ครอบคลุมทั้งสองกรณีด้วยเช่นกัน

7.5.2 Pseudocode

Pseudocode: Recursive Binary Search

$RBSearch(A, x, \ell, r)$:

```
if  $\ell > r$  then return 0
 $m \leftarrow \lfloor (\ell + r) / 2 \rfloor$ 
if  $A[m] = x$  then return  $m$ 
if  $x < A[m]$  then return  $RBSearch(A, x, \ell, m - 1)$ 
else return  $RBSearch(A, x, m + 1, r)$ 
```

7.5.3 Termination

Claim 7.5.1 (Termination of $RBSearch$). สำหรับทุก input ที่อยู่ในโดเมนข้างต้น ฟังก์ชัน $RBSearch(A, x, \ell, r)$ จะหยุดทำงาน

Proof idea. วัดขนาดปัญหาด้วยความยาวช่วงค้นหา

$$n = r - \ell + 1.$$

ถ้าช่วงว่างแล้วก็หยุดทันที ถ้ายังไม่ว่าง recursive call ครั้งถัดไปจะไปยังช่วงที่สั้นกว่าช่วงเดิมจริง

Proof. พิจารณา

$$n = r - \ell + 1.$$

ถ้า $\ell > r$, algorithm เข้ากรณีฐานและคืนค่า 0 ทันที จึงหยุดทำงาน

ถ้า $\ell \leq r$, algorithm คำนวณ

$$m = \left\lfloor \frac{\ell + r}{2} \right\rfloor.$$

ถ้า $A[m] = x$ ก็หยุดทันทีเช่นกัน

ถ้าไม่เจอ จะเกิด recursive call เพียงกรณีใดกรณีหนึ่ง:

$$RBSearch(A, x, \ell, m - 1) \quad \text{หรือ} \quad RBSearch(A, x, m + 1, r).$$

ในกรณีแรก ความยาวช่วงใหม่คือ

$$(m - 1) - \ell + 1 = m - \ell < n.$$

ในกรณีที่สอง ความยาวช่วงใหม่คือ

$$r - (m + 1) + 1 = r - m < n.$$

ดังนั้นทุกครั้งที่เกิด recursive call ขนาดปัญหาจะลดลงจริง และเพราะขนาดปัญหาเป็นจำนวนเต็มไม่ลบ จึงไม่สามารถลดลงต่อไปเรื่อย ๆ ได้ไม่สิ้นสุด ท้ายที่สุด algorithm ต้องเข้าสู่กรณีฐานและหยุดทำงาน $\square$

7.5.4 Correctness

Theorem 7.5.2 (Correctness of RBSearch). ให้ A เป็นลิสต์ sorted และให้

$$out = RBSearch(A, x, \ell, r)$$

โดยที่

$$1 \leq \ell \leq r + 1 \leq |A| + 1.$$

แล้ว *out* ตรงตาม spec ของ Binary Search

Proof idea. พิสูจน์ด้วย strong induction บนความยาวช่วง

$$n = r - \ell + 1.$$

เหตุผลสำคัญคือ เราต้องการสมมติว่า recursive call ทุกตัวที่เกิดกับปัญหาที่สั้นกว่าเดิมทำงานถูกต้อง แล้วใช้ข้อเท็จจริงเรื่อง sortedness เพื่อแสดงว่าครั้งที่เราโยนทิ้งไปนั้นไม่มีคำตอบจริง ๆ

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ sorted A , ทุกค่า x , และทุกดัชนี ℓ, r ที่ทำให้

$$1 \leq \ell \leq r + 1 \leq |A| + 1 \quad \text{และ} \quad r - \ell + 1 = n,$$

ถ้าเรียก

$$out = RBSearch(A, x, \ell, r),$$

แล้ว *out* ตรงตาม spec ของ Binary Search

เราจะพิสูจน์ว่า $P(n)$ จริงสำหรับทุก $n \geq 0$

ขั้นฐาน: $n = 0$

เมื่อ $r - \ell + 1 = 0$ จะได้ว่า $\ell = r + 1$ ดังนั้นช่วง $[\ell..r]$ เป็นช่วงว่าง algorithm คืนค่า 0 ทันที และแน่นอนว่าไม่มีดัชนี i โดยอยู่ในช่วงนี้ที่ทำให้ $A[i] = x$ ดังนั้น output ตรงตาม spec จึงได้ว่า $P(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$P(0), P(1), \dots, P(k)$$

เป็นจริงทั้งหมด เราต้องพิสูจน์ว่า $P(k + 1)$ เป็นจริง

พิจารณา input ใด ๆ ที่มี

$$r - \ell + 1 = k + 1.$$

ดังนั้น $\ell \leq r$ และช่วงไม่ว่าง algorithm คำนวณ

$$m = \left\lfloor \frac{\ell + r}{2} \right\rfloor.$$

พิจารณาเป็นสามกรณี

กรณีที่ 1: $A[m] = x$

algorithm คืนค่า m ทันที และชัดเจนว่า

$$\ell \leq m \leq r, \quad A[m] = x.$$

ดังนั้น output ตรงตามข้อ (2) ของ spec

กรณีที่ 2: $x < A[m]$

algorithm เรียก

$$RBSearch(A, x, \ell, m - 1).$$

ช่วงใหม่มีความยาว

$$m - \ell < k + 1,$$

ดังนั้นใช้สมมติฐานขั้นอุปนัยได้

แต่เพียงแค่ว่า recursive call ถูกต้องในช่วง $[\ell..m - 1]$ ยังไม่พอ เราต้องอธิบายเพิ่มว่า ในช่วง $[m..r]$ ไม่มีทางมีสมาชิกเท่ากับ x

เพราะ A sorted และ $x < A[m]$, สำหรับทุก j ที่ $m \leq j \leq r$ จะมี

$$A[j] \geq A[m] > x.$$

ดังนั้นในช่วง $[m..r]$ ไม่มีสมาชิกเท่ากับ x

เพราะฉะนั้น:

- ถ้า recursive call คืนค่า 0, ก็แปลว่าในช่วง $[\ell..m - 1]$ ไม่มีสมาชิกเท่ากับ x เมื่อรวมกับการที่ช่วง $[m..r]$ ก็ไม่มี x , จะได้ว่าในช่วงเดิม $[\ell..r]$ ไม่มี x ดังนั้น output ตรงตามข้อ (1) ของ spec
- ถ้า recursive call คืนดัชนี $out \neq 0$, จากสมมติฐานขั้นอุปนัยจะมี

$$\ell \leq out \leq m - 1 \leq r$$

และ

$$A[out] = x.$$

ดังนั้น output ตรงตามข้อ (2) ของ spec

กรณีที่ 3: $x > A[m]$

algorithm เรียก

$$RBSearch(A, x, m + 1, r).$$

ช่วงใหม่มีความยาว

$$r - m < k + 1,$$

จึงใช้สมมติฐานขั้นอุปนัยได้อีกครั้ง

และเพราะ A sorted กับ $x > A[m]$, สำหรับทุก j ที่ $\ell \leq j \leq m$ จะมี

$$A[j] \leq A[m] < x.$$

ดังนั้นช่วง $[\ell..m]$ ไม่มีสมาชิกเท่ากับ x

จากนี้ให้เหตุผลแบบเดียวกับกรณีก่อนหน้า: ถ้า recursive call ไม่พบ x ในช่วง $[m + 1..r]$, ก็ไม่มี x ในช่วงเดิมทั้งหมด และถ้า recursive call ค้นดัชนีหนึ่งตำแหน่ง ดัชนีนั้นก็อยู่ในช่วงเดิมและทำให้ $A[out] = x$

ดังนั้นในทุกกรณี $P(k + 1)$ เป็นจริง สรุปด้วย strong induction ว่า $RBSearch$ ถูกต้อง $\square$

7.6 ตัวอย่าง recursive algorithm เพิ่มเติม

ต่อไปเราจะดู recursive algorithm ที่เรียบง่ายกว่า เพื่อให้เห็นแพตเทิร์นของ correctness proof ชัดขึ้น แม้โครงของมันจะไม่ซับซ้อนเท่า MergeSort หรือ Binary Search

7.6.1 Power

นิยาม

นิยามฟังก์ชัน Power แบบ recursion โดย

$$Power(a, 0) = 1, \quad Power(a, n) = a \cdot Power(a, n - 1) \quad (n \geq 1).$$

Pseudocode: Power

```
Power( $a, n$ ):  
  if  $n = 0$  then return 1  
  else return  $a \cdot \text{Power}(a, n - 1)$ 
```

Correctness

Claim 7.6.1 (Correctness of Power). สำหรับทุก $a \in \mathbb{R}$ และทุก $n \in \mathbb{N}$,

$$\text{Power}(a, n) = a^n.$$

Proof. ให้

$$P(n) : \text{Power}(a, n) = a^n$$

สำหรับค่าคงที่ $a \in \mathbb{R}$

ขั้นฐาน: เมื่อ $n = 0$, algorithm คืนค่า 1, จึงได้

$$\text{Power}(a, 0) = 1 = a^0.$$

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$\text{Power}(a, k) = a^k.$$

เราต้องพิสูจน์ว่า

$$\text{Power}(a, k + 1) = a^{k+1}.$$

จาก algorithm,

$$\text{Power}(a, k + 1) = a \cdot \text{Power}(a, k).$$

แทนด้วยสมมติฐานขั้นอุปนัย จะได้

$$\text{Power}(a, k + 1) = a \cdot a^k = a^{k+1}.$$

จึงพิสูจน์ได้ครบ

□

7.6.2 SumList

นิยาม

ให้ลิสต์

$$A = (a_1, a_2, \dots, a_n)$$

และให้ $\text{tail}(A)$ หมายถึงลิสต์ที่เหลือหลังตัดสมาชิกตัวแรกออก

นิยามฟังก์ชัน SumList แบบ recursion โดย

$$\text{SumList}([]) = 0, \quad \text{SumList}([a_1, \dots, a_n]) = a_1 + \text{SumList}([a_2, \dots, a_n]).$$

Pseudocode: SumList

```
SumList(A):  
  if  $|A| = 0$  then return 0  
  else return  $A[1] + \text{SumList}(\text{tail}(A))$ 
```

Correctness

Claim 7.6.2 (Correctness of SumList). สำหรับทุกลิสต์จำนวนจริง

$$A = (a_1, \dots, a_n),$$

มี

$$\text{SumList}(A) = a_1 + a_2 + \dots + a_n,$$

และถ้า A เป็นลิสต์ว่าง ให้ผลรวมเป็น 0

Proof. ให้

$$P(n)$$

แทนข้อความว่า:

สำหรับทุกลิสต์ A ที่มีความยาว n , ค่าของ $\text{SumList}(A)$ เท่ากับผลรวมของสมาชิกทุกตัวใน A

ขั้นฐาน: $n = 0$

ลิสต์ต้องเป็นลิสต์ว่าง algorithm คืนค่า 0 แทนที่ ซึ่งตรงกับนิยามของผลรวมของลิสต์ว่าง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า $P(k)$ เป็นจริง พิจารณาลิสต์ A ที่มีความยาว $k + 1$ เขียนได้เป็น

$$A = [a_1, a_2, \dots, a_{k+1}].$$

ดังนั้น

$$\text{tail}(A) = [a_2, \dots, a_{k+1}]$$

มีความยาว k

จาก algorithm,

$$\text{SumList}(A) = a_1 + \text{SumList}(\text{tail}(A)).$$

จากสมมติฐานขั้นอุปนัย,

$$\text{SumList}(\text{tail}(A)) = a_2 + \dots + a_{k+1}.$$

จึงได้

$$\text{SumList}(A) = a_1 + (a_2 + \dots + a_{k+1}) = a_1 + a_2 + \dots + a_{k+1}.$$

ดังนั้น $P(k + 1)$ เป็นจริง

□

7.7 Pattern Summary

เมื่อมองทั้ง MergeSort, Binary Search, Power, และ SumList ร่วมกัน จะเห็นแพตเทิร์นกลางของ recursive correctness proof ซ้ำขึ้นมาก

Pattern ของ Recursive Correctness Proof

เมื่อต้องการพิสูจน์ความถูกต้องของ recursive algorithm ให้ถามตัวเองเป็นลำดับดังนี้

1. **Input** ที่อนุญาตคืออะไร
ต้องกำหนดโดเมนให้ชัดก่อน
2. **Output** ที่ถูกต้องต้องเป็นแบบไหน
นี่คือ spec
3. **จะวัดขนาดปัญหาด้วยอะไร**
เช่น ความยาวลิสต์, ความยาวช่วง, หรือจำนวนสมาชิก
4. **recursive call** ไปสู่ปัญหาที่เล็กลงจริงหรือไม่
นี่คือหัวใจของ termination
5. **ถ้าสมมติว่า recursive call ถูกต้องแล้ว**
อัลกอริทึมใช้ผลย่อยเหล่านั้นอย่างไรเพื่อสร้างคำตอบของปัญหาเดิม
6. **ถ้ามีขั้น combine**
ต้องพิสูจน์ให้ชัดว่าขั้น combine นั้นไม่ทำลายความถูกต้อง

กล่าวอีกแบบหนึ่ง ในบทอุปนัย เราพิสูจน์ว่า ``ข้อความ" เป็นจริงสำหรับทุกขนาด แต่ในบทนี้ เราใช้แนวคิดเดียวกันเพื่อพิสูจน์ว่า ``อัลกอริทึม" ทำงานถูกต้องสำหรับทุกขนาดของ input

แบบฝึกหัดท้ายบท

Exercise 7.7.1. จงเขียนพิสูจน์ termination ของ *RBSearch* โดยไม่ใช้อุปนัย แต่ใช้เหตุผลว่าความยาวช่วงค้นหาลดลงทุกครั้งจนต้องเข้าสู่กรณีฐาน

Exercise 7.7.2. ในพิสูจน์ correctness ของ *RBSearch*, อธิบายให้ละเอียดว่า sortedness ถูกใช้ตรงไหน และถ้าเอาเงื่อนไข sorted ออกไป ส่วนใดของบทพิสูจน์จะพังทันที

Exercise 7.7.3. จงพิสูจน์ correctness ของฟังก์ชัน

$$\text{Len}([]) = 0, \quad \text{Len}([a_1, \dots, a_n]) = 1 + \text{Len}([a_2, \dots, a_n]).$$

กล่าวคือพิสูจน์ว่า $\text{Len}(A) = |A|$ สำหรับทุกลิสต์ A

Exercise 7.7.4. พิจารณาข้อความต่อไปนี้:

“ถ้า recursive algorithm มี base case ถูกต้อง และ recursive call ถูกต้อง ก็เพียงพอแล้วที่จะสรุปว่าอัลกอริทึมทั้งหมดถูกต้อง”

จงอธิบายว่าข้อความนี้จริงหรือเท็จ พร้อมยกเหตุผลจากตัวอย่างในบทนี้ว่าเป็นระบบ

บทที่ 8

Recurrent Relation

8.1 Introduction to Recurrence Relations

ในหลายปัญหาของคณิตศาสตร์ไม่ต่อเนื่องและอัลกอริทึม เรามักไม่ได้คำตอบออกมาเป็นสูตรปิดตั้งแต่ต้น แต่ได้ความสัมพันธ์ที่บอกว่า ``คำตอบของปัญหขนาดปัจจุบัน" เขียนในรูปของ ``คำตอบของปัญหขนาดเล็กลง" ได้ อย่างไร ความสัมพันธ์ลักษณะนี้เรียกว่า *recurrence relation*

Definition 8.1.1 (recurrence relation). สมการที่นิยามพจน์ $T(n)$ โดยอ้างอิงพจน์ก่อนหน้า เช่น

$$T(n) = T(n-1) + 2n - 1 \quad \text{หรือ} \quad T(n) = 2T(n/2) + n - 1.$$

แนวคิดสำคัญคือ *recurrence relation* ยังไม่ใช่คำตอบสุดท้าย แต่เป็นการบอกโครงสร้างการเกิดของคำตอบ ดังนั้นเมื่อเห็น *recurrence* สิ่งที่เราสนใจต่อมาก็คือคำถาม 4 ข้อ: ข้อความนี้แกะออกอย่างไร, มีรูปแบบซ่อนอยู่หรือไม่, เค้า *closed form* ได้ไหม, และเมื่อเค้าได้แล้วจะพิสูจน์อย่างไรให้ถูกต้อง

เป้าหมายของส่วนนี้

- แกะ *recurrence* ด้วยการแทนค่าถอยกลับ
- มองหา *pattern*
- เค้า *closed form*
- เตรียมไปพิสูจน์ *closed form* ด้วย *induction*

8.1.1 ตัวอย่างที่ 1: recurrence แบบง่าย

เราจะเริ่มจากตัวอย่างที่ง่ายที่สุดก่อน เพื่อให้เห็นกลไกของการ ``คลี่ recurrence" โดยไม่ต้องมีรายละเอียดของอัลกอริทึมเข้ามาปะปน

Example 8.1.2. กำหนดให้

$$T(0) = 0, \quad T(n) = T(n-1) + (2n-1) \quad (n \geq 1).$$

จงหา closed form ของ $T(n)$

Proof idea. แทนค่าถอยกลับไปเรื่อย ๆ จนสุดที่ฐาน $T(0)$ แล้วสังเกตว่าพจน์ที่ถูกบวกเข้ามาคือ

$$1, 3, 5, \dots, 2n-1$$

ซึ่งเป็นผลบวกของจำนวนคี่ n พจน์แรก

Proof. เริ่มจาก recurrence:

$$T(n) = T(n-1) + (2n-1).$$

แทน $T(n-1)$ ด้วย recurrence เดิมอีกครั้ง จะได้

$$T(n) = (T(n-2) + (2(n-1)-1)) + (2n-1).$$

ดังนั้น

$$T(n) = T(n-2) + (2n-3) + (2n-1).$$

แทนต่ออีกหนึ่งครั้ง:

$$T(n) = T(n-3) + (2n-5) + (2n-3) + (2n-1).$$

ทำเช่นนี้ต่อไปเรื่อย ๆ จะเห็นรูปแบบว่า

$$T(n) = T(0) + 1 + 3 + 5 + \dots + (2n-1).$$

เพราะ $T(0) = 0$ จึงได้

$$T(n) = 1 + 3 + 5 + \dots + (2n-1).$$

แต่ผลบวกของจำนวนคือ n พจน์แรกเท่ากับ n^2 ดังนั้น

$$T(n) = n^2.$$

□

กล่าวอีกแบบหนึ่ง recurrence ข้อนี้อธิบายว่าค่า $T(n)$ เกิดจากการนำ $T(n-1)$ ไปบวก “ชั้นใหม่” ที่มีขนาด $2n-1$ และเมื่อรวมชั้นทั้งหมดตั้งแต่ 1 ถึง n จึงได้กำลังสองพอดี

Exercise 8.1.3. เมื่อเดา closed form ได้แล้ว จงพิสูจน์ด้วย induction ว่าสูตร

$$T(n) = n^2$$

ถูกต้องจริงสำหรับทุก $n \in \mathbb{N}$

8.1.2 ตัวอย่างที่ 2: Merge Sort recurrence (กรณี n เป็นกำลังของ 2)

ตัวอย่างถัดไปมาจากอัลกอริทึม และสำคัญมากเพราะเป็น recurrence ที่มีรูป “แก้ปัญหาย่อยสองอัน แล้วบวกงานที่ต้องทำตอนรวมคำตอบ”

ให้ $T(n)$ แทนจำนวน comparisons มากที่สุดของ Merge Sort โดยสมมติว่า n เป็นกำลังของ 2 จะได้ recurrence

$$T(1) = 0, \quad T(n) = 2T(n/2) + (n-1) \quad (n \geq 2).$$

ภาพรวมของ Merge Sort

ทบทวนแนวคิดของ Merge Sort:

1. ถ้าลิสต์มีขนาด 1 ก็ sorted อยู่แล้ว
2. ถ้าลิสต์มีขนาดมากกว่า 1 ให้แบ่งออกเป็นสองครึ่ง
3. เรียงแต่ละครึ่งแบบ recursion
4. นำสองลิสต์ที่ sorted แล้วมา merge กลับเข้าด้วยกัน

สิ่งสำคัญคือ recurrence นี้ไม่ได้เกิดจากการเดาสุ่ม แต่เกิดจากการอ่านโครงสร้างของอัลกอริทึมตรง ๆ: มี recursive call สองครั้งบนข้อมูลขนาด $n/2$ และมีค่าใช้จ่ายเพิ่มตอน merge อีกหนึ่งส่วน

Example 8.1.4 (Merge step). อธิบายว่าทำไมการ merge ลิสต์ sorted สองลิสต์ที่มีสมาชิกทั้งหมดรวมกัน n ตัว ใช้ comparisons มากที่สุดไม่เกิน $n-1$ ครั้ง และทำไม recurrence relation จึงเป็น

$$T(n) = 2T(n/2) + (n-1), \quad T(1) = 0.$$

Proof idea. ในขั้น merge แต่ละครั้ง เราดูหัวของสองลิสต์แล้วเลือกตัวที่น้อยกว่าออกมาใส่ผลลัพธ์ ทุก comparison ทำให้มีสมาชิกถูกปล่อยออกจากหัวลิสต์อย่างน้อยหนึ่งตัว และทันทีที่ลิสต์ใดลิสต์หนึ่งหมด สมาชิกที่เหลือของอีกลิสต์ไม่ต้องเปรียบเทียบอีกแล้ว จึงใช้ comparisons มากที่สุดไม่เกิน $n - 1$ ครั้ง

Proof. ให้ลิสต์ sorted สองลิสต์มีความยาวรวมกันเป็น n ทุกครั้งที่เปรียบเทียบหัวของสองลิสต์ เราจะย้ายสมาชิกออกไปสู่ลิสต์ผลลัพธ์หนึ่งตัว ดังนั้นจำนวนสมาชิกที่ยัง “ค้าง” อยู่จะลดลงทีละหนึ่ง

เมื่อใดก็ตามที่ลิสต์หนึ่งหมด สมาชิกทั้งหมดของอีกลิสต์สามารถต่อท้ายเข้าผลลัพธ์ได้ทันที โดยไม่ต้องเปรียบเทียบอีก ดังนั้นในกรณีแย่ที่สุด เราจะต้องเปรียบเทียบจนเหลือสมาชิกค้างอยู่เพียงหนึ่งตัวสุดท้าย ทำให้จำนวน comparisons มากที่สุดไม่เกิน $n - 1$

นี่สำหรับ Merge Sort บนลิสต์ขนาด n :

- recursive call ครั้งหนึ่ง ใช้มากที่สุด $T(n/2)$
- recursive call ครั้งที่สอง ใช้มากที่สุด $T(n/2)$
- ขั้น merge ใช้มากที่สุด $n - 1$

จึงได้

$$T(n) = 2T(n/2) + (n - 1)$$

และฐานคือ

$$T(1) = 0$$

เพราะลิสต์ขนาด 1 ไม่ต้องเปรียบเทียบเลย

□

8.1.3 Plug-and-chug สำหรับ Merge Sort

ตอนนี้เป้าหมายของเราไม่ใช่เพียงอ่าน recurrence ให้เป็น แต่ต้องคลี่มันออกเพื่อหารูปแบบของคำตอบ

Example 8.1.5. จงแทนค่าถอยกลับเพื่อหา pattern ของ $T(n)$:

$$T(n) = 2T(n/2) + (n - 1), \quad T(1) = 0$$

คำแนะนำ: ใช้ $n = 2^k$

Proof idea. ถ้าเขียน $n = 2^k$ จำนวนครั้งที่เราหารสองได้ก่อนถึง 1 คือ k ครั้งพอดี จึงทำให้รูปแบบของการแทนค่าชัดขึ้นมาก

Proof. ให้ $n = 2^k$ เริ่มจาก

$$T(n) = 2T(n/2) + (n - 1).$$

แทนค่าถอยกลับอีกครั้ง:

$$T(n) = 2 \left(2T(n/4) + \left(\frac{n}{2} - 1 \right) \right) + (n - 1) = 2^2 T(n/4) + n - 2 + n - 1.$$

จึงได้

$$T(n) = 2^2 T(n/4) + 2n - 3.$$

แทนต่อไปอีกหนึ่งขั้น:

$$T(n) = 2^3 T(n/8) + 3n - 7.$$

จะเห็น pattern ว่าหลังแทนไป i ขั้น ได้

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in - (2^i - 1).$$

เมื่อ $n = 2^k$ และเลือก $i = k$ จะได้ว่า

$$\frac{n}{2^k} = 1, \quad 2^k = n.$$

ดังนั้น

$$T(n) = 2^k T(1) + kn - (2^k - 1).$$

ใช้ฐาน $T(1) = 0$ และ $2^k = n$ จึงได้

$$T(n) = kn - n + 1.$$

เมื่อ $k = \log_2 n$ จะได้ closed form เป็น

$$T(n) = n \log_2 n - n + 1.$$

ดังนั้น Merge Sort มีการเติบโตระดับ

$$T(n) = \Theta(n \log n).$$

□

Example 8.1.6. จงพิสูจน์ closed form ที่ได้ด้วย induction

Proof idea. เพราะเราเดาสูตรจากกรณี $n = 2^k$ การพิสูจน์ที่เป็นธรรมชาติที่สุดคือพิสูจน์บนตัวแปร k โดยกำหนด

$$S(k) : T(2^k) = k2^k - 2^k + 1.$$

Proof. ให้

$$S(k) : T(2^k) = k2^k - 2^k + 1.$$

ขั้นฐาน: เมื่อ $k = 0$,

$$T(2^0) = T(1) = 0$$

และด้านขวาเป็น

$$0 \cdot 2^0 - 2^0 + 1 = 0 - 1 + 1 = 0.$$

ดังนั้น $S(0)$ เป็นจริง

ขั้นอุปนัย: ให้ $k \geq 0$ และสมมติว่า

$$T(2^k) = k2^k - 2^k + 1.$$

ต้องการพิสูจน์ว่า

$$T(2^{k+1}) = (k+1)2^{k+1} - 2^{k+1} + 1.$$

จาก recurrence,

$$T(2^{k+1}) = 2T(2^k) + (2^{k+1} - 1).$$

แทนสมมติฐานขั้นอุปนัยลงไป:

$$T(2^{k+1}) = 2(k2^k - 2^k + 1) + 2^{k+1} - 1.$$

จัดรูปได้เป็น

$$T(2^{k+1}) = k2^{k+1} - 2^{k+1} + 2 + 2^{k+1} - 1 = k2^{k+1} + 1.$$

และ

$$(k+1)2^{k+1} - 2^{k+1} + 1 = k2^{k+1} + 2^{k+1} - 2^{k+1} + 1 = k2^{k+1} + 1.$$

ดังนั้น

$$T(2^{k+1}) = (k+1)2^{k+1} - 2^{k+1} + 1.$$

จึงได้ว่า $S(k+1)$ เป็นจริง

สรุปด้วย induction ว่า

$$T(2^k) = k2^k - 2^k + 1$$

สำหรับทุก $k \geq 0$ หรือเท่ากับ

$$T(n) = n \log_2 n - n + 1$$

เมื่อ n เป็นกำลังของ 2

□

8.2 Binary Search recurrence

ตัวอย่างก่อนหน้าของ Merge Sort มีลักษณะว่า แยกปัญหาออกเป็นสองส่วนย่อย แก่ทั้งสองส่วน แล้วค่อยรวมคำตอบกลับ แต่ Binary Search ต่างออกไป: มันยังแบ่งปัญหาเหมือนกัน แต่หลังจากเปรียบเทียบกับค่ากลางแล้วเราทิ้งไปได้ครึ่งหนึ่งทันที ดังนั้น recursive call จะเหลือเพียงด้านเดียว

Pseudocode: Recursive Binary Search

```
RBSearch( $A, x, \ell, r$ ):  
    if  $\ell > r$  then return 0  
     $m \leftarrow \lfloor (\ell + r)/2 \rfloor$   
    if  $A[m] = x$  then return  $m$   
    if  $x < A[m]$  then return RBSearch( $A, x, \ell, m - 1$ )  
    else return RBSearch( $A, x, m + 1, r$ )
```

Example 8.2.1. หา 5 จาก sorted array $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]$

จากตัวอย่างนี้จะเห็นว่า ทุกครั้งที่เปรียบเทียบกับค่ากลาง เราตัดทิ้งสมาชิกไปประมาณครึ่งหนึ่ง จึงเป็นธรรมชาติที่จะได้ recurrence ที่เกี่ยวกับ $n/2$

8.2.1 การวิเคราะห์

“ในกรณีแย่ที่สุด Binary Search ใช้การเปรียบเทียบกี่ครั้ง?”

ตั้งฟังก์ชันต้นทุน

ให้ $B(n)$ แทนจำนวน comparisons มากที่สุดที่ Binary Search ใช้กับลิสต์ sorted ขนาด n เพื่อให้ง่าย เราจะเริ่มจากกรณีที่ n เป็นกำลังของ 2 และใช้วิธีตัดแบ่งครึ่งเลย

Example 8.2.2. เพราะเหตุใด recurrence ของ Binary Search จึงควรเป็น

$$B(1) = 1, \quad B(n) = B(n/2) + 1 \quad (n \geq 2)?$$

Proof idea. ในกรณีแย่ที่สุด เราเปรียบเทียบกับสมาชิกตรงกลางก่อนหนึ่งครั้ง แล้วถ้ายังไม่พบ จะเหลือปัญหาย่อยเพียงครึ่งเดียวให้ทำต่อ

Proof. เมื่อลิสต์มีขนาด 1 ในกรณีแย่ที่สุดเราต้องเปรียบเทียบกับสมาชิกตัวนั้นหนึ่งครั้ง จึงได้

$$B(1) = 1.$$

สำหรับ $n \geq 2$, Binary Search จะเลือกค่ากลางขึ้นมาเปรียบเทียบกับก่อนหนึ่งครั้ง ถ้ายังไม่พบคำตอบ เพราะลิสต์ถูกเรียงไว้แล้ว เราารู้ทันทีว่าควรไปค้นต่อเพียงฝั่งซ้ายหรือฝั่งขวา ซึ่งมีขนาดประมาณ $n/2$

ดังนั้นต้นทุนในกรณีแย่ที่สุดจึงเท่ากับ

ต้นทุนของปัญหาย่อยขนาด $n/2$ + comparison ปัจจุบันอีก 1 ครั้ง

หรือเขียนเป็น

$$B(n) = B(n/2) + 1.$$

□

8.2.2 Plug-and-chug

Example 8.2.3. จงแทนค่าถอยกลับ:

$$B(n) = B(n/2) + 1.$$

Proof idea. เนื่องจากแต่ละชั้นหารขนาดปัญหาด้วย 2 จำนวนครั้งที่ทำได้ก่อนถึง 1 จึงควรสัมพันธ์กับ $\log_2 n$

Proof. เริ่มจาก

$$B(n) = B(n/2) + 1.$$

แทนค่าต่อ:

$$B(n) = B(n/4) + 2.$$

อีกหนึ่งขั้น:

$$B(n) = B(n/8) + 3.$$

จึงเห็น pattern ว่าหลังแทนไป i ชั้น

$$B(n) = B\left(\frac{n}{2^i}\right) + i.$$

เมื่อ $n = 2^k$ และเลือก $i = k$ จะได้

$$\frac{n}{2^k} = 1.$$

ดังนั้น

$$B(n) = B(1) + k = 1 + k.$$

และเพราะ $k = \log_2 n$ จึงได้ว่า

$$B(n) = \log_2 n + 1.$$

ดังนั้น Binary Search โตรระดับ

$$B(n) = \Theta(\log n).$$

สิ่งที่ควรได้

ถ้า $n = 2^k$ จะได้

$$B(n) = \log_2 n + 1.$$

ดังนั้น Binary Search โตรระดับ

$$B(n) = \Theta(\log n).$$

Exercise 8.2.4. เปรียบเทียบ Merge Sort กับ Binary Search:

1. เหตุใดทั้งสองตัวอย่างจึงเป็น **recursion** เหมือนกัน แต่ได้อัตราการเติบโตไม่เหมือนกัน
2. ใน recurrence ของ Merge Sort มีส่วนใดที่ "แพงเพิ่ม" เมื่อเทียบกับ Binary Search

8.3 From Recurrence to Closed Form and Back Again

ถึงจุดนี้ควรเห็นลำดับการทำงานของหัวข้อนี้ชัดเจนแล้ว:

1. เริ่มจากปัญหาหรืออัลกอริทึม
2. อ่านโครงสร้างของมันให้ออก แล้วเขียนเป็น recurrence
3. แทนค่าถอยกลับเพื่อหา pattern
4. เคา closed form
5. ใช้ induction พิสูจน์ว่าสูตรที่ได้นั้นถูกต้องจริง

กล่าวอีกแบบหนึ่ง recurrence relation เป็นเหมือน "ภาษากลาง" ระหว่างตัวปัญหากับสูตรคำตอบ มันช่วยให้เราเห็นว่าคำตอบเกิดขึ้นอย่างไรที่ละขั้น ก่อนจะสรุปออกมาเป็นสูตรที่อ่านง่ายกว่าในตอนท้าย

ข้อสรุปของบทนี้

recurrence relation ไม่ได้มีเป้าหมายเพียงให้เราแทนค่าถอยกลับได้ แต่มีเป้าหมายให้เราเห็นว่า

- สูตรเกิดจากโครงสร้างของปัญหาอย่างไร
- closed form ที่เราได้ควรพิสูจน์กลับด้วย induction เสมอ
- recurrence คนละแบบ อาจให้การเติบโตต่างกันมาก

แม้จะดูคล้ายกันว่า ``เรียกตัวเองกับปัญหาที่เล็กลง"

ภาค IV

ทฤษฎีกราฟเบื้องต้น

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 9

ทฤษฎีกราฟเบื้องต้น

ในช่วงก่อนหน้านี้ของหนังสือ เราได้ฝึกมองคณิตศาสตร์ผ่านภาษาของเซต ความสัมพันธ์ ฟังก์ชัน การนิยามแบบเวียนเกิด และการพิสูจน์ด้วยวิธีต่าง ๆ เมื่อเครื่องมือเหล่านี้เริ่มชัดเจนแล้ว ก็ถึงเวลาหันมาศึกษาโครงสร้างชนิดหนึ่งที่สำคัญมากทั้งในคณิตศาสตร์ดิสครีตและวิทยาการคอมพิวเตอร์ นั่นคือ *กราฟ*

เหตุผลที่กราฟสำคัญมากก็เพราะมันเป็นภาษาที่เรียบง่ายแต่ทรงพลังสำหรับอธิบาย *ความเชื่อมโยง* ระหว่างสิ่งต่าง ๆ ในหลายปัญหา เราไม่ได้สนใจเพียงว่าวัตถุมีอะไรอยู่บ้าง แต่สนใจด้วยว่าวัตถุเหล่านั้นเชื่อมถึงกันอย่างไร ตัวอย่างเช่น เครือข่ายคอมพิวเตอร์ เส้นทางคมนาคม ความสัมพันธ์ระหว่างรายวิชาที่เป็นวิชาบังคับก่อน โครงสร้างของลิงก์ในเว็บเพจ หรือแม้แต่สถานะและการเปลี่ยนสถานะของระบบบางชนิด ล้วนสามารถถูกทำให้เรียบลงมาอยู่ในรูปของกราฟได้

สิ่งที่ควรสังเกตคือ กราฟไม่ใช่เพียง “รูปที่มีจุดกับเส้น” แต่เป็นวัตถุทางคณิตศาสตร์ที่มีนิยามชัดเจน และเมื่อนิยามชัดเจนแล้ว เราก็สามารถพิสูจน์คุณสมบัติต่าง ๆ ของมันได้อย่างเป็นระบบ ในบทนี้ เราจะเริ่มจากกราฟแบบง่ายที่สุดก่อน คือกราฟไม่มีทิศทางและไม่มีเส้นซ้ำ เพื่อสร้างภาษาพื้นฐานให้มั่นคง จากนั้นจึงค่อยขยับไปสู่แนวคิดที่ลึกขึ้นในส่วนถัดไปของบท

9.1 กราฟอย่างง่ายและภาษาพื้นฐาน

เมื่อเริ่มเรียนกราฟ สิ่งสำคัญที่สุดไม่ใช่การจำศัพท์ให้ได้เร็วที่สุด แต่คือการมองให้ออกว่า กราฟเป็นแบบจำลองของ “สิ่งของ” และ “ความเชื่อมโยงระหว่างสิ่งของ” ในระดับนามธรรม เรามักไม่สนใจรูปร่างของการวาดบนกระดาษมากนัก แต่สนใจว่าใครบ้างเป็นจุดยอด และมีเส้นเชื่อมระหว่างจุดยอดคู่ใดบ้าง

กล่าวอีกแบบหนึ่ง กราฟเป็นภาษาสำหรับบอกว่า

มีวัตถุอะไรอยู่ในระบบนี้บ้าง และวัตถุใดเชื่อมโยงกับวัตถุใด

แนวคิดนี้เรียบง่ายมาก แต่เพียงพอสำหรับสร้างทฤษฎีที่กว้างและลึกอย่างยิ่ง ดังนั้นในส่วนแรกของบท เราจะเริ่มจากนิยามพื้นฐานที่สุดก่อน แล้วค่อยดูว่าจากนิยามเพียงไม่กี่ข้อ เราสามารถให้เหตุผลเชิงพิสูจน์ได้มากเพียงใด

9.1.1 Simple graph, vertex, edge

กราฟที่เราจะเริ่มต้นศึกษาก่อนคือ **simple graph** ซึ่งเป็นแบบที่ง่ายที่สุดและเหมาะสมมากสำหรับวางรากฐานของภาษาในบทนี้ คำว่า “simple” มีความหมายเชิงเทคนิคว่า เราจะยังไม่อนุญาตสิ่งที่ทำให้ภาพซับซ้อนเกินความจำเป็น เช่นเส้นเชื่อมจากจุดยอดกลับมาหาตัวเอง หรือเส้นเชื่อมซ้ำหลายเส้นระหว่างจุดยอดคู่เดียวกัน

Definition 9.1.1 (simple graph). กราฟอย่างง่าย หรือ **simple graph** คือคู่อันดับ

$$G = (V, E)$$

โดยที่

- V เป็นเซตของ จุดยอด (vertices)
- E เป็นเซตของ เส้นเชื่อม (edges)

และเส้นเชื่อมแต่ละเส้นเป็นเซตที่มีจุดยอดสองตัวที่ต่างกันเป็นสมาชิก กล่าวคือ

$$E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}.$$

นิยามนี้ควรอ่านอย่างช้า ๆ เพราะมีรายละเอียดหลายอย่างซ่อนอยู่

อย่างแรก เส้นเชื่อมใน simple graph ถูกเขียนในรูป $\{u, v\}$ ซึ่งเป็นเซตไม่ใช่คู่อันดับ ดังนั้น $\{u, v\} = \{v, u\}$ ความหมายคือเส้นเชื่อมในกราฟชนิดนี้ ไม่มีทิศทาง การเชื่อมระหว่าง u กับ v จึงเป็นเรื่องเดียวกับการเชื่อมระหว่าง v กับ u

อย่างที่สอง เงื่อนไข $u \neq v$ บอกว่าเราไม่อนุญาต self-loop หรือเส้นที่เชื่อมจุดยอดเข้ากับตัวเอง และเนื่องจาก E เป็น เซต ของเส้นเชื่อม จึงไม่สามารถมีเส้นซ้ำระหว่างจุดยอดคู่ใดได้ นี่คือเหตุผลที่เราเรียกมันว่า simple graph

Example 9.1.2. ให้

$$V = \{a, b, c, d, e\}$$

และ

$$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}, \{d, e\}\}.$$

แล้ว $G = (V, E)$ เป็น simple graph

เรามองกราฟนี้ในเชิงรูปภาพ เรามักวาดจุดยอด a, b, c, d, e เป็นจุด แล้วลากเส้นเชื่อมตามสมาชิกของ E

อย่างไรก็ตาม สิ่งที่เราควรจำคือ รูปที่วาดเป็นเพียง ตัวแทนเชิงภาพ ไม่ใช่นิยามของกราฟเอง กราฟตัวเดียวกันอาจถูกวาดได้หลายแบบ トラバドที่ยังสะท้อนชุดของจุดยอดและเส้นเชื่อมเดิม

จุดนี้สำคัญมากสำหรับผู้เริ่มต้น เพราะบางครั้งนักศึกษาไม่รู้สึกรู้ว่ากราฟสองรูป ``ดูไม่เหมือนกัน" จึงคิดว่าเป็นคนละกราฟ ทั้งที่แท้จริงอาจต่างกันเพียงการจัดวางตำแหน่งบนหน้ากระดาษ แต่มีข้อมูลเชิงโครงสร้างเหมือนกันทุกประการ

เมื่อเริ่มคุ้นกับภาษาแล้ว เรามักพูดว่าเส้นเชื่อม $\{u, v\}$ *incident* กับจุดยอด u และ v หรือพูดว่าจุดยอด u และ v *adjacent* กัน ถ้ามีเส้นเชื่อม $\{u, v\} \in E$ ศัพท์เหล่านี้จะถูกใช้อยู่เสมอในหัวข้อถัด ๆ ไป จึงควรเริ่มคุ้นกับมันตั้งแต่นี้

9.1.2 ดีกรีของจุดยอด

เมื่อเรารู้แล้วว่ากราฟประกอบด้วยจุดยอดและเส้นเชื่อม คำถามพื้นฐานถัดไปคือ จุดยอดแต่ละตัวเชื่อมกับจุดยอดอื่นอยู่ที่เส้น แนวคิดนี้นำไปสู่คำว่า ดีกรี ของจุดยอด

Definition 9.1.3 (degree). ให้ $G = (V, E)$ เป็น simple graph และให้ $v \in V$ ดีกรีของจุดยอด v เขียนแทนด้วย $\deg(v)$ หมายถึงจำนวนเส้นเชื่อมทั้งหมดใน E ที่ incident กับ v

ถ้ามองอย่างไม่เป็นทางการ ดีกรีคือ ``จำนวนเพื่อนบ้าน" ของจุดยอดนั้น หรือจำนวนเส้นที่ออกจากจุดยอดนั้นในกราฟไม่มีทิศทางแบบง่าย ดังนั้นดีกรีเป็นตัวเลขที่สรุปข้อมูลเชิงท้องถิ่นของจุดยอดได้อย่างกะทัดรัดมาก

Example 9.1.4. พิจารณากราฟ

$$V = \{a, b, c, d, e\}, \quad E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}, \{d, e\}\}.$$

จะได้ว่า

$$\deg(a) = 2, \quad \deg(b) = 2, \quad \deg(c) = 3, \quad \deg(d) = 2, \quad \deg(e) = 1.$$

สิ่งที่ควรฝึกให้ชัดตั้งแต่ต้นคือ ดีกรีไม่ใช่จำนวนจุดยอดทั้งหมดในกราฟ แต่เป็นจำนวนเส้นเชื่อมที่ ``แตะ" จุดยอดนั้น ดังนั้นแม้กราฟจะมีจุดยอดจำนวนมาก บางจุดยอดก็อาจมีดีกรีต่ำมากได้ หรือแม้แต่มียดีกรีเป็นศูนย์ก็ได้ ในกรณีที่จุดยอดนั้นไม่มีเส้นเชื่อมกับใครเลย จุดยอดเช่นนี้มักเรียกว่า **isolated vertex**

แนวคิดเรื่องดีกรีดูเรียบง่าย แต่จะกลายเป็นเครื่องมือสำคัญมากในทฤษฎีกราฟ เพราะหลายทฤษฎีบทพื้นฐานเริ่มต้นจากการรวมดีกรีของจุดยอดทั้งหมดเข้าด้วยกัน แล้วตีความผลรวมนี้ในอีกมุมหนึ่ง ตัวอย่างแรกและสำคัญที่สุดคือ Handshake Lemma ซึ่งเราจะพิสูจน์ในหัวข้อถัดไป

9.1.3 Handshake Lemma และการนับแบบสองทาง

หนึ่งในหลักคิดที่ทรงพลังมากในคณิตศาสตร์ดิสครีตคือ การนับสิ่งเดียวกันสองวิธี หรือ *double counting* แนวคิดคือ ถ้ามีวัตถุหรือเหตุการณ์ชุดหนึ่งที่เราสงสัย เราสามารถนับจำนวนของมันได้สองมุมมอง แล้วตั้งสองคำตอบนั้นให้เท่ากัน วิธีคิดเช่นนี้มักให้ผลลัพธ์ที่ทั้งสวยและมีความหมายเชิงโครงสร้าง

ในกรณีของกราฟ สิ่งที่เราจะนับคือ “จำนวนปลายเส้นเชื่อมทั้งหมด” ฟังดูเป็นคำพูดง่าย ๆ แต่กลับนำไปสู่ทฤษฎีบทพื้นฐานที่สำคัญมาก

Theorem 9.1.5 (Handshake Lemma). ถ้า $G = (V, E)$ เป็น simple graph แล้ว

$$\sum_{v \in V} \deg(v) = 2|E|.$$

ชื่อ Handshake Lemma มาจากภาพเปรียบเทียบง่าย ๆ ว่า ถ้าในห้องหนึ่งมีคนหลายคนจับมือกัน การนับจำนวนการจับมือทีละคน จะทำให้การจับมือแต่ละครั้งถูกนับสองครั้งเสมอ หนึ่งครั้งจากมุมมองของคนแรก และอีกหนึ่งครั้งจากมุมมองของคนที่สอง

Idea ให้นับ “จำนวนปลายเส้นเชื่อมทั้งหมด” สองวิธี: ถ้านับทีละจุดยอด จะได้ผลรวมของดีกรี แต่ถ้านับทีละเส้นเชื่อม แต่ละเส้นมีปลายอยู่สองด้าน จึงนับได้ $2|E|$

บทพิสูจน์. พิจารณา simple graph $G = (V, E)$

เราจะนับจำนวนปลายเส้นเชื่อมทั้งหมดในกราฟนี้สองวิธี

วิธีแรก นับจากมุมมองของจุดยอด: ที่จุดยอด v มีเส้นเชื่อม incident อยู่ $\deg(v)$ เส้น ดังนั้นเมื่อรวมทุกจุดยอดเข้าด้วยกัน จำนวนปลายเส้นเชื่อมทั้งหมดที่นับได้คือ

$$\sum_{v \in V} \deg(v).$$

วิธีที่สอง นับจากมุมมองของเส้นเชื่อม: เส้นเชื่อมแต่ละเส้นใน simple graph เชื่อมจุดยอดสองตัว จึงมีปลายสองด้านเสมอ ดังนั้นถ้ามีเส้นเชื่อมทั้งหมด $|E|$ เส้น จำนวนปลายเส้นเชื่อมทั้งหมดคือ

$$2|E|.$$

เนื่องจากทั้งสองวิธีเป็นการนับสิ่งเดียวกัน จึงต้องได้ผลลัพธ์เท่ากัน ดังนั้น

$$\sum_{v \in V} \deg(v) = 2|E|.$$

□

ทฤษฎีบทนี้สั้นมาก แต่มีความสำคัญมากเกินกว่าจะมองข้าม เพราะมันเป็นตัวอย่างคลาสสิกของการพิสูจน์แบบ double counting และยังบอกความสัมพันธ์โดยตรงระหว่างข้อมูลสองระดับของกราฟ: ระดับของจุดยอดและระดับของเส้นเชื่อม

อีกสิ่งหนึ่งที่ควรสังเกตคือ ข้างขวาของสมการคือ $2|E|$ ซึ่งเป็นจำนวนคู่เสมอ ดังนั้นผลรวมดีกรีของจุดยอดทั้งหมดต้องเป็นจำนวนคู่เสมอ ผลตามเล็ก ๆ นี้นำไปสู่ข้อสังเกตที่มีประโยชน์มากอีกข้อหนึ่ง

Corollary 9.1.6. ใน simple graph ใด ๆ จำนวนจุดยอดที่มีดีกรีคี่เป็นจำนวนคู่

Idea ผลรวมดีกรีทั้งหมดเป็นจำนวนคู่ และผลรวมของจำนวนคู่หลายจำนวนยังคงเป็นจำนวนคู่ ดังนั้นจำนวนพจน์ที่เป็นจำนวนคี่จึงต้องมีจำนวนเป็นคู่

บทพิสูจน์. จาก Handshake Lemma เรารู้ว่า

$$\sum_{v \in V} \deg(v) = 2|E|$$

ซึ่งเป็นจำนวนคู่

ตอนนี้แยกจุดยอดออกเป็นสองกลุ่ม: จุดยอดที่มีดีกรีคู่ และจุดยอดที่มีดีกรีคี่

ผลรวมของดีกรีจากกลุ่มดีกรีคู่อ้อมเป็นจำนวนคู่ ดังนั้นผลรวมของดีกรีจากกลุ่มดีกรีคี่ก็ต้องเป็นจำนวนคู่เช่นกัน แต่ผลรวมของจำนวนคี่หลายจำนวนจะเป็นจำนวนคู่ก็ต่อเมื่อมีจำนวนพจน์ที่เป็นจำนวนคี่ จึงสรุปได้ว่า จำนวนจุดยอดที่มีดีกรีคี่ต้องเป็นจำนวนคู่ □

ผลตามนี้สำคัญมากในเชิงการใช้งาน เพราะทำให้เราตรวจสอบความเป็นไปได้ของข้อมูลบางชุดได้ทันที ตัวอย่างเช่น ถ้ามีใครบอกว่ามีกราฟหนึ่งที่มีจุดยอดดีกรีที่อยู่สามจุด เราสามารถตอบได้ทันทีว่าเป็นไปไม่ได้ โดยไม่ต้องวาดกราฟใด ๆ เลย

ในภาพรวม หัวข้อนี้ควรทำให้ผู้อ่านเริ่มเห็นลักษณะเฉพาะของทฤษฎีกราฟ: นิยามพื้นฐานอาจดูเรียบง่ายมาก แต่เมื่อผสมกับการให้เหตุผลที่ถูกต้อง เราสามารถได้ข้อสรุปที่มีพลังและใช้งานได้จริงอย่างรวดเร็ว และนี่เองคือรูป

แบบของคณิตศาสตร์ที่สคริปต์ที่เราจะได้พบซ้ำอีกหลายครั้งในบทต่อไป

9.2 Walks, Paths, Cycles, and Connectivity

เมื่อเริ่มมองกราฟผ่านภาษาเรื่องตึกรี้ เรากำลังสนใจข้อมูลเชิง “ท้องถิ่น” ของแต่ละจุดยอดเป็นหลัก กล่าวคือ เราถามว่าจุดยอดหนึ่งเชื่อมกับใครบ้าง และเชื่อมอยู่ที่เส้น แต่ในหลายปัญหาที่สำคัญจริง ๆ สิ่งที่เราต้องการรู้ไม่ใช่เพียงว่า “ใครติดกับใคร” หากเป็นว่า

จากจุดยอดหนึ่ง เราเดินไปถึงอีกจุดยอดหนึ่งได้หรือไม่
ถ้าไปได้ ไปอย่างไร
และการเดินนั้นวนกลับมาเป็นวงได้หรือไม่

คำถามลักษณะนี้้นำเราไปสู่แนวคิดเรื่อง walk, path, cycle และท้ายที่สุดคือความคิดที่สำคัญมากของบทนี้ นั่นคือ *ความต่อเนื่อง* ของกราฟ

สิ่งที่ควรสังเกตตั้งแต่นั้นคือ หัวข้อเหล่านี้ไม่ใช่ศัพท์หลายคำที่วางเรียงกันเฉย ๆ แต่เป็นลำดับของแนวคิดที่ค่อย ๆ เพิ่มขึ้น walk เป็นแนวคิดที่ยืดหยุ่นที่สุด เพราะอนุญาตให้เดินซ้ำได้ path เป็น walk ที่ตัดความซ้ำที่ไม่จำเป็นออก cycle ทำให้เราเริ่มเห็นโครงสร้างแบบ “วง” และเมื่อเราถามว่าทุกคู่จุดยอดเชื่อมถึงกันหรือไม่ เราก็มาถึงแนวคิดของ connected graph ในที่สุด

9.2.1 Walk, path, closed walk, cycle

ให้ $G = (V, E)$ เป็น simple graph สมมติว่าเรายืนอยู่ที่จุดยอดหนึ่งแล้วต้องการ “เดิน” ไปตามเส้นเชื่อมของกราฟ สิ่งที่เป็นธรรมชาติที่สุดก็คือการบันทึกลำดับของจุดยอดที่เราเดินผ่าน トラバใดที่จุดยอดที่อยู่ติดกันในลำดับนั้นมีเส้นเชื่อมต่อกันจริง แนวคิดพื้นฐานที่สุดนี้เรียกว่า **walk**

Definition 9.2.1 (walk). ให้ $G = (V, E)$ เป็น simple graph **walk** จาก v_0 ไป v_k คือ ลำดับของจุดยอด

$$v_0, v_1, \dots, v_k$$

ที่ทำให้

$$\{v_{i-1}, v_i\} \in E \quad \text{สำหรับทุก } i = 1, 2, \dots, k.$$

จำนวน k เรียกว่า *ความยาว (length) ของ walk* นี้

นิยามนี้ควรอ่านให้เห็นภาพว่า เราไม่ได้กำหนดให้จุดยอดทั้งหมดในลำดับต้องต่างกัน และไม่ได้กำหนดว่าเส้นเชื่อม

แต่ละเส้นต้องถูกใช้เพียงครั้งเดียว ดังนั้น walk จึงเป็นแนวคิดที่ยอมให้ ``ย้อนกลับ" ``อ้อม" หรือ ``เดินวน" ได้ทั้งหมด ในแง่ this walk คือความหมายกว้างที่สุดของการเคลื่อนที่ในกราฟ

Example 9.2.2. ถ้าในกราฟมีเส้นเชื่อม

$$\{a, b\}, \{b, c\}, \{c, d\}, \{b, d\},$$

แล้ว

$$a, b, c, b, d$$

เป็น walk จาก a ไป d ที่มีความยาว 4

ตัวอย่างนี้สำคัญเพราะช่วยให้เห็นทันทีว่า walk อนุญาตให้จุดยอดซ้ำได้ ที่นี้จุดยอด b ปรากฏสองครั้ง แต่ลำดับนี้ก็ยังเป็น walk ที่ถูกต้องอยู่ เพราะทุกคู่ที่ติดกันในลำดับมีเส้นเชื่อมรองรับจริง

ในหลายสถานการณ์ เราต้องการการเดินทางที่ ``ประหยัด" กว่านั้น คือไม่วนกลับไปที่เคยโดยไม่จำเป็น แนวคิดนี้นำไปสู่คำว่า **path**

Definition 9.2.3 (path). **path** คือ walk ที่ไม่มีจุดยอดซ้ำ

ดังนั้น path จึงเป็น walk ที่ตัดการย้อนกลับหรือการวนซ้ำที่ไม่จำเป็นออกไป ถ้ามองอย่างไม่เป็นทางการ path คือการเดินทางจากจุดหนึ่งไปอีกจุดหนึ่งโดยไม่กลับไปเหยียบจุดเดิมซ้ำ เพราะฉะนั้นทุก path ย่อมเป็น walk แต่ไม่ใช่ทุก walk จะเป็น path

จุดเล็ก ๆ ที่ควรระวังคือ ลำดับที่มีจุดยอดเพียงตัวเดียว เช่น

$$v$$

ก็ถือเป็น path ได้เช่นกัน และมีความยาว 0 เพราะไม่มีข้อใดในนิยามที่ห้ามกรณีนี้ รายละเอียดเช่นนี้จะมีประโยชน์เมื่อเราเริ่มพูดถึง connectivity อย่างเป็นทางการ

เมื่อเราเริ่มสนใจการเดินทางที่ ``กลับมาจุดเดิม" เราจะได้แนวคิดของ **closed walk**

Definition 9.2.4 (closed walk). **closed walk** คือ walk ที่จุดยอดเริ่มต้นและจุดยอดสุดท้ายเป็นจุดเดียวกัน

ตัวอย่างเช่น

$$a, b, c, a$$

เป็น closed walk ความยาว 3 เพราะเริ่มที่ a และกลับมาจบที่ a อีกครั้ง

อย่างไรก็ตาม ไม่ใช่ทุก closed walk จะเป็นโครงสร้างที่น่าสนใจในทางทฤษฎีกราฟ เพราะบางครั้ง closed walk อาจมีการวกไปวนมาซ้ำ ๆ มากเกินไป สิ่งที่เราสนใจมากกว่าคือ closed walk ที่เป็น ``วงแท้ ๆ" ซึ่งไม่มีการซ้ำของจุดยอดระหว่างทาง แนวคิดนี้คือ **cycle**

Definition 9.2.5 (cycle). cycle คือ closed walk ที่มีความยาวเป็นบวก และไม่มีจุดยอดซ้ำ นอกจากจุดเริ่มต้นกับจุดสุดท้าย

นิยามนี้ทำให้เห็นว่าคำว่า cycle เข้มกว่า closed walk มาก เพราะ cycle ต้องเป็นการเดินที่กลับมาจุดเดิม โดยไม่วกกลับไปแตะจุดเดิมระหว่างทางเลย ยกเว้นการกลับมาปิดวงที่จุดเริ่มต้นเท่านั้น

Example 9.2.6. ลำดับ

a, b, c, a

เป็น cycle แต่ลำดับ

a, b, c, b, a

เป็นเพียง closed walk ไม่เป็น cycle เพราะจุดยอด b ปรากฏซ้ำระหว่างทาง

จากนิยามทั้งหมดนี้ มีความสัมพันธ์พื้นฐานที่ควรจำให้แม่นดังนี้

ความสัมพันธ์ของแนวคิดพื้นฐาน

- ทุก path เป็น walk
- ไม่ใช่ทุก walk จะเป็น path
- ทุก cycle เป็น closed walk
- ไม่ใช่ทุก closed walk จะเป็น cycle

สิ่งที่ควรสังเกตคือ เราเริ่มจากแนวคิดที่กว้างที่สุด แล้วค่อยเพิ่มเงื่อนไขเข้าไปเพื่อได้วัตถุที่มีโครงสร้างมากขึ้น วิธีคิดแบบนี้พบได้บ่อยมากในคณิตศาสตร์บริสุทธิ์: เริ่มจากนิยามกว้าง แล้วค่อยแยกกรณีย่อยที่มีสมบัติพิเศษและน่าสนใจมากกว่า

9.2.2 จาก walk ไปสู่ path

เมื่อเริ่มเห็นความต่างระหว่าง walk กับ path คำถามตามธรรมชาติที่ควรถามต่อคือ: ถ้าเรารู้เพียงว่าจาก u ไป v มี walk อยู่สักเส้นหนึ่ง เราจะมั่นใจได้หรือไม่ว่ามี path จาก u ไป v ด้วย

คำตอบคือ ได้ และนี่เป็นผลลัพธ์พื้นฐานที่สำคัญมาก เพราะมันบอกเราว่า ถ้าจุดยอดสองจุด “ไปถึงกันได้” เราย่อมนตัดส่วนที่วนเกินจำเป็นออก จนเหลือการเดินทางแบบไม่ซ้ำได้เสมอ

Theorem 9.2.7. ถ้ามี walk จากจุดยอด u ไปยังจุดยอด v แล้วจะมี path จาก u ไปยัง v ด้วย

Idea เลือก walk จาก u ไป v ที่สั้นที่สุด ถ้า walk นี้ยังมีจุดยอดซ้ำ เราจะตัดช่วงที่วนซ้ำออกได้ และได้ walk ที่สั้นกว่าเดิม ซึ่งขัดกับการเลือกให้สั้นที่สุด ดังนั้น walk ที่สั้นที่สุดต้องไม่มีจุดยอดซ้ำ จึงเป็น path

บทพิสูจน์. สมมติว่ามี walk จาก u ไปยัง v ในบรรดา walk ทั้งหมดจาก u ไป v เลือกมาหนึ่งเส้นที่มีความยาวน้อยที่สุด เขียนเป็น

$$v_0, v_1, \dots, v_k$$

โดยที่ $v_0 = u$ และ $v_k = v$

เราจะพิสูจน์ว่า walk นี้เป็น path กล่าวคือไม่มีจุดยอดซ้ำ

สมมติย้อนแย้งว่า walk นี้มีจุดยอดซ้ำ นั่นคือมีดัชนี $i < j$ ที่ทำให้

$$v_i = v_j.$$

พิจารณาลำดับใหม่ที่ได้จากการตัดช่วง

$$v_i, v_{i+1}, \dots, v_j$$

ออกไป จะได้ลำดับ

$$v_0, v_1, \dots, v_i, v_{j+1}, \dots, v_k.$$

ลำดับใหม่นี้ยังคงเป็น walk จาก u ไป v เพราะ $v_i = v_j$ จึงทำให้การเชื่อมต่อหลังการตัดยังคงถูกต้อง แต่ลำดับใหม่นี้มีความยาวสั้นกว่าเดิม ซึ่งขัดกับสมมติฐานที่ว่าเราเลือก walk เดิมให้สั้นที่สุดแล้ว

ดังนั้น walk ที่สั้นที่สุดจะมีจุดยอดซ้ำไม่ได้ จึงเป็น path □

ทฤษฎีบทนี้สำคัญมากเพราะทำให้เวลาเราพูดว่า “จาก u ไป v ไปถึงกันได้” เรามักไม่ต้องกังวลมากกว่าจะเป็น walk หรือ path เพราะถ้ามี walk อยู่ ก็ย่อมมี path อยู่ด้วยเสมอ

ในเชิงความคิด นี่คือผลลัพธ์ที่บอกว่า การวนซ้ำในกราฟไม่ได้จำเป็นต่อการเชื่อมจุดต้นทางกับปลายทาง ถ้าเป้าหมายของเรามีเพียง “ไปให้ถึง” เราสามารถลบส่วนที่วกวนทิ้งไปได้จนเหลือโครงที่เรียบง่ายกว่า

9.2.3 Connected graph และ connected component

เมื่อเรารู้แล้วว่าจุดยอดสองจุดอาจเชื่อมถึงกันผ่าน path ได้ คำถามที่เป็นธรรมชาติมากขึ้นในระดับของกราฟทั้งกราฟคือ

จุดยอดทุกคู่ในกราฟนี้เชื่อมถึงกันหรือไม่

ถ้าคำตอบคือใช่ เราจะเรียกกราฟนั้นว่า **connected**

Definition 9.2.8 (connected graph). กราฟ G เรียกว่า **connected** ถ้าสำหรับทุกคู่ของจุดยอด $u, v \in V(G)$ มี path จาก u ไป v

ด้วยทฤษฎีบทในหัวข้อก่อน เราสามารถอ่านนิยามนี้แบบสมมูลได้ด้วยว่า กราฟ G **connected** ก็ต่อเมื่อสำหรับทุกคู่ของจุดยอด u, v มี walk จาก u ไป v แต่ในทางนิยาม คำว่า **path** มักเหมาะกว่า เพราะสื่อถึงการเชื่อมโยงที่ไม่ฟุ่มเฟือยและตรงกับ intuition มากกว่า

Example 9.2.9. กราฟเส้นตรง

$$a - b - c - d - e$$

เป็น **connected graph** เพราะไม่ว่าเลือกจุดยอดสองจุดใด เราสามารถเดินตามเส้นเชื่อมไปถึงกันได้เสมอ

ในทางกลับกัน ถ้ากราฟมีส่วนที่แยกขาดออกจากกัน เช่นมีจุดยอดกลุ่มหนึ่งเชื่อมกันเอง และอีกกลุ่มหนึ่งเชื่อมกันเอง แต่ไม่มีเส้นเชื่อมข้ามกลุ่ม กราฟนั้นย่อมไม่ **connected**

Example 9.2.10. ถ้า

$$V = \{1, 2, 3, 4, 5, 6, 7\}$$

และ

$$E = \{\{1, 2\}, \{2, 3\}, \{3, 1\}, \{4, 5\}, \{6, 7\}\},$$

แล้วกราฟนี้ไม่ **connected** เพราะจุดยอด 1 ไม่สามารถไปถึงจุดยอด 4 ได้

เมื่อกราฟไม่ **connected** มักมีประโยชน์มากที่จะมองว่า กราฟนั้นประกอบด้วย “ก้อนที่เชื่อมกันภายใน” แต่ไม่เชื่อมถึงกันข้ามก้อน แนวคิดนี้คือ **connected component**

Definition 9.2.11 (connected component). **Connected component** ของกราฟ G คือกราฟย่อยที่ **connected** และไม่สามารถเติมจุดยอดจากกราฟเดิมเข้าไปได้อีก โดยยังคง **connected** อยู่

ถ้ามองในเชิง intuition **connected component** คือ “ก้อนเชื่อมต่อที่ใหญ่ที่สุด” ภายในกราฟ ทุกจุดยอดใน **component** เดียวกันไปถึงกันได้ แต่ถ้าหยิบจุดยอดจากคนละ **component** มา จะไปถึงกันไม่ได้

สำหรับตัวอย่างก่อนหน้า กราฟมีสาม **connected components** คือ กลุ่ม $\{1, 2, 3\}$, กลุ่ม $\{4, 5\}$, และกลุ่ม $\{6, 7\}$

สิ่งที่ควรสังเกตคือ **connected components** ไม่ได้เกิดขึ้นแบบเลื่อนลอย แต่จริง ๆ แล้วมันมาจากความสัมพันธ์ธรรมชาติบนเซตของจุดยอด: สองจุดยอดถือว่า “อยู่ด้วยกัน” ถ้ามี path เชื่อมระหว่างกัน เราสามารถทำให้แนวคิดนี้เป็นข้อความทางคณิตศาสตร์ได้ดังนี้

Definition 9.2.12. ให้ $G = (V, E)$ เป็นกราฟ นิยามความสัมพันธ์ $\sim$ บนเซต V โดยให้

$$u \sim v \iff \text{มี path จาก } u \text{ ไป } v.$$

ความสัมพันธ์นี้มีความสำคัญมาก เพราะมันอธิบาย connected component ได้อย่างเป็นระบบ

Proposition 9.2.13. ความสัมพันธ์ $\sim$ เป็นความสัมพันธ์สมมูลบน V

Idea ต้องตรวจสอบสมบัติ: reflexive, symmetric, และ transitive ซึ่งในบริบทของ path ทั้งหมดมีความหมายเชิงรูปภาพที่ชัดเจนมาก

บทพิสูจน์. เราจะพิสูจน์ว่า $\sim$ มีสมบัติ reflexive, symmetric, และ transitive

Reflexive: ให้ $u \in V$ ลำดับที่มีจุดยอดเพียงตัวเดียว u เป็น path ความยาว 0 จาก u ไป u ดังนั้น $u \sim u$

Symmetric: ถ้า $u \sim v$ แปลว่ามี path จาก u ไป v แต่ใน simple graph เส้นเชื่อมไม่มีทิศทาง ดังนั้นเมื่อกลับลำดับของ path เดิม เราจะได้ path จาก v ไป u จึงได้ว่า $v \sim u$

Transitive: ถ้า $u \sim v$ และ $v \sim w$ แปลว่ามี path จาก u ไป v และมี path จาก v ไป w เมื่อนำสองลำดับนี้มาต่อกัน จะได้ walk จาก u ไป w จากทฤษฎีบทก่อนหน้านี้ว่า ถ้ามี walk แล้วมี path จึงสรุปได้ว่ามี path จาก u ไป w ดังนั้น $u \sim w$

จึงสรุปได้ว่า $\sim$ เป็นความสัมพันธ์สมมูล □

ผลลัพธ์นี้สำคัญมาก เพราะบอกว่า connected components ของกราฟ แท้จริงแล้วก็คือ ชั้นสมมูล ของความสัมพันธ์ “ไปถึงกันได้” นั่นเอง ดังนั้น components ต่าง ๆ จึงแบ่งจุดยอดของกราฟออกเป็นกลุ่มที่ไม่ทับกัน และทุกจุดยอดต้องอยู่ใน component ใด component หนึ่งเสมอ

ภาพรวมที่ควรจำ

- walk คือการเดินตามเส้นเชื่อม โดยอนุญาตให้ซ้ำได้
- path คือ walk ที่ไม่มีจุดยอดซ้ำ
- cycle คือ closed walk ที่ไม่ซ้ำจุดยอด ยกเว้นจุดเริ่มต้นกับจุดสุดท้าย
- ถ้ามี walk ระหว่างสองจุดยอด ก็มี path ระหว่างสองจุดยอด
- connected graph คือกราฟที่ทุกคู่จุดยอดมี path เชื่อมกัน
- connected components คือก้อนเชื่อมต่อที่ใหญ่ที่สุดของกราฟ

จากจุดนี้ไป เราจะเริ่มใช้ภาษาของ path, cycle, และ connectedness เป็นเครื่องมือหลักในการพิสูจน์ทฤษฎีบทเกี่ยวกับกราฟ โดยเฉพาะในหัวข้อเรื่อง bridge, forest, และ tree ซึ่งจะเห็นชัดขึ้นว่าศัพท์พื้นฐานในส่วนนี้ไม่ได้มีไว้เพียงเพื่อเรียกชื่อสิ่งต่าง ๆ แต่เป็นภาษาหลักของการให้เหตุผลในทฤษฎีกราฟทั้งหมด

9.3 Bridges

เมื่อเรารู้จัก walk, path, cycle และ connected graph แล้ว เราก็เริ่มมีภาษามากพอที่จะถามคำถามเชิงโครงสร้างได้จริงจังขึ้น ตัวอย่างเช่น ถ้าเราตัดเส้นเชื่อมเส้นหนึ่งออกจากกราฟ กราฟจะยังเชื่อมต่ออยู่หรือไม่ หรือถ้ากราฟไม่มี cycle เลย โครงสร้างของมันจะมีหน้าตาแบบใด

คำถามเหล่านี้เป็นจุดเริ่มต้นของแนวคิดเรื่อง **bridge** (ที่จะนำไปสู่เรื่อง **forest** และ **tree** ในบทถัดไป) ซึ่งเป็นหัวใจสำคัญของทฤษฎีกราฟระดับต้น เพราะวัตถุเหล่านี้อยู่กึ่งกลางระหว่าง “กราฟทั่วไป” กับ “โครงสร้างที่มีระเบียบมากพอจะพิสูจน์อะไรสวย ๆ ได้” โดยเฉพาะ **tree** นั้นปรากฏอยู่ทั่วทั้งคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ไม่ว่าจะเป็นโครงสร้างข้อมูล การค้นหา การแยกวิเคราะห์พจน์ การจัดลำดับชั้นของข้อมูล หรือการอธิบายการเชื่อมโยงที่ไม่มีวงวน

สิ่งที่ควรสังเกตคือ แนวคิดทั้งสามนี้ไม่ได้แยกขาดจากกัน **bridge** เป็นภาษาสำหรับบอกว่า “เส้นเชื่อมเส้นใดสำคัญต่อความต่อเนื่องของกราฟ” **forest** เป็นกราฟที่ไม่มี **cycle** และ **tree** เป็นกรณีพิเศษที่กราฟทั้ง **connected** และไม่มี **cycle** ดังนั้นส่วนนี้ของบทจะค่อย ๆ พาเราไปจากการวิเคราะห์เส้นเชื่อมหนึ่งเส้น ไปสู่การเข้าใจโครงสร้างทั้งกราฟในภาพรวม

9.3.1 Bridge และความสัมพันธ์กับ cycle

ในกราฟบางเส้นเชื่อม ถ้าเราลบออกไปแล้วกราฟแทบไม่เปลี่ยนอะไรสำคัญ เพราะยังมีทางอ้อมไปได้ แต่ในบางกรณี การลบเส้นเชื่อมเพียงเส้นเดียวกลับทำให้กราฟแยกออกเป็นหลายส่วนทันที เส้นเชื่อมชนิดหลังนี้มีบทบาทพิเศษมาก และเรียกว่า **bridge**

Definition 9.3.1 (bridge). ให้ $G = (V, E)$ เป็นกราฟ และให้ $e \in E$ เราเรียก e ว่า **bridge** ถ้าเมื่อเอาเส้นเชื่อม e ออกจากกราฟแล้ว จำนวน **connected components** ของกราฟเพิ่มขึ้น

นิยามนี้ควรตีความในภาษาที่ตรงไปตรงมาว่า **bridge** คือเส้นเชื่อมที่ “รับภาระการเชื่อมต่อ” อยู่จริง ถ้าตัดมันทิ้งจะมีจุดยอดบางคู่ที่เดิมไปถึงกันได้ แต่หลังจากตัดแล้วไปถึงกันไม่ได้อีก

Example 9.3.2. พิจารณากราฟที่มีเส้นเชื่อม

$$\{a, b\}, \{b, c\}, \{c, a\}, \{c, d\}, \{d, e\}.$$

เส้นเชื่อม $\{c, d\}$ และ $\{d, e\}$ เป็น **bridge** แต่เส้นเชื่อม $\{a, b\}, \{b, c\}, \{c, a\}$ ไม่เป็น **bridge** เพราะเส้นทั้งสามอยู่ในสามเหลี่ยม เมื่อลบเส้นใดเส้นหนึ่งออก ยังมีทางอ้อมผ่านอีกสองเส้นได้

ตัวอย่างนี้ชี้ให้เห็น **intuition** ที่สำคัญมาก: ถ้าเส้นเชื่อมเส้นหนึ่งอยู่บน **cycle** มันมักไม่ใช่ **bridge** เพราะ **cycle** ให้ทางอ้อมกลับไปยังปลายอีกด้านหนึ่งได้เสมอ ในทางกลับกัน ถ้าเส้นเชื่อมเส้นหนึ่งไม่ได้อยู่บน **cycle** ใดเลย มันมักเป็นเส้นที่ “ขาดไม่ได้” จริง ความคิดนี้นำไปสู่ทฤษฎีบทพื้นฐานต่อไปนี้

Theorem 9.3.3 (Bridge iff not on a cycle). ให้ G เป็นกราฟ และให้ $e \in E(G)$ แล้ว e เป็น bridge ก็ต่อเมื่อ e ไม่ได้อยู่บน cycle ใด ๆ ของกราฟ

ทฤษฎีบทนี้ควรจำให้แม่น เพราะมันเป็นหนึ่งใน characterization พื้นฐานที่สุดของ bridge และยังแสดงความเชื่อมโยงโดยตรงระหว่างสองแนวคิดที่ดูต่างกันมากในตอนแรก: “ความเปราะบางของการเชื่อมต่อ” กับ “การมีอยู่ของวงในกราฟ”

Claim ถ้า e อยู่บน cycle แล้ว e ไม่เป็น bridge

Idea ถ้า e อยู่บน cycle เมื่อเอา e ออก ปลายทั้งสองของ e ยังเดินอ้อมไปตามส่วนที่เหลือของ cycle ได้ ดังนั้นกราฟจะไม่ขาดจากกันเพราะการลบ e เพียงเส้นเดียว

บทพิสูจน์. สมมติว่า $e = \{u, v\}$ อยู่บน cycle หนึ่งในการาฟ G พิจารณากราฟใหม่ $G - e$ ที่ได้จากการลบเส้นเชื่อม e ออก

เพราะ e อยู่บน cycle บน cycle เดิมจะมีเส้นทางอีกด้านหนึ่งจาก u ไป v ที่ไม่ใช่เส้นเชื่อม e ดังนั้นในกราฟ $G - e$ ยังมี path จาก u ไป v

ตอนนี้พิจารณาจุดยอดสองจุดใด ๆ ที่เดิมเชื่อมถึงกันได้ ใน G ถ้า path เดิมของมันไม่ได้ใช้ e ก็ยังใช้ path เดิมนั้นได้ใน $G - e$ แต่ถ้า path เดิมใช้ e เราสามารถแทนช่วงที่ผ่าน e ด้วยทางอ้อมบน cycle ได้ จึงยังได้ walk จากต้นทางไปปลายทางใน $G - e$ และจากทฤษฎีบทก่อนหน้านี้ ถ้ามี walk ก็มี path

ดังนั้นการลบ e ไม่ได้ทำให้กราฟแตกออกเป็น component เพิ่มขึ้น จึงสรุปว่า e ไม่เป็น bridge □

Claim ถ้า e ไม่ได้อยู่บน cycle ใด ๆ แล้ว e เป็น bridge

Idea ถ้าลบ e แล้วปลายทั้งสองด้านยังเชื่อมถึงกันได้ เราจะนำ path ใหม่นั้นมารวมกับ e แล้วเกิด cycle ที่มี e อยู่ ซึ่งขัดกับสมมติฐาน

บทพิสูจน์. ให้ $e = \{u, v\}$ และสมมติว่า e ไม่ได้อยู่บน cycle ใด ๆ เราจะพิสูจน์ว่า e เป็น bridge

สมมติย้อนแย้งว่า e ไม่เป็น bridge นั่นหมายความว่าเมื่อลบ e ออกจากกราฟแล้ว จำนวน connected components ไม่เพิ่มขึ้น โดยเฉพาะอย่างยิ่ง u และ v ยังต้องเชื่อมถึงกันได้ในการาฟ $G - e$

ดังนั้นในกราฟ $G - e$ มี path จาก u ไป v เมื่อเรานำ path นี้มารวมกับเส้นเชื่อม $e = \{u, v\}$ จะได้ closed walk และเพราะ path ดังกล่าวไม่มีการซ้ำจุดยอด closed walk ที่ได้จะบรรจุ cycle หนึ่งที่มีเส้นเชื่อม e อยู่ด้วย แต่นี่ขัดกับสมมติฐานที่ว่า e ไม่ได้อยู่บน cycle ใด ๆ ดังนั้นสมมติฐานที่ว่า e ไม่เป็น bridge ต้องเป็นเท็จ จึงสรุปว่า e เป็น bridge $\square$

ทฤษฎีบทนี้มีประโยชน์มากในทางปฏิบัติ เพราะเวลาจะตรวจว่าเส้นเชื่อมเส้นหนึ่งเป็น bridge หรือไม่ เราไม่จำเป็นต้องกลับไปเช็คจำนวน connected components ทุกครั้ง หลายครั้งการดูว่าเส้นนั้นอยู่บน cycle หรือไม่ ยากกว่า และให้ภาพโครงสร้างชัดกว่ามาก

อย่างไรก็ตาม ความสำคัญของ bridge ไม่ได้จบอยู่เพียงการใช้ตรวจว่าเส้นเชื่อมเส้นใด "สำคัญ" ต่อความต่อเนื่องของกราฟ สิ่งที่น่าสนใจกว่านั้นคือ แนวคิดเรื่อง bridge ทำให้เราเริ่มมองเห็นความสัมพันธ์ที่ลึกซึ้ง ระหว่าง ความต่อเนื่อง และ การไม่มี cycle กล่าวคือ เมื่อเราพิจารณากราฟที่เส้นเชื่อมทุกเส้นล้วนมีบทบาทสำคัญจริง หรือกราฟที่ไม่มีเส้นเชื่อมใดอยู่บน cycle เลย เรากำลังค่อย ๆ เข้าใกล้โครงสร้างชนิดพิเศษมากชนิดหนึ่งของทฤษฎีกราฟ ซึ่งทั้งเรียบง่ายและทรงพลังอย่างยิ่ง

กล่าวอีกแบบหนึ่ง หัวข้อเรื่อง bridge ในบทนี้ทำหน้าที่เหมือนประตูเชื่อม จากภาษาพื้นฐานของกราฟทั่วไป ไปสู่การศึกษาโครงสร้างที่มีระเบียบมากขึ้น เพราะทันทีที่เราถามต่อว่า "แล้วถ้ากราฟไม่มี cycle เลยจะเกิดอะไรขึ้น" หรือ "แล้วถ้ากราฟเชื่อมต่อกันครบแต่ไม่มีความซ้ำซ้อนของเส้นทางเลยล่ะ" เราก็จะเข้าสู่โลกของ forests และ trees ทันที

ด้วยเหตุนี้ จากจุดนี้ไป เราจะหยุดมองกราฟเพียงในฐานะโครงสร้างทั่วไป แล้วหันมาศึกษากราฟชนิดพิเศษที่สำคัญที่สุดชนิดหนึ่งในคณิตศาสตร์ดิสครีตและวิทยาการคอมพิวเตอร์ นั่นคือ ต้นไม้ ซึ่งเป็นโครงสร้างที่อยู่เบื้องหลังทั้งเรื่องการค้นหา โครงสร้างข้อมูล การจัดลำดับชั้น และการให้เหตุผลเกี่ยวกับการเชื่อมต่อที่ไม่มีวงวน

บทที่ 10

Trees

บทก่อนหน้านี้ให้ภาษาพื้นฐานของทฤษฎีกราฟแก่เราแล้ว ไม่ว่าจะเป็นจุดยอด เส้นเชื่อม ดีกรี แนวคิดเรื่อง walk, path, cycle, connectedness รวมถึง bridge ซึ่งช่วยให้เราเห็นว่า เส้นเชื่อมบางเส้นมีบทบาทสำคัญต่อโครงสร้างของกราฟมากเพียงใด ในบทนี้ เราจะก้าวต่อไปอีกขั้น โดยมุ่งสนใจกราฟที่มีระเบียบมากกว่ากราฟทั่วไป กล่าวคือกราฟที่ไม่มีวงวน และในกรณีที่เชื่อมต่อกันครบถ้วนด้วย จะกลายเป็นโครงสร้างที่เรียกว่า **tree**

tree เป็นหนึ่งในวัตถุพื้นฐานที่สุดของทฤษฎีกราฟ และมีบทบาทกว้างขวางมากในวิทยาการคอมพิวเตอร์ เหตุผลสำคัญก็คือ มันเป็นโครงสร้างที่ “เชื่อมต่อกันพอดี” มีทางไปถึงกันได้ครบ แต่ไม่มีความซ้ำซ้อนของเส้นทางในรูปของ cycle จึงเหมาะอย่างยิ่งกับการแทนโครงสร้างลำดับชั้น การแทน dependency การค้นหา และการพิสูจน์เชิงโครงสร้างในหัวข้ออื่น ๆ ต่อไป

ในบทนี้ เราจะเริ่มจากการแยกความต่างระหว่าง forest กับ tree ให้ชัดเจน จากนั้นจึงศึกษาคุณสมบัติพื้นฐานของต้นไม้ และค่อย ๆ พัฒนาไปสู่ characterizations หลายแบบของ tree ซึ่งจะทำให้เราเห็นว่า แม้ต้นไม้จะนิยามได้อย่างสั้น ๆ ว่า “connected และไม่มี cycle” แต่มันสามารถถูกอธิบายได้จากหลายมุมมองที่สมมูลกันอย่างน่าประทับใจ

10.1 Forests และ Trees

ก่อนจะเข้าสู่ทฤษฎีบทสำคัญของต้นไม้ เราควรเริ่มจากคำศัพท์พื้นฐานสองคำนี้ให้ชัดเจนก่อน เพราะแม้จะเกี่ยวข้องกันอย่างใกล้ชิด แต่มีบทบาทไม่เหมือนกัน

10.1.1 Forest และ tree

หลังจากเข้าใจบทบาทของ cycle ในการทำให้เส้นเชื่อม ``ไม่เปราะบาง" คำถามถัดไปที่เป็นธรรมชาติมากคือ จะเกิดอะไรขึ้นถ้ากราฟไม่มี cycle เลย กราฟประเภทนี้เรียกว่า **forest** และถ้ามันยัง **connected** ด้วย เราจะได้วัตถุสำคัญที่สุดชนิดหนึ่งในทฤษฎีกราฟ นั่นคือ **tree**

Definition 10.1.1 (forest). กราฟที่ไม่มี cycle เรียกว่า **forest**

Definition 10.1.2 (tree). กราฟที่ **connected** และไม่มี cycle เรียกว่า **tree**

นิยามนี้สั้นมาก แต่ควรอ่านให้เห็นภาพเชิงโครงสร้างอย่างชัดเจน **forest** คือกราฟที่ประกอบด้วย ``ส่วนที่เป็นต้นไม้" หลายส่วนได้ กล่าวคืออาจไม่ **connected** ก็ได้ ส่วน **tree** คือกรณีที่มีเพียงส่วนเดียวและเชื่อมต่อกันหมด

ดังนั้นเราจึงสรุป intuition ได้ง่าย ๆ ว่า

มุมมองที่ควรจำ

$\text{tree} = \text{connected} + \text{acyclic}$

$\text{forest} = \text{acyclic}$ แต่ไม่จำเป็นต้อง **connected**

คำว่า ``tree" อาจทำให้คิดถึงรูปที่มีรากและกิ่งก้าน แต่ในทฤษฎีกราฟเบื้องต้น tree ยังเป็นเพียงกราฟไม่มีทิศทางที่ **connected** และไม่มี cycle เรายังไม่ต้องเลือกจุดใดเป็น root และยังไม่ต้องพูดถึง parent หรือ child ใดๆ ก็ตาม ชื่อ ``tree" ก็มีเหตุผลทางภาพมาก เพราะถ้าวาดกราฟชนิดนี้ มันมักมีลักษณะคล้ายกิ่งไม้ที่แผ่ออกไปโดยไม่มีวงวน

Example 10.1.3. กราฟเส้นตรง

$$a - b - c - d - e$$

เป็น tree เพราะมัน **connected** และไม่มี cycle

Example 10.1.4. กราฟสามเหลี่ยม

$$a - b - c - a$$

ไม่เป็น tree เพราะแม้จะ **connected** แต่มี cycle

Example 10.1.5. กราฟที่ประกอบด้วยเส้นตรงสองเส้นแยกจากกัน แต่ละส่วนไม่มี cycle ดังนั้นเป็น forest แต่ไม่เป็น tree เพราะกราฟทั้งก้อนไม่ **connected**

จุดนี้ควรระวังข้อสับสนที่พบบ่อยมาก: ผู้เรียนมักเผลอคิดว่า ``ไม่มี cycle" เพียงอย่างเดียวพอที่จะเป็น tree แต่จริง ๆ ยังต้องมี **connected** ด้วย หรือในทางกลับกัน บางคนเห็นว่ากราฟ **connected** แล้วก็คิดว่าเป็น tree ทันที ทั้งที่อาจมี cycle ซ่อนอยู่

ดังนั้นเวลาเจอกราฟหนึ่งกราฟ ถ้าต้องการตัดสินว่าเป็น tree หรือไม่ ควรถามสองคำถามแยกกันเสมอ:

1. connected หรือไม่
2. มี cycle หรือไม่

ต้องผ่านทั้งสองข้อพร้อมกันจึงจะเป็น tree

อีกมุมหนึ่งที่สำคัญคือ forest สามารถมองได้ว่าเป็น *disjoint union of trees* กล่าวคือแต่ละ connected component ของ forest จะเป็น tree เพราะเมื่อ component หนึ่ง connected อยู่แล้ว และทั้งกราฟไม่มี cycle component นั้นก็ย่อมไม่มี cycle ด้วย จึงเข้าเงื่อนไขของ tree ทันที

Proposition 10.1.6. ทุก connected component ของ forest เป็น tree

บทพิสูจน์. ให้ F เป็น forest และให้ C เป็น connected component หนึ่งของ F โดยนิยามของ connected component, C เป็น connected graph อีกทั้ง C เป็นกราฟย่อยของ F ดังนั้น C จะมี cycle ไม่ได้ เพราะถ้า C มี cycle แล้ว F ก็จะมี cycle ด้วย ซึ่งขัดกับการที่ F เป็น forest

จึงได้ว่า C connected และ acyclic ดังนั้น C เป็น tree □

ผลลัพธ์นี้สำคัญเพราะทำให้คำว่า forest ไม่ได้เป็นเพียงนิยามลอย ๆ แต่มีความหมายชัดว่า มันคือกราฟที่ประกอบจาก tree หลายต้นวางแยกกัน

10.1.2 คุณสมบัติพื้นฐานของต้นไม้

tree เป็นกราฟที่มีโครงสร้างพิเศษมาก และสิ่งที่ทำให้มันสำคัญในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ก็คือ คุณสมบัติพื้นฐานจำนวนมากของมันสวย เรียบง่าย และพิสูจน์ได้จากนิยามโดยตรง เราจะเริ่มจากคุณสมบัติที่สำคัญที่สุดข้อหนึ่งก่อน:

Theorem 10.1.7. ใน tree ระหว่างจุดยอดสองจุดใด ๆ จะมี path เพียงเส้นเดียว

ทฤษฎีบทนี้เป็นหัวใจสำคัญของ intuition เรื่อง tree เพราะมันบอกว่าใน tree ไม่มีทั้ง “จุดที่ไม่ถึงกัน” และ “ทางเลือกอ้อมที่ทำให้เกิดวง” ดังนั้นถ้าจะเดินจาก u ไป v เส้นทางจะถูกบังคับไว้อย่างเป็นเอกลักษณ์

Idea อย่างน้อยหนึ่งเส้นมีแน่ เพราะ tree เป็น connected ส่วนจะมีได้ไม่เกินหนึ่งเส้น เพราะถ้ามีสองเส้นที่ต่างกัน เมื่อนำมารวมกันจะเกิด cycle ซึ่งขัดกับการที่ tree ไม่มี cycle

บทพิสูจน์. ให้ T เป็น tree และให้ $u, v \in V(T)$

เนื่องจาก T เป็น connected จึงมี path จาก u ไป v อย่างน้อยหนึ่งเส้น

เราจะพิสูจน์ว่ามีได้ไม่เกินหนึ่งเส้น สมมติว่ามี path จาก u ไป v อยู่สองเส้นที่ต่างกัน พิจารณาจุดแรกที่สอง path นี้เริ่มแยกจากกัน และจุดถัดไปที่มันกลับมาพบกันอีกครั้ง ส่วนของสอง path ระหว่างจุดทั้งสองนี้จะให้ทางเดินสองทางที่ต่างกัน เมื่อนำมารวมกันจะเกิด cycle ในกราฟ

แต่นี้ขัดกับการที่ T เป็น tree ซึ่งไม่มี cycle ดังนั้นระหว่าง u และ v จะมี path ได้เพียงเส้นเดียว $\square$

ผลลัพธ์นี้มีความหมายมากในทางการใช้งาน เพราะมันอธิบายว่าทำไม tree จึงเหมาะกับการแทนโครงสร้างลำดับชั้น หรือโครงสร้างที่ไม่ต้องการความกำกวมของเส้นทาง เช่น parse tree หรือ search tree ถ้ามีสองเส้นทางต่างกันไปยังจุดเดียวกัน โครงสร้างนั้นก็จะไม่เป็น tree อีกต่อไป

คุณสมบัติพื้นฐานอีกข้อหนึ่งที่สำคัญมากคือความสัมพันธ์ระหว่างจำนวนจุดยอดกับจำนวนเส้นเชื่อม

Theorem 10.1.8. ถ้า $T = (V, E)$ เป็น tree แล้ว

$$|E| = |V| - 1.$$

ทฤษฎีบทนี้เป็นข้อความที่สำคัญมากจนแทบถือเป็นลายเซ็นของ tree เพราะมันบอกว่า tree มีเส้นเชื่อม “พอดี” คือมีมากพอที่จะทำให้ connected แต่ไม่มากเกินไปจนเกิด cycle

มีหลายวิธีพิสูจน์ผลลัพธ์นี้ แต่สำหรับหนังสือเล่มนี้ วิธีที่เป็นธรรมชาติที่สุดคือพิสูจน์ด้วยอุปนัยตามจำนวนจุดยอด อาศัยข้อเท็จจริงสำคัญว่า tree ทุกต้นที่มีอย่างน้อยสองจุดยอด ต้องมีจุดยอดที่มีดีกรี 1 อย่างน้อยหนึ่งจุด ซึ่งมักเรียกว่า leaf

ก่อนพิสูจน์ทฤษฎีบทหลัก เราพิสูจน์ lemma นี้ก่อน

Lemma 10.1.9. ถ้า T เป็น tree ที่มีจุดยอดอย่างน้อยสองจุด แล้ว T มีจุดยอดดีกรี 1 อย่างน้อยสองจุด

Idea เลือก path ที่ยาวที่สุดใน tree ปลายทั้งสองด้านของ path นี้จะต่อออกไปไม่ได้อีก ไม่เช่นนั้น path จะยาวขึ้นได้ ดังนั้นปลายทั้งสองต้องมดีกรี 1

บทพิสูจน์. ให้ T เป็น tree ที่มีอย่างน้อยสองจุดยอด เลือก path ใน T ที่มีความยาวมากที่สุด เขียนเป็น

$$v_0, v_1, \dots, v_k.$$

เราจะแสดงว่า v_0 และ v_k มดีกรี 1

พิจารณา v_0 แน่นอนว่า v_0 มดีกรีอย่างน้อย 1 เพราะมันเชื่อมกับ v_1

ถ้า $\deg(v_0) \geq 2$ จะมีจุดยอด $w \neq v_1$ ที่เชื่อมกับ v_0 ถ้า w อยู่บน path เดิม จะเกิด cycle ขึ้น ซึ่งขัดกับการที่ T ไม่มี cycle ถ้า w ไม่อยู่บน path เดิม เราจะได้ path ใหม่

$$w, v_0, v_1, \dots, v_k$$

ซึ่งยาวกว่าเดิม ขัดกับการเลือก path ให้ยาวที่สุด

ดังนั้น $\deg(v_0) = 1$ ด้วยเหตุผลเดียวกันจะได้ว่า $\deg(v_k) = 1$ จึงสรุปว่า T มีจุดยอดดีกรี 1 อย่างน้อยสองจุด □

ตอนนี้เราพร้อมพิสูจน์ผลลัพธ์หลักแล้ว

บทพิสูจน์. เราจะพิสูจน์ด้วยอุปนัยบนจำนวนจุดยอด $n = |V|$

ขั้นฐาน: ถ้า $n = 1$ tree มีจุดยอดหนึ่งจุดและไม่มีเส้นเชื่อมเลย ดังนั้น

$$|E| = 0 = 1 - 1 = |V| - 1.$$

ขั้นอุปนัย: สมมติว่าข้อความเป็นจริงสำหรับ tree ทุกต้นที่มีจุดยอด n จุด และให้ T เป็น tree ที่มีจุดยอด $n + 1$ จุด

จาก lemma ข้างต้น T มี leaf จุดหนึ่ง ให้เรียกจุดยอดนั้นว่า v และให้ e เป็นเส้นเชื่อม $\square\square$ ที่ incident กับ v

ลบ v และ e ออกจาก T จะได้กราฟใหม่ T' ซึ่งยังคง connected อยู่และไม่มี cycle จึงยังเป็น tree อีกทั้ง T' มีจุดยอด n จุด

จากสมมติฐานขั้นอุปนัย

$$|E(T')| = |V(T')| - 1 = n - 1.$$

เมื่อกลับมาที่ T เราเพิ่มจุดยอดกลับมาอีกหนึ่งจุด และเพิ่มเส้นเชื่อมกลับมาอีกหนึ่งเส้น จึงได้ว่า

$$|E(T)| = (n - 1) + 1 = n$$

และ

$$|V(T)| = n + 1.$$

ดังนั้น

$$|E(T)| = n = (n + 1) - 1 = |V(T)| - 1.$$

จึงสรุปว่าข้อความเป็นจริงสำหรับ tree ทุกต้น

□

ทฤษฎีบทนี้สำคัญมาก และในภายหลังเราจะเห็นว่ามันไม่ได้เป็นเพียงผลที่ “tree ทำให้จริง” แต่กลับเป็นหนึ่งใน characterization ของ tree ด้วย กล่าวคือ ถ้ากราฟตัวหนึ่ง connected และมี $|E| = |V| - 1$ มันจะต้องเป็น tree หรือถ้ากราฟตัวหนึ่งไม่มี cycle และมี $|E| = |V| - 1$ มันก็ต้อง connected เช่นกัน อย่างไรก็ตาม ในตอนนี้เรายังเก็บภาพเหล่านั้นไว้ก่อน เพื่อให้โครงสร้างพื้นฐานของ tree ชัดเจนที่ละขั้น

สรุปแล้ว ในส่วนนี้เราได้ภาพสำคัญสามขั้นดังนี้

- bridge บอกว่าเส้นเชื่อมใดเป็นเส้นที่ขาดไม่ได้ต่อความต่อเนื่อง
- forest คือกราฟที่ไม่มี cycle
- tree คือ forest ที่ connected และมีโครงสร้างที่เรียบง่ายมากพอจะทำให้เส้นทางระหว่างทุกคู่จุดหยุดเป็นเอกลักษณ์

และนี่คือเหตุผลว่าทำไม tree จึงเป็นหนึ่งในวัตถุพื้นฐานที่สุดของทฤษฎีกราฟ: มันเป็นกราฟที่เชื่อมต่อกันครบ แต่ไม่มีความซ้ำซ้อนของเส้นทางเลย ซึ่งทำให้เหมาะอย่างยิ่งทั้งต่อการพิสูจน์ทางคณิตศาสตร์ และต่อการนำไปใช้เป็นแบบจำลองในวิทยาการคอมพิวเตอร์

10.2 Characterizations of Trees

ในหัวข้อก่อนหน้า เรานิยาม tree ว่าเป็นกราฟที่ *connected* และ *ไม่มี cycle* นิยามนี้เหมาะสมมากในฐานะจุดเริ่มต้น เพราะมันบอกภาพเชิงโครงสร้างของต้นไม้ได้ตรงที่สุด: ต้นไม้คือกราฟที่ทุกจุดหยุดเชื่อมถึงกันได้ แต่ไม่มีความซ้ำซ้อนของเส้นทางในลักษณะของวงปิด

อย่างไรก็ตาม เมื่อศึกษาทฤษฎีกราฟต่อไป เราจะพบว่า tree สามารถถูกอธิบายได้หลายแบบ และแต่ละแบบก็เน้นมุมมองคนละด้านของโครงสร้างเดียวกัน บางแบบเน้นเรื่องเส้นทาง บางแบบเน้นเรื่องการตัดเส้นเชื่อม บางแบบเน้นเรื่องการเพิ่มเส้นเชื่อม และบางแบบเน้นเพียงความสัมพันธ์เชิงจำนวนระหว่าง $|V|$ และ $|E|$

การมีคำอธิบายหลายแบบเช่นนี้ไม่ได้เป็นเพียงความสวยงามทางทฤษฎี แต่มีคุณค่ามากในทางปฏิบัติด้วย เพราะในโจทย์หนึ่ง เราอาจตรวจว่า graph เป็น tree ได้ง่ายที่สุดจากการดู path แต่อีกโจทย์หนึ่งอาจสะดวกกว่ามากที่จะมองผ่าน bridge หรือผ่านสมการ $|E| = |V| - 1$ ข้อความที่ให้ภาพต่างกันแต่สมมูลกันเช่นนี้เรียกว่า **characterizations** ของวัตถุนั้น

ในส่วนนี้ เราจะค่อย ๆ พิสูจน์ characterization หลัก ๆ ของ tree ก่อนจะรวบรวมทั้งหมดเข้าด้วยกันในตอนท้าย เพื่อให้เห็นว่าแม้ข้อความเหล่านี้จะหน้าตาต่างกัน แต่แท้จริงแล้วกำลังอธิบายโครงสร้างชนิดเดียวกันอยู่

10.2.1 Path uniqueness

หนึ่งในคุณสมบัติที่สำคัญที่สุดของ tree คือ ระหว่างจุดยอดสองจุดใด ๆ จะมี path เพียงเส้นเดียว ผลลัพธ์นี้เราได้เห็นไปแล้วในหัวข้อก่อนหน้า แต่ตอนนี้เราจะมองมันในฐานะ characterization อย่างเต็มตัว กล่าวคือไม่เพียงพิสูจน์ว่า tree มีสมบัตินี้ แต่จะพิสูจน์กลับด้วยว่าถ้ากราฟมีสมบัตินี้แล้ว กราฟนั้นต้องเป็น tree

Theorem 10.2.1. ให้ $G = (V, E)$ เป็นกราฟ แล้วข้อความต่อไปนี้สมมูลกัน

1. G เป็นต้นไม้
2. ระหว่างจุดยอดสองจุดใด ๆ ใน G มี path เพียงเส้นเดียว

บทพิสูจน์. เราพิสูจน์ทีละทิศทาง

(1) $\Rightarrow$ (2): สมมติว่า G เป็นต้นไม้ จากนิยามของ tree, G connected ดังนั้นสำหรับทุกคู่ของจุดยอด $u, v \in V$ มี path จาก u ไป v อย่างน้อยหนึ่งเส้น

ถ้ามี path จาก u ไป v อยู่สองเส้นที่ต่างกัน พิจารณาจุดแรกที่สอง path นี้เริ่มแยกจากกัน และจุดถัดไปที่มันกลับมาพบกันอีกครั้ง ส่วนย่อยระหว่างจุดทั้งสองนี้จะให้ทางเดินสองทางที่แตกต่างกัน เมื่อนำมารวมกันจะเกิด cycle ซึ่งขัดกับการที่ G ไม่มี cycle ดังนั้น path ดังกล่าวมีได้เพียงหนึ่งเส้น

(2) $\Rightarrow$ (1): สมมติว่าระหว่างจุดยอดสองจุดใด ๆ ใน G มี path เพียงเส้นเดียว

ก่อนอื่น G ต้อง connected เพราะนิยามของสมมติฐานบอกอยู่แล้วว่า ทุกคู่จุดยอดมี path เชื่อมกัน

เหลือเพียงพิสูจน์ว่า G ไม่มี cycle สมมติย้อนแย้งว่า G มี cycle เลือกจุดยอดสองจุดต่างกัน u และ v บน cycle นั้น การเดินไปตาม cycle สองทิศทางจะให้ path จาก u ไป v สองเส้นที่ต่างกัน ซึ่งขัดกับสมมติฐานที่ว่าระหว่างทุกคู่จุดยอดมี path เพียงเส้นเดียว

ดังนั้น G connected และไม่มี cycle จึงเป็น tree □

characterization นี้สำคัญมาก เพราะทำให้เราเห็นว่า tree ไม่ได้เป็นเพียงกราฟที่ “ไม่มีวง” แต่เป็นกราฟที่มีโครงสร้างของเส้นทางเรียบง่ายที่สุดเท่าที่จะเป็นไปได้ คือไปถึงกันได้เสมอ แต่ไปได้เพียงทางเดียวเท่านั้น

10.2.2 $|E| = |V| - 1$

ในหัวข้อก่อนหน้า เราพิสูจน์แล้วว่า ถ้า T เป็นต้นไม้ แล้ว

$$|E(T)| = |V(T)| - 1.$$

ผลลัพธ์นี้สำคัญมากจนหลายคนเผลอจำว่า ``graph ที่มี $|E| = |V| - 1$ ต้องเป็น tree" แต่ข้อความนี้ *ยังไม่จริงถ้าไม่มีเงื่อนไขอื่นร่วมด้วย* เพราะกราฟอาจมี $|E| = |V| - 1$ ได้ทั้งที่ไม่ connected หรือมี cycle ดังนั้นถ้าจะใช้สมการนี้เป็น characterization เราต้องเสริมเงื่อนไขอีกด้านหนึ่งเข้าไปด้วย โดยมีสองรูปแบบที่ใช้บ่อยที่สุด:

- connected และมี $|E| = |V| - 1$
- acyclic และมี $|E| = |V| - 1$

เราจะพิสูจน์ทั้งสองแบบ

Theorem 10.2.2. ให้ $G = (V, E)$ เป็นกราฟที่ connected แล้ว G เป็นต้นไม้ ก็ต่อเมื่อ

$$|E| = |V| - 1.$$

บทพิสูจน์. ($\Rightarrow$): ถ้า G เป็นต้นไม้ เราได้พิสูจน์ไปแล้วในหัวข้อก่อนหน้านี้ว่า

$$|E| = |V| - 1.$$

($\Leftarrow$): สมมติว่า G connected และมี

$$|E| = |V| - 1.$$

เราจะพิสูจน์ว่า G ไม่มี cycle

สมมติย้อนแย้งว่า G มี cycle เลือกเส้นเชื่อมเส้นหนึ่ง e บน cycle นั้น จากทฤษฎีบทในหัวข้อก่อนหน้านี้ เส้นเชื่อมที่อยู่บน cycle ไม่เป็น bridge ดังนั้นเมื่อลบ e ออก กราฟยังคง connected อยู่

กราฟใหม่ $G - e$ จึงเป็น connected graph ที่มีจุดยอดเท่าเดิม แต่มีเส้นเชื่อมลดลงหนึ่งเส้น ดังนั้น

$$|E(G - e)| = |E| - 1 = (|V| - 1) - 1 = |V| - 2.$$

แต่กราฟ connected ใด ๆ ต้องมีเส้นเชื่อมอย่างน้อย $|V| - 1$ เส้น ซึ่งขัดแย้งกัน

ดังนั้น G ไม่มี cycle เมื่อรวมกับการที่ G connected จึงได้ว่า G เป็นต้นไม้ □

ทฤษฎีบทนี้บอกว่า ในบรรดากราฟที่ connected ทั้งหมด tree คือกราฟที่มีเส้นเชื่อมน้อยที่สุดเท่าที่จะเป็นไปได้ ถ้าน้อยกว่านี้อีกเพียงเส้นเดียว กราฟจะไม่ connected แล้วทันที

ต่อไปเป็นอีกเวอร์ชันหนึ่งที่สมมูลกัน แต่เน้นมุมมองจากฝั่ง acyclic แทน

Theorem 10.2.3. ให้ $G = (V, E)$ เป็นกราฟที่ไม่มี cycle แล้ว G เป็นต้นไม้ ก็ต่อเมื่อ

$$|E| = |V| - 1.$$

บทพิสูจน์. ($\Rightarrow$): ถ้า G เป็นต้นไม้ ย่อมมี

$$|E| = |V| - 1.$$

($\Leftarrow$): สมมติว่า G ไม่มี cycle และมี

$$|E| = |V| - 1.$$

ให้ connected components ของ G มีทั้งหมด c ส่วน แต่ละ component เป็นกราฟ connected ที่ไม่มี cycle จึงเป็น tree

ถ้า component ที่ i มีจุดยอด v_i จุด จะมีเส้นเชื่อม $v_i - 1$ เส้น เมื่อรวมทุก component เข้าด้วยกัน จะได้

$$|E| = \sum_{i=1}^c (v_i - 1) = \sum_{i=1}^c v_i - c = |V| - c.$$

แต่สมมติฐานให้ว่า $|E| = |V| - 1$ ดังนั้น

$$|V| - c = |V| - 1,$$

จึงได้ว่า $c = 1$

ดังนั้น G มี connected component เพียงส่วนเดียว กล่าวคือ G connected เมื่อรวมกับการที่ G ไม่มี cycle จึงได้ว่า G เป็นต้นไม้ □

เวอร์ชันนี้มีประโยชน์มาก เพราะมันบอกว่าในบรรดากราฟที่ไม่มี cycle ทั้งหมด tree คือกราฟที่ “มากที่สุดเท่าที่จะเชื่อมต่อได้” ถ้าเส้นเชื่อมน้อยกว่านี้อีก ก็ต้องแยกเป็นหลาย component

10.2.3 Minimal connected graphs

ทฤษฎีบทในหัวข้อก่อนหน้านี้ให้ intuition สำคัญมากกว่า tree เป็นกราฟ connected ที่มีเส้นเชื่อม “พอดี” ไม่มากเกินไป มุมมองนี้สามารถถูกเขียนใหม่ในรูปที่เน้นการลบเส้นเชื่อมได้ดังนี้: ใน tree ไม่มีเส้นเชื่อมเส้นใดที่ลบออกแล้วกราฟยัง connected อยู่ กล่าวอีกแบบหนึ่ง tree เป็น *minimal connected graph*

Theorem 10.2.4. ให้ $G = (V, E)$ เป็นกราฟ แล้วข้อความต่อไปนี้สมมูลกัน

1. G เป็นต้นไม้
2. G connected และเส้นเชื่อมทุกเส้นของ G เป็น bridge

บทพิสูจน์. (1) $\Rightarrow$ (2): สมมติว่า G เป็นต้นไม้ ดังนั้น G connected

ให้ $e \in E$ เป็นเส้นเชื่อมใด ๆ เพราะ G ไม่มี cycle จึงมีได้แน่ว่า e ไม่ได้อยู่บน cycle ใด ๆ จากทฤษฎีบทเรื่อง bridge และ cycle จึงสรุปได้ว่า e เป็น bridge

ดังนั้นเส้นเชื่อมทุกเส้นของ G เป็น bridge

(2) $\Rightarrow$ (1): สมมติว่า G connected และเส้นเชื่อมทุกเส้นเป็น bridge เราจะพิสูจน์ว่า G ไม่มี cycle

สมมติย้อนแย้งว่า G มี cycle แล้วเส้นเชื่อมทุกเส้นบน cycle นั้นจะไม่เป็น bridge ซึ่งขัดกับสมมติฐานที่ว่าเส้นเชื่อมทุกเส้นเป็น bridge ดังนั้น G ไม่มี cycle

เมื่อรวมกับการที่ G connected จึงได้ว่า G เป็นต้นไม้ □

characterization นี้ให้ภาพที่สวยงาม: ใน tree เส้นเชื่อมทุกเส้นมีความจำเป็นต่อการคงความต่อเนื่องของกราฟ ไม่มีเส้นใดเป็น "เส้นเกิน" ถ้ามีเส้นเกินแม้เพียงเส้นเดียว เส้นนั้นย่อมอยู่บน cycle และลบออกได้โดยที่กราฟยัง connected อยู่ ซึ่งจะขัดกับความเป็น minimal connected

10.2.4 Maximal acyclic graphs

มุมมองแบบ dual กับหัวข้อก่อนหน้านี้คือ แทนที่จะมอง tree เป็นกราฟ connected ที่ลบอะไรออกไม่ได้อีก เรามอง tree เป็นกราฟ acyclic ที่เพิ่มอะไรเข้าไปอีกไม่ได้ เพราะถ้าเพิ่มเส้นเชื่อมใหม่เมื่อไร จะเกิด cycle ทันที แนวคิดนี้เรียกว่า *maximal acyclic graph*

Theorem 10.2.5. ให้ $G = (V, E)$ เป็นกราฟ แล้วข้อความต่อไปนี้สมมูลกัน

1. G เป็นต้นไม้
2. G ไม่มี cycle และถ้าเพิ่มเส้นเชื่อมใหม่ระหว่างจุดยอดสองจุดที่เดิมยังไม่ adjacent กัน กราฟที่ได้จะมี cycle เสมอ

บทพิสูจน์. (1) $\Rightarrow$ (2): สมมติว่า G เป็นต้นไม้ ดังนั้น G ไม่มี cycle อยู่แล้ว

ให้ u และ v เป็นจุดยอดสองจุดที่ยังไม่ adjacent กัน เพราะ G เป็น tree ระหว่าง u และ v มี path เพียงเส้นเดียว เมื่อเพิ่มเส้นเชื่อมใหม่ $\{u, v\}$ เข้าไป เส้นเชื่อมใหม่นี้รวมกับ path เดิมจาก u ไป v จะเกิด cycle ทันที ดังนั้น G เป็น maximal acyclic

(2) $\Rightarrow$ (1): สมมติว่า G ไม่มี cycle และเป็น maximal acyclic เราจะพิสูจน์ว่า G connected

สมมติย้อนแย้งว่า G ไม่ connected เลือกจุดยอด u และ v จากคนละ connected component เนื่องจากไม่มี path ระหว่าง u กับ v ถ้าเราเพิ่มเส้นเชื่อมใหม่ $\{u, v\}$ เข้าไป กราฟที่ได้จะยังไม่มี cycle เพราะเส้นใหม่เพียงเชื่อมสอง component เข้าด้วยกัน ไม่ได้ปิดวงกับ path เดิมใด ๆ

แต่นี้ขัดกับสมมติฐานที่ว่าเมื่อเพิ่มเส้นเชื่อมใหม่แล้วต้องเกิด cycle เสมอ ดังนั้น G ต้อง connected

เมื่อรวมกับการที่ G ไม่มี cycle จึงได้ว่า G เป็นต้นไม้ $\square$

characterization นี้ให้ภาพอีกด้านหนึ่งของ tree ที่สำคัญมาก: tree เป็นกราฟที่ "กว้างที่สุดเท่าที่จะกว้างได้" โดยยังไม่มี cycle ถ้าเพิ่มเส้นเชื่อมใหม่แม้เพียงเส้นเดียว ความเป็น acyclic จะหายไปทันที

10.2.5 ความสมมูลของ characterization ต่าง ๆ ของ tree

ตอนนี้เราได้เห็นแล้วว่า tree สามารถถูกอธิบายได้หลายแบบ แต่ละแบบเน้นมุมมองต่างกัน:

- มุมมองเรื่องเส้นทาง
- มุมมองเรื่องจำนวนเส้นเชื่อม
- มุมมองเรื่องการลบเส้นเชื่อม
- มุมมองเรื่องการเพิ่มเส้นเชื่อม

เราสามารถรวบรวมทั้งหมดเป็นทฤษฎีบทเดียวได้ดังนี้

Theorem 10.2.6 (Characterizations of trees). ให้ $G = (V, E)$ เป็นกราฟ แล้วข้อความต่อไปนี้สมมูลกัน

1. G เป็นต้นไม้
2. ระหว่างจุดยอดสองจุดใด ๆ ของ G มี path เพียงเส้นเดียว
3. G connected และ $|E| = |V| - 1$
4. G connected และเส้นเชื่อมทุกเส้นของ G เป็น bridge
5. G ไม่มี cycle และเมื่อเพิ่มเส้นเชื่อมใหม่ระหว่างจุดยอดสองจุดที่เดิมยังไม่ adjacent กัน จะเกิด cycle เสมอ
6. G ไม่มี cycle และ $|E| = |V| - 1$

บทพิสูจน์. จากหัวข้อก่อนหน้าและหัวข้อนี้ เราได้พิสูจน์ความสมมูลต่อไปนี้แล้ว:

$$(1) \iff (2),$$

$$(1) \iff (3),$$

$$(1) \iff (4),$$

$$(1) \iff (5),$$

$$(1) \iff (6).$$

ดังนั้นข้อความทั้งหกข้อจึงสมมูลกันทั้งหมด $\square$

ทฤษฎีบทนี้มีความสำคัญมาก เพราะมันสอนเราว่า ในคณิตศาสตร์ วัตถุหนึ่งชนิดอาจไม่มี “คำจำกัดความที่ดีที่สุดเพียงคำเดียว” แต่มีหลายคำอธิบายที่สมมูลกัน และแต่ละคำอธิบายเหมาะกับสถานการณ์ต่างกัน

ถ้าโจทย์ถามเรื่องเส้นทาง characterization แบบ path uniqueness อาจตรงที่สุด

ถ้าโจทย์ให้จำนวนจุดยอดและจำนวนเส้นเชื่อมมา เงื่อนไข $|E| = |V| - 1$ ร่วมกับ connected หรือ acyclic อาจใช้สะดวกที่สุด

ถ้าโจทย์เกี่ยวกับการลบเส้นเชื่อม มุมมองแบบ minimal connected graph มักเป็นธรรมชาติที่สุด

และถ้าโจทย์เกี่ยวกับการเพิ่มเส้นเชื่อม มุมมองแบบ maximal acyclic graph จะให้ intuition ที่ชัดเจนกว่า

ดังนั้นการรู้ characterization หลายแบบของ tree จึงไม่ใช่การจำข้อความหลายประโยคที่พูดเรื่องเดียวกันซ้ำ ๆ แต่คือการมีเครื่องมือหลายชิ้นสำหรับมองวัตถุเดียวกันจากหลายมุม ซึ่งเป็นทักษะสำคัญมากในคณิตศาสตร์ดิสครีตและในการพิสูจน์โดยทั่วไป

เมื่อมองย้อนกลับไปทั้งบทนี้ จะเห็นว่า tree ไม่ได้สำคัญเพียงเพราะมันเป็นกราฟชนิดหนึ่งที่นิยามได้สั้น ๆ ว่า “connected และไม่มี cycle” แต่สำคัญเพราะมันเป็นจุดที่แนวคิดหลายเส้นของทฤษฎีกราฟมาบรรจบกัน เราเริ่มจากภาษาของ path และ cycle ขยับไปสู่ bridge และ forest จากนั้นจึงเห็นว่าโครงสร้างเดียวกันนี้ สามารถถูกอธิบายได้ทั้งในฐานะกราฟที่เส้นทางมีเอกลักษณ์ กราฟที่มีเส้นเชื่อมพอดิ กราฟที่ลบเส้นเชื่อมใดออกไม่ได้ก็ โดยยังคง connected หรือกราฟที่เพิ่มเส้นเชื่อมใดเข้าไปไม่ได้ก็ โดยยังคง acyclic ภาพทั้งหมดนี้ช่วยให้เราเห็นว่า tree เป็นโครงสร้างที่เรียบง่าย แต่มีระเบียบภายในสูงมาก

ในอีกด้านหนึ่ง บทนี้ยังทำหน้าที่เป็นตัวอย่างสำคัญของวิธีคิดในคณิตศาสตร์ดิสครีตด้วย กล่าวคือ วัตถุหนึ่งชนิด อาจไม่มีคำอธิบายที่ “ดีที่สุด” เพียงแบบเดียว แต่มีหลาย characterization ที่สมมูลกัน และแต่ละแบบเหมาะกับคำถามคนละชนิด นี่คือเหตุผลว่าทำไมการศึกษาด้านนี้จึงมีคุณค่ามากกว่าการรู้จักกราฟชนิดหนึ่งเพิ่มขึ้น เพราะมันฝึกให้เราเห็นความสัมพันธ์ระหว่างนิยาม ทฤษฎีบท และ representation ที่ต่างกันของโครงสร้างเดียวกัน

อย่างไรก็ตาม จนถึงตอนนี้ เรายังมองกราฟเป็นหลักผ่าน ภาพเชิงโครงสร้าง เราพูดถึงจุดยอด เส้นเชื่อม เส้นทาง วง

ความต่อเนื่อง และต้นไม้ ซึ่งเป็นมุมมองที่เป็นธรรมชาติที่สุดเมื่อเริ่มเรียนทฤษฎีกราฟ แต่เมื่อเราต้องการคำนวณกับกราฟอย่างเป็นระบบมากขึ้น หรือต้องการให้คอมพิวเตอร์จัดการข้อมูลของกราฟ การวาดรูปหรือการอธิบายด้วยภาษาของ path และ cycle เพียงอย่างเดียวอาจไม่เพียงพออีกต่อไป เราจึงต้องการภาษาที่เป็นเชิงพีชคณิตมากขึ้น และสามารถเก็บข้อมูลของกราฟไว้ในรูปที่นำไปคำนวณต่อได้สะดวก

ด้วยเหตุนี้ จากบทถัดไป เราจะค่อย ๆ เปลี่ยนมุมมองจาก ``กราฟในฐานะโครงสร้างที่มองเห็นเป็นจุดและเส้น" ไปสู่ ``กราฟในฐานะวัตถุที่แทนได้ด้วยเมทริกซ์และความสัมพันธ์" ซึ่งจะเปิดประตูไปสู่การเชื่อม graph theory เข้ากับ algebra directed graphs และ relation theory อย่างเป็นระบบมากขึ้น

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 11

Algebra ของ Graph

ในบทก่อนหน้านี้ เราศึกษากราฟผ่านมุมมองเชิงโครงสร้างเป็นหลัก เราเริ่มจากภาษาพื้นฐานของ simple graphs ไปสู่ walk, path, cycle, connectedness, จากนั้นจึงศึกษาต้นไม้ในฐานะกราฟที่ connected และไม่มี cycle มุมมองเหล่านี้ช่วยให้เราเข้าใจรูปร่างเชิงตรรกะของกราฟได้ดีมาก เพราะทำให้เห็นว่าโครงสร้างชนิดหนึ่งเชื่อมโยงกันอย่างไร มีวงหรือไม่ และเส้นทางระหว่างจุดยอดต่าง ๆ มีลักษณะอย่างไร

อย่างไรก็ตาม ในหลายสถานการณ์ เราไม่ได้ต้องการเพียง “เข้าใจรูปของกราฟ” แต่ต้องการแทนกราฟให้อยู่ในรูปที่คำนวณได้สะดวกด้วย โดยเฉพาะเมื่อกราฟมีขนาดใหญ่ เมื่อเราต้องการให้คอมพิวเตอร์จัดการกับมัน หรือเมื่อเราต้องการเชื่อมทฤษฎีกราฟเข้ากับพีชคณิตเชิงเส้น วิธีแทนกราฟในรูปของรูปวาดอาจไม่ใช่เครื่องมือที่เหมาะสมที่สุดอีกต่อไป

บทนี้จึงมีเป้าหมายต่างจากสองบทก่อนหน้านี้เล็กน้อย ถ้าบทกราฟเบื้องต้นและบทต้นไม้เน้น โครงสร้าง บทนี้จะเน้น ตัวแทน และ การคำนวณ เราจะศึกษาว่ากราฟสามารถถูกแทนด้วย adjacency matrix ได้อย่างไร กำลังของเมทริกซ์เหล่านี้บอกข้อมูลอะไรเกี่ยวกับ walks กราฟมีทิศทางเปลี่ยนภาพของเมทริกซ์อย่างไร และเหตุใด relation บนเซตจำกัดจึงสามารถถูกมองได้ทั้งในฐานะ directed graph และในฐานะ 0-1 matrix กล่าวอีกแบบหนึ่ง บทนี้คือจุดที่ graph theory เริ่มเชื่อมกับ algebra อย่างจริงจัง

11.1 Adjacency Matrices

ในหัวข้อก่อนหน้านี้ เราศึกษากราฟผ่านภาษาของจุดยอด เส้นเชื่อม เส้นทาง และความต่อเนื่อง ซึ่งเป็นมุมมองเชิงโครงสร้างโดยตรง อย่างไรก็ตาม เมื่อทฤษฎีกราฟเริ่มเชื่อมกับพีชคณิตเชิงเส้น หรือเมื่อเราต้องการให้คอมพิวเตอร์จัดการข้อมูลของกราฟอย่างเป็นระบบ เรามักต้องการวิธีแทนกราฟให้อยู่ในรูปที่คำนวณได้สะดวกกว่าการวาดรูป แนวคิดสำคัญที่สุดอย่างหนึ่งในจุดนี้คือ adjacency matrix

เมทริกซ์ชนิดนี้เป็นตัวอย่างที่ดีมากของความคิดในคณิตศาสตร์ที่สคริตว่า

โครงสร้างเดียวกันอาจถูกมองได้หลาย representation

กราฟแบบเดียวกัน ถ้ามองเชิงรูปภาพ เราเห็นจุดและเส้น แต่ถ้ามองเชิงเมทริกซ์ เราจะเห็นตาราง 0 และ 1 ที่เข้ารหัสว่า จุดยอดคู่ใดเชื่อมกันอยู่ และเมื่อเปลี่ยนมุมมองเป็นเมทริกซ์แล้ว เครื่องมือทางพีชคณิตจำนวนมากก็เริ่มเข้ามาจับบทบาทได้ทันที

สิ่งที่ทำให้ adjacency matrix น่าสนใจมาก ไม่ใช่เพียงเพราะมันเป็นวิธีเก็บข้อมูลของกราฟ แต่เพราะมันสะพานเชื่อมระหว่าง “รูปทรงของกราฟ” กับ “การคำนวณด้วยเมทริกซ์” โดยเฉพาะอย่างยิ่ง กำลังของ adjacency matrix จะช่วยให้เรานับจำนวน walks ได้ ซึ่งเป็นข้อเท็จจริงที่สวยงามและสำคัญมากในทฤษฎีกราฟ

11.1.1 Adjacency matrix ของกราฟไม่มีทิศทาง

ให้ $G = (V, E)$ เป็น simple graph และสมมติว่าเราจัดลำดับจุดยอดของกราฟเป็น

$$V(G) = \{v_1, v_2, \dots, v_n\}.$$

เมื่อจุดยอดถูกเรียงลำดับแล้ว เราสามารถสร้างเมทริกซ์ขนาด $n \times n$ เพื่อบันทึกว่าแต่ละคู่ของจุดยอดเชื่อมกันหรือไม่

Definition 11.1.1 (adjacency matrix ของ simple graph). ให้ $G = (V, E)$ เป็น simple graph และให้

$$V(G) = \{v_1, v_2, \dots, v_n\}.$$

เมทริกซ์ $A_G = (a_{ij})$ ขนาด $n \times n$ เรียกว่า **adjacency matrix** ของ G ถ้า

$$a_{ij} = \begin{cases} 1 & \text{ถ้า } \{v_i, v_j\} \in E(G), \\ 0 & \text{ถ้า } \{v_i, v_j\} \notin E(G). \end{cases}$$

นิยามนี้ควรอ่านอย่างระมัดระวัง เพราะมีรายละเอียดสำคัญหลายอย่างซ่อนอยู่

อย่างแรก adjacency matrix ของ simple graph เป็นเมทริกซ์แบบ 0-1 กล่าวคือทุก entry มีได้เพียง 0 หรือ 1 เท่านั้น ค่า 1 หมายถึง “มีเส้นเชื่อม” และค่า 0 หมายถึง “ไม่มีเส้นเชื่อม”

อย่างที่สอง เพราะ simple graph ไม่มีทิศทาง เราจึงมี

$$\{v_i, v_j\} = \{v_j, v_i\}.$$

ดังนั้นจะได้ทันทีว่า

$$a_{ij} = a_{ji}$$

สำหรับทุก i, j ซึ่งหมายความว่า adjacency matrix ของกราฟไม่มีทิศทางเป็น เมทริกซ์สมมาตร (symmetric matrix)

อย่างที่สาม เพราะ simple graph ไม่มี self-loop จึงไม่มีเส้นเชื่อมจากจุดยอดกลับมาหาตัวเอง ดังนั้น

$$a_{ii} = 0 \quad \text{สำหรับทุก } i.$$

กล่าวคือ entry บนเส้นทแยงมุมหลักเป็นศูนย์ทั้งหมด

Example 11.1.2. ให้ G เป็นกราฟที่มี

$$V(G) = \{1, 2, 3, 4, 5\}$$

และ

$$E(G) = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}\}.$$

ถ้าเรียงจุดยอดตามลำดับ 1, 2, 3, 4, 5 adjacency matrix ของ G คือ

$$A_G = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

จากตัวอย่างนี้ ผู้อ่านควรเริ่มเห็นว่า adjacency matrix ไม่ได้บันทึก ``หน้าตาการวาดกราฟ" แต่บันทึก ข้อมูลการเชื่อมกันของจุดยอด ในรูปแบบที่เป็นตาราง และเมื่อเขียนในรูปแบบนี้ เราสามารถใช้สายตาอ่านคุณสมบัติบางอย่างของกราฟได้ทันที เช่นความสมมาตรของเมทริกซ์ หรือการไม่มี loop จากค่าบนแนวทแยง

อย่างไรก็ตาม มีข้อควรระวังสำคัญอย่างหนึ่งคือ adjacency matrix ขึ้นกับลำดับการเรียงจุดยอด ถ้าเราเปลี่ยนการเรียงจุดยอด เมทริกซ์ที่ได้ก็จะเปลี่ยนไปด้วย แม้กราฟเชิงโครงสร้างจะยังเป็นกราฟเดิม

ดังนั้นเวลาพูดถึง adjacency matrix เราจึงต้องไม่ลืมว่า มันเป็น representation ของกราฟเมื่อจุดยอดถูกจัดลำดับแล้ว ไม่ใช่สิ่งที่แยกขาดจากการตั้งชื่อหรือการเรียงลำดับจุดยอด

11.1.2 การอ่านข้อมูลกราฟจากเมทริกซ์

เมื่อเรามี adjacency matrix อยู่ในมือแล้ว คำถามต่อไปคือ เราจะอ่านข้อมูลอะไรของกราฟออกจากเมทริกซ์ได้บ้าง และอ่านอย่างไรให้ถูกต้อง

ข้อแรกที่เราเห็นได้ทันทีคือ entry a_{ij} บอกความเป็น adjacency ระหว่างจุดยอด v_i และ v_j กล่าวคือ

$$a_{ij} = 1 \iff v_i \text{ และ } v_j \text{ เชื่อมกันด้วยเส้นเชื่อมหนึ่งเส้น.}$$

ดังนั้นแต่ละแถวและแต่ละคอลัมน์ของเมทริกซ์ จึงเป็นเหมือน "ลายเซ็นของเพื่อนบ้าน" ของจุดยอดนั้น

สิ่งที่สำคัญมากอีกอย่างหนึ่งคือ ดีกรีของจุดยอดสามารถอ่านได้จากผลรวมในแถวหรือคอลัมน์

Proposition 11.1.3. ให้ G เป็น simple graph และ $A_G = (a_{ij})$ เป็น adjacency matrix ของ G แล้วสำหรับทุก i ,

$$\deg(v_i) = \sum_{j=1}^n a_{ij}.$$

บทพิสูจน์. จากนิยามของ adjacency matrix entry a_{ij} มีค่าเป็น 1 ก็ต่อเมื่อมีเส้นเชื่อมระหว่าง v_i กับ v_j และมีค่าเป็น 0 ในกรณีอื่น ดังนั้นเมื่อรวมค่าในแถวที่ i เข้าด้วยกัน เรากำลังนับจำนวนจุดยอดทั้งหมดที่เชื่อมกับ v_i ซึ่งก็คือดีกรีของ v_i จึงได้ว่า

$$\deg(v_i) = \sum_{j=1}^n a_{ij}.$$

□

เนื่องจากเมทริกซ์เป็น symmetric ผลรวมในคอลัมน์ที่ i ก็ให้ค่าเดียวกันเช่นกัน ดังนั้นสำหรับกราฟไม่มีทิศทางจะมองดีกรีผ่านแถวหรือผ่านคอลัมน์ก็ได้

Example 11.1.4. พิจารณาเมทริกซ์

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

ผลรวมของแถวที่ 4 คือ

$$0 + 1 + 1 + 0 + 1 = 3$$

ดังนั้นจุดยอด v_4 มีดีกรี 3

นอกจากนี้แล้ว รูปแบบของเมทริกซ์ยังให้ข้อมูลเชิงคุณภาพอื่นได้อีก ตัวอย่างเช่น ถ้าเมทริกซ์ไม่ symmetric เราย่อมรู้ทันทีว่ามันไม่ใช่ adjacency matrix ของ simple graph ไม่มีทิศทาง หรือถ้ามีค่า 1 ปรากฏบนแนวทแยงหลัก ก็แปลว่ากำลังทำงานกับกราฟที่มี loop ไม่ใช่ simple graph ในความหมายที่เราใช้ในบทนี้

อีกสิ่งหนึ่งที่ควรระวังคือ แม้ adjacency matrix จะเก็บข้อมูลของกราฟได้ครบ แต่การ “อ่านกราฟ” จากเมทริกซ์ต้องทำอย่างมีวินัย ผู้เรียนจำนวนมากมักเผลอสับสนระหว่าง แถวที่ i กับจุดยอด v_i หรือมองข้ามความจริงที่ว่าค่า a_{ij} และ a_{ji} เป็นข้อมูลเดียวกันในกราฟไม่มีทิศทาง ดังนั้นเมื่อใช้เมทริกซ์ ควรยึดการอ่านตามนิยามอย่างเคร่งครัดเสมอ

ในภาพรวม adjacency matrix ให้ representation ของกราฟที่เหมาะสมมากกับการคำนวณ เพราะมันแปลงข้อมูลเชิงโครงสร้างให้กลายเป็นวัตถุทางพีชคณิต และเมื่อเราเริ่มคูณเมทริกซ์นี้กับตัวเอง ความหมายเชิงกราฟที่น่าสนใจมากก็จะเริ่มปรากฏ

11.1.3 กำลังของ adjacency matrix และการนับจำนวน walks

หัวข้อที่สำคัญที่สุดของส่วนนี้คือ กำลังของ adjacency matrix ไม่ได้เป็นเพียงการคำนวณทางพีชคณิตที่ทำไปตามสูตร แต่มีความหมายเชิงกราฟที่ลึกและชัดมาก: มันใช้ นับจำนวน walks

ก่อนจะกล่าวทฤษฎีบททั่วไป ลองเริ่มจากความคิดง่าย ๆ ก่อน entry a_{ij} ของ A บอกว่ามี walk ความยาว 1 จาก v_i ไป v_j หรือไม่ เพราะ walk ความยาว 1 ก็คือเส้นเชื่อมหนึ่งเส้นนั่นเอง ถัดมา entry ของ A^2 จะเกี่ยวกับการเดินสองก้าว และ entry ของ A^k จะเกี่ยวกับการเดิน k ก้าว ทฤษฎีบทต่อไปนี้จะทำให้ความคิดนี้แม่นยำอย่างสมบูรณ์

Theorem 11.1.5 (Walk counting theorem). ให้ G เป็นกราฟที่มี adjacency matrix เท่ากับ A แล้วสำหรับทุกจำนวนเต็มบวก k entry ตำแหน่ง (i, j) ของเมทริกซ์ A^k เท่ากับจำนวน walks ความยาว k จาก v_i ไป v_j

ทฤษฎีบทนี้สำคัญมาก และควรหยุดทำความเข้าใจช้า ๆ เพราะมันเป็นจุดที่พีชคณิตของเมทริกซ์เริ่มสะท้อนโครงสร้างของกราฟโดยตรง สัญลักษณ์

$$(A^k)_{ij}$$

ไม่ได้เป็นเพียงตัวเลขจากการคูณเมทริกซ์ซ้ำ ๆ แต่เป็นจำนวนวิธีเดินจาก v_i ไป v_j โดยใช้ขอบทั้งหมด k เส้น

กรณี $k = 2$ ลองดูกรณีแรกที่ไม่เป็นเรื่องเล็กน้อย คือ A^2 เรามี

$$(A^2)_{ij} = \sum_{r=1}^n a_{ir}a_{rj}.$$

พจน์ $a_{ir}a_{rj}$ จะมีค่าเป็น 1 ก็ต่อเมื่อ

$$a_{ir} = 1 \quad \text{และ} \quad a_{rj} = 1,$$

ซึ่งหมายความว่า มีเส้นเชื่อมจาก v_i ไป v_r และมีเส้นเชื่อมจาก v_r ไป v_j ดังนั้น v_r จึงทำหน้าที่เป็น จุดยอด
ตัวกลาง ของ walk ความยาว 2 จาก v_i ไป v_j

เมื่อรวมทุกค่า r ผลรวมนี้จึงนับจำนวนจุดยอดตัวกลางทั้งหมดที่ทำให้เดินสองก้าวจาก v_i ไป v_j ได้ กล่าวคือมัน
นับจำนวน walks ความยาว 2 นั้นเอง

บทพิสูจน์.[Proof for the case $k = 2$] ให้ $A = (a_{ij})$ เป็น adjacency matrix ของกราฟ G จากนิยามการ
คูณเมทริกซ์

$$(A^2)_{ij} = \sum_{r=1}^n a_{ir}a_{rj}.$$

สำหรับดัชนี r ใด ๆ พจน์ $a_{ir}a_{rj}$ มีค่าเป็น 1 ก็ต่อเมื่อ $\{v_i, v_r\} \in E$ และ $\{v_r, v_j\} \in E$ ซึ่งเท่ากับว่ามี
walk

$$v_i, v_r, v_j$$

ความยาว 2 จาก v_i ไป v_j

ถ้าสำหรับค่า r ใดเงื่อนไขนี้ไม่เป็นจริง พจน์นั้นจะเป็น 0

ดังนั้นผลรวม

$$\sum_{r=1}^n a_{ir}a_{rj}$$

จึงนับจำนวน walks ความยาว 2 จาก v_i ไป v_j สรุปว่า

$$(A^2)_{ij}$$

เท่ากับจำนวน walks ความยาว 2 จาก v_i ไป v_j

□

กรณีทั่วไปของ k พิสูจน์ได้ด้วยอุปนัยบน k แต่ในขั้นต้น สิ่งสำคัญกว่า proof เติมรูปแบบคือการเข้าใจความหมาย
ของข้อความนี้ให้แม่นยำ เพราะมันจะถูกใช้ซ้ำในทั้งทฤษฎีกราฟ การคำนวณ และในมุมมองที่เชื่อมโยงไปสู่ relation
ภายหลัง

Example 11.1.6. ให้

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

ค่า $(A^2)_{14}$ หมายถึงจำนวน walks ความยาว 2 จาก v_1 ไป v_4

ถ้ามองจากกราฟ เราต้องหาจุดยอดตัวกลาง v_r ที่ทำให้

$$v_1 \rightarrow v_r \rightarrow v_4$$

เดินได้จริง ในกรณีนี้มีเพียง v_2 เท่านั้น ดังนั้น

$$(A^2)_{14} = 1.$$

Example 11.1.7. ค่า $(A^2)_{22}$ มีความหมายว่า จำนวน walks ความยาว 2 ที่เริ่มจาก v_2 แล้วกลับมาที่ v_2

ในกราฟไม่มีทิศทางแบบง่าย walk ความยาว 2 ที่เริ่มและจบที่ v_2 ต้องมีรูป

$$v_2, v_r, v_2$$

โดยที่ v_r เป็นเพื่อนบ้านของ v_2 ดังนั้นจำนวน walks เช่นนี้จึงเท่ากับดีกรีของ v_2

ข้อนี้สะท้อนอีกมุมหนึ่งของความจริงที่ว่า

$$(A^2)_{ii} = \deg(v_i)$$

สำหรับ simple graph

จุดที่ควรระวังมากคือ ทฤษฎีบทนี้พูดถึง walks ไม่ใช่ paths กล่าวคืออนุญาตให้ผ่านจุดยอดซ้ำหรือเส้นเชื่อมซ้ำได้ ดังนั้น entry ของ A^k จึงนับจำนวนการเดิน k ก้าวทั้งหมด ไม่ใช่จำนวนเส้นทางแบบไม่ซ้ำ

ความต่างนี้สำคัญมาก เพราะหลายครั้งผู้เริ่มต้นเห็นคำว่า “เดินจาก v_i ไป v_j ” แล้วเผลอคิดไปถึง path โดยอัตโนมัติ แต่ในความจริง การคูณเมทริกซ์แบบปกตินับโครงสร้างที่กว้างกว่า path อยู่มาก

ข้อควรจำ

$$(A^k)_{ij} = \text{จำนวน walks ความยาว } k \text{ จาก } v_i \text{ ไป } v_j.$$

คำว่า walk อนุญาตให้ซ้ำได้

ถ้าต้องการนับ path จะยากกว่านี้มาก และโดยทั่วไปไม่สามารถอ่านตรง ๆ จาก A^k ได้

สรุปแล้ว adjacency matrix ให้มุมมองใหม่กับกราฟในสามระดับพร้อมกัน:

- ระดับแรก มันบันทึกว่าใครเชื่อมกับใคร
- ระดับที่สอง มันทำให้เราอ่านข้อมูลพื้นฐานอย่างตึกได้
- ระดับที่สาม มันทำให้การคูณเมทริกซ์กลายเป็นเครื่องมือสำหรับนับ walks ในกราฟ

นี่คือหนึ่งในตัวอย่างที่สวยงามที่สุดของการที่ representation ทางพีชคณิต ไม่ได้เพียง “แทนข้อมูลเดิม” แต่เปิดประตูให้เราเห็นคุณสมบัติใหม่ของโครงสร้างเดิมได้ด้วย และในหัวข้อถัดไป แนวคิดนี้จะยิ่งชัดเจนขึ้นอีกเมื่อเราขยับจากกราฟไม่มีทิศทาง ไปสู่ directed graphs และความสัมพันธ์ ซึ่ง adjacency matrix จะกลายเป็นภาษากลางที่สำคัญอย่างยิ่ง

11.2 Directed Graphs

ในหัวข้อก่อนหน้านี้ เราศึกษากราฟไม่มีทิศทางและ adjacency matrix โดยเน้นภาพว่าเส้นเชื่อม $\{u, v\}$ เป็นเพียงการบอกว่า จุดยอดสองจุด “เชื่อมกัน” โดยไม่สนว่าการเชื่อมนั้นมีทิศทางหรือไม่ ด้วยเหตุนี้ adjacency matrix ของกราฟไม่มีทิศทางจึงเป็นเมทริกซ์สมมาตร เพราะความสัมพันธ์ระหว่าง u กับ v เป็นเรื่องเดียวกันกับความสัมพันธ์ระหว่าง v กับ u

อย่างไรก็ตาม ในหลายปัญหาที่สำคัญทั้งในคณิตศาสตร์ดิสครีตและวิทยาการคอมพิวเตอร์ ความเชื่อมโยงมี *ทิศทาง* เป็นส่วนหนึ่งของความหมาย ตัวอย่างเช่น ถ้าวิชาหนึ่งเป็นวิชาบังคับก่อนของอีกวิชาหนึ่ง เราจะถือว่า “บังคับก่อน” กับ “ถูกบังคับหลัง” เป็นเรื่องเดียวกันไม่ได้ หรือถ้าในเครือข่ายหนึ่งมีลูกศรจาก u ไป v ก็ไม่ได้แปลว่าจะย้อนจาก v กลับไป u ได้เสมอ ในสถานการณ์เช่นนี้ กราฟไม่มีทิศทางไม่เพียงพออีกต่อไป เราจึงต้องใช้ **directed graph** หรือ **digraph**

สิ่งที่ควรสังเกตคือ directed graph ไม่ได้เป็นเพียงกราฟธรรมดาที่เติมหัวลูกศรเข้าไป แต่มันเปลี่ยนภาษาทางคณิตศาสตร์ของโครงสร้างนั้นด้วย ในกราฟไม่มีทิศทาง เราแทนเส้นเชื่อมด้วยเซต $\{u, v\}$ แต่เมื่อมีทิศทาง เราต้องแทนการเชื่อมด้วย *ordered pair* คือ (u, v) ซึ่งต่างจาก (v, u) โดยทั่วไป และเมื่อภาษาเปลี่ยนจากเซตไปเป็น ordered pairs adjacency matrix ก็เปลี่ยนความหมายตามไปด้วยเช่นกัน

11.2.1 Directed graph และ ordered pairs

กราฟมีทิศทางเป็นเครื่องมือสำหรับอธิบายสถานการณ์ที่ การเชื่อมจากจุดยอดหนึ่งไปยังอีกจุดยอดหนึ่ง ไม่ได้สมมาตรกันเสมอ ดังนั้นวัตถุพื้นฐานของมันจึงไม่ใช่ ``เส้นเชื่อมระหว่างสองจุด" ในความหมายแบบไม่มีทิศทาง แต่เป็น ``ลูกศรจากจุดหนึ่งไปยังอีกจุดหนึ่ง"

Definition 11.2.1 (directed graph). **Directed graph** หรือ **digraph** คือโครงสร้าง

$$D = (V, E)$$

โดยที่ V เป็นเซตของจุดยอด และ

$$E \subseteq V \times V.$$

สมาชิกของ E เรียกว่า **directed edges** หรือ **arcs**

นิยามนี้ควรอ่านอย่างระมัดระวัง เพราะมีความแตกต่างจาก **simple graph** ที่สำคัญมาก: ในกรณีนี้ เส้นเชื่อมไม่ได้เป็นเซต $\{u, v\}$ แต่เป็น **ordered pair** (u, v) ซึ่งหมายถึงมีลูกศรจาก u ไป v

ดังนั้นโดยทั่วไป

$$(u, v) \neq (v, u),$$

และการที่ $(u, v) \in E$ ไม่ได้บังคับว่า $(v, u) \in E$ ด้วย นี่คือจุดที่ทำให้ directed graph สามารถแทนความสัมพันธ์ที่มีทิศทางได้จริง

Example 11.2.2. ถ้า

$$V = \{a, b, c, d\}$$

และ

$$E = \{(a, b), (a, d), (b, c), (c, a), (d, b)\},$$

เราจะวาดกราฟนี้เป็นลูกศร

$$a \rightarrow b, \quad a \rightarrow d, \quad b \rightarrow c, \quad c \rightarrow a, \quad d \rightarrow b.$$

ในกราฟนี้ การมีลูกศร $a \rightarrow b$ ไม่ได้แปลว่าต้องมีลูกศร $b \rightarrow a$

อีกประเด็นหนึ่งที่ควรกล่าวตั้งแต่ต้นคือ ใน directed graph เรามักยอมให้มี **loop** กล่าวคือลูกศรจากจุดยอดกลับมาหาตัวเอง เช่น $(c, c) \in E$ ซึ่งมีความหมายว่ามี arc จาก c ไป c เอง รายละเอียดนี้ต่างจาก **simple graph** ที่เราเคยนิยามไว้ก่อนหน้านี้ ซึ่งไม่อนุญาต loop

เมื่อเริ่มคุ้นกับ directed graph แล้ว เรามักต้องการวัดว่าแต่ละจุดยอดมีลูกศรเข้าและออกมากน้อยเพียงใด ถ้ามีลูกศรออกจาก v ไปยังจุดยอดอื่นหลายเส้น เราจะมองว่า v มี **out-degree** สูง และถ้ามีลูกศรจากจุดยอดอื่นวิ่ง

เข้ามาที่ v หลายเส้น เราจะมองว่า v มี **in-degree** สูง แม้ในส่วนนี้เรายังไม่จำเป็นต้องใช้นิยามอย่างเป็นทางการมากนัก แต่ภาพนี้จะมีประโยชน์ทันทีเมื่อเราเปลี่ยนไปมอง digraph ผ่าน adjacency matrix เพราะผลรวมของแถวและคอลัมน์จะสะท้อน out-degree และ in-degree ตามลำดับ

11.2.2 Adjacency matrix ของ directed graph

เมื่อเรามี directed graph อยู่ในมือ เราก็ต้องการ representation เชิงเมทริกซ์เช่นเดียวกับกราฟไม่มีทิศทางหลัก คิดพื้นฐานยังเหมือนเดิม: entry ตำแหน่ง (i, j) จะบอกว่า มีการเชื่อมจาก v_i ไป v_j หรือไม่ แต่เพราะตอนนี้การเชื่อมมีทิศทาง เมทริกซ์ที่ได้จึงไม่จำเป็นต้อง symmetric อีกต่อไป

Definition 11.2.3 (adjacency matrix ของ directed graph). ให้ $D = (V, E)$ เป็น directed graph และให้

$$V(D) = \{v_1, v_2, \dots, v_n\}.$$

เราเรียกเมทริกซ์ $A_D = (a_{ij})$ ขนาด $n \times n$ ว่า **adjacency matrix** ของ D ถ้า

$$a_{ij} = \begin{cases} 1 & \text{ถ้า } (v_i, v_j) \in E(D), \\ 0 & \text{ถ้า } (v_i, v_j) \notin E(D). \end{cases}$$

นิยามนี้เกือบเหมือนกรณีกราฟไม่มีทิศทางทุกอย่าง ยกเว้นจุดสำคัญที่สุดคือ ตอนนี้ a_{ij} พูดยถึงลูกศรจาก v_i ไป v_j ไม่ใช่เพียงการที่สองจุดยอดเชื่อมกันแบบไม่สนทิศทาง ดังนั้นข้อมูลในตำแหน่ง (i, j) และ (j, i) อาจต่างกันก็ได้ โดยสมบูรณ์

Example 11.2.4. ให้ directed graph D มีจุดยอด

$$V(D) = \{a, b, c, d\}$$

และมี arcs

$$E(D) = \{(a, b), (a, d), (b, c), (c, a), (c, c), (d, b)\}.$$

ถ้าเรียงจุดยอดตามลำดับ a, b, c, d adjacency matrix ของ D คือ

$$A_D = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

จากตัวอย่างนี้ มีข้อสังเกตสำคัญหลายอย่างที่ควรเห็นทันที

อย่างแรก เมทริกซ์นี้ไม่ symmetric เพราะ $a_{12} = 1$ แต่ $a_{21} = 0$ ซึ่งสะท้อนว่ามีลูกศรจาก a ไป b แต่ไม่มีลูกศรจาก b ไป a

อย่างที่สอง entry บนเส้นทแยงมุมหลักอาจเป็น 1 ได้ เช่น $a_{33} = 1$ ซึ่งบอกว่ามี loop ที่จุดยอด c

อย่างสาม ผลรวมของแถวที่ i ให้จำนวนลูกศรที่ออกจาก v_i หรือ out-degree ของ v_i ขณะที่ผลรวมของคอลัมน์ที่ j ให้จำนวนลูกศรที่วิ่งเข้ามาที่ v_j หรือ in-degree ของ v_j

ดังนั้นถ้า $A_D = (a_{ij})$ เราจะได้ว่า

$$\deg^+(v_i) = \sum_{j=1}^n a_{ij}, \quad \deg^-(v_i) = \sum_{j=1}^n a_{ji}.$$

ข้อเท็จจริงนี้ช่วยให้เราอ่านข้อมูลเชิงทิศทางของ digraph ได้โดยตรงจากเมทริกซ์ ซึ่งต่างจากกราฟไม่มีทิศทางที่ผลรวมแถวกับผลรวมคอลัมน์ให้ความหมายเดียวกัน

สรุปสั้น ๆ ก็คือ adjacency matrix ของ directed graph เป็น **0--1 matrix** เช่นเดียวกับกราฟไม่มีทิศทาง แต่โดยทั่วไปจะ

- ไม่ symmetric
- มีค่าบนเส้นทแยงมุมเป็น 1 ได้ถ้ามี loop
- แถวและคอลัมน์มีความหมายต่างกัน

และความต่างเหล่านี้ทั้งหมดเกิดจากข้อเท็จจริงพื้นฐานเพียงข้อเดียว: (v_i, v_j) เป็น ordered pair ไม่ใช่เซตสองสมาชิก

11.2.3 Directed walks และการตีความเมทริกซ์

เมื่อมี directed graph แล้ว เราย่อมต้องการนิยามการเดินบนกราฟเช่นเดียวกับกราฟไม่มีทิศทาง แต่คราวนี้การเดินจะต้อง *เคารพทิศของลูกศร* ด้วย เราไม่สามารถย้อนลูกศรกลับทางได้ตามใจ

Definition 11.2.5 (directed walk). ให้ $D = (V, E)$ เป็น directed graph **directed walk** จาก v_0 ไป v_k คือ ลำดับของจุดยอด

$$v_0, v_1, \dots, v_k$$

ที่ทำให้

$$(v_{i-1}, v_i) \in E \quad \text{สำหรับทุก } i = 1, 2, \dots, k.$$

จำนวน k เรียกว่า **ความยาว** ของ directed walk นี้

นิยามนี้เหมือนนิยามของ walk ในกราฟไม่มีทิศทางเกือบทุกอย่าง ยกเว้นตรงที่เงื่อนไขการเชื่อมต่อเป็น ordered

pair ตามทิศของลูกศร ดังนั้นถ้ามีเพียง $(u, v) \in E$ แต่ไม่มี $(v, u) \in E$ เราสามารถเดินจาก u ไป v ได้ แต่ไม่สามารถเดินจาก v กลับไป u ด้วย arc เดียวกันได้

Example 11.2.6. ถ้า digraph มี arcs

$$(1, 2), (2, 3), (3, 1), (2, 4),$$

แล้ว

$$1, 2, 3, 1$$

เป็น directed walk ความยาว 3 แต่ลำดับ

$$1, 3, 2$$

ไม่เป็น directed walk ถ้าไม่มีลูกศร $(1, 3)$ และ $(3, 2)$ รองรับ

ตอนนี้คำถามสำคัญคือ กำลังของ adjacency matrix ของ directed graph ยังตีความแบบเดียวกับกราฟไม่มีทิศทางได้หรือไม่ คำตอบคือ ได้ โดยเปลี่ยนจากการนับ walks ทั่วไป มาเป็นการนับ *directed walks* แทน

Theorem 11.2.7 (Walk counting theorem for directed graphs). ให้ D เป็น directed graph ที่มี adjacency matrix เท่ากับ A แล้วสำหรับทุกจำนวนเต็มบวก k entry ตำแหน่ง (i, j) ของเมทริกซ์ A^k เท่ากับจำนวน directed walks ความยาว k จาก v_i ไป v_j

ความหมายของทฤษฎีบทนี้ควรอ่านให้ชัดว่า

$$(A^k)_{ij}$$

ไม่ได้เพียงบอกว่ามีทางจาก v_i ไป v_j หรือไม่ แต่บอกว่ามี *directed walks* ความยาว k อยู่กี่เส้น ดังนั้นมันเป็นการนับจำนวน ไม่ใช่เพียงการตรวจการมีอยู่

กรณี $k = 2$ เพื่อให้ intuition ชัดที่สุด พิจารณา

$$(A^2)_{ij} = \sum_{r=1}^n a_{ir}a_{rj}.$$

พจน์ $a_{ir}a_{rj}$ มีค่าเป็น 1 ก็ต่อเมื่อมีลูกศร

$$v_i \rightarrow v_r \quad \text{และ} \quad v_r \rightarrow v_j.$$

ดังนั้นดัชนี r แต่ละตัวกำลังทำหน้าที่เป็นจุดยอดตัวกลางของ directed walk ความยาว 2 จาก v_i ไป v_j เมื่อรวมทุกค่า r เราจึงได้จำนวน directed walks ความยาว 2 ทั้งหมด

บทพิสูจน์.[Proof for the case $k = 2$] ให้ $A = (a_{ij})$ เป็น adjacency matrix ของ digraph D จากนิยาม การคูณเมทริกซ์

$$(A^2)_{ij} = \sum_{r=1}^n a_{ir}a_{rj}.$$

สำหรับดัชนี r ใด ๆ พจน์ $a_{ir}a_{rj}$ มีค่าเป็น 1 ก็ต่อเมื่อ

$$a_{ir} = 1 \quad \text{และ} \quad a_{rj} = 1,$$

ซึ่งเท่ากับว่ามี arcs

$$(v_i, v_r) \in E \quad \text{และ} \quad (v_r, v_j) \in E.$$

นั่นคือมี directed walk

$$v_i \rightarrow v_r \rightarrow v_j$$

ความยาว 2 จาก v_i ไป v_j

ถ้าสำหรับค่า r ใดเงื่อนไขไม่เป็นจริง พจน์นั้นจะเป็น 0 ดังนั้นผลรวม

$$\sum_{r=1}^n a_{ir}a_{rj}$$

จึงนับจำนวน directed walks ความยาว 2 จาก v_i ไป v_j สรุปว่า

$$(A^2)_{ij}$$

เท่ากับจำนวน directed walks ความยาว 2 จาก v_i ไป v_j

□

ในกรณีทั่วไปของ k หลักคิดเหมือนเดิมทุกประการ เพียงแต่จุดยอดตัวกลางมีหลายตำแหน่งมากขึ้น และการคูณเมทริกซ์ซ้ำ ๆ กำลังเก็บข้อมูลว่าเราสามารถต่อ arcs ตามทิศทางได้ที่รูปแบบ

Example 11.2.8. ถ้า

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

แล้วค่า $(A^2)_{14}$ หมายถึงจำนวน directed walks ความยาว 2 จาก v_1 ไป v_4

จากเมทริกซ์จะเห็นว่า v_1 ไปได้ที่ v_2 และ v_3 จากนั้น

$$v_2 \rightarrow v_4 \quad \text{และ} \quad v_3 \rightarrow v_4$$

ทำได้ทั้งคู่ ดังนั้นมี directed walks ความยาว 2 จาก v_1 ไป v_4 อยู่ 2 เส้น และจึงได้ว่า

$$(A^2)_{14} = 2.$$

สิ่งที่ควรระวังมากคือ เหมือนกราฟไม่มีทิศทางทุกประการ: เมทริกซ์กำลังนับ **walks** ไม่ใช่ **paths** ดังนั้นอนุญาตให้ผ่านจุดยอดซ้ำหรือ arc ซ้ำได้ ยิ่งเมื่อมี loop อยู่ใน digraph จำนวน directed walks ความยาวมาก ๆ ก็ยิ่งเติบโตได้เร็วมาก เพราะเราสามารถวนอยู่ที่เดิมแล้วเดินต่อได้หลายแบบ

ภาพรวมที่ควรจำ

- directed graph ใช้ ordered pairs (u, v) แทน arcs
- adjacency matrix ของ directed graph ไม่จำเป็นต้อง symmetric
- แถวของเมทริกซ์สะท้อนลูกศรที่ออกจากจุดยอด
- คอลัมน์ของเมทริกซ์สะท้อนลูกศรที่วิ่งเข้าจุดยอด
- $(A^k)_{ij}$ ให้จำนวน directed walks ความยาว k จาก v_i ไป v_j

หัวข้อนี้มีความสำคัญมากกว่าการเพิ่มศัพท์ใหม่อีกชุดหนึ่ง เพราะ directed graphs เป็นจุดเชื่อมสำคัญระหว่าง graph theory กับ relation theory ในหัวข้อถัดไป เราจะเห็นว่าความสัมพันธ์บนเซตจำกัด สามารถถูกมองเป็น directed graph ได้โดยตรง และ adjacency matrix ก็จะถูกกลายเป็นภาษากลาง ที่ช่วยให้เราอ่าน คำนวณ และให้เหตุผลเกี่ยวกับ relations ได้อย่างเป็นระบบมากขึ้น

11.3 ความสัมพันธ์ในมุมมองของกราฟมีทิศทาง

ในบทก่อนหน้านี้ เราได้เห็นแล้วว่า directed graph เป็นภาษาที่เหมาะสมมากสำหรับอธิบาย “ความเชื่อมโยงที่มีทิศทาง” ระหว่างวัตถุต่าง ๆ และ adjacency matrix ของ directed graph ก็ช่วยให้เราเก็บข้อมูลดังกล่าวในรูปแบบที่คำนวณได้สะดวก หัวข้อนี้จะพาเราไปอีกก้าวหนึ่ง โดยชี้ให้เห็นว่าแนวคิดเรื่อง **ความสัมพันธ์** หรือ **relation** บนเซตจำกัด สามารถถูกมองเป็น directed graph ได้โดยตรง และเมื่อมองเช่นนั้น การคำนวณเกี่ยวกับ relation ก็สามารถทำผ่านเมทริกซ์ได้อย่างเป็นระบบ

สิ่งที่ควรสังเกตคือ นี่ไม่ใช่การเริ่มหัวข้อ relation ใหม่จากศูนย์ เพราะเราเคยปูพื้นเรื่อง relation ในฐานะเซตย่อยของผลคูณคาร์ทีเซียนมาแล้ว แต่ในส่วนนี้ ผู้เขียนต้องการเน้น **representation** เป็นหลัก กล่าวคือ relation หนึ่งตัว สามารถแทนได้อย่างน้อยสามแบบที่สมมูลกัน:

1. เป็นเซตของ ordered pairs
2. เป็น directed graph
3. เป็นเมทริกซ์ 0-1

เมื่อเห็นความสมมูลของทั้งสามมุมมองนี้ชัด ผู้อ่านจะสามารถสลับภาษาได้อย่างยืดหยุ่นมากขึ้น และจะเข้าใจได้ดีขึ้นว่าเหตุใดการคูณเมทริกซ์แบบ Boolean จึงสอดคล้องกับ composition ของ relations

11.3.1 Relation as a subset of $A \times A$

ให้ A เป็นเซตหนึ่ง ความสัมพันธ์บน A เป็นวิธีอย่างเป็นทางการในการบอกว่า สมาชิกบางคู่ของ A "เกี่ยวข้องกัน" ในลักษณะที่เราสนใจ กล่าวอีกแบบหนึ่ง relation เป็นเครื่องมือสำหรับเลือก ordered pairs บางคู่จาก $A \times A$ ขึ้นมา แล้วประกาศว่าคู่เหล่านั้นเป็นคู่ที่ "อยู่ในความสัมพันธ์"

Definition 11.3.1 (relation บนเซต). ให้ A เป็นเซต **relation** บน A คือสับเซตของ

$$A \times A.$$

ถ้า R เป็น relation บน A แล้ว $(a, b) \in R$ หมายความว่า a มีความสัมพันธ์ R กับ b

นิยามนี้ดูสั้นมาก แต่มีสาระสำคัญซ่อนอยู่หลายจุด

อย่างแรก relation เป็นเรื่องของ *ordered pairs* ไม่ใช่เพียงการเลือกสมาชิกสองตัวโดยไม่สนใจลำดับ ดังนั้นโดยทั่วไป

$$(a, b) \in R$$

ไม่ได้บังคับว่า

$$(b, a) \in R$$

ด้วย จุดนี้สำคัญมาก เพราะมันทำให้ relation สามารถแทนสถานการณ์ที่มีทิศทางได้ตามธรรมชาติ เช่น "เป็นวิชาบังคับก่อนของ" "น้อยกว่าหรือเท่ากับ" หรือ "มีลูกศรไปหา"

อย่างที่สอง relation ตัวหนึ่งถูกกำหนดสมบูรณ์ทันทีเมื่อเรารู้ว่า ordered pairs ใดอยู่ในมันบ้าง ดังนั้นสำหรับเซตจำกัด การเขียน relation เป็นรายชื่อของ ordered pairs จึงเป็นวิธีแทนที่ตรงไปตรงมาที่สุด

Example 11.3.2. ให้

$$A = \{1, 2, 3, 4\}$$

และให้

$$R = \{(1, 2), (1, 4), (2, 3), (3, 3), (4, 2)\}.$$

แล้ว R เป็น relation บน A

จากตัวอย่างนี้ เราสามารถอ่านได้ทันทีว่า $(1, 2) \in R$, $(1, 4) \in R$, $(3, 3) \in R$ แต่ $(2, 1) \notin R$ และ $(2, 4) \notin R$ ดังนั้น relation นี้ไม่สมมาตร และยังมี loop ที่ 3 ในความหมายว่า $(3, 3) \in R$

ตรงนี้ควรสังเกตด้วยว่า เมื่อ A เป็นเซตจำกัดที่มีสมาชิก n ตัว เซต $A \times A$ จะมี ordered pairs ทั้งหมด n^2 คู่ ดังนั้น relation หนึ่งตัวบน A ก็คือการเลือกบางคู่จาก n^2 คู่ นั้นขึ้นมา ด้วยเหตุนี้ จำนวน relation ทั้งหมดบนเซตที่มีสมาชิก n ตัวจึงเท่ากับ

$$2^{n^2},$$

เพราะแต่ละ ordered pair มีสองทางเลือก: จะอยู่ใน relation หรือไม่อยู่ใน relation ข้อสังเกตนี้ทำให้เห็นอีกครั้งว่า relation เป็นวัตถุเชิงโครงสร้างที่ตรงไปตรงมาอย่างมากในภาษาของเซต

11.3.2 Directed graph representation ของ relation

เมื่อ relation ถูกนิยามเป็นเซตของ ordered pairs การเปลี่ยนมันให้เป็น directed graph จึงแทบไม่ต้องทำอะไรเพิ่มเลย เรานำสมาชิกของเซต A มาเป็นจุดยอด และนำ ordered pair แต่ละคู่ใน relation มาเป็นลูกศร จากตัวหน้าไปยังตัวหลัง

Definition 11.3.3 (directed graph representation ของ relation). ให้ R เป็น relation บนเซต A **directed graph representation** ของ R คือ directed graph $D_R = (A, E)$ ที่กำหนดโดย

$$E = R.$$

กล่าวคือ เราวาดลูกศรจาก a ไป b ก็ต่อเมื่อ

$$(a, b) \in R.$$

นิยามนี้สำคัญมาก เพราะมันบอกว่า relation กับ directed graph ไม่ได้เป็นเพียงวัตถุที่ "คล้ายกัน" แต่ในกรณีของ relation บนเซตจำกัด พวกมันแทบจะเป็นข้อมูลชุดเดียวกันที่เขียนคนละภาษาเท่านั้น

Example 11.3.4. ให้

$$A = \{1, 2, 3, 4\}$$

และ

$$R = \{(1, 2), (1, 4), (2, 3), (3, 3), (4, 2)\}.$$

directed graph ของ R มีจุดยอด 1, 2, 3, 4 และมีลูกศร

$$1 \rightarrow 2, \quad 1 \rightarrow 4, \quad 2 \rightarrow 3, \quad 3 \rightarrow 3, \quad 4 \rightarrow 2.$$

ข้อดีของมุมมองแบบ directed graph คือ มันทำให้คุณสมบัติหลายอย่างของ relation กลายเป็นภาพที่อ่านออกได้ง่ายด้วยสายตา ตัวอย่างเช่น

- ถ้า relation เป็น reflexive จุดยอดทุกตัวต้องมี loop
- ถ้า relation เป็น symmetric ทุกลูกศร $a \rightarrow b$ ต้องมีลูกศร $b \rightarrow a$ คู่กัน
- ถ้า relation เป็น antisymmetric ลูกศรสองทิศทางพร้อมกันเกิดได้เฉพาะเมื่อเป็น loop

แม้ในส่วนนี้เรายังไม่ลงลึกเรื่องสมบัติเหล่านี้มากนัก แต่การเห็น relation เป็น directed graph จะช่วยให้การตีความคุณสมบัติต่าง ๆ ในบท relation ชัดขึ้นมากในภายหลัง

อีกสิ่งหนึ่งที่เราควรเน้นคือ directed graph representation ของ relation ไม่ใช่เพียงภาพสวยงามประกอบการสอน แต่มันเป็น representation ที่พร้อมใช้งานทางคณิตศาสตร์จริง เพราะทันทีที่เราพูดถึงการมีเส้นทางยาวสองขั้นใน directed graph เราก็กำลังเข้าใจแนวคิดเรื่อง composition ของ relation แล้ว ซึ่งจะกลับมาในหัวข้อย่อยถัดไป

11.3.3 0--1 matrix representation ของ relation

นอกจากมอง relation เป็น directed graph ได้แล้ว เรายังมอง relation เป็นเมทริกซ์ 0--1 ได้อีกด้วย มุมมองนี้สำคัญมาก เพราะทำให้คำถามเชิงโครงสร้างเกี่ยวกับ relation ถูกแปลงเป็นการคำนวณเชิงเมทริกซ์ได้

Definition 11.3.5 (0--1 matrix representation ของ relation). ให้ $A = \{a_1, a_2, \dots, a_n\}$ เป็นเซตจำกัด และให้ R เป็น relation บน A เมทริกซ์ $M_R = (m_{ij})$ ขนาด $n \times n$ เรียกว่า **matrix representation** ของ R ถ้า

$$m_{ij} = \begin{cases} 1 & \text{ถ้า } (a_i, a_j) \in R, \\ 0 & \text{ถ้า } (a_i, a_j) \notin R. \end{cases}$$

ผู้อ่านควรเห็นทันทีว่า นี่คือ adjacency matrix ของ directed graph representation ของ R นั่นเอง ดังนั้น relation, directed graph, และ 0--1 matrix จึงเป็นสาม representation ของข้อมูลชุดเดียวกันโดยตรง

Example 11.3.6. สำหรับ relation

$$R = \{(1, 2), (1, 4), (2, 3), (3, 3), (4, 2)\}$$

บนเซต $A = \{1, 2, 3, 4\}$ เมทริกซ์ของ R คือ

$$M_R = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

การอ่านเมทริกซ์นี้ควรทำอย่างมีวินัย: แถวที่ i และคอลัมน์ที่ j กำลังพูดถึง ordered pair (a_i, a_j) ไม่ใช่ unordered pair ดังนั้นตำแหน่ง (i, j) และ (j, i) จึงมีความหมายต่างกันโดยทั่วไป

จาก representation นี้ เราสามารถอ่านคุณสมบัติบางอย่างของ relation ได้ทันที เช่น

- ถ้า entry ทุกตัวบนแนวทแยงหลักเป็น 1 แสดงว่า relation เป็น reflexive
- ถ้าเมทริกซ์ symmetric แสดงว่า relation เป็น symmetric
- ถ้า entry (i, j) และ (j, i) ไม่เคยเป็น 1 พร้อมกัน ยกเว้นเมื่อ $i = j$ ก็สะท้อนลักษณะของ antisymmetry

แน่นอนว่าในหนังสือเล่มนี้ ผู้เขียนไม่ได้ต้องการให้ผู้อ่านจำรายการตรวจเมทริกซ์แบบกลไก แต่ต้องการให้เห็นว่า เหตุใดการตรวจเช่นนั้นจึงสมเหตุสมผล และคำตอบก็คือ เพราะเมทริกซ์กำลังแทน ordered pairs ของ relation โดยตรงนั่นเอง

อีกประเด็นหนึ่งที่ต้องจำคือ เมทริกซ์ของ relation ขึ้นกับลำดับการเรียงสมาชิกของเซต A เช่นเดียวกับ adjacency matrix ของกราฟ ดังนั้นถ้าเราเปลี่ยนลำดับของ $a_1, a_2, \dots, a_n$ เมทริกซ์ที่ได้ก็จะเปลี่ยนไป แม้ relation เดิมจะยังเป็น relation ตัวเดิมก็ตาม

11.3.4 Composition และ powers of relations

เมื่อ relation ถูกมองเป็น directed graph ได้ แนวคิดเรื่อง **composition** ของ relation ก็มีความหมายเชิงภาพที่เป็นธรรมชาติอย่างยิ่ง: มันคือการต่อเส้นทางที่ละช่วง

ให้ R และ S เป็น relations บนเซต A การที่ (x, z) อยู่ใน $R \circ S$ หมายความว่า มีจุดกลาง y บางตัว ที่ทำให้เดินจาก x ไป y ผ่าน S แล้วจาก y ไป z ผ่าน R ได้ ดังนั้น composition จึงเป็นการ “รวมสองก้าวให้เป็นก้าวเชิงความสัมพันธ์หนึ่งก้าว”

Definition 11.3.7 (composition ของ relations). ให้ R และ S เป็น relations บนเซต A เราให้นิยาม

$$R \circ S$$

เป็น relation บน A โดยกำหนดว่า

$$(x, z) \in R \circ S$$

ก็ต่อเมื่อมี $y \in A$ บางตัวที่ทำให้

$$(x, y) \in S \quad \text{และ} \quad (y, z) \in R.$$

จุดที่นักศึกษา มักสับสนคือ ลำดับการเขียน เพราะ

$$R \circ S$$

หมายถึงทำ S ก่อน แล้วค่อยทำ R ดังนั้นเวลาตีความด้วย directed graph เราควรอ่านว่า

$$x \rightarrow y \rightarrow z$$

โดยก้าวแรกมาจาก S และก้าวที่สองมาจาก R

กรณีพิเศษที่สำคัญมากคือ เมื่อ relation เดียวกันถูก compose กับตัวเอง เราจะได้ powers ของ relation

Definition 11.3.8 (powers of a relation). ให้ R เป็น relation บนเซต A นิยาม

$$R^2 = R \circ R,$$

และโดยทั่วไป

$$R^k = R \circ R \circ \cdots \circ R$$

โดยมี R ทั้งหมด k ตัว

ความหมายเชิงกราฟของ R^2 ง่ายมาก:

$$(x, z) \in R^2$$

ก็ต่อเมื่อมี directed walk ความยาว 2 จาก x ไป z ใน directed graph ของ R และโดยทั่วไป

$$(x, z) \in R^k$$

ก็ต่อเมื่อมี directed walk ความยาว k จาก x ไป z

Example 11.3.9. ให้

$$R = \{(1, 2), (2, 3), (1, 4), (4, 2), (3, 3)\}$$

บนเซต $A = \{1, 2, 3, 4\}$ จะเห็นว่า

$$(1, 3) \in R^2$$

เพราะมีจุดกลาง 2 ที่ทำให้

$$(1, 2) \in R \quad \text{และ} \quad (2, 3) \in R.$$

ในทำนองเดียวกัน

$$(1, 2) \in R^2$$

เพราะมีเส้นทางสองชั้น

$$1 \rightarrow 4 \rightarrow 2.$$

จุดนี้ควรสังเกตให้ชัดว่า $(x, z) \in R^2$ ไม่จำเป็นต้องหมายความว่า $(x, z) \in R$ เพราะ R^2 กำลังบอกว่า “ไปได้ในสองชั้น” ไม่ใช่ “มีลูกศรตรงอยู่แล้ว” นี่เป็นความแตกต่างที่สำคัญมาก และเป็นแหล่งของความเข้าใจผิดบ่อยครั้งในช่วงต้น

11.3.5 Boolean product กับความหมายแบบมี/ไม่มี

เมื่อ relation ถูกแทนด้วย 0--1 matrix คำถามตามธรรมชาติคือ เราจะคำนวณ composition ของ relation ด้วยเมทริกซ์ได้อย่างไร คำตอบคือใช้ **Boolean product** ซึ่งต่างจากการคูณเมทริกซ์แบบปกติเล็กน้อยตรงที่เราไม่ได้สนใจว่า “มีทางไปกี่ทาง” แต่สนใจเพียงว่า “มีหรือไม่มีทางไป”

ให้ $M = (m_{ij})$ และ $N = (n_{ij})$ เป็นเมทริกซ์ 0--1 เมื่อคูณแบบปกติ entry (i, j) ของ MN จะเป็นผลรวมของจำนวนเส้นทางผ่านตัวกลางต่าง ๆ ดังนั้นมันเหมาะกับการ นับจำนวน walks แต่สำหรับ composition ของ relation สิ่งที่เราต้องการรู้มีเพียงว่า มีดัชนีตัวกลาง r บางตัวหรือไม่ ที่ทำให้

$$m_{ir} = 1 \quad \text{และ} \quad n_{rj} = 1.$$

ถ้ามีสักตัวเดียวก็ถือว่า relation composed แล้วมีคู่ (i, j) อยู่แน่นอน เราไม่จำเป็นต้องรู้ว่าทั้งหมดกี่ทาง

Definition 11.3.10 (Boolean product). ให้ $M = (m_{ij})$ และ $N = (n_{ij})$ เป็นเมทริกซ์ 0--1 ขนาดที่คูณกันได้ เราให้นิยาม **Boolean product** $M \odot N$ โดย

$$(M \odot N)_{ij} = \bigvee_{r=1}^n (m_{ir} \wedge n_{rj}),$$

ซึ่งหมายความว่า entry ตำแหน่ง (i, j) เป็น 1 ก็ต่อเมื่อมีดัชนี r บางตัวที่ทำให้

$$m_{ir} = 1 \quad \text{และ} \quad n_{rj} = 1.$$

แม้สัญลักษณ์ $\vee$ และ $\wedge$ จะมาจากตรรกศาสตร์ แต่ความหมายเชิง relation ของนิยามนี้ตรงมาก: เรากำลังถามว่ามีหรือไม่มีจุดกลางที่ทำให้เดินได้สองช่วงต่อกัน

Theorem 11.3.11. ถ้า R และ S เป็น relations บนเซตจำกัด A แล้วเมทริกซ์ของ $R \circ S$ เท่ากับ Boolean product ของเมทริกซ์ของ R และ S กล่าวคือ

$$M_{R \circ S} = M_S \odot M_R$$

หรือขึ้นกับ convention การเรียง composition ที่ใช้

ในหนังสือเล่มนี้ สิ่งที่เราควรจำให้แม่นไม่ใช่รูปสูตรเพียงอย่างเดียว แต่คือความหมายของมัน: entry (i, j) ใน Boolean product จะเป็น 1 ก็ต่อเมื่อ (a_i, a_j) อยู่ใน composed relation จริง นั่นคือมี directed walk ตามจำนวนขั้นที่เหมาะสม อย่างน้อยหนึ่งเส้น

เพื่อป้องกันความสับสน ผู้เขียนขอย้ำความแตกต่างระหว่างการคูณเมทริกซ์สองแบบดังนี้

สองมุมมองของการคูณเมทริกซ์

- คูณแบบปกติ ใช้เมื่อต้องการรู้ว่า มีกี่ walks
- คูณแบบ Boolean ใช้เมื่อต้องการรู้ว่า มีหรือไม่มี walk หรือ relation แบบประกอบ

Example 11.3.12. ให้

$$M_R = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}.$$

ถ้าคำนวณ M_R^2 แบบปกติ entry บางตำแหน่งอาจได้ค่า 2 ซึ่งหมายความว่า มี directed walks ความยาว 2 อยู่สองเส้น แต่ถ้าคำนวณแบบ Boolean entry เดียวกันจะถูกอ่านเพียงว่าเป็น 1 เพราะเราสนใจเพียงว่ามีอย่างน้อยหนึ่งเส้นหรือไม่

สิ่งนี้สำคัญมากในเชิงแนวคิด เพราะ relation ไม่ได้บันทึก "จำนวนวิธี" ที่คู่หนึ่งสัมพันธ์กัน แต่มันบันทึกเพียงว่า คู่ดังกล่าวอยู่ใน relation หรือไม่เท่านั้น ดังนั้น Boolean product จึงเป็นการคูณที่สอดคล้องกับธรรมชาติของ relation มากกว่า

สรุปแล้ว หัวข้อนี้ควรทำให้ผู้อ่านเห็นภาพใหญ่ต่อไปนี้อย่างชัดเจน: relation บนเซตจำกัด directed graph บนเซตของจุดยอดเดียวกัน และ 0--1 matrix ที่อ้างถึงเซตนั้นตามลำดับ เป็นเพียงสามภาษาในการเขียนโครงสร้างเดียวกัน เมื่อเราคิดเชิงภาพ เราอาจใช้ directed graph เมื่อเราต้องการความเป็นทางการเชิงเซต เราอาจใช้ ordered pairs และเมื่อเราต้องการคำนวณ composition หรือ powers อย่างเป็นระบบ เมทริกซ์ โดยเฉพาะ Boolean product จะกลายเป็นเครื่องมือที่ทรงพลังมากที่สุด

Final Draft
Handout version
Phaphontee Yamchote

ภาค V

คอมบินาทอริกเบื้องต้น

Final Draft
Handout version
Phaphontee Yamchote

บทที่ 12

กฎการนับเบื้องต้น

คณิตศาสตร์เชิงการนับ หรือ *combinatorics* เป็นหัวข้อที่ทำให้เราเห็นชัดมากกว่า คณิตศาสตร์ดิสครีตไม่ได้มีแก่นอยู่กับการคำนวณเชิงตัวเลขเพียงอย่างเดียว แต่มีแก่นอยู่กับการ *จัดโครงสร้างของความเป็นไปได้* และ *อธิบายเหตุผลของการนับ* อย่างเป็นระบบ ในหลายโจทย์ เราไม่ได้ต้องการเพียงคำตอบสุดท้ายว่า “มีกี่วิธี” แต่ต้องการเข้าใจด้วยว่า เหตุใดจำนวนดังกล่าวจึงออกมาเท่านั้น เราแบ่งกรณีอย่างไร อะไรคือหนึ่งหน่วยของสิ่งที่เรากำลังนับ และเรามั่นใจได้อย่างไรว่าการนับนั้นไม่ตกหล่นและไม่นับซ้ำ

ในระดับต้น ผู้เรียนมักรู้สึกหัวข้อนี้เต็มไปด้วยสูตร เช่น แฟกทอเรียล การเรียงสับเปลี่ยน และการจัดกลุ่ม แต่ถ้าลองให้ลึกลงไป สูตรเหล่านี้ไม่ได้เป็นจุดเริ่มต้นของการคิด ตรงกันข้าม มันเป็นเพียงผลสรุปที่เกิดขึ้น *ภายหลัง* เมื่อเราเข้าใจวิธีวางแผนการนับได้ชัดแล้ว ด้วยเหตุนี้ บทนี้จึงตั้งใจเริ่มจากสิ่งที่พื้นฐานที่สุดก่อน คือการมองการนับในฐานะการวางแผน แล้วค่อยพัฒนาไปสู่หลักการบวก หลักการคูณ จากนั้นจึงต่อยอดสู่ *permutations* และ *combinations* ซึ่งเป็นเครื่องมือพื้นฐานของโจทย์การนับจำนวนมาก

กล่าวอีกแบบหนึ่ง บทนี้ทำหน้าที่วาง *ภาษาพื้นฐานของการนับ* ถ้าภาษาในบทนี้ชัด สูตรจำนวนมากในบทถัดไปจะไม่ใช่ว่าที่ต้องจำแยกเป็นรายข้อ แต่จะดูเป็นผลที่เกิดขึ้นอย่างเป็นธรรมชาติจากการให้เหตุผลที่ถูกต้อง ผู้เขียนจึงอยากให้ผู้อ่านอ่านบทนี้ด้วยท่าทีเดียวกับเวลาศึกษาการพิสูจน์: อย่าเพิ่งถามว่าต้องใช้สูตรอะไร แต่ให้ถามก่อนว่า

สิ่งที่กำลังนับคืออะไร

ผลลัพธ์หนึ่งแบบของโจทย์นี้หน้าตาเป็นอย่างไร

และควรวางแผนการนับโดยแยกกรณีหรือโดยแบ่งเป็นขั้นตอน

12.1 บทนำ: การนับในฐานะการวางแผน

ก่อนเข้าสู่สูตรและเทคนิคเฉพาะทาง ผู้เขียนอยากย้ำภาพใหญ่ของบทนี้ให้ชัดเจนก่อน: การนับที่ดีไม่ใช่การเดาสูตรให้ถูก แต่คือการออกแบบกระบวนการนับให้เป็นระบบ ในหลายโจทย์ที่ดูซับซ้อน คำตอบสามารถหาได้โดยอาศัยเพียงการแยกกรณีอย่างมีวินัย หรือการแบ่งงานออกเป็นขั้นตอนย่อยที่ชัดเจน แล้วใช้หลักพื้นฐานเพียงไม่กี่ข้อเท่านั้น

ความคิดนี้สำคัญมาก เพราะถ้าผู้เรียนเริ่มจากการมอง combinatorics เป็นเพียงคลังสูตร เวลาพบโจทย์ที่หน้าตาไม่ตรงกับตัวอย่างที่เคยเห็น ก็มักจะไม่ว่าจะเริ่มจากตรงไหน แต่ถ้าเริ่มจากการมองว่าโจทย์การนับทุกข้อคือปัญหาของการจัดระเบียบความเป็นไปได้ เราจะค่อย ๆ เห็นว่า แม้โจทย์จะเปลี่ยนบริบทไปเรื่อย ๆ แกนกลางของวิธีคิดกลับไม่เปลี่ยนมากนัก

12.1.1 การนับไม่ใช่การจำสูตร

ผู้เรียนจำนวนมากคุ้นเคยกับ combinatorics ผ่านโจทย์ประเภท “มีคน n คนเรียงกันได้กี่แบบ” หรือ “เลือกของ r ชิ้นจาก n ชิ้นได้กี่วิธี” แล้วก็ได้รับสูตรสำเร็จรูปอย่างรวดเร็ว เช่น $n!$, $P(n, r)$, หรือ $\binom{n}{r}$ การเรียนแบบนี้ทำให้รู้สึกเหมือนหัวข้อการนับเป็นเรื่องของการจับคู่โจทย์กับสูตร แต่ความจริงแล้วสูตรเหล่านั้นเป็นเพียง *ผลลัพธ์* ของเหตุผลเชิงการนับ ไม่ใช่จุดเริ่มต้นของเหตุผลนั้น

ตัวอย่างเช่น ถ้าถามว่ามีวิธีเรียงคน 5 คนในแนวเส้นตรงกี่วิธี เราสามารถตอบว่า $5!$ ก็จริง แต่ถ้าถามต่อว่าเหตุใดจึงเป็น $5!$ คำตอบที่แท้จริงไม่ได้อยู่ที่ “เพราะสูตรบอกเช่นนั้น” แต่อยู่ที่การวางแผนว่า ตำแหน่งแรกมีตัวเลือก 5 คน ตำแหน่งถัดไปเหลือ 4 คน ตำแหน่งถัดไปเหลือ 3 คน และดำเนินเช่นนี้ต่อไปจนหมด กล่าวคือสูตรเกิดจากการนับแบบเป็นขั้นตอน ไม่ใช่เกิดขึ้นมาลอย ๆ

ด้วยเหตุนี้ ในบทนี้เราจะพยายามชะลอการใช้สูตรให้นานที่สุดเท่าที่ทำได้ ไม่ใช่เพราะสูตรไม่มีประโยชน์ แต่เพราะผู้เรียนควรเห็นก่อนว่า ถ้าไม่มีสูตรอยู่ตรงหน้า เราจะยังสามารถคิดและออกแบบการนับได้หรือไม่ ท่าทีแบบนี้มีความสำคัญมาก เพราะช่วยให้เมื่อพบโจทย์ใหม่หรือโจทย์ที่มีเงื่อนไขซับซ้อน เราจะไม่ได้ติดอยู่กับการพยายามจำว่ามันคล้ายสูตรใด แต่จะย้อนกลับมาถามที่โครงสร้างของโจทย์แทน

12.1.2 มุมมองแบบ set-theoretic และการนับจำนวนสมาชิก

อีกมุมมองหนึ่งที่ช่วยให้ combinatorics เป็นคณิตศาสตร์มากขึ้น คือการมองการนับว่าเป็นการหาจำนวนสมาชิกของเซต แทนที่จะพูดเพียงว่า “มีกี่วิธี” เราสามารถถามให้ชัดเจนว่า

เซตของผลลัพธ์ที่เป็นไปได้ทั้งหมดคืออะไร

เมื่อเขียนเซตนี้ได้ชัด ปัญหาการนับก็กลายเป็นการหาคาร์ดินาลของเซตนั้น

ตัวอย่างเช่น ถ้าต้องการนับจำนวนวิธีเลือกตัวอักษรหนึ่งตัวจากเซต A หรือเซต B เราสามารถมองได้ว่า เรากำลังหาจำนวนสมาชิกของเซต

$$A \cup B.$$

ถ้า A และ B ไม่มีสมาชิกร่วมกัน การนับนี้ก็ตรงกับหลักการบวกทันที ในทำนองเดียวกัน ถ้าต้องการนับจำนวนวิธีเลือกสิ่งหนึ่งจาก A และอีกสิ่งหนึ่งจาก B เรากำลังนับจำนวนสมาชิกของเซต

$$A \times B.$$

ซึ่งจะนำไปสู่หลักการคูณ

มุมมองแบบ set-theoretic มีประโยชน์อย่างยิ่ง เพราะมันบังคับให้เราระบุให้ชัดว่า "ผลลัพธ์หนึ่งแบบ" ในโจทย์นี้หน้าตาเป็นอย่างไร เป็นสมาชิกเดี่ยว เป็นคู่ลำดับ เป็นลิสต์ เป็นเซตย่อย หรือเป็นวัตถุชนิดอื่น เมื่อจุดนี้ชัดขึ้น ความคลุมเครือจำนวนมากในการนับก็จะหายไป

ที่สำคัญยิ่งกว่านั้น มุมมองนี้ยังช่วยให้เราเชื่อมโยง combinatorics เข้ากับเนื้อหาที่เรียนมาก่อนหน้านี้ในหนังสือได้โดยตรง ไม่ว่าจะเป็นเรื่องเซต ผลคูณคาร์ทีเซียน ความสัมพันธ์ของสมาชิก หรือแม้แต่การพิสูจน์ความเท่ากันของจำนวนสองจำนวน ผ่านการสร้าง correspondence หรือการนับสิ่งเดียวกันสองวิธี ดังนั้นการนับจึงไม่ได้เป็นหัวข้อโดดเดี่ยว แต่เป็นอีกพื้นที่หนึ่งที่ภาษาเรื่องเซตและโครงสร้างทำงานได้อย่างเป็นธรรมชาติ

12.1.3 โจทย์การนับกับการออกแบบกระบวนการนับ

ถ้าจะสรุปวิธีคิดที่สำคัญที่สุดของบทนี้ในช่วงต้น ผู้เขียนอยากให้อ่านไว้ว่า โจทย์การนับแทบทุกข้อเริ่มจากการตัดสินใจอย่างใดอย่างหนึ่งต่อไปนี้:

1. จะแยกโจทย์ออกเป็นหลายกรณีที่ไม่ทับซ้อนกันหรือไม่
2. หรือจะมองว่าโจทย์นี้เป็นกระบวนการหลายขั้นตอนที่ต้องทำต่อเนื่องกัน

คำถามแรกนำเราไปสู่หลักการบวก ส่วนคำถามที่สองนำเราไปสู่หลักการคูณ

ตัวอย่างเช่น ถ้าพาสเวิร์ดหนึ่งชุดยาว 3 หรือ 4 ตัวอักษร คีย์เวิร์ดสำคัญคือคำว่า "หรือ" เพราะมันบอกว่าเรากำลังนับแบบแยกกรณี: กรณีความยาว 3 และกรณีความยาว 4 แต่ถ้าถามว่ามีพาสเวิร์ดยาว 4 ตัวอักษรกี่แบบ เรามักจะนับแบบเป็นขั้นตอน: ตำแหน่งที่หนึ่งเลือกอะไร ตำแหน่งที่สองเลือกอะไร และต่อไปจนครบทุกตำแหน่ง

จะเห็นว่าโจทย์การนับที่ดีไม่ได้บังคับให้เราคิดในแนวเดียวเสมอไป หลายครั้งโจทย์เดียวกันอาจต้องใช้ทั้งหลักการบวกและหลักการคูณร่วมกัน หรืออาจต้องจัดระเบียบการนับใหม่หลายครั้งก่อนจะได้รูปที่นับได้ง่าย ด้วยเหตุนี้ ผู้เรียนจึงควรฝึกมองโจทย์การนับเหมือนการออกแบบอัลกอริทึมขนาดเล็ก: เป้าหมายไม่ใช่เพียงคำตอบ แต่คือการ

ออกแบบแผนการนับที่ครบ ไม่ซ้ำ และอธิบายได้

12.2 หลักการนับพื้นฐาน

จากภาพรวมในบทนำ ตอนนี้เราพร้อมจะเข้าสู่หลักการพื้นฐานที่สุดของ combinatorics คือ **หลักการบวก** และ **หลักการคูณ** แม้หลักการทั้งสองจะดูเรียบง่ายมาก แต่แท้จริงแล้วเป็นแกนกลางของการนับแทบทุกแบบในบทนี้ สูตรต่าง ๆ ที่จะเรียนต่อจากนี้ ไม่ว่าจะเป็นการเรียงสับเปลี่ยน การจัดกลุ่ม หรือสัมประสิทธิ์ทวินาม ล้วนมีรากอยู่ที่สองหลักการนี้ทั้งสิ้น

จึงควรมองหวัข้อนี้ไม่ใช่เพียงในฐานะ ``ของง่ายก่อนของยาก" แต่ในฐานะการฝึกภาษาพื้นฐานของการนับ หากเข้าใจสองหลักการนี้อย่างถูกต้อง โจทย์การนับจำนวนมากจะกลายเป็นเพียงการตัดสินใจว่า ตอนนี้ควรบวกหรือควรคูณ หรือควรบวกในชั้นนอกแล้วคูณในชั้นใน หรือควรคูณก่อนแล้วค่อยแยกกรณีมาบวกทีหลัง

12.2.1 หลักการบวก

หลักการบวกใช้ในสถานการณ์ที่เรากำลังเลือกจากหลายกรณี โดยแต่ละกรณี **ทับซ้อนกันไม่ได้** และเมื่อเลือกกรณีใดกรณีหนึ่งแล้ว งานก็ถือว่าเสร็จสมบูรณ์ในตัวเอง

ในภาษาพูดแบบไม่เป็นทางการ หลักการบวกคือหลักของคำว่า

``ทำอย่างนี้ หรือ ทำอย่างนั้น"

โดยต้องระวังว่าคำว่า ``หรือ" ที่ใช้ตรงนี้ เป็นการแยกกรณีที่ไม่มีผลลัพธ์ร่วมกัน มิฉะนั้นจะเกิดการนับซ้ำ

เราสามารถเขียนหลักการบวกอย่างกระชับได้ดังนี้: ถ้ามีทางเลือกสองทาง ทางแรกทำได้ p วิธี ทางที่สองทำได้ q วิธี และไม่มีวิธีใดที่ถูกนับอยู่ทั้งสองทางพร้อมกัน จำนวนวิธีทั้งหมดคือ

$$p + q.$$

ในภาษาเรื่องเซต นี่คือข้อความเดียวกับผลลัพธ์พื้นฐานว่า ถ้า A และ B เป็นเซตที่ไม่ทับกัน กล่าวคือ $A \cap B = \emptyset$ แล้ว

$$|A \cup B| = |A| + |B|.$$

สิ่งที่ควรสังเกตคือ สูตรนี้ดูเหมือนเล็กมาก แต่บอกเงื่อนไขสำคัญไว้ครบ: เราต้องมั่นใจว่าเซตของผลลัพธ์สองส่วน **ไม่ทับกัน** หากทับกันเมื่อใด การบวกตรง ๆ จะทำให้นับซ้ำทันที

Example 12.2.1. ถ้ามหาวิทยาลัยแห่งหนึ่งมีนักศึกษาริชาเอกคณิตศาสตร์ 33 คน และนักศึกษาริชาเอกวิทยาการคอมพิวเตอร์ 40 คน ถ้าต้องการเลือกนักศึกษาริชาหนึ่งคนจากสองสาขาริชานี้ จันวนวิธีเลือกคือ

$$33 + 40 = 73.$$

สิ่งสำคัญของตัวอย่างนี้ไม่ใช่คำตอบ 73 แต่คือการเห็นว่ารากำลังนับโดยแยกเป็นสองกรณีที่ไม่ทับกัน: เลือกจากสาขาแรก หรือเลือกจากสาขาที่สอง และแต่ละการเลือกจบในตัวเองแล้ว

หลักการบวกยังขยายไปได้โดยตรงสู่หลายกรณี ถ้า $A_1, \dots, A_m$ เป็นเซตที่ไม่มีสองเซตใดทับกันเลย แล้วจะได้ว่า

$$|A_1 \cup A_2 \cup \dots \cup A_m| = |A_1| + |A_2| + \dots + |A_m|.$$

ในภาษาของโยทย์การนับ นี่หมายความว่า ถ้าเราสามารถแบ่งผลลัพธ์ทั้งหมดออกเป็น m กรณีที่ไม่ซ้ำกันได้ จันวนทั้งหมดก็เท่ากับผลรวมของจันวนในแต่ละกรณี

ดังนั้น เวลาจะใช้หลักการบวก คำถามสำคัญที่สุดไม่ใช่เพียงว่า “มีคำว่าริชาหรือไม่มี” แต่คือ

กรณีที่แยกออกมานั้นไม่ทับกันจริงหรือไม่

ถ้าตอบไม่ได้อย่างมั่นใจ ก็ควรระวังไว้ก่อนว่ารากำลังนับซ้ำอยู่

12.2.2 หลักการคูณ

ถ้าหลักการบวกเป็นหลักของการแยกกรณี หลักการคูณก็เป็นหลักของการประกอบขั้นตอน มันใช้ในสถานการณ์ที่ผลลัพธ์หนึ่งแบบเกิดจากการตัดสินใจหลายขั้นตอนต่อเนื่องกัน และเราต้องทำทุกขั้นตอนจึงจะได้ผลลัพธ์สมบูรณ์หนึ่งแบบ

ในภาษาพูดแบบง่าย หลักการคูณคือหลักของคำว่า

“ทำขั้นตอนนี้ แล้ว ทำขั้นตอนถัดไป”

ถ้าขั้นตอนแรกทำได้ p วิธี และหลังจากเลือกวิธีใดในขั้นตอนแรกแล้ว ขั้นตอนที่สองยังทำได้ q วิธีเสมอ จันวนวิธีทั้งหมดคือ

$$pq.$$

เงื่อนไขที่สำคัญมากของหลักการคูณคือ จันวนทางเลือกในขั้นตอนถัดไปต้องสม่ำเสมอตามที่เรากล่าวอ้าง หรืออย่างน้อยถ้าไม่สม่ำเสมอ เราต้องจัดกรณีใหม่ก่อนจะคูณ จุดนี้เป็นสาเหตุที่ผู้เรียนมักใช้หลักการคูณผิด เพราะเห็น

ว่ามีหลายขั้นตอนแล้วรับประกันทันที โดยยังไม่ได้ตรวจว่าจำนวนตัวเลือกในแต่ละขั้นขึ้นอยู่กับสิ่งที่เลือกมาก่อนหรือไม่

ในภาษาเรื่องเซต หลักการคูณสอดคล้องกับข้อเท็จจริงว่า ถ้า A และ B เป็นเซต แล้วผลคูณคาร์ทีเซียนของมันมีจำนวนสมาชิกเท่ากับ

$$|A \times B| = |A| \cdot |B|.$$

นี่สะท้อนความคิดโดยตรงว่า สมาชิกของ $A \times B$ คือคู่อันดับ (a, b) ซึ่งเกิดจากการเลือกสมาชิกหนึ่งตัวจาก A และอีกหนึ่งตัวจาก B

Example 12.2.2. ถ้าต้องการเลือกนักศึกษาวิชาเอกคณิตศาสตร์หนึ่งคนจากทั้งหมด 33 คน และเลือกนักศึกษาวิชาเอกวิทยาการคอมพิวเตอร์อีกหนึ่งคนจากทั้งหมด 40 คน จำนวนวิธีเลือกทั้งหมดคือ

$$33 \cdot 40.$$

ในตัวอย่างนี้ คีย์เวิร์ดสำคัญไม่ใช่ “หรือ” แต่เป็น “และ” กล่าวคือผลลัพธ์หนึ่งแบบประกอบด้วยการเลือกสองส่วนพร้อมกัน จึงเหมาะกับหลักการคูณ

ในกรณีทั่วไป ถ้างานหนึ่งประกอบด้วย m ขั้นตอน โดยขั้นตอนที่ 1 ทำได้ r_1 วิธี ขั้นตอนที่ 2 ทำได้ r_2 วิธี และดำเนินต่อไปจนถึงขั้นตอนที่ m ทำได้ r_m วิธี แล้วจำนวนวิธีทั้งหมดคือ

$$r_1 r_2 \cdots r_m.$$

แต่การใช้สูตรนี้อย่างปลอดภัย ต้องผ่านการตรวจให้ชัดเจนว่า โครงสร้างของโจทย์เป็นการเลือกแบบเป็นขั้นตอนจริง และจำนวนวิธีในแต่ละขั้นสอดคล้องกับสิ่งที่กล่าวอ้าง

12.2.3 การนับผ่าน Cartesian product

แม้หลักการคูณจะถูกอธิบายผ่านภาษาของ “ขั้นตอน” แต่อีกมุมมองหนึ่งที่สำคัญมากคือการมองผ่านผลคูณคาร์ทีเซียน มุมมองนี้ช่วยทำให้การนับมีสถานะเป็นคณิตศาสตร์ที่ชัดเจน ไม่ใช่เพียง heuristic ทางความคิด

ถ้าผลลัพธ์หนึ่งแบบของโจทย์สามารถถูกบันทึกได้ในรูปคู่อันดับ เช่น (a, b) โดยที่ $a \in A$ และ $b \in B$ เรากำลังนับสมาชิกของเซต $A \times B$ โดยตรง และจึงได้ทันทีว่า

$$|A \times B| = |A||B|.$$

มุมมองนี้ขยายต่อไปได้อย่างเป็นธรรมชาติ ถ้าผลลัพธ์หนึ่งแบบประกอบด้วยการตัดสินใจ m ขั้นตอน และสามารถแทน

ได้เป็นลำดับ

$$(a_1, a_2, \dots, a_m)$$

โดยที่ $a_i \in A_i$ เรากำลังนับสมาชิกของเซต

$$A_1 \times A_2 \times \dots \times A_m$$

และจึงได้ว่า

$$|A_1 \times A_2 \times \dots \times A_m| = |A_1| |A_2| \dots |A_m|.$$

ข้อดีของการเขียนโจทย์ในรูป Cartesian product คือ มันบังคับให้เราเห็นให้ชัดว่า หนึ่งผลลัพธ์ประกอบด้วยข้อมูลอะไรบ้าง ลำดับมีความสำคัญหรือไม่ และแต่ละขั้นของการเลือกเป็นอิสระจากกันแค่ไหน สิ่งเหล่านี้จะสำคัญมากขึ้นเรื่อย ๆ เมื่อเราไปถึงเรื่องการเรียงสับเปลี่ยนและการจัดกลุ่มในหัวข้อถัดไป

12.2.4 การแยกกรณีและการนับแบบเป็นขั้นตอน

เมื่อมองในภาพใหญ่ หลักการบวกและหลักการคูณไม่ได้แยกขาดจากกัน แต่เป็นสองรูปแบบพื้นฐานของการวางแผนการนับ โจทย์จำนวนมากใน combinatorics ที่ดูซับซ้อน แท้จริงแล้วเกิดจากการผสมสองรูปแบบนี้เข้าด้วยกัน

บางครั้งเราต้องแยกโจทย์ออกเป็นกรณีใหญ่ก่อน แล้วภายในแต่ละกรณีจึงใช้หลักการคูณ เช่น พาสเวิร์ดที่ยาว 3 หรือ 4 ตัวอักษร: เราต้องบวกจำนวนกรณีความยาว 3 กับกรณีความยาว 4 แต่ในแต่ละกรณีความยาวคงที่ เรามักคูณจำนวนตัวเลือกของแต่ละตำแหน่ง

ในทางกลับกัน บางโจทย์อาจดูเหมือนเป็นกระบวนการหลายขั้นตอน แต่เมื่อพิจารณาให้ดี บางขั้นตอนต้องถูกแยกกรณีก่อนจึงจะนับได้ถูกต้อง เช่น ถ้าจำนวนตัวเลือกในขั้นตอนถัดไปขึ้นอยู่กับสิ่งที่เลือกในขั้นปัจจุบัน เราอาจต้องแบ่งกรณีตามสิ่งที่เลือกนั้นก่อน แล้วจึงใช้หลักการคูณภายในแต่ละกรณี

ดังนั้น ความชำนาญในการนับไม่ได้อยู่ที่การจำว่าหลักการบวกคืออะไร หรือหลักการคูณคืออะไรเพียงลำพัง แต่อยู่ที่การมองโจทย์ให้ออกว่า ควรจัดระเบียบความเป็นไปได้อย่างไร กล่าวอีกแบบหนึ่ง โจทย์การนับหลายข้อมีคำตอบที่ถูก เพราะผู้แก้เลือก representation ของกระบวนการนับได้เหมาะสม ไม่ใช่เพราะจำสูตรได้มากกว่า

12.2.5 ข้อควรระวัง: การนับซ้ำและการแบ่งกรณีไม่ครบ

ความผิดพลาดที่พบบ่อยที่สุดใน combinatorics ไม่ได้เกิดจากการคูณหรือการบวกเลขผิด แต่เกิดจากการจัดโครงสร้างของการนับผิด โดยเฉพาะสองแบบต่อไปนี้:

1. การนับซ้ำ ผลลัพธ์หนึ่งแบบถูกนับมากกว่าหนึ่งครั้ง เพราะกรณีที่แยกออกมาไม่ได้ไม่ทับกันจริง หรือ

กระบวนการสร้างผลลัพธ์หนึ่งแบบทำได้หลายทางแต่เราไม่ได้ขดเชยให้ถูกต้อง

2. การแบ่งกรณีไม่ครบ เรานับได้เพียงบางส่วนของผลลัพธ์ทั้งหมด เพราะลืมนับบางแบบไป หรือออกแบบขั้นตอนที่ไม่ครอบคลุมความเป็นไปได้ทุกชนิด

สองข้อผิดพลาดนี้สำคัญมาก เพราะคำตอบที่ได้อาจดูสมเหตุสมผล แต่ผิดตั้งแต่ระดับโครงสร้าง และที่สำคัญคือยิ่งโจทย์ซับซ้อนมากขึ้น ความเสี่ยงของความผิดพลาดสองชนิดนี้ก็ยิ่งสูงขึ้น

ตัวอย่างเช่น ถ้าเราต้องการนับจำนวนวิธีเลือกนักศึกษาหนึ่งคนจากชมรม A หรือชมรม B แต่มีนักศึกษาบางคนอยู่ทั้งสองชมรม การบวก $|A| + |B|$ ตรง ๆ จะทำให้นับคนนั้นซ้ำ หรือถ้าเรานับจำนวนเต็มสี่หลักที่มีคุณสมบัติบางอย่าง โดยแบ่งตามเลขหลักแรก แต่ลืมนับที่เลขหลักแรกเป็นศูนย์ซึ่งไม่ถือเป็นจำนวนสี่หลัก การวางแผนการนับก็จะคลาดเคลื่อนได้ทันที

เพราะฉะนั้น ก่อนลงมือคำนวณจริง ควรหยุดถามตัวเองเสมอว่า

ผลลัพธ์หนึ่งแบบถูกนับได้มากกว่าหนึ่งทางหรือไม่
และผลลัพธ์ทุกแบบที่ควรนับถูกครอบคลุมครบแล้วหรือยัง

ถ้าตอบสองคำถามนี้ได้อย่างมั่นใจ กระบวนการนับก็มักจะมั่นคงขึ้นมาก และนี่คือเหตุผลว่าทำไม combinatorics จึงเป็นการฝึก reasoning ที่ดีมาก: คำตอบที่ถูกไม่ได้มาจากการกดสูตร แต่มาจากการควบคุมโครงสร้างของการนับให้ครบและไม่ซ้ำ ซึ่งเป็นทักษะพื้นฐานเกี่ยวกับการเขียนพิสูจน์ที่ดีนั่นเอง

12.3 Permutations: การเรียงสับเปลี่ยน

หลังจากที่เราได้ฝึกใช้หลักการบวกและหลักการคูณเป็นเครื่องมือพื้นฐานของการนับแล้ว ขั้นตอนต่อไปที่เป็นธรรมชาติมากคือการศึกษาสถานการณ์ที่ผลลัพธ์หนึ่งแบบไม่ได้ขึ้นกับเพียงว่า “เลือกใครบ้าง” แต่ขึ้นกับด้วยว่า “วางหรือเรียงสิ่งทีเลือกมาอย่างไร” แนวคิดนี้คือหัวใจของ การเรียงสับเปลี่ยน หรือ *permutations*

สิ่งที่ควรเข้าใจตั้งแต่ต้นคือ การเรียงสับเปลี่ยนไม่ใช่หัวข้อที่ตัดขาดจากหลักการบวกและหลักการคูณ แต่เป็นผลลัพธ์ของการนำสองหลักการนั้นมาใช้กับปัญหาที่ *ลำดับมีความสำคัญ* เมื่อเราพูดว่าเรียงสิ่งของ r ชิ้นจากสิ่งของทั้งหมด n ชิ้น เรากำลังพูดถึงกระบวนการเลือกทีละตำแหน่ง และแต่ละตำแหน่งมีจำนวนตัวเลือกที่ลดลงตามของที่ถูกรวบรวมไปแล้ว ดังนั้นสูตรที่ได้ในหัวข้อนี้จึงควรถูกมองว่าเป็น สูตรที่สรุปมาจากการวางแผนการนับ ไม่ใช่สูตรที่ควรท่องจำโดยไม่มีภาพของเหตุผลอยู่เบื้องหลัง

12.3.1 การเรียงสับเปลี่ยนเชิงเส้นของสิ่งที่ไม่ซ้ำกัน

เริ่มจากกรณีพื้นฐานที่สุดก่อน สมมติว่าเรามีสิ่งของ n ชิ้นที่แตกต่างกันทั้งหมด และต้องการนำบางส่วนหรือทั้งหมดมาเรียงในแนวเส้นตรง คำว่า “เรียงในแนวเส้นตรง” มีความหมายสำคัญตรงที่ ตำแหน่งที่หนึ่งกับตำแหน่ง

ที่สองไม่เท่ากัน ดังนั้นถ้าใช้สิ่งของชุดเดียวกันแต่สลับลำดับกัน เราถือว่าเป็นคนละผลลัพธ์

ตัวอย่างเช่น ถ้ามีตัวอักษร a, b, c แล้วรูปแบบ

abc และ bac

ถือว่าต่างกัน แม้จะใช้ตัวอักษรชุดเดียวกันทั้งหมด เพราะตำแหน่งของตัวอักษรไม่เหมือนกัน จุดนี้เองที่ทำให้ permutations แตกต่างจาก combinations อย่างมีนัยสำคัญ และเป็นเหตุผลว่าทำไมเราจึงควรแยกสองหัวข้อนี้ออกจากกันอย่างชัดเจน

Definition 12.3.1. ให้ $A = \{a_1, a_2, \dots, a_n\}$ เป็นเซตของสิ่งของ n ชิ้นที่แตกต่างกัน และให้ $0 \leq r \leq n$ การเรียงสับเปลี่ยน r ชิ้น ของเซต A คือการจัดสมาชิก r ตัวจาก A เรียงเป็นลำดับในแนวเส้นตรง โดยไม่ให้มีการใช้สมาชิกตัวเดิมซ้ำ

หากต้องการเขียนให้เป็นภาษาของเซตอย่างชัดเจน เราสามารถมองการเรียงสับเปลี่ยน r ชิ้นได้ว่าเป็นลำดับ

$$(x_1, x_2, \dots, x_r)$$

โดยที่แต่ละ $x_i \in A$ และ $x_i \neq x_j$ เมื่อ $i \neq j$ นิยามนี้ทำให้เห็นชัดเจนที่ว่า permutation เป็นวัตถุชนิด "ลำดับ" ไม่ใช่เพียงเซตของสิ่งที่ถูกเลือก

12.3.2 จำนวนวิธีเรียงสับเปลี่ยน $P(n, r)$

เมื่อมองการเรียงสับเปลี่ยนเป็นการเติมของลงในตำแหน่งที่ละตำแหน่ง จำนวนวิธีนับจะเกิดขึ้นอย่างเป็นธรรมชาติจากหลักการคูณ

สำหรับตำแหน่งแรก มีสิ่งของให้เลือกทั้งหมด n ชิ้น เมื่อเลือกตำแหน่งแรกไปแล้ว ตำแหน่งที่สองเหลือให้เลือก $n - 1$ ชิ้น ตำแหน่งที่สามเหลือ $n - 2$ ชิ้น และดำเนินเช่นนี้ไปเรื่อย ๆ จนถึงตำแหน่งที่ r ซึ่งจะเหลือให้เลือก

$$n - r + 1$$

ชิ้น

ดังนั้นจำนวนวิธีเรียงสับเปลี่ยน r ชิ้นจากของ n ชิ้น จึงเท่ากับ

$$n(n-1)(n-2) \cdots (n-r+1).$$

Definition 12.3.2. เราเขียนจำนวนวิธีเรียงสับเปลี่ยน r ชิ้นจากสิ่งของ n ชิ้นที่แตกต่างกันด้วยสัญลักษณ์

$$P(n, r).$$

จากการให้เหตุผลข้างต้น เราจึงได้สูตรสำคัญดังนี้

Theorem 12.3.3. สำหรับจำนวนเต็ม n, r ที่ $0 \leq r \leq n$,

$$P(n, r) = n(n-1)(n-2) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

บทพิสูจน์. การเรียงสับเปลี่ยน r ชั้นเป็นกระบวนการ r ขั้นตอน โดยขั้นตอนที่หนึ่งมีตัวเลือก n วิธี ขั้นตอนที่สองมีตัวเลือก $n-1$ วิธี ... และขั้นตอนที่ r มีตัวเลือก $n-r+1$ วิธี ดังนั้นจากหลักการคูณ จำนวนวิธีทั้งหมดเท่ากับ

$$n(n-1) \cdots (n-r+1).$$

อีกด้านหนึ่ง

$$n! = n(n-1) \cdots (n-r+1)(n-r)!,$$

จึงได้ว่า

$$P(n, r) = \frac{n!}{(n-r)!}.$$

□

สิ่งที่ควรสังเกตคือ สูตรนี้ไม่ได้มีความหมายว่า “เจอโจทย์อะไรก็แทนค่าลงไป” แต่ควรตีความกลับได้ด้วยว่า โจทย์นั้นต้องมีลักษณะเป็นการเรียงลำดับ r ตำแหน่ง จากตัวเลือก n ตัว โดยไม่ซ้ำ ถ้าโจทย์ไม่ตรงโครงนี้ เราก็ไม่ควรใช้สูตรนี้ตรง ๆ

12.3.3 การเรียงสับเปลี่ยนเต็มชุดและแฟกทอเรียล

กรณีพิเศษที่สำคัญที่สุดของ $P(n, r)$ คือกรณีที่ $r = n$ กล่าวคือเราใช้สิ่งของทั้งหมดมาเรียงครบทุกชั้น ในกรณีนี้จะได้ว่า

$$P(n, n) = n!.$$

ดังนั้นแฟกทอเรียลจึงควรถูกมองว่าเป็น จำนวนวิธีเรียงสิ่งของ n ชั้นที่แตกต่างกันทั้งหมดในแนวเส้นตรง

Definition 12.3.4. สำหรับจำนวนเต็มบวก n ,

$$n! = n(n-1)(n-2) \cdots 2 \cdot 1.$$

และกำหนดโดยอนุโลมว่า

$$0! = 1.$$

การนิยาม $0! = 1$ อาจดูแปลกในตอนแรก แต่มีเหตุผลทั้งเชิงพีชคณิตและเชิง combinatorics ในมุมมองการนับ การเรียงสิ่งของศูนย์ชิ้นมีอยู่เพียงหนึ่งแบบ คือ “ไม่ต้องเรียงอะไรเลย” ดังนั้นจึงสมเหตุสมผลที่จะกำหนดว่า $0! = 1$

Example 12.3.5. จำนวนวิธีจัดคน 6 คนให้นั่งเรียงกันในแถวตรงเท่ากับ

$$6! = 720.$$

สิ่งที่ควรระวังคือ ในโจทย์จำนวนมาก คำว่า “เรียง” ไม่จำเป็นต้องปรากฏอย่างชัดเจน แต่โครงสร้างของโจทย์อาจเทียบเท่ากับการเรียงก็ได้ เช่น การสร้างรหัส การเติมตัวเลขลงในหลัก หรือการกำหนดว่าใครอยู่ตำแหน่งใด ดังนั้นเวลาเห็นแฟกทอเรียลหรือ $P(n, r)$ ควรถามตัวเองก่อนว่า ผลลัพธ์หนึ่งแบบในโจทย์นี้คือ “ลำดับ” จริงหรือไม่

12.3.4 การเรียงสับเปลี่ยนแบบวงกลม

จนถึงตอนนี้ เราเรียงสิ่งของในแนวเส้นตรงมาตลอด แต่ในหลายสถานการณ์ เช่น การนั่งล้อมโต๊ะกลม แนวคิดเรื่อง “ตำแหน่งซ้ายสุด” หรือ “ตำแหน่งแรก” ไม่มีอยู่โดยธรรมชาติ สิ่งนี้ทำให้การนับแบบวงกลมต่างจากการนับแบบเส้นตรงอย่างสำคัญ

ประเด็นหลักคือ ถ้าเรานำการเรียงเชิงเส้นมาล้อมเป็นวงกลม รูปแบบหลายแบบที่ต่างกันในเชิงเส้นตรง จะกลายเป็นรูปแบบเดียวกันเมื่อมองบนวงกลม เพราะการหมุนทั้งหมดไม่ทำให้การจัดเรียงเปลี่ยนไปในเชิงโครงสร้าง

ตัวอย่างเช่น การเรียง

$$(a, b, c, d), \quad (b, c, d, a), \quad (c, d, a, b), \quad (d, a, b, c)$$

ถ้านำไปวางรอบโต๊ะกลม ทั้งสี่แบบนี้ถือว่าเป็นรูปแบบเดียวกัน เพราะต่างกันเพียงการหมุนทั้งหมด

ดังนั้นถ้ามีสิ่งของ n ชิ้นที่แตกต่างกันทั้งหมด จำนวนวิธีจัดเรียงเป็นวงกลมจะน้อยกว่าการเรียงเชิงเส้น n เท่า จึงได้ว่า

Theorem 12.3.6. จำนวนวิธีจัดเรียงสิ่งของ n ชิ้นที่แตกต่างกันรอบวงกลมเท่ากับ

$$(n - 1)!.$$

บทพิสูจน์. เริ่มจากการเรียงสิ่งของทั้ง n ชิ้นในแนวเส้นตรง ซึ่งมี $n!$ วิธี แต่การเรียงเชิงเส้นแต่ละ n แบบที่ได้จาก

การหมุนตำแหน่งทั้งชุด จะให้การจัดเรียงบนวงกลมแบบเดียวกัน ดังนั้นจำนวนแบบบนวงกลมคือ

$$\frac{n!}{n} = (n-1)!$$

□

วิธีคิดอีกแบบหนึ่งที่มักใช้ง่ายในทางปฏิบัติคือ “ตรึง” สิ่งของหนึ่งชิ้นไว้กับที่ก่อน แล้วเรียงของที่เหลืออีก $n-1$ ชิ้นรอบมัน ซึ่งก็ให้คำตอบเดียวกันคือ $(n-1)!$

ในทำนองเดียวกัน ถ้าต้องการเลือกสิ่งของ r ชิ้นจาก n ชิ้นมาเรียงเป็นวงกลม เราสามารถเริ่มจากการเรียงเชิงเส้น r ชิ้น ซึ่งมี $P(n, r)$ วิธี แล้วหารด้วย r เพราะการหมุนครบหนึ่งรอบให้รูปแบบวงกลมเดียวกัน จึงได้ว่า

$$Q(n, r) = \frac{P(n, r)}{r}.$$

12.3.5 การเรียงสับเปลี่ยนของสิ่งที่มีของซ้ำ

จนถึงตอนนี้ เราสมมติว่าสิ่งของทุกชิ้นแตกต่างกัน แต่ในหลายปัญหา สิ่งของบางชิ้นอาจถือว่าเหมือนกัน เช่นตัวอักษรซ้ำในคำ หรือวัตถุที่มีสีเดียวกันและไม่แยกแยะชิ้นรายตัว ในกรณีเช่นนี้ การใช้ $n!$ ตรง ๆ จะนับซ้ำทันที เพราะการสลับสิ่งของที่เหมือนกันไม่ควรถือว่าได้รูปแบบใหม่

สมมติว่ามีสิ่งของทั้งหมด n ชิ้น โดยแบ่งเป็น k ประเภท และประเภทที่ i มีอยู่ n_i ชิ้น เมื่อ

$$n_1 + n_2 + \cdots + n_k = n.$$

ถ้าเรามองทุกชิ้นว่าแยกจากกันก่อน จะเรียงได้ $n!$ วิธี แต่การสลับของในประเภทเดียวกันไม่เปลี่ยนรูปแบบจริง ดังนั้นเราต้องหารจำนวนการนับซ้ำออก

สำหรับประเภทที่หนึ่ง มีการสลับกันเองได้ $n_1!$ แบบ ประเภทที่สองมี $n_2!$ แบบ ... ประเภทที่ k มี $n_k!$ แบบ

จึงได้สูตรสำคัญดังนี้

Theorem 12.3.7. จำนวนวิธีเรียงสิ่งของ n ชิ้น เมื่อมีของซ้ำโดยแบ่งเป็น k ประเภท และประเภทที่ i มี n_i ชิ้น เท่ากับ

$$\frac{n!}{n_1! n_2! \cdots n_k!}.$$

Example 12.3.8. คำว่า MISSISSIPPI มีตัวอักษรทั้งหมด 11 ตัว โดยมี M หนึ่งตัว, I สี่ตัว, S สี่ตัว และ P สองตัว ดังนั้นจำนวนวิธีเรียงตัวอักษรที่แตกต่างกันทั้งหมดคือ

$$\frac{11!}{1! 4! 4! 2!}.$$

จุดที่ควรสังเกตคือ สูตรนี้เป็นอีกตัวอย่างหนึ่งของการขดเคยการนับซ้ำ กล่าวคือเราเริ่มจากนับทุกอย่างเหมือนกันจะแตกต่างกันหมด แล้วจึงหารจำนวนวิธีที่แท้จริงแล้วไม่ควรถูกนับแยกออก ดังนั้นแนวคิดของ ``อะไรนับซ้ำอยู่" จึงยังคงสำคัญมากเหมือนเดิม

12.3.6 โจทย์ประยุกต์ของ permutations

ถึงตรงนี้ ผู้อ่านควรเห็นแล้วว่า permutations ไม่ได้เป็นเพียงเรื่องของการเรียงคนเป็นแถว แต่เป็นเครื่องมือสำหรับโจทย์หลากหลายชนิด เช่น การสร้างรหัส การนับจำนวนเต็มที่มีข้อจำกัดบางอย่าง การนั่งรอบโต๊ะ หรือการจัดวางวัตถุที่มีสิ่งของซ้ำกัน

อย่างไรก็ตาม สิ่งที่เราควรระวังมากคือ *permutation* เป็นเครื่องมือ ไม่ใช่ชนิดของโจทย์ โจทย์หนึ่งข้ออาจใช้ permutations เพียงบางส่วนของกระบวนการนับ แล้วต้องอาศัยหลักการบวกหรือหลักการคูณร่วมด้วย ตัวอย่างเช่น ถ้าต้องการนับจำนวนจำนวนเต็มที่มีบางหลักต้องเป็นเลขคู่ และบางหลักห้ามซ้ำ เรามักต้องแยกกรณีตามหลักสุดท้ายก่อน แล้วจึงใช้ permutation กับหลักที่เหลือ

ดังนั้นเวลาพบโจทย์จริง ไม่ควรถามเพียงว่า ``ข้อนี้เป็น permutation หรือไม่" แต่ควรถามให้ละเอียดกว่านั้นว่า

อะไรคือผลลัพธ์หนึ่งแบบของโจทย์นี้
ลำดับมีความสำคัญหรือไม่
มีของซ้ำหรือไม่
วางเป็นเส้นตรงหรือวงกลม
และต้องผสมหลักการบวกหรือคูณเข้าไปตรงไหนบ้าง

12.4 Combinations: การจัดกลุ่ม

หลังจากศึกษา permutations แล้ว ขั้นต่อไปที่เป็นธรรมชาติมากคือการพิจารณาสถานการณ์ที่เราไม่ได้สนใจลำดับอีกต่อไป แต่สนใจเพียงว่า *เลือกใครบ้าง* หรือ *มีอะไรอยู่ในกลุ่มบ้าง* แนวคิดนี้คือ *combinations* หรือ การจัดกลุ่ม

หัวข้อนี้มีความสำคัญมาก เพราะผู้เรียนมักสับสนกับ permutations ในช่วงแรก และจริง ๆ แล้วความต่างระหว่างสองเรื่องนี้ เป็นหนึ่งในจุดแยกที่สำคัญที่สุดของ combinatorics ทั้งบท ถ้าแยกตรงนี้ออกได้ชัด สูตรจำนวนมากในบทนี้จะมีความหมายขึ้นทันที แต่ถ้ายังสับสนว่าลำดับสำคัญหรือไม่ การแทนสูตรจะผิดได้อย่างง่ายมาก

12.4.1 ความแตกต่างระหว่างการเรียงกับการเลือก

ใจกลางของความต่างระหว่าง permutation และ combination อยู่ที่คำถามเดียว:

ลำดับของสิ่งที่เลือกมีความสำคัญหรือไม่

ถ้าลำดับมีความสำคัญ เช่น เลือกผู้ชนะอันดับหนึ่ง อันดับสอง และอันดับสาม หรือจัดตัวอักษรลงในตำแหน่ง เรากำลังทำงานกับ permutations

แต่ถ้าลำดับไม่สำคัญ เช่น เลือกนักศึกษา 3 คนไปเป็นตัวแทน โดยไม่สนใจใครถูกเลือกก่อนหรือหลัง เรากำลังทำงานกับ combinations

ตัวอย่างเช่น ถ้าเลือก a, b, c มาจากเซตหนึ่ง แล้วพิจารณา

$$(a, b, c), (b, a, c), (c, a, b)$$

ทั้งสามแบบถือว่า *ต่างกัน* ในโลกของ permutations แต่ถือว่าเป็น *แบบเดียวกัน* ในโลกของ combinations เพราะใน combination เราสนใจเพียงว่าเลือกสมาชิกชุดเดียวกันหรือไม่

นี่คือจุดที่ควรถามตัวเองทุกครั้งก่อนเริ่มนับ: ผลลัพธ์หนึ่งแบบของโจทย์นี้ควรถูกแทนเป็น

- ลำดับ
- หรือเซตย่อย

ถ้าตอบข้อนี้ได้ถูก โจทย์จำนวนมากจะเริ่มชัดเจนขึ้น

12.4.2 จำนวนวิธีจัดกลุ่ม $\binom{n}{r}$

สมมติว่าเรามีสิ่งของ n ชิ้นที่แตกต่างกันทั้งหมด และต้องการเลือกมา r ชิ้นโดยไม่สนใจลำดับ เราจะเรียกสิ่งนี้ว่า **combination of r elements from n** หรือ **การจัดกลุ่ม r ชิ้นจาก n ชิ้น**

Definition 12.4.1. ให้ A เป็นเซตที่มีสมาชิก n ตัว การจัดกลุ่ม r ชิ้นจาก A คือการเลือกสมาชิก r ตัวของ A มาเป็นหนึ่งกลุ่ม โดยไม่สนใจลำดับภายในกลุ่ม จำนวนวิธีทั้งหมดเขียนแทนด้วย

$$\binom{n}{r} \quad \text{หรือ} \quad C(n, r).$$

ในภาษาของเซต $\binom{n}{r}$ คือจำนวนเซตย่อยที่มีสมาชิก r ตัวของเซตที่มีสมาชิก n ตัว ดังนั้น combination จึงควรถูกมองว่าเกี่ยวข้องโดยตรงกับเรื่องเซตย่อย ไม่ใช่เรื่องลำดับ

Theorem 12.4.2. สำหรับ $0 \leq r \leq n$,

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}.$$

แม้สูตรนี้เป็นสูตรมาตรฐานมาก แต่สิ่งสำคัญคือการเข้าใจว่า ตัวหาร $r!$ ปรากฏขึ้นมาเพราะอะไร และคำตอบก็คือ: เมื่อเราเริ่มจาก permutations แล้วตัดความสำคัญของลำดับออก แต่ละกลุ่ม r ตัวเดียวกันจะถูกนับซ้ำอยู่ $r!$ ครั้ง เพราะสมาชิกทั้ง r ตัวสามารถสลับลำดับกันได้ $r!$ แบบ โดยยังแทนกลุ่มเดิมอยู่

12.4.3 ความสัมพันธ์ระหว่าง permutations และ combinations

วิธีที่เป็นธรรมชาติที่สุดในการเชื่อมสองแนวคิดนี้คือ เริ่มจากการเลือกสมาชิก r ตัวมาก่อน แล้วจึงจัดเรียงสมาชิกที่เลือกมา หรือกลับกัน เริ่มจากการจัดเรียง r ตัวเลย แล้วค่อยลืบลำดับทิ้งไป

ถ้าเริ่มจาก combinations เรามีวิธีเลือกกลุ่ม r ตัวอยู่ $\binom{n}{r}$ วิธี และเมื่อเลือกมาแล้ว สมาชิกในกลุ่มนั้นสามารถเรียงกันได้อีก $r!$ วิธี ดังนั้นจำนวน permutations ทั้งหมดคือ

$$P(n, r) = \binom{n}{r} r!.$$

จากสมการนี้ก็จัดรูปกลับได้ว่า

$$\binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!}.$$

ความสัมพันธ์นี้สำคัญมาก เพราะทำให้เห็นชัดว่า combinations กับ permutations ไม่ใช่สูตรคนละโลก แต่เป็นวัตถุเดียวกันที่ต่างกันตรงว่า เราจะถือว่าการสลับลำดับภายในกลุ่มเป็นสิ่งใหม่หรือไม่

12.4.4 การเลือกภายใต้เงื่อนไข

ในโจทย์จริง การเลือกมักไม่ใช่การเลือกอย่างอิสระทั้งหมด แต่อาจมีเงื่อนไขกำกับอยู่ เช่น ต้องมีบุคคลบางประเภทอยู่ในกลุ่ม ห้ามเลือกคนสองคนพร้อมกัน หรือต้องผ่านจุดที่กำหนดในโจทย์เส้นทาง สถานการณ์เหล่านี้ยังคงอยู่ในโลกของ combinations ได้ แต่ต้องอาศัยการวางแผนการนับมากขึ้น

ตัวอย่างเช่น ถ้าต้องการเชิญเพื่อน 6 คนจากเพื่อนทั้งหมด 10 คน โดยมีพี่น้องคูหนึ่งที่ถ้าจะเชิญต้องเชิญมาทั้งคู่ เราสามารถแยกกรณีได้ว่า

1. ไม่เชิญทั้งสองคนนี้เลย

2. เชิญทั้งสองคนนี้มาด้วยทั้งคู่

แล้วจึงใช้ combinations นับในแต่ละกรณี ก่อนนำมาบวกกันด้วยหลักการบวก

ในทำนองเดียวกัน โจทย์เส้นทางบนตารางจำนวนเต็ม ซึ่งดูเหมือนไม่เกี่ยวกับ “การเลือกคน” ก็สามารถตีความเป็น combination ได้ เพราะเส้นทางจาก $(0, 0)$ ไป (m, n) ที่เดินได้เฉพาะขึ้นกับขวา เท่ากับการเลือกตำแหน่งของก้าวขึ้นจากก้าวทั้งหมด $m + n$ ก้าว จึงมีจำนวนเท่ากับ

$$\binom{m+n}{n}.$$

ตัวอย่างเช่นนี้สำคัญมาก เพราะแสดงให้เห็นว่า combinations เป็นภาษาเชิงโครงสร้าง ไม่ใช่เพียงสูตรสำหรับโจทย์ที่มีคำว่า “เลือก” ปรากฏอยู่ตรง ๆ

12.4.5 การแบ่งคนหรือสิ่งของออกเป็นกลุ่ม

อีกเรื่องหนึ่งที่มักปรากฏตามมาจาก combinations พื้นฐานคือ การแบ่งสิ่งของหรือคนออกเป็นหลายกลุ่ม ซึ่งมีความละเอียดอ่อนกว่าการเลือกเพียงกลุ่มเดียวมาก

ตัวอย่างเช่น ถ้าต้องการแบ่งนักเรียน 7 คนออกเป็นสามกลุ่ม โดยมีกลุ่มขนาด 3, 2, 2 โจทย์นี้ไม่ใช่เพียงการเลือกกลุ่มขนาด 3 แล้วเลือกอีกกลุ่มขนาด 2 ตรง ๆ เท่านั้น แต่ต้องระวังด้วยว่า สองกลุ่มที่มีขนาด 2 เท่ากันนั้น ควรถูกนับว่าเป็นคนละกลุ่มหรือเป็นกลุ่มแบบเดียวกัน

ถ้าเริ่มจากการเลือกกลุ่มขนาด 3 ก่อน จะมี

$$\binom{7}{3}$$

วิธี จากนั้นเลือกกลุ่มขนาด 2 จากคนที่เหลืออีก 4 คน จะมี

$$\binom{4}{2}$$

วิธี และอีกสองคนที่เหลือถูกบังคับเป็นกลุ่มสุดท้าย แต่ถ้าสองกลุ่มขนาด 2 ไม่มีป้ายชื่อ การนับนี้จะซ้ำอยู่ $2!$ เท่า เพราะการสลับสองกลุ่มนั้นไม่ทำให้การแบ่งเปลี่ยนไปจริง

โจทย์ลักษณะนี้เป็นจุดที่ combinatorics เริ่มต้องการความระวังมากขึ้น เพราะแม้ผู้เรียนจะรู้สูตร $\binom{n}{r}$ แล้ว ก็ยังอาจผิดได้ง่ายหากยังไม่ชัดว่า ผลลัพธ์หนึ่งแบบของโจทย์กำหนดให้แยกกลุ่มเหล่านั้นออกจากกันหรือไม่

12.4.6 ข้อควรระวัง: กลุ่มที่มีป้ายชื่อและไม่มีป้ายชื่อ

นี่เป็นข้อผิดพลาดที่พบบ่อยมาก และควรเน้นให้ชัดเป็นพิเศษ: ในการแบ่งคนหรือสิ่งของออกเป็นหลายกลุ่ม ต้องถามก่อนเสมอว่า *กลุ่มเหล่านั้นมีป้ายชื่อหรือไม่*

ถ้ากลุ่มมีป้ายชื่อ เช่น แบ่งนักเรียนออกเป็น กลุ่ม A, กลุ่ม B, และกลุ่ม C การสลับสมาชิกทั้งก่อนจากกลุ่ม A ไปกลุ่ม B ถือว่าได้ผลลัพธ์ใหม่ เพราะชื่อของกลุ่มต่างกัน

แต่ถ้ากลุ่มไม่มีป้ายชื่อ เช่น แบ่งนักเรียนออกเป็นสามทีมเฉย ๆ โดยไม่ได้ตั้งชื่อทีมหรือแยกบทบาทของแต่ละทีม การสลับสองกลุ่มทั้งก่อน ไม่ควรถือว่าเป็นผลลัพธ์ใหม่ เพราะเราเพียงเปลี่ยนชื่อเรียกภายนอกเท่านั้น ไม่ได้เปลี่ยนการแบ่งจริง

นี่เป็นความแตกต่างที่สำคัญมาก และเป็นสาเหตุของการนับซ้ำในโจทย์แบ่งกลุ่มแทบทั้งหมด ดังนั้นเวลาทำโจทย์ลักษณะนี้ ผู้เรียนควรตรวจให้ชัดเสมอว่า

เรากำลังนับ ``วิธีเลือกกลุ่ม" หรือกำลังนับ ``วิธีแบ่งออกเป็นกลุ่มที่มีบทบาทต่างกัน"

ถ้าตอบคำถามนี้ได้ชัด เราจะรู้ทันทีว่าควรหารด้วยแฟกทอเรียลของจำนวนกลุ่มที่สลับกันได้หรือไม่ และนี่คืออีกตัวอย่างหนึ่งที่แสดงให้เห็นว่า *combinatorics* ไม่ใช่เรื่องของการเรียงสับเปลี่ยน แต่เป็นเรื่องของการนิยามให้ชัดว่าผลลัพธ์หนึ่งแบบในโจทย์นี้หมายถึงอะไรกันแน่

12.5 Combinatorial Proofs and Double Counting

เมื่อเรียน *combinatorics* ไปได้ระยะหนึ่ง ผู้เรียนมักเริ่มเห็นว่าโจทย์การนับไม่ได้มีเป้าหมายเพียงเพื่อหาคำตอบเชิงตัวเลข แต่ยังสามารถใช้เป็น *วิธีพิสูจน์* ได้ด้วย กล่าวคือ แทนที่เราจะพิสูจน์เอกลักษณ์หนึ่งด้วยการจัดรูปพีชคณิตอย่างเดียว เราอาจเลือกมองว่า พจน์ทั้งสองข้างของสมการนั้นกำลังนับ ``สิ่งเดียวกัน" แต่คนละวิธี เมื่อเป็นเช่นนั้น ถ้าทั้งสองข้างนับวัตถุชุดเดียวกันได้ถูกต้องจริง สองข้างก็ต้องเท่ากัน

วิธีคิดเช่นนี้เรียกว่า **combinatorial proof** และในหลายกรณีจะอยู่ในรูปของสิ่งที่เรียกว่า **double counting** หรือการนับสิ่งเดียวกันสองวิธี เสน่ห์ของวิธีนี้อยู่ตรงที่ มันไม่ได้เพียงบอกว่าเอกลักษณ์ใดเป็นจริง แต่ยังอธิบายด้วยว่า เหตุใดมันจึงเป็นจริงในเชิงโครงสร้าง พูดอีกแบบหนึ่ง พิสูจน์เชิงการนับที่ดีไม่ได้เป็นเพียงการแทนสัญลักษณ์ แต่เป็นการเปิดเผยความหมายที่ซ่อนอยู่ของสมการนั้น

ในหัวข้อนี้ เราจะค่อย ๆ ศึกษาหลักคิดของ *combinatorial proof* เริ่มจากแนวคิดทั่วไปของการนับสองวิธี แล้วจึงดูเอกลักษณ์พื้นฐานอย่าง

$$\binom{n}{r} = \binom{n}{n-r} \quad \text{และ} \quad \binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r},$$

ก่อนจะขยายไปสู่ตัวอย่างที่ซับซ้อนขึ้น โดยเฉพาะตัวอย่างที่เกี่ยวกับการเรียงสับเปลี่ยนและการจัดกลุ่ม ซึ่งจะช่วยให้เห็นว่าการพิสูจน์เชิงการนับ เป็นส่วนหนึ่งของการให้เหตุผลทางคณิตศาสตร์อย่างแท้จริง ไม่ใช่เพียงเทคนิคเสริมเฉพาะบทนี้เท่านั้น

12.5.1 การพิสูจน์ด้วยการนับสองวิธี

หลักคิดที่เรียบง่ายที่สุดของ combinatorial proof คือ: ถ้าเรามีเซตของวัตถุบางชนิดอยู่หนึ่งเซต และสามารถนับจำนวนสมาชิกของเซตนั้นได้สองวิธี ผลลัพธ์ที่ได้จากทั้งสองวิธีย่อมต้องเท่ากัน

ในเชิงนามธรรม ถ้า S เป็นเซตของวัตถุที่เราสนใจ แล้วเราพิสูจน์ได้ว่า

$$|S| = A \quad \text{และ} \quad |S| = B,$$

เราย่อมสรุปได้ทันทีว่า

$$A = B.$$

นี่คือรูปแบบพื้นฐานที่สุดของการพิสูจน์แบบ double counting

สิ่งสำคัญที่ควรสังเกตคือ จุดยากของวิธีนี้ไม่ได้อยู่ที่การบวกหรือลบเลข แต่อยู่ที่การเลือกเซตของวัตถุที่จะนับให้เหมาะสม และการอธิบายให้ชัดว่า พจน์แต่ละข้างของสมการที่ต้องการพิสูจน์ กำลังนับวัตถุชนิดนั้นอย่างไร ถ้าเลือกเซตไม่เหมาะสม พิสูจน์อาจดูฝืนหรือไม่เห็นภาพ แต่ถ้าเลือกได้ดี สมการที่ดูแปลกในตอนแรกอาจกลายเป็นเรื่องธรรมดาทันที

ยกตัวอย่างอย่างง่าย สมมติว่าเราต้องการพิสูจน์ว่า

$$P(n, n) = P(n, k) P(n - k, n - k).$$

แทนที่จะจัดรูปจากสูตรแฟกทอเรียลโดยตรง เราสามารถมองว่า ทั้งสองข้างกำลังนับจำนวนวิธีเรียงของ n ชิ้นทั้งหมดในแนวเส้นตรง

ด้านซ้าย $P(n, n)$ นับตรง ๆ: เรียงของทั้ง n ชิ้นเลย

ส่วนด้านขวา $P(n, k) P(n - k, n - k)$ นับโดยแยกเป็นสองช่วง: เรียง k ตำแหน่งแรกก่อน จากนั้นเรียงของที่เหลือนอีก $n - k$ ชิ้นในตำแหน่งที่เหลือ ทั้งสองวิธีให้ผลลัพธ์สุดท้ายเป็นการเรียงของ n ชิ้นครบทุกชิ้นเหมือนกัน จึงต้องได้จำนวนเท่ากัน

ตัวอย่างนี้แสดงให้เห็นลักษณะสำคัญของ combinatorial proof: เราไม่ได้เริ่มจาก “จะจัดรูปพีชคณิตอย่างไร” แต่เริ่มจาก “สองข้างนี้นับอะไรอยู่”

12.5.2 $\binom{n}{r} = \binom{n}{n-r}$

หนึ่งในเอกลักษณ์พื้นฐานที่สุดของ combinations คือ

$$\binom{n}{r} = \binom{n}{n-r}.$$

ถ้ามองจากสูตร เราตรวจสอบได้ไม่ยากว่า

$$\binom{n}{r} = \frac{n!}{r!(n-r)!} = \frac{n!}{(n-r)!r!} = \binom{n}{n-r}.$$

แต่การตรวจแบบนี้ยังไม่บอกว่า เหตุใดเอกลักษณ์นี้จึงควรเป็นจริงในเชิงความหมาย

มุมมองเชิง combinatorics ของสมการนี้ตรงไปตรงมามาก: การเลือกสมาชิก r ตัวจากเซตที่มี n ตัว สมมูลกับการเลือกสมาชิกที่ *ไม่ถูกเลือก* ซึ่งมีอยู่ $n - r$ ตัว

Theorem 12.5.1. สำหรับทุกจำนวนเต็ม n, r ที่ $0 \leq r \leq n$,

$$\binom{n}{r} = \binom{n}{n-r}.$$

บทพิสูจน์. พิจารณาเซต A ที่มีสมาชิก n ตัว เราจะนับจำนวนวิธีเลือกกลุ่มย่อยขนาด r ของ A

วิธีที่หนึ่ง นับตรง ๆ ได้จำนวน

$$\binom{n}{r}.$$

วิธีที่สอง ทุกครั้งที่เราเลือกสมาชิก r ตัว สมาชิกที่เหลือซึ่งไม่ได้ถูกเลือกจะมีอยู่ $n - r$ ตัว ในทางกลับกัน ถ้าเรารู้ว่าไม่ได้เลือกสมาชิกตัวใดบ้างจำนวน $n - r$ ตัว เราก็ย่อมรู้ทันทีว่าสมาชิกที่ถูกเลือกคืออะไร

ดังนั้นการเลือกสมาชิก r ตัว ก็กับการเลือกสมาชิกที่เหลือ $n - r$ ตัว เป็นกระบวนการที่สอดคล้องกันแบบหนึ่งต่อหนึ่ง จึงมีจำนวนเท่ากัน นั่นคือ

$$\binom{n}{r} = \binom{n}{n-r}.$$

□

เอกลักษณ์นี้มักถูกเรียกว่า **symmetry identity** เพราะสะท้อนความสมมาตรของการเลือกกับการไม่เลือก และยังเป็นตัวอย่างที่ดีมากของการพิสูจน์แบบ combinatorial: เราไม่ได้จัดสูตร แต่เราตีความโจทย์เดิมใหม่ในอีกมุมหนึ่ง

12.5.3 Pascal identity

อีกเอกลักษณ์หนึ่งที่สำคัญมากและปรากฏแทบทุกครั้งเมื่อศึกษาสัมประสิทธิ์ทวินามคือ

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}.$$

ถ้าอ่านอย่างผิวเผิน สมการนี้อาจดูเหมือนเป็นเพียงสูตร recursive ของ combinations แต่ในความจริง มันมีความหมายเชิงการนับที่เป็นธรรมชาติมาก: เมื่อจะเลือก r คนจาก n คน เราสามารถแยกกรณีตามว่า "มีบุคคลพิเศษคนหนึ่งถูกเลือกหรือไม่"

Theorem 12.5.2 (Pascal identity). สำหรับทุกจำนวนเต็ม n, r ที่ $1 \leq r \leq n$,

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}.$$

บทพิสูจน์. พิจารณาเซต A ที่มีสมาชิก n ตัว และเลือกสมาชิกตัวหนึ่งมาเป็นสมาชิกพิเศษ เรียกว่า a

เราจะนับจำนวนวิธีเลือกกลุ่มย่อยขนาด r ของ A

วิธีที่หนึ่ง นับตรง ๆ ได้จำนวน

$$\binom{n}{r}.$$

วิธีที่สอง แยกเป็นสองกรณีที่ไม่ทับกัน:

1. **กรณีที่เลือก a เข้ามาด้วย** เมื่อเลือก a แล้ว เราต้องเลือกสมาชิกเพิ่มอีก $r - 1$ ตัวจากสมาชิกที่เหลืออีก $n - 1$ ตัว จึงมีจำนวนวิธี

$$\binom{n-1}{r-1}.$$

2. **กรณีที่ไม่เลือก a** ในกรณีนี้ เราต้องเลือกสมาชิก r ตัวทั้งหมดจากสมาชิกที่เหลืออีก $n - 1$ ตัว จึงมีจำนวนวิธี

$$\binom{n-1}{r}.$$

สองกรณีนี้ไม่ทับกันและครอบคลุมทุกความเป็นไปได้ ดังนั้นจากหลักการบวก จำนวนวิธีทั้งหมดคือ

$$\binom{n-1}{r-1} + \binom{n-1}{r}.$$

จึงได้ว่า

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r}.$$

□

สิ่งที่ควรจำจากพิสูจน์นี้ไม่ใช่แค่ตัวสมการ แต่คือแพทเทิร์นของการคิด: เมื่อจะนับอะไรบางอย่าง การเลือกวัตถุพิเศษหนึ่งตัวขึ้นมา แล้วแยกกรณีตามว่า “อยู่” หรือ “ไม่อยู่” มักเป็นเทคนิคที่ทรงพลังมาก

12.5.4 เกล็ดที่เกิตจากการตีความโจทย์เดียวกันสองแบบ

เมื่อเข้าใจ symmetry identity และ Pascal identity แล้ว สิ่งสำคัญถัดไปคือการเห็นว่า double counting ไม่ได้จำกัดอยู่เพียงการพิสูจน์สองเอกลักษณ์นี้ แต่เป็นวิธีคิดทั่วไป กล่าวคือเมื่อใดก็ตามที่เราหาโจทย์หนึ่งโจทย์ ซึ่งสามารถตีความได้สองแบบ เราก็มีโอกาสได้เอกลักษณ์ใหม่ขึ้นมา

ตัวอย่างมาตรฐานที่สำคัญมากคือ

$$\sum_{i=0}^r \binom{m}{i} \binom{n}{r-i} = \binom{m+n}{r}.$$

สมการนี้อาจดูเหมือนผลรวมที่ซับซ้อน แต่ถ้าอ่านในเชิงการนับ มันมีความหมายง่ายมาก: สมมติว่าเรามีคนทั้งหมด $m+n$ คน โดยแบ่งเป็นสองกลุ่มย่อย กลุ่มแรกมี m คน กลุ่มที่สองมี n คน ถ้าต้องการเลือกคนทั้งหมด r คน เราสามารถนับได้สองวิธี

วิธีแรก เลือกตรง ๆ จากคนทั้งหมด $m+n$ คน ได้

$$\binom{m+n}{r}.$$

วิธีที่สอง แยกตามจำนวนคนที่เลือกมาจากกลุ่มแรก ถ้าเลือกจากกลุ่มแรกมา i คน ก็ต้องเลือกจากกลุ่มที่สองอีก $r-i$ คน จึงมีจำนวนวิธี

$$\binom{m}{i} \binom{n}{r-i}.$$

เมื่อรวมทุกค่าที่เป็นไปได้ของ i จึงได้ผลรวมทางซ้าย

สมการนี้จึงไม่ใช่สิ่งลึกลับ แต่เป็นผลจากการนับงานเดียวกันสองมุมมอง และนี่คือธรรมชาติของ combinatorial proof อย่างแท้จริง

ตัวอย่างอีกแบบหนึ่งที่พบบ่อยคือการใช้ทฤษฎีบททวินาม เช่น

$$\sum_{r=0}^n \binom{n}{r} = 2^n.$$

ข้อนี้อาจพิสูจน์จากการแทน $x = y = 1$ ใน $(x + y)^n$ ได้ แต่ในเชิง combinatorics มันพูดว่า: จำนวนเซตย่อยทั้งหมดของเซตที่มีสมาชิก n ตัวมีค่าเท่ากับ 2^n

ด้านซ้ายแยกนับตามขนาดของเซตย่อย: มีเซตย่อยขนาด 0, ขนาด 1, ..., ขนาด n

ด้านขวานับโดยมองทีละสมาชิก: สำหรับสมาชิกแต่ละตัว มีสองทางเลือกคือ จะอยู่หรือไม่อยู่ในเซตย่อย จึงมีทั้งหมด 2^n แบบ

จะเห็นว่าเอกลักษณ์เดียวกัน ถ้ามองต่างมุมก็ให้ความเข้าใจต่างกัน และมุมมองเชิง combinatorics มักให้ intuition ที่คมกว่า

12.5.5 ตัวอย่างของ double counting ในปัญหาการจัดเรียงและการเลือก

เพื่อให้เห็นว่าการนับสองวิธีไม่ใช่เรื่องจำกัดยู่แค่สูตรของ combinations ลองพิจารณาตัวอย่างจาก permutations และ combinations ร่วมกัน

Theorem 12.5.3. สำหรับ $0 \leq r \leq n$,

$$P(n, r) = \binom{n}{r} r!.$$

บทพิสูจน์. เราจะนับจำนวนวิธีเรียงสมาชิก r ตัวจากเซตที่มีสมาชิก n ตัว

วิธีที่หนึ่ง นับตรง ๆ ได้จำนวน

$$P(n, r).$$

วิธีที่สอง เริ่มจากเลือกสมาชิกที่จะถูกนำมาใช้ก่อนจำนวน r ตัว ซึ่งทำได้

$$\binom{n}{r}$$

วิธี จากนั้นจึงนำสมาชิกที่เลือกมาเรียงในแนวเส้นตรง ซึ่งทำได้

$$r!$$

วิธี

ดังนั้นจากหลักการคูณ จำนวนวิธีทั้งหมดคือ

$$\binom{n}{r} r!.$$

เพราะทั้งสองวิธีนับวัตถุเดียวกัน จึงได้ว่า

$$P(n, r) = \binom{n}{r} r!.$$

□

พิสูจน์นี้มีคุณค่ามาก เพราะช่วยเชื่อมสองหัวข้อก่อนหน้านี้เข้าด้วยกันอย่างชัดเจน: permutation คือ combination ที่เพิ่มลำดับกลับเข้าไป

อีกตัวอย่างหนึ่งที่น่าสนใจคือ

$$n \cdot n! = (n + 1 - 1)(n!)$$

ในรูปนี้อาจดูธรรมดาเกินไป แต่ถ้าออกแบบโจทย์ดี ๆ มันอาจกลายเป็น combinatorial proof ได้ เช่นการนับจำนวนวิธีเลือกหนึ่งตำแหน่งพิเศษในลำดับของ n คนที่ถูกเรียงแล้ว หรือการนับจำนวนวิธีสร้างลำดับโดยระบุสมาชิกที่อยู่ตำแหน่งต้นทางหรือปลายทาง ตัวอย่างประเภทนี้ช่วยให้ผู้เรียนเริ่มมองเห็นว่า double counting ไม่ได้ถูกจำกัดด้วยรายการเอกลักษณ์ที่มีในตำรา แต่เป็นวิธีคิดที่ยืดหยุ่นมาก

ในการทำงานเดียวกัน โจทย์แบ่งคนเป็นกลุ่ม หรือเลือกคณะกรรมการภายใต้เงื่อนไข ก็มักมีคำอธิบายสองแบบได้เช่นกัน เช่น เราอาจนับตามจำนวนผู้หญิงที่ถูกเลือก หรืออาจนับตามจำนวนผู้ชายที่ไม่ได้ถูกเลือก ถ้าทั้งสองแบบพูดถึงเหตุการณ์เดียวกัน เราก็ได้เอกลักษณ์ใหม่ขึ้นมา

ด้วยเหตุนี้ การฝึก combinatorial proof จึงไม่ควรถูกมองว่าเป็นเพียงส่วนเสริมของบท แต่เป็นการฝึกมองสมการว่า “กำลังนับอะไรอยู่” ซึ่งเป็นทักษะที่ลึกและมีประโยชน์มาก เพราะช่วยให้เราไม่เพียงพิสูจน์เอกลักษณ์ได้ แต่ยังเข้าใจที่มาของมันด้วย

แนวคิดที่ควรจำ

เมื่อจะทำ combinatorial proof ควรถามตัวเองอย่างน้อยสามข้อ:

1. ฉันกำลังนับวัตถุใด
2. พจน์ด้านซ้ายของสมการนับวัตถุนี้อย่างไร
3. พจน์ด้านขวาของสมการนับวัตถุเดียวกันนี้อย่างไร

ถ้าตอบสามข้อนี้ได้ชัด พิสูจน์เชิงการนับมักจะตามมาได้อย่างเป็นธรรมชาติ

สรุปแล้ว double counting เป็นหนึ่งในรูปแบบการให้เหตุผลที่สวยงามที่สุดใน combinatorics เพราะมันแสดงให้เห็น

เห็นว่า ความเท่ากันของสองนิพจน์ อาจไม่ได้เกิดจากการจัดรูปเชิงสัญลักษณ์เท่านั้น แต่เกิดจากการที่ทั้งสองนิพจน์กำลังอธิบายความจริงเชิงโครงสร้างเดียวกัน และมุมมองแบบนี้นี่เอง ที่จะพาเราเข้าสู่หัวข้อถัดไปเรื่องสัมประสิทธิ์ทวินามและทฤษฎีบททวินาม ด้วยความเข้าใจที่ลึกกว่าเพียงการคำนวณตามสูตร

เมื่อมองย้อนกลับไปทั้งบทนี้ จะเห็นว่าหัวข้อที่ผ่านมาแม้จะเริ่มแตกแขนงออกไป ตั้งแต่หลักการบวกและหลักการคูณ ไปจนถึง **permutations** และ **combinations** แต่แก่นของทั้งหมดกลับเป็นเรื่องเดียวกัน นั่นคือการตอบคำถามให้ชัดว่า ผลลัพธ์หนึ่งแบบของโจทย์คืออะไร ลำดับมีความสำคัญหรือไม่ เรากำลังเลือกจากหลายกรณีที่ไม่ทับกัน หรือกำลังประกอบผลลัพธ์ขึ้นจากหลายขั้นตอนต่อเนื่องกัน และมีการนับซ้ำซ้อนอยู่หรือไม่

นี่คือเหตุผลที่ผู้เขียนอยากให้มองบทนี้ ไม่ใช่ในฐานะรายการสูตรพื้นฐานที่ต้องรีบจำให้ครบ แต่ในฐานะบทฝึก *วินัยของการนับ* ถ้าผู้อ่านเริ่มเห็นได้ว่า สูตรอย่าง $n!$, $P(n, r)$, หรือ $\binom{n}{r}$ ไม่ได้เกิดขึ้นลอย ๆ แต่เกิดจากการวางแผนการนับอย่างเป็นระบบ บทนี้ก็ได้ทำหน้าที่สำคัญของมันแล้ว เพราะในระยะยาว ความเข้าใจเชิงโครงสร้างเช่นนี้มีค่ามากกว่าการจำสูตรแยกเป็นข้อ ๆ มาก

อย่างไรก็ตาม **combinatorics** ไม่ได้หยุดอยู่เพียงการนับแบบพื้นฐานเท่านั้น เมื่อภาษาของการเรียงและการเลือกเริ่มมั่นคงแล้ว เราสามารถใช้การนับเป็นเครื่องมือสำหรับพิสูจน์เอกลักษณ์ ใช้สัมประสิทธิ์ทวินามเพื่อเชื่อม **combinatorics** เข้ากับ **algebra** ใช้การเลือกตำแหน่งเพื่ออธิบายเส้นทางบนตาราง และใช้สมการจำนวนเต็มเพื่ออธิบายโจทย์การแจกของได้ด้วย ดังนั้นจากบทถัดไป เราจะค่อย ๆ ขยับจาก “กฎพื้นฐานของการนับ” ไปสู่ “หัวข้อเพิ่มเติมเกี่ยวกับการนับ” ซึ่งจะทำให้เห็นว่า การนับไม่ใช่เพียงเทคนิคสำหรับหาคำตอบเชิงจำนวน แต่เป็นภาษาอีกแบบหนึ่งของการให้เหตุผลทางคณิตศาสตร์

บทที่ 13

หัวข้อเพิ่มเติมเกี่ยวกับการนับ

ในบทก่อนหน้า เราได้วางรากฐานของการนับไว้แล้ว ไม่ว่าจะเป็นหลักการบวก หลักการคูณ การเรียงสับเปลี่ยน และการจัดกลุ่ม เครื่องมือเหล่านี้ทำให้เราเริ่มตอบโจทย์พื้นฐานได้อย่างเป็นระบบ และที่สำคัญกว่านั้น ทำให้เราเห็นว่า “สูตรการนับ” จำนวนมาก แท้จริงแล้วเป็นผลสรุปจากการวางแผนการนับอย่างระมัดระวัง

บทนี้จะต่อยอดจากฐานดังกล่าว แต่จุดเน้นจะเปลี่ยนไปเล็กน้อย ถ้าบทก่อนหน้ามุ่งฝึก กฎพื้นฐานของการนับ บทนี้จะมุ่งฝึก การใช้มุมมองเชิงการนับในบริบทที่ลึกซึ้ง เราจะเห็นว่า การนับสามารถทำหน้าที่เป็นวิธีพิสูจน์ได้ สามารถเชื่อมกับทฤษฎีบททวินามและพหุนามได้ สามารถอธิบายเส้นทางบนตารางในฐานปัญหาการเลือกตำแหน่งได้ และสามารถแปลโจทย์การแจกของให้กลายเป็นปัญหาเรื่อง multisets หรือคำตอบจำนวนเต็มของสมการได้

กล่าวอีกแบบหนึ่ง บทนี้ไม่ได้มีเป้าหมายจะเพิ่มคลังสูตรให้มากขึ้นเฉย ๆ แต่ต้องการทำให้ผู้อ่านเห็นว่า วัตถุเดียวกันอาจถูกมองได้หลาย representation และเมื่อเปลี่ยน representation ให้เหมาะสม โจทย์ที่ดูยากในรูปเดิมอาจกลายเป็นเรื่องตรงไปตรงมาทันที เพราะฉะนั้น หัวใจของบทนี้จึงไม่ใช่การจำเทคนิคเฉพาะข้อ แต่คือการฝึกมองว่า

สองนิพจน์นี้กำลังนับสิ่งเดียวกันอยู่หรือไม่

พจน์ในพหุนามนี้กำลังสะท้อนการเลือกอะไร

เส้นทางนี้สามารถแปลเป็นลำดับของทางเลือกได้อย่างไร

และโจทย์การแจกของนี้ควรถูกมองเป็นฟังก์ชัน เป็นการเลือกซ้ำ หรือเป็นคำตอบจำนวนเต็ม

ถ้าบทก่อนหน้าช่วยให้ผู้อ่าน “นับเป็น” บทนี้ก็จะมีเป้าหมายจะช่วยให้ผู้อ่าน “มองการนับเป็นภาษา” มากขึ้น และเมื่อมองได้ในระดับนั้น combinatorics จะไม่ใช่เพียงเครื่องมือสำหรับแก้โจทย์ แต่จะกลายเป็นอีกวิธีหนึ่งของการคิดและการพิสูจน์ในคณิตศาสตร์ที่สวยงาม

13.1 Binomial Coefficients and the Binomial Theorem

ในหัวข้อที่ผ่านมา เราได้ศึกษาจำนวน

$$\binom{n}{r}$$

ในฐานะจำนวนวิธีเลือกสมาชิก r ตัวจากเซตที่มีสมาชิก n ตัว และยังได้เห็นด้วยว่าเอกลักษณ์หลายอย่างของ $\binom{n}{r}$ สามารถพิสูจน์ได้ด้วยเหตุผลเชิงการนับ จุดนี้สำคัญมาก เพราะมันทำให้เราเห็นว่า binomial coefficient ไม่ใช่เพียงสัญลักษณ์ที่เกิดจากสูตรแพททอเรียล แต่เป็นวัตถุทางคณิตศาสตร์ที่มีความหมายเชิงโครงสร้างอยู่จริง

อย่างไรก็ตาม จำนวน $\binom{n}{r}$ ยังปรากฏอีกบริบทหนึ่งที่สำคัญไม่แพ้กัน คือในการกระจายพหุนามทวินาม

$$(x + y)^n.$$

เมื่อเรากระจายพหุนามนี้ออกมา สัมประสิทธิ์ที่ปรากฏหน้าพจน์ต่าง ๆ จะเป็นจำนวนชนิดเดียวกับที่เราใช้แทนการเลือกสมาชิก นี่จึงเป็นเหตุผลว่าทำไม $\binom{n}{r}$ จึงถูกเรียกว่า **สัมประสิทธิ์ทวินาม** (binomial coefficient) การเชื่อมโยงระหว่าง “การเลือก” กับ “สัมประสิทธิ์ของพจน์” เป็นหนึ่งในภาพที่สวยงามที่สุดของ combinatorics เพราะมันแสดงให้เห็นว่าโครงสร้างเชิงการนับและโครงสร้างเชิงพีชคณิต กำลังอธิบายสิ่งเดียวกันอยู่คนละภาษา

ในส่วนนี้ เราจะค่อย ๆ พัฒนาภาพดังกล่าวให้ชัดขึ้น เริ่มจากการย้ำความหมายของ $\binom{n}{r}$ ในฐานะจำนวนวิธีเลือก จากนั้นจึงพิสูจน์ทฤษฎีบททวินาม แล้วใช้ทฤษฎีบทนี้เป็นเครื่องมือในการอ่านสัมประสิทธิ์ของพหุนาม รวมถึงพิสูจน์เอกลักษณ์พื้นฐานต่าง ๆ ของสัมประสิทธิ์ทวินามในอีกมุมมองหนึ่ง

13.1.1 สัมประสิทธิ์ทวินามในฐานะจำนวนวิธีเลือก

ให้ A เป็นเซตที่มีสมาชิก n ตัว จำนวน

$$\binom{n}{r}$$

ถูกนิยามให้เป็นจำนวนวิธีเลือกสมาชิก r ตัวจาก A โดยไม่สนใจลำดับของสมาชิกที่เลือก ในความหมายนี้ $\binom{n}{r}$ คือจำนวนของเซตย่อยขนาด r ของเซตที่มีสมาชิก n ตัว

ดังนั้น ถ้ามองอย่างเป็นทางการ

$$\binom{n}{r}$$

คือจำนวนสมาชิกของเซต

$$\{X \subseteq A \mid |X| = r\}.$$

มุมมองนี้สำคัญมาก เพราะมันทำให้เราอ่านสัญลักษณ์นี้ได้ว่า ไม่ใช่เพียง “ค่าเชิงสูตร” แต่คือ “จำนวนของวัตถุบางชนิด” อย่างแท้จริง

จากหัวข้อก่อนหน้านี้ เราทราบแล้วว่า

$$\binom{n}{r} = \frac{n!}{r!(n-r)!} \quad \text{เมื่อ } 0 \leq r \leq n.$$

และมักกำหนดเพิ่มเติมว่า

$$\binom{n}{r} = 0 \quad \text{เมื่อ } r < 0 \text{ หรือ } r > n.$$

การกำหนดเช่นนี้ทำให้เอกลักษณ์หลายอย่างเขียนได้กระชับและสม่ำเสมอขึ้น

สิ่งที่ควรเน้นอีกครั้งคือ สูตรแฟกทอเรียลข้างต้นไม่ใช่จุดเริ่มต้นของความหมาย แต่เป็นผลลัพธ์ที่สรุปมาจากความหมายเชิงการเลือก ดังนั้นเมื่อไปพบ $\binom{n}{r}$ ในบริบทอื่น โดยเฉพาะในบริบทของพหุนาม เราควรยังคงมองเห็นอยู่เสมอว่า เบื้องหลังของมันคือจำนวนวิธีเลือกตำแหน่งหรือเลือกปัจจัยบางตัวนั่นเอง

13.1.2 ทฤษฎีบททวินาม

ตอนนี้พิจารณาพหุนาม

$$(x+y)^n = (x+y)(x+y) \cdots (x+y),$$

ซึ่งเป็นผลคูณของปัจจัย n ตัวที่เหมือนกัน เมื่อเรากระจายผลคูณนี้ออก พจน์แต่ละพจน์จะเกิดจากการเลือกสมาชิกหนึ่งตัวจากแต่ละวงเล็บ กล่าวคือ จากวงเล็บแต่ละตัว เราเลือกได้ว่าจะหยิบ x หรือหยิบ y

เพราะฉะนั้น ถ้าต้องการให้เกิดพจน์

$$x^{n-r}y^r,$$

เราต้องเลือก y มาจากวงเล็บทั้งหมด r วงเล็บ และเลือก x จากวงเล็บที่เหลืออีก $n-r$ วงเล็บ จำนวนวิธีที่ทำเช่นนี้ได้คือจำนวนวิธีเลือกว่ามีวงเล็บใดบ้าง r วงเล็บที่เราจะหยิบ y ซึ่งก็คือ

$$\binom{n}{r}.$$

นี่เองคือเหตุผลเชิง combinatorics ของทฤษฎีบททวินาม

Theorem 13.1.1 (Binomial Theorem). สำหรับจำนวนเต็มบวก n ใด ๆ

$$(x+y)^n = \binom{n}{0}x^n + \binom{n}{1}x^{n-1}y + \binom{n}{2}x^{n-2}y^2 + \cdots + \binom{n}{n-1}xy^{n-1} + \binom{n}{n}y^n.$$

หรือเขียนอย่างย่อได้ว่า

$$(x + y)^n = \sum_{r=0}^n \binom{n}{r} x^{n-r} y^r.$$

บทพิสูจน์. พิจารณาการกระจาย

$$(x + y)^n = (x + y)(x + y) \cdots (x + y)$$

ซึ่งมีทั้งหมด n ปัจจัย

พจน์หนึ่งพจน์ในการกระจายนี้เกิดจากการเลือกสมาชิกหนึ่งตัวจากแต่ละปัจจัย ดังนั้นถ้าต้องการให้ได้พจน์

$$x^{n-r} y^r,$$

เราต้องเลือก y จากปัจจัยทั้งหมด r ตัว และเลือก x จากปัจจัยที่เหลืออีก $n - r$ ตัว

จำนวนวิธีเลือกกว่าปัจจัยใดบ้าง r ตัวที่จะให้ y เท่ากับจำนวนวิธีเลือก r ตำแหน่งจากทั้งหมด n ตำแหน่ง ซึ่งมีค่าเท่ากับ

$$\binom{n}{r}.$$

ดังนั้นสัมประสิทธิ์ของพจน์ $x^{n-r} y^r$ ในการกระจาย $(x+y)^n$ จึงเป็น $\binom{n}{r}$ เมื่อพิจารณาทุกค่า $r = 0, 1, \dots, n$ จะได้ว่า

$$(x + y)^n = \sum_{r=0}^n \binom{n}{r} x^{n-r} y^r.$$

□

พิสูจน์นี้เป็นตัวอย่างที่ดีมากของการเชื่อม algebra กับ combinatorics เพราะสัมประสิทธิ์ของพหุนามไม่ได้ถูกหาโดยการจัดรูปเพียงอย่างเดียว แต่ถูกอธิบายผ่าน “จำนวนวิธีเลือกตำแหน่ง” ซึ่งเป็นแกนกลางของหัวข้อ combinations ที่เราเพิ่งเรียนมา

13.1.3 การอ่านสัมประสิทธิ์จากการกระจายพหุนาม

หลังจากมีทฤษฎีบทวินามแล้ว สิ่งสำคัญถัดไปคือการฝึกอ่านว่า สัมประสิทธิ์ของพจน์ใดพจน์หนึ่งในพหุนามที่กระจายออกมา ควรตีความอย่างไร ในกรณีพื้นฐานที่สุด ถ้าเรากำลังพิจารณา

$$(x + y)^n,$$

สัมประสิทธิ์ของพจน์

$$x^{n-r}y^r$$

คือ

$$\binom{n}{r}.$$

ดังนั้นเวลาพบพจน์ชนิดนี้ สิ่งแรกที่ควรถามคือ พจน์ดังกล่าวเกิดจากการเลือก y มากี่ครั้งจากทั้งหมด n ปัจจัย เมื่อรู้จำนวนครั้งนั้นแล้ว สัมประสิทธิ์ก็จะตามมาโดยตรง

Example 13.1.2. ในพหุนาม

$$(x + y)^5$$

สัมประสิทธิ์ของพจน์ x^2y^3 คือ

$$\binom{5}{3} = 10,$$

เพราะต้องเลือก y จากทั้งหมด 5 ปัจจัยมา 3 ครั้ง

อย่างไรก็ตาม ในใจที่ยังจริง พหุนามที่พบมักไม่ได้อยู่ในรูป $(x + y)^n$ ตรง ๆ แต่อาจอยู่ในรูปที่ซับซ้อนกว่า เช่น

$$(ax + by)^n \quad \text{หรือ} \quad (u + v)^n$$

ที่ u และ v เป็นนิพจน์อื่น ในกรณีเช่นนี้ หลักคิดยังคงเหมือนเดิม เพียงแต่ต้องระวังว่าพจน์ที่เลือกแต่ละครั้งมีค่าสัมประสิทธิ์และดีกรีติดมาด้วย

ตัวอย่างเช่น ถ้าพิจารณา

$$(2x + y^2)^5,$$

แล้วต้องการหาสัมประสิทธิ์ของพจน์ x^2y^6 เราควรเริ่มจากถามก่อนว่า เพื่อให้ได้ x^2 เราต้องเลือก $2x$ มาทั้งหมดกี่ครั้ง คำตอบคือ 2 ครั้ง และเพื่อให้ได้ y^6 เราต้องเลือก y^2 ทั้งหมด 3 ครั้ง ดังนั้นโครงสร้างของการเลือกคือ: เลือก $2x$ จาก 5 ปัจจัยมา 2 ปัจจัย และเลือก y^2 จากอีก 3 ปัจจัยที่เหลือ

จำนวนวิธีเลือกตำแหน่งเช่นนี้คือ

$$\binom{5}{2}.$$

แต่ทุกครั้งที่เลือก $2x$ มาหนึ่งครั้ง จะมีตัวคูณ 2 ติดมาด้วย ดังนั้นเมื่อเลือก $2x$ ทั้งหมด 2 ครั้ง จะได้ตัวคูณเพิ่มอีก 2^2 จึงสรุปว่าสัมประสิทธิ์ของ x^2y^6 คือ

$$\binom{5}{2} 2^2 = 10 \cdot 4 = 40.$$

ตัวอย่างนี้สำคัญเพราะชี้ให้เห็นว่า การอ่านสัมประสิทธิ์จากการกระจายพหุนาม ไม่ใช่เพียงการแทนค่าลงสูตร แต่

คือการแยกให้ออกว่า พจน์เป้าหมายเกิดจากการเลือกองค์ประกอบใดจากแต่ละปัจจัย และมีตัวคูณอะไรสะสมมาด้วยบ้าง

13.1.4 เอกลักษณ์พื้นฐานของสัมประสิทธิ์ทวินาม

แม้เราได้พิสูจน์เอกลักษณ์บางอย่างของ $\binom{n}{r}$ ไปแล้วในหัวข้อ combinatorial proofs แต่เมื่อเข้าสู่บริบทของทฤษฎีบททวินาม เอกลักษณ์เหล่านี้เริ่มมีชีวิตในอีกมุมหนึ่ง เพราะมันสามารถถูกอ่านได้ทั้งในฐานะข้อเท็จจริงเชิงการเลือก และในฐานะข้อเท็จจริงเชิงพหุนาม

เอกลักษณ์พื้นฐานที่สุดได้แก่

$$\binom{n}{0} = 1, \quad \binom{n}{n} = 1,$$

ซึ่งสะท้อนว่า การเลือกไม่มีสมาชิกเลยหรือเลือกสมาชิกทั้งหมด มีได้เพียงวิธีเดียวเท่านั้น

ถัดมาคือ symmetry identity

$$\binom{n}{r} = \binom{n}{n-r},$$

ซึ่งสะท้อนว่าการเลือก r ตัว สมมูลกับการเลือกตัวที่ไม่ถูกเลือกอีก $n-r$ ตัว

อีกเอกลักษณ์ที่สำคัญมากคือ Pascal identity

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r},$$

ซึ่งสรุปได้จากการแยกกรณีว่ามีสมาชิกพิเศษตัวหนึ่งถูกเลือกหรือไม่

เอกลักษณ์เหล่านี้อาจดูเหมือนเป็นเพียงรายการข้อเท็จจริง แต่จริง ๆ แล้วเป็น "ไวยากรณ์พื้นฐาน" ของ binomial coefficients และจะกลับมาปรากฏซ้ำแล้วซ้ำอีก ทั้งใน combinatorics, ในพหุนามทวินาม, และในหัวข้อถัด ๆ ไปของคณิตศาสตร์ดีสครีต

นอกจากนี้ เมื่อมองผ่านทฤษฎีบททวินาม เรายังได้เอกลักษณ์สำคัญอีกชุดหนึ่งอย่างเป็นธรรมชาติ เช่น

$$\sum_{r=0}^n \binom{n}{r} = 2^n.$$

สมการนี้บอกว่า ผลรวมของสัมประสิทธิ์ทั้งหมดในแถวที่ n ของ Pascal's triangle มีค่าเท่ากับ 2^n และยังมีความหมายเชิง combinatorics ว่า จำนวนเซตย่อยทั้งหมดของเซตที่มีสมาชิก n ตัวเท่ากับ 2^n

อีกสมการหนึ่งที่สำคัญคือ

$$\sum_{r=0}^n (-1)^r \binom{n}{r} = 0 \quad \text{เมื่อ } n \geq 1.$$

สมการนี้สะท้อนการหักล้างกันระหว่างสัมประสิทธิ์ที่มีเครื่องหมายสลับบวกและลบ และจะมีบทบาทมากในเอกลักษณ์ที่เกี่ยวข้องกับผลรวมเชิง alternating

13.1.5 การใช้ทฤษฎีบททวินามในการพิสูจน์เอกลักษณ์เชิงการจัด

จุดแข็งของทฤษฎีบททวินามไม่ได้อยู่แค่การกระจายพหุนาม แต่อยู่ที่มันเป็นเครื่องมือที่ทรงพลังมากในการพิสูจน์เอกลักษณ์เกี่ยวกับ $\binom{n}{r}$ หลายครั้งเอกลักษณ์ที่ดูซับซ้อนในเชิงผลรวม กลับได้มาทันทีเมื่อเราแทนค่า x และ y อย่างเหมาะสมใน

$$(x + y)^n = \sum_{r=0}^n \binom{n}{r} x^{n-r} y^r.$$

เริ่มจากเอกลักษณ์พื้นฐานที่สุดก่อน ถ้าแทน $x = 1$ และ $y = 1$ จะได้ว่า

$$(1 + 1)^n = \sum_{r=0}^n \binom{n}{r} 1^{n-r} 1^r,$$

หรือก็คือ

$$2^n = \sum_{r=0}^n \binom{n}{r}.$$

นี่คือวิธีพิสูจน์ที่สั้นมาก แต่สะท้อนความสัมพันธ์โดยตรงระหว่างทฤษฎีบททวินามกับผลรวมของสัมประสิทธิ์ทวินาม

ถัดมา ถ้าแทน $x = 1$ และ $y = -1$ จะได้ว่า

$$(1 - 1)^n = \sum_{r=0}^n \binom{n}{r} (-1)^r,$$

ดังนั้นสำหรับ $n \geq 1$,

$$0 = \sum_{r=0}^n (-1)^r \binom{n}{r}.$$

เมื่อมีสองสมการนี้แล้ว เราสามารถรวมและลบกันเพื่อหาผลรวมของสัมประสิทธิ์ตำแหน่งคู่และตำแหน่งคี่ได้ด้วย

กล่าวคือ จาก

$$\sum_{r=0}^n \binom{n}{r} = 2^n$$

และ

$$\sum_{r=0}^n (-1)^r \binom{n}{r} = 0$$

จะได้ว่า

$$\binom{n}{0} + \binom{n}{2} + \binom{n}{4} + \cdots = \binom{n}{1} + \binom{n}{3} + \binom{n}{5} + \cdots = 2^{n-1}.$$

ผลลัพธ์นี้มีความหมายสวยงามมาก เพราะมันบอกว่าผลรวมของสัมประสิทธิ์ลำดับคู่ เท่ากับผลรวมของสัมประสิทธิ์ลำดับคี่พอดี

อีกเอกลักษณ์หนึ่งที่สำคัญมากคือ

$$\sum_{i=0}^r \binom{m}{i} \binom{n}{r-i} = \binom{m+n}{r},$$

ซึ่งมักเรียกว่า *Vandermonde's identity* เอกลักษณ์นี้พิสูจน์ได้เชิง combinatorics โดยตรงจากการเลือก r คนจากสองกลุ่มย่อย แต่ในอีกมุมหนึ่งก็อธิบายได้ผ่านทฤษฎีบททวินามเช่นกัน โดยเริ่มจากข้อเท็จจริงว่า

$$(1+x)^m (1+x)^n = (1+x)^{m+n}.$$

ถ้ากระจายด้านซ้ายและอ่านสัมประสิทธิ์ของ x^r เราจะได้

$$\sum_{i=0}^r \binom{m}{i} \binom{n}{r-i},$$

ขณะที่ถ้าอ่านสัมประสิทธิ์ของ x^r จากด้านขวา จะได้

$$\binom{m+n}{r}.$$

จึงสรุปว่า

$$\sum_{i=0}^r \binom{m}{i} \binom{n}{r-i} = \binom{m+n}{r}.$$

ตัวอย่างนี้สำคัญมาก เพราะแสดงให้เห็นว่าวิธีคิดแบบ combinatorial proof และวิธีคิดแบบพหุนามไม่ได้แข่งขันกัน แต่กลับช่วยกันอธิบายสมการเดียวกันจากคนละมุม บางครั้งการนับให้ intuition ดีกว่า บางครั้งพหุนามทำให้

การคำนวณสั้นกว่า และเมื่อสองมูมนี้นำมาประกบกัน ความเข้าใจของเราต่อ binomial coefficients ก็จะสมบูรณ์ยิ่งขึ้น

ภาพรวมที่ควรจำ

จำนวน

$$\binom{n}{r}$$

มีสองความหมายหลักที่ต้องเห็นพร้อมกันเสมอ:

1. เป็นจำนวนวิธีเลือกสมาชิก r ตัวจากทั้งหมด n ตัว
2. เป็นสัมประสิทธิ์ของพจน์ $x^{n-r}y^r$ ในการกระจาย $(x + y)^n$

เมื่อเห็นสองภาพนี้เชื่อมกันได้ชัด เอกลักษณ์จำนวนมากของสัมประสิทธิ์ทวินามจะกลายเป็นเรื่องที่น่าสนใจ ไม่ใช่เพียงเรื่องที่ต้องจำ

สรุปแล้ว หัวข้อนี้ทำหน้าที่เป็นสะพานสำคัญระหว่างสองโลก: โลกของการเลือกและการนับ กับโลกของพหุนามและสัมประสิทธิ์ และการเชื่อมโยงนี้เองที่จะทำให้ binomial coefficient กลายเป็นวัตถุที่ทรงพลังมากกว่าการเป็นเพียงคำตอบของโจทย์ combination ธรรมดา

13.2 เส้นทางบนตารางและการตีความแบบ combinations

หลังจากเรียนเรื่อง combinations และสัมประสิทธิ์ทวินามแล้ว มีโจทย์ชนิดหนึ่งที่คุณควรลองว่าเป็นสนามฝึกความคิดที่สำคัญมาก นั่นคือโจทย์ **เส้นทางบนตารางจุดจำนวนเต็ม** เพราะโจทย์เหล่านี้ดูเหมือนเป็นเรื่อง "เดินจากจุดหนึ่งไปอีกจุดหนึ่ง" แต่แท้จริงแล้วแก่นของมันคือการเลือกตำแหน่งของก้าวชนิดต่าง ๆ ดังนั้นมันจึงเป็นปัญหา combinations ในอีกภาษาหนึ่ง

สิ่งที่ทำให้หัวข้อนี้สำคัญมากคือ มันช่วยให้ผู้อ่านเห็นอย่างเป็นรูปธรรมว่า จำนวน

$$\binom{n}{r}$$

ไม่ได้มีความหมายเพียงว่า "เลือกสมาชิก r ตัวจาก n ตัว" แต่ยังอาจหมายถึง "เลือกว่าก้าวชนิดหนึ่งจะไปอยู่ในตำแหน่งใดบ้าง" เมื่อมองแบบนี้ได้ โจทย์เส้นทางจำนวนมากจะกลายเป็นโจทย์การเลือกที่ตรงไปตรงมาอย่างน่าประหลาด

ตลอดหัวข้อนี้ เราจะพิจารณาเส้นทางบนตารางที่เดินจากจุดหนึ่งไปยังอีกจุดหนึ่ง โดยอนุญาตให้เดินเพียงสองทิศที่ทำให้เข้าใกล้จุดหมาย เช่น เดินไปทางขวาและเดินขึ้น ซึ่งมักเรียกว่า **lattice paths** แบบโมโนโทนิค ข้อจำกัดนี้สำคัญมาก เพราะเมื่อห้ามถอยหลัง เส้นทางหนึ่งเส้นจะถูกกำหนดได้ทั้งหมดจาก "ลำดับของก้าว" ที่ใช้

13.2.1 การนับ lattice paths

เริ่มจากโจทย์พื้นฐานที่สุดก่อน: มีทั้งหมดกี่วิธีในการเดินจากจุด $(0, 0)$ ไปยังจุด (m, n) ถ้าในแต่ละก้าวเดินได้เพียง

$$R = \text{ขวา} (1, 0) \quad \text{หรือ} \quad U = \text{ขึ้น} (0, 1)$$

เท่านั้น

คำถามนี้ดูเหมือนเป็นปัญหาเชิงเรขาคณิต แต่เมื่อพิจารณาให้ดี เส้นทางทุกเส้นจาก $(0, 0)$ ไป (m, n) จำเป็นต้องใช้ก้าวไปทางขวาทั้งหมด m ก้าว และก้าวขึ้นทั้งหมด n ก้าว เพราะต้องเพิ่มพิกัด x ขึ้น m หน่วย และเพิ่มพิกัด y ขึ้น n หน่วยพอดี

ดังนั้นเส้นทางหนึ่งเส้นจึงสามารถแทนได้ด้วยลำดับของสัญลักษณ์ ที่มี R อยู่ m ตัว และ U อยู่ n ตัว รวมทั้งทั้งหมด $m + n$ ตำแหน่ง เช่นถ้าจะเดินจาก $(0, 0)$ ไป $(3, 2)$ เส้นทาง

$$RURRU$$

หมายถึง ขวา ขึ้น ขวา ขวา ขึ้น และกำหนดเส้นทางได้ครบถ้วน

ตรงนี้เองที่ทำให้โจทย์เส้นทางถูกแปลเป็นโจทย์ combinations ได้ทันที: เราต้องเลือกว่าจะวางก้าวขึ้น n ก้าวไว้ในตำแหน่งใดบ้างจากทั้งหมด $m + n$ ตำแหน่ง หรือจะเลือกตำแหน่งของก้าวขวา m ก้าวก็ได้เช่นกัน

Theorem 13.2.1. จำนวนเส้นทางจาก $(0, 0)$ ไป (m, n) ที่เดินได้เฉพาะทางขวาและทางขึ้น เท่ากับ

$$\binom{m+n}{n} = \binom{m+n}{m}.$$

บทพิสูจน์. เส้นทางทุกเส้นจาก $(0, 0)$ ไป (m, n) ประกอบด้วยก้าวทั้งหมด $m + n$ ก้าว โดยในจำนวนนี้ต้องเป็นก้าวขวา m ก้าว และก้าวขึ้น n ก้าว

ดังนั้นการกำหนดเส้นทางหนึ่งเส้น เท่ากับการเลือกตำแหน่ง n ตำแหน่งจากทั้งหมด $m + n$ ตำแหน่ง เพื่อวางก้าวขึ้น เมื่อเลือกตำแหน่งของก้าวขึ้นแล้ว ตำแหน่งที่เหลือจะต้องเป็นก้าวขวาโดยอัตโนมัติ

จึงมีจำนวนเส้นทางทั้งหมดเท่ากับ

$$\binom{m+n}{n}.$$

และโดยอาศัย symmetry identity ของ combinations จะได้เท่ากับ

$$\binom{m+n}{m}$$

เช่นกัน

□

Example 13.2.2. จำนวนเส้นทางจาก $(0, 0)$ ไป $(11, 5)$ ที่เดินได้เฉพาะขึ้นและขวา มีค่าเท่ากับ

$$\binom{16}{5} = \binom{16}{11}.$$

สิ่งที่ควรสังเกตคือ ในพิสูจน์นี้เราไม่ได้นับเส้นทางจากภาพวาดโดยตรง แต่เปลี่ยนปัญหาไปเป็นการเลือกตำแหน่งในลำดับ นี่คือหัวใจของทั้งหัวข้อนี้ และเป็นทักษะที่สำคัญมากใน **combinatorics** โดยทั่วไป: เมื่อโจทย์รูปหนึ่งน่ายาก ให้พยายามเปลี่ยน **representation** ไปเป็นรูปที่นับง่ายกว่า

13.2.2 การบังคับให้ผ่านจุดที่กำหนด

เมื่อเข้าใจกรณีพื้นฐานแล้ว โจทย์ที่ซับซ้อนขึ้นตามธรรมชาติก็คือ ถ้าต้องการบังคับให้เส้นทางผ่านจุดที่กำหนด จำนวนเส้นทางจะเป็นเท่าใด

สมมติว่าเราต้องการนับเส้นทางจาก $(0, 0)$ ไป (m, n) โดยกำหนดว่าต้องผ่านจุด (a, b) ที่อยู่ในช่วง

$$0 \leq a \leq m, \quad 0 \leq b \leq n.$$

เพราะเราอนุญาตให้เดินได้เฉพาะขึ้นกับขวา ถ้าเส้นทางจะผ่าน (a, b) จริง มันย่อมแบ่งออกเป็นสองช่วงอย่างเป็นธรรมชาติ:

1. ช่วงจาก $(0, 0)$ ไป (a, b)
2. ช่วงจาก (a, b) ไป (m, n)

และทั้งสองช่วงนี้เป็นอิสระจากกันในการนับ

จำนวนเส้นทางในช่วงแรกคือ

$$\binom{a+b}{a} \quad \text{หรือ} \quad \binom{a+b}{b}$$

เพราะต้องใช้ก้าวขวา a ก้าว และก้าวขึ้น b ก้าว

ส่วนช่วงหลังจาก (a, b) ไป (m, n) ต้องใช้ก้าวขวาอีก $m - a$ ก้าว และก้าวขึ้นอีก $n - b$ ก้าว จึงมีจำนวนเส้นทางเท่ากับ

$$\binom{(m-a) + (n-b)}{m-a} = \binom{(m-a) + (n-b)}{n-b}.$$

ดังนั้นจากหลักการคูณ เราจึงได้ผลลัพธ์ต่อไปนี้

Theorem 13.2.3. จำนวนเส้นทางจาก $(0, 0)$ ไป (m, n) ที่เดินได้เฉพาะขึ้นและขวา และต้องผ่านจุด (a, b) เท่ากับ

$$\binom{a+b}{a} \binom{m+n-a-b}{m-a}.$$

Example 13.2.4. จำนวนเส้นทางจาก $(0, 0)$ ไป $(11, 5)$ ที่ต้องผ่านจุด $(4, 3)$ มีค่าเท่ากับ

$$\binom{7}{4} \binom{9}{7} = \binom{7}{3} \binom{9}{2}.$$

แนวคิดนี้ขยายต่อไปได้ทันทีถ้าต้องผ่านหลายจุด トラバใดที่จุดเหล่านั้นเรียงลำดับกันได้ในรูปแบบที่เส้นทางโมนโทนิกผ่านได้จริง เช่น

$$(0, 0) \rightarrow (a_1, b_1) \rightarrow (a_2, b_2) \rightarrow \cdots \rightarrow (m, n),$$

โดยที่พิกัดเพิ่มขึ้นตามลำดับ จำนวนเส้นทางจะเท่ากับผลคูณของจำนวนเส้นทางในแต่ละช่วงย่อย เพราะเรากำลังใช้หลักการเดียวกันซ้ำหลายครั้ง

13.2.3 การบังคับให้ผ่านเส้นหรือช่วงที่กำหนด

นอกจากการบังคับให้ผ่าน "จุด" เรายังอาจบังคับให้เส้นทางผ่าน "เส้น" หรือ "ช่วงของเส้นทาง" ที่กำหนดไว้ล่วงหน้าได้ด้วย แก่นของความคิดยังเหมือนเดิม: เมื่อส่วนหนึ่งของเส้นทางถูกกำหนดตายตัวแล้ว สิ่งที่ต้องนับจริง ๆ คือจำนวนวิธีเติมส่วนก่อนหน้าและส่วนหลังจากนั้น

เริ่มจากกรณีง่ายที่สุดก่อน สมมติว่าต้องการบังคับให้เส้นทางจาก $(0, 0)$ ไป (m, n) ผ่านเส้นเชื่อมแนวนอนที่เชื่อมระหว่าง

$$(a, b) \quad \text{และ} \quad (a+1, b).$$

เพราะเส้นนี้ถูกกำหนดไว้ชัดเจน เส้นทางใดก็ตามที่ผ่านเส้นนี้ จะต้อง

1. เดินจาก $(0, 0)$ ไปถึง (a, b)
2. ใช้ก้าวขวหนึ่งก้าวจาก (a, b) ไป $(a+1, b)$
3. เดินต่อจาก $(a+1, b)$ ไปถึง (m, n)

ช่วงกลางมีเพียงหนึ่งวิธีเพราะถูกบังคับไว้แล้ว ดังนั้นสิ่งที่ต้องนับคือช่วงแรกและช่วงสุดท้าย

จึงได้ว่า จำนวนเส้นทางดังกล่าวเท่ากับ

$$\binom{a+b}{a} \binom{(m-a-1) + (n-b)}{m-a-1}.$$

ในการทำงานเดียวกัน ถ้าต้องผ่านเส้นเชื่อมแนวตั้งระหว่าง

$$(a, b) \quad \text{และ} \quad (a, b + 1),$$

จำนวนเส้นทางจะเท่ากับ

$$\binom{a+b}{a} \binom{(m-a) + (n-b-1)}{m-a}.$$

Example 13.2.5. จำนวนเส้นทางจาก $(0, 0)$ ไป $(11, 5)$ ที่ต้องผ่านเส้นเชื่อมระหว่าง $(2, 3)$ และ $(3, 3)$ มีค่าเท่ากับ

$$\binom{5}{2} \binom{11}{8} = \binom{5}{3} \binom{11}{3}.$$

หากสิ่งที่ถูกบังคับไม่ใช่เพียงเส้นเดียว แต่เป็น ช่วงของเส้นทาง ที่กำหนดตายตัว เช่นเส้นทางย่อยจากจุด P ไปยังจุด Q ซึ่งมีลำดับของก้าวถูกระบุไว้ครบแล้ว หลักคิดก็ยิ่งเหมือนเดิม: เราเพียงนับจำนวนวิธีเดินจากจุดเริ่มต้นไปถึง P แล้วคูณด้วยจำนวนวิธีเดินจาก Q ไปยังจุดหมาย เพราะช่วงจาก P ไป Q ไม่ต้องนับอีก มันถูกกำหนดไว้แล้วหนึ่งแบบ

อย่างไรก็ตาม ควรระวังความแตกต่างระหว่างสองโจทย์ต่อไปนี้ให้ดี:

1. "ต้องผ่านเส้นทางย่อยที่กำหนดไว้อย่างตายตัว"
2. "ต้องผ่านอย่างน้อยหนึ่งเส้นจากชุดของหลายเส้น"

กรณีแรกมักนับได้ด้วยการแบ่งเป็นช่วงแล้วคูณตรง ๆ แต่กรณีหลังอาจเกิดการนับซ้ำได้ เพราะเส้นทางหนึ่งเส้นอาจผ่านหลายเส้นในชุดเดียวกัน จึงต้องใช้การแยกกรณีอย่างระมัดระวังมากขึ้น หรืออาจต้องพึ่งเทคนิคอื่นในบทถัดไป เช่น inclusion--exclusion

13.2.4 การแปลปัญหาเส้นทางให้เป็นปัญหาการเลือกตำแหน่ง

หัวใจสำคัญที่สุดของทั้งหัวข้อนี้ไม่ใช่สูตรเฉพาะข้อใดข้อหนึ่ง แต่คือ วิธีแปลโจทย์ จากปัญหาเชิงเส้นทางไปเป็นปัญหาเชิงการเลือก เมื่อแปลได้ถูก การนับมักจะง่ายลงมาก

หลักคิดพื้นฐานมีดังนี้: เส้นทางโมนโทนิคหนึ่งเส้น สามารถแทนได้ด้วยลำดับของก้าว เช่นลำดับของ R และ U ดังนั้นการนับเส้นทางจึงเท่ากับการนับลำดับเหล่านี้ และการนับลำดับเช่นนี้ก็คือการเลือกตำแหน่งให้กับก้าวชนิดหนึ่งจากทั้งหมดนั่นเอง

พูดอีกแบบหนึ่ง โจทย์เส้นทางจาก $(0, 0)$ ไป (m, n) ไม่ต่างอะไรกับคำถามว่า

จะเลือกตำแหน่ง n ตำแหน่งจากทั้งหมด $m + n$ ตำแหน่ง เพื่อวางก้าวขึ้นได้กี่วิธี

หรือจะเลือกตำแหน่งของก้าวขวา m ตำแหน่งก็ได้เช่นกัน

เมื่อมีเงื่อนไขเพิ่มเติม เราก็มักแปลเงื่อนไขนั้นกลับมาเป็นเงื่อนไขเกี่ยวกับตำแหน่งของก้าว ตัวอย่างเช่น

- การบังคับให้ผ่านจุด (a, b) เท่ากับการบอกว่า หลังจากเดินไป $a + b$ ก้าวแรก ต้องมีก้าวขวายู่ a ครั้ง และก้าวขึ้นอยู่ b ครั้งพอดี
- การบังคับให้ผ่านเส้นเชื่อมหนึ่งเส้น เท่ากับการบอกว่าที่ตำแหน่งบางจุดในลำดับ ต้องมีรูปแบบของก้าวที่แน่นอนเกิดขึ้น
- การบังคับให้ผ่านช่วงเส้นทางที่กำหนด เท่ากับการตรึงช่วงหนึ่งของลำดับก้าวไว้ล่วงหน้า

นี่คือเหตุผลที่โจทย์เส้นทางบนตารางเป็นตัวอย่างที่ดีมากของ combinations มันสอนให้เราเห็นว่า การเลือกไม่ได้จำกัดอยู่เพียงการเลือกสมาชิกจากเซตแบบตรง ๆ แต่ยังอาจเป็นการเลือกตำแหน่ง เลือกเวลาเกิดเหตุการณ์ หรือเลือกว่าปัจจัยใดจะมีบทบาทแบบใดในการสร้างวัตถุหนึ่งขึ้น

หลักคิดที่ควรจำ

เมื่อพบโจทย์เส้นทางบนตาราง ควรถามตัวเองตามลำดับดังนี้:

1. เส้นทางหนึ่งเส้นต้องประกอบด้วยก้าวชนิดใดบ้าง และอย่างละกี่ก้าว
2. เส้นทางนั้นสามารถแทนด้วยลำดับของสัญลักษณ์อะไรได้
3. การนับลำดับดังกล่าวเทียบเท่ากับการเลือกตำแหน่งใดจากทั้งหมดกี่ตำแหน่ง
4. ถ้ามีเงื่อนไขเพิ่มเติม สามารถแบ่งเส้นทางออกเป็นช่วงย่อยได้หรือไม่

ถ้าแปลโจทย์ได้ถึงขั้นนี้ ปัญหาส่วนใหญ่จะกลายเป็นโจทย์ combinations โดยตรง

สรุปแล้ว เส้นทางบนตารางเป็นหัวข้อที่มีคุณค่ามาก เพราะมันช่วยยืนยันอีกครั้งว่า combinatorics ไม่ใช่เรื่องของ การจำสูตรเฉพาะโจทย์ แต่เป็นเรื่องของการเปลี่ยนมุมมองให้เห็นวัตถุหนึ่งชนิด ในฐานะผลของการเลือกอย่างเป็นระบบ และเมื่อมองได้เช่นนี้ โจทย์ที่ดูเป็นเรขาคณิตก็อาจกลายเป็นโจทย์เลือกตำแหน่งธรรมดา ๆ ได้ทันที

13.3 Distribution Problems: โจทย์การแจกของ

หลังจากศึกษา permutations, combinations, และสัมประสิทธิ์ทวินามแล้ว มีหัวข้อหนึ่งที่เราควรดูมองว่าเป็นการ ต่อยอดอย่างเป็นธรรมชาติมาก นั่นคือ โจทย์การแจกของ โจทย์ประเภทนี้มักถามในรูปว่า จะนำของบางชนิดไป แจกให้คนหลายคน หรือใส่ลงในกล่องหลายใบ หรือแบ่งออกเป็นหลายส่วนได้กี่วิธี แม้อ้อยคำในโจทย์จะเปลี่ยนไป ได้มาก แต่แกนกลางของปัญหามักอยู่ที่คำถามเดียวกันคือ

เรากำลังแจกอะไร

ผู้รับหรือกล่องแตกต่างกันหรือไม่

ของที่แจกแตกต่างกันหรือไม่

และเราสนใจเพียงจำนวนที่แต่ละคนได้รับ หรือสนใจด้วยว่าใครได้ชิ้นไหน

หัวข้อนี้สำคัญมาก เพราะมันเป็นจุดที่ผู้เรียนเริ่มเห็นชัดว่า ปัญหาการนับแบบเดียวกัน สามารถถูกมองได้หลายภาษา บางครั้งมองเป็นฟังก์ชัน บางครั้งมองเป็นเซตย่อย บางครั้งมองเป็นลำดับของตัวเลือก และบางครั้งมองเป็นคำตอบจำนวนเต็มของสมการ การเปลี่ยนมุมมองให้เหมาะกับโจทย์ จึงเป็นทักษะหลักของหัวข้อนี้

13.3.1 ทำไมโจทย์การแจกจึงเป็นหัวข้อถัดจาก combinations

ถ้ามองในเชิงโครงสร้าง โจทย์การแจกไม่ได้แยกขาดจาก combinations แต่เป็นการขยายโจทย์การเลือกให้กว้างขึ้น ใน combinations เรามักถามว่า จะเลือกสมาชิก r ตัวจากทั้งหมด n ตัวได้กี่วิธี ซึ่งแก่นของเรื่องคือการตอบว่า ใครถูกเลือกและใครไม่ถูกเลือก

แต่เมื่อเข้าสู่โจทย์การแจก เราไม่ได้หยุดอยู่เพียงการเลือก เราต้องตอบต่อด้วยว่า ของแต่ละชิ้นไปอยู่กับใคร หรือแต่ละคนได้ของกี่ชิ้น ดังนั้นโจทย์การแจกจึงอาจมองได้ว่าเป็น *การเลือกที่มีโครงสร้างมากขึ้น* เพราะเราไม่ได้เพียงถามว่าจะได้อยู่ในผลลัพธ์ แต่ถามด้วยว่าองค์ประกอบต่าง ๆ ถูกจัดวางสัมพันธ์กันอย่างไร

ยิ่งไปกว่านั้น หัวข้อนี้ยังเป็นสะพานเชื่อมสำคัญระหว่างหลายแนวคิดที่เรียนมาแล้ว ถ้าของแตกต่างกันและผู้รับแตกต่างกัน เรามักมองโจทย์ผ่านฟังก์ชันหรือหลักการคูณ ถ้าของเหมือนกันแต่ผู้รับแตกต่างกัน เรามักมองผ่านคำตอบจำนวนเต็มไม่ลบ และเมื่อโจทย์อยู่ในรูปของการเลือกซ้ำจากชนิดของของ เราจะพบว่ามันเชื่อมกลับไปที่ combinations และ binomial coefficients อีกครั้ง ดังนั้นหัวข้อนี้จึงไม่ใช่ส่วนเสริม แต่เป็นพื้นที่ที่ทำให้สูตรและมุมมองจากหัวข้อก่อนหน้านี้มารวมตัวกันอย่างชัดเจน

13.3.2 มุมมองของโจทย์การแจก: ฟังก์ชัน เซตย่อย ลำดับ และคำตอบจำนวนเต็ม

หนึ่งในเหตุผลที่โจทย์การแจกดูยากในตอนแรก คือมันมีหลาย representation พร้อมกัน ถ้าไม่แยกให้ออกว่า โจทย์ปัจจุบันควรถูกมองในภาษาใด เรามักจะไม่แน่ใจว่าควรเริ่มนับจากตรงไหน เพื่อให้ภาพชัดขึ้น ลองพิจารณาว่าโจทย์การแจกหนึ่งข้อ อาจถูกมองได้อย่างน้อยสี่แบบต่อไปนี้

1) มองเป็นฟังก์ชัน ถ้ามีของ n ชิ้นที่แตกต่างกัน และมีผู้รับ k คนที่แตกต่างกัน การแจกของแต่ละชิ้นให้ผู้รับหนึ่งคน เท่ากับการกำหนดฟังก์ชันจากเซตของของไปยังเซตของผู้รับ ในมุมมองนี้ คำถามว่า “ของแต่ละชิ้นไปอยู่กับใคร” กลายเป็นคำถามว่า “แต่ละสมาชิกในโดเมนถูกส่งไปยังสมาชิกใดในโคโดเมน”

2) มองเป็นเซตย่อย บางครั้งเราไม่ได้สนใจของทีละชิ้น แต่สนใจว่าใครบ้างได้รับของบางชนิด หรือเลือกตำแหน่งใดบ้างให้ทำหน้าที่พิเศษ ในกรณีนี้ ปัญหามักยุบลงเป็นการเลือกเซตย่อยขนาดที่กำหนด ซึ่งเชื่อมโดยตรงกับ combinations

3) มองเป็นลำดับของทางเลือก ถ้าเราหยิบของที่ละชิ้นแล้วตัดสินใจว่าจะให้ใคร โจทย์อาจถูกมองเป็นกระบวนการหลายชั้น ของชั้นแรกมีผู้รับให้เลือกกี่คน ของชั้นที่สองมีผู้รับให้เลือกกี่คน และต่อไปเรื่อย ๆ มุมมองนี้เชื่อมกับหลักการคูณอย่างชัดเจน

4) มองเป็นคำตอบจำนวนเต็ม ถ้าของที่แจกเหมือนกัน สิ่งที่สำคัญไม่ใช่ว่าชิ้นที่หนึ่งหรือชิ้นที่สองไปอยู่กับใคร เพราะของทุกชิ้นแยกกันไม่ได้ สิ่งที่เหลือให้สนใจคือ แต่ละคนได้ไปกี่ชิ้น ดังนั้นปัญหาจึงกลายเป็นการหาคำตอบจำนวนเต็มไม่ลบของสมการอย่าง

$$x_1 + x_2 + \cdots + x_k = n,$$

โดยที่ x_i แทนจำนวนของคนที่ i ได้รับ

ทั้งสี่มุมมองนี้ไม่ได้แข่งขันกัน แต่ช่วยกันอธิบายโจทย์คนละแบบ งานสำคัญของผู้แก้จึงไม่ใช่เพียงจำสูตร แต่ต้องรู้ว่าจะเปลี่ยน representation ไปใช้ภาษาใดจึงจะนับได้ง่ายที่สุด

13.3.3 แจกของที่แตกต่างกันให้คนที่แตกต่างกัน

เริ่มจากกรณีพื้นฐานที่สุดก่อน: มีของ n ชิ้นที่แตกต่างกัน และมีคน k คนที่แตกต่างกัน ถ้าต้องการแจกของทุกชิ้นให้คนเหล่านี้ โดยแต่ละชิ้นต้องมีผู้รับเพียงคนเดียว และยังไม่ใส่เงื่อนไขเพิ่มเติม จะมีวิธีแจกทั้งหมดกี่วิธี

วิธีคิดที่เป็นธรรมชาติที่สุดคือ พิจารณาของที่ละชิ้น ของชิ้นแรกเลือกผู้รับได้ k คน ของชิ้นที่สองก็เลือกผู้รับได้ k คนเช่นกัน และเป็นเช่นนี้ไปทุกชิ้น ดังนั้นจากหลักการคูณ จำนวนวิธีทั้งหมดคือ

$$k^n.$$

Theorem 13.3.1. จำนวนวิธีแจกของ n ชิ้นที่แตกต่างกันให้คน k คนที่แตกต่างกัน เมื่อแต่ละชิ้นต้องถูกแจกให้คนใดคนหนึ่ง เท่ากับ

$$k^n.$$

บทพิสูจน์. สำหรับของแต่ละชิ้น มีผู้รับที่เป็นไปได้ k คน และการเลือกผู้รับของแต่ละชิ้นเป็นอิสระจากกัน ดังนั้นจากหลักการคูณ จะมีวิธีแจกทั้งหมด

$$k \cdot k \cdots k = k^n$$

โดยมีตัวคูณทั้งหมด n ตัว □

ในมุมมองเชิงฟังก์ชัน ผลลัพธ์นี้พูดว่า จำนวนฟังก์ชันจากเซตที่มีสมาชิก n ตัว ไปยังเซตที่มีสมาชิก k ตัว มีค่า

เท่ากับ k^n มุมมองนี้มีประโยชน์มาก เพราะจะช่วยให้เราอ่านเงื่อนไขเพิ่มเติมในโจทย์ เป็นเงื่อนไขเกี่ยวกับฟังก์ชันได้อย่างเป็นระบบ

Example 13.3.2. มีหนังสือ 5 เล่มที่แตกต่างกัน และมีนักศึกษา 3 คนที่แตกต่างกัน ถ้าต้องการแจกหนังสือทุกเล่มให้นักศึกษาคนใดคนหนึ่ง จำนวนวิธีแจกเท่ากับ

$$3^5.$$

จุดที่ควรสังเกตคือ ผลลัพธ์นี้ไม่สนใจว่าบางคนอาจไม่ได้อะไรเลย หรือบางคนอาจได้หลายชิ้น ทุกความเป็นไปได้ถูกอนุญาตทั้งหมด ดังนั้นถ้าโจทย์มีเงื่อนไขเพิ่มว่าใครต้องได้หรือห้ามได้ เราต้องปรับการนับต่อไปอีก

13.3.4 แจกของที่แตกต่างกันโดยมีเงื่อนไขเรื่องคนต้องได้ของหรือห้ามได้ของ

เมื่อเริ่มมีเงื่อนไข โจทย์การแจกของที่ดูเหมือนง่าย จะเริ่มต้องการการวางแผนมากขึ้นทันที เงื่อนไขที่พบบ่อยที่สุดมีสองชนิดคือ

1. มีบางคน ห้าม ได้ของบางชิ้นหรือห้ามได้ของเลย
2. มีบางคน ต้อง ได้ของอย่างน้อยหนึ่งชิ้น

หัวใจของการแก้โจทย์อยู่ที่การอ่านเงื่อนไขให้ชัดว่า มันไปลดทางเลือกของของแต่ละชิ้น หรือกำลังบังคับโครงสร้างของการแจกทั้งชุด

กรณีห้ามได้ของ ถ้ากำหนดว่าคนหนึ่งคนห้ามได้รับของใดเลย เราก็เพียงตัดคนคนนั้นออกจากตัวเลือกของผู้รับ เช่น ถ้ามีของ n ชิ้นและคน k คน แต่คนที่ k ห้ามได้ของเลย จำนวนวิธีแจกจะเหลือ

$$(k-1)^n.$$

เพราะของแต่ละชิ้นเลือกผู้รับได้เพียง $k-1$ คน

ในทำนองเดียวกัน ถ้าของบางชิ้นมีผู้รับที่อนุญาตไม่เท่ากัน เราต้องมองทีละชิ้นและใช้หลักการคูณอย่างระมัดระวัง ไม่ใช่รีบเขียน k^n ตรง ๆ

กรณีต้องได้อย่างน้อยหนึ่งชิ้น ถ้ากำหนดว่าคนหนึ่งคนต้องได้ของอย่างน้อยหนึ่งชิ้น วิธีหนึ่งที่เป็นธรรมชาติคือใช้วิธีนับทั้งหมด แล้วลบกรณีที่คนคนนั้นไม่ได้อะไรเลย เช่น ถ้ามีคนพิเศษหนึ่งคนที่ต้องได้อย่างน้อยหนึ่งชิ้น จำนวนวิธีแจกจะเท่ากับ

$$k^n - (k-1)^n.$$

เพราะ

- k^n คือจำนวนวิธีแจกทั้งหมด
- $(k-1)^n$ คือจำนวนวิธีแจกที่คนพิเศษไม่ได้อะไรเลย

Example 13.3.3. มีของ 6 ชิ้นที่แตกต่างกัน และมีเด็ก 4 คนที่แตกต่างกัน ถ้าต้องการแจกของทั้งหมด โดยกำหนดว่าเด็กคนที่ 1 ต้องได้อย่างน้อยหนึ่งชิ้น จำนวนวิธีแจกคือ

$$4^6 - 3^6.$$

อย่างไรก็ตาม ถ้าโจทย์บังคับว่า *ทุกคน* ต้องได้อย่างน้อยหนึ่งชิ้น ปัญหาจะซับซ้อนขึ้นมาก เพราะกรณีที่ต้องลบออกมีการทับซ้อนกัน นั่นคืออาจมีหลายคนที่ไม่ได้อะไรพร้อมกัน ซึ่งต้องใช้เทคนิคเพิ่มเติมในการจัดการ ในบริบทของบทนี้ ผู้เขียนต้องการให้ผู้อ่านเห็นก่อนว่า เงื่อนไขเรื่อง “ต้องได้” และ “ห้ามได้” กำลังแปลเป็นการควบคุมฟังก์ชันหรือการตัดตัวเลือกอย่างไร ก่อนจะไปสู่เทคนิคที่ลึกกว่านั้นในโอกาสต่อไป

13.3.5 แจกของที่เหมือนกันให้คนที่แตกต่างกัน

ตอนนี้ลองเปลี่ยนชนิดของของบ้าง สมมติว่าเรามีลูกอม n เม็ดที่เหมือนกันหมด และมีเด็ก k คนที่แตกต่างกัน คำถามคือจะมีวิธีแจกได้กี่วิธี

จุดสำคัญคือ เมื่อลูกอมทุกเม็ดเหมือนกัน เราไม่สามารถติดตามได้ว่า “เม็ดไหนไปอยู่กับใคร” เพราะการสับลูกอมสองเม็ดไม่ทำให้ผลลัพธ์เปลี่ยนไปจริง สิ่งเดียวที่ยังมีความหมายคือ เด็กแต่ละคนได้รับไปกี่เม็ด ดังนั้นเราจึงเปลี่ยนปัญหาการแจกให้เป็นปัญหาใหม่ว่า จะหาจำนวนคำตอบจำนวนเต็มไม่ลบของสมการ

$$x_1 + x_2 + \cdots + x_k = n$$

ได้กี่คำตอบ โดยที่ x_i แทนจำนวนลูกอมที่เด็กคนที่ i ได้รับ

นี่คือจุดเปลี่ยนสำคัญของหัวข้อการแจกของ: เมื่อของเหมือนกัน **representation** แบบฟังก์ชันไม่เหมาะสมอีกต่อไป แต่ **representation** แบบคำตอบจำนวนเต็มจะเป็นภาษาที่เหมาะสมกว่า

Example 13.3.4. ถ้ามีลูกอม 7 เม็ดเหมือนกัน และมีเด็ก 3 คนที่แตกต่างกัน การแจกหนึ่งแบบอาจเขียนแทนได้ด้วย

$$(2, 4, 1),$$

ซึ่งหมายความว่าเด็กคนที่หนึ่งได้ 2 เม็ด คนที่สองได้ 4 เม็ด และคนที่สามได้ 1 เม็ด

ดังนั้นโจทย์การแจกของเหมือนกันให้คนที่แตกต่างกัน จึงกลายเป็นโจทย์มาตรฐานของ **combinatorics** คือการนับคำตอบจำนวนเต็มไม่ลบของสมการเชิงเส้น ซึ่งจะถูกแก้ได้อย่างสวยงามด้วยเทคนิค **Stars and Bars**

13.3.6 Stars and Bars และการนับคำตอบจำนวนเต็มไม่ลบ

เทคนิค **Stars and Bars** เป็นหนึ่งในเครื่องมือที่สำคัญที่สุดของโจทย์การแจกของ และควรถูกมองว่าเป็นรูปแบบหนึ่งของการแปลงปัญหา จาก “การแจกของเหมือนกัน” ไปเป็น “การเลือกตำแหน่ง” อีกครั้ง

สมมติว่าเราต้องการนับจำนวนคำตอบจำนวนเต็มไม่ลบของสมการ

$$x_1 + x_2 + \cdots + x_k = n.$$

เราจะตีความ n เป็นจำนวนดาว * ทั้งหมด n ดวง และใช้เส้นคั่น $k - 1$ เส้นแบ่ง | เพื่อแบ่งดาวเหล่านี้ออกเป็น k กลุ่ม กลุ่มที่หนึ่งมีจำนวนดาวเท่ากับ x_1 , กลุ่มที่สองมีจำนวนดาวเท่ากับ $x_2, \dots$, กลุ่มที่ k มีจำนวนดาวเท่ากับ x_k

ตัวอย่างเช่น

$$** \mid **** \mid *$$

แทนคำตอบ

$$(x_1, x_2, x_3) = (2, 4, 1).$$

ดังนั้นการหาจำนวนคำตอบของสมการ เท่ากับการหาจำนวนวิธีวางแท่ง $k - 1$ แท่ง ลงในลำดับที่มีสัญลักษณ์ทั้งหมด $n + k - 1$ ตำแหน่ง โดยเลือกกว่าตำแหน่งใดเป็นแท่ง ที่เหลือเป็นดาว

Theorem 13.3.5 (Stars and Bars). จำนวนคำตอบจำนวนเต็มไม่ลบของสมการ

$$x_1 + x_2 + \cdots + x_k = n$$

เท่ากับ

$$\binom{n+k-1}{k-1} = \binom{n+k-1}{n}.$$

บทพิสูจน์. พิจารณาดาว n ดวง และแท่ง $k - 1$ แท่ง เมื่อวางเรียงสัญลักษณ์ทั้งหมดเข้าด้วยกัน จะมีตำแหน่งรวมทั้งหมด $n + k - 1$ ตำแหน่ง

การเลือกกว่าตำแหน่งใดเป็นแท่ง จะกำหนดการแบ่งดาวออกเป็น k กลุ่มโดยอัตโนมัติ และจึงกำหนดค่าของ

$$x_1, x_2, \dots, x_k$$

ได้พอดี ในทางกลับกัน คำตอบจำนวนเต็มไม่ลบแต่ละชุด ก็สร้างลำดับของดาวและแท่งได้เพียงแบบเดียว

ดังนั้นจำนวนคำตอบทั้งหมด จึงเท่ากับจำนวนวิธีเลือกตำแหน่ง $k - 1$ ตำแหน่งจากทั้งหมด $n + k - 1$ ตำแหน่ง

ซึ่งมีค่าเท่ากับ

$$\binom{n+k-1}{k-1}.$$

□

นี่คือสูตรพื้นฐานที่สุดของการแจกของเหมือนกันให้คนที่แตกต่างกัน และเป็นตัวอย่างที่ดีมากของแนวคิดในบท
นี้ว่า หลายโจทย์ที่ดูไม่เกี่ยวกับ combinations แท้จริงแล้วเป็น combinations เมื่อเปลี่ยน representation ให้
เหมาะสม

13.3.7 กรณีที่ต้องมีอย่างน้อยหนึ่งชิ้น: การลดรูปด้วยการหัก quota ขั้นต่ำ

ในหลายโจทย์ เราไม่ได้อนุญาตให้บางคนได้ศูนย์ชิ้น แต่ต้องการให้ทุกคนหรือบางคนได้อย่างน้อยหนึ่งชิ้น ในกรณี
เช่นนี้ เทคนิคที่เป็นธรรมชาติที่สุดคือ หัก quota ขั้นต่ำออกก่อน แล้วค่อยนับปริมาณที่เหลือด้วย Stars and Bars

สมมติว่าเราต้องการหาจำนวนคำตอบจำนวนเต็มบวกของสมการ

$$x_1 + x_2 + \cdots + x_k = n$$

โดยที่

$$x_i \geq 1 \quad \text{สำหรับทุก } i.$$

ให้กำหนดตัวแปรใหม่โดย

$$y_i = x_i - 1.$$

แล้วจะได้ว่า $y_i \geq 0$ และ

$$y_1 + y_2 + \cdots + y_k = n - k.$$

ดังนั้นจำนวนคำตอบของสมการเดิม จึงเท่ากับจำนวนคำตอบจำนวนเต็มไม่ลบของสมการใหม่ ซึ่งมีค่าเป็น

$$\binom{(n-k) + k - 1}{k-1} = \binom{n-1}{k-1}.$$

Theorem 13.3.6. จำนวนคำตอบจำนวนเต็มบวกของสมการ

$$x_1 + x_2 + \cdots + x_k = n$$

เท่ากับ

$$\binom{n-1}{k-1}.$$

วิธีคิดนี้ไม่ได้จำกัดอยู่ที่กรณีขั้นต่ำเท่ากับ 1 เท่านั้น ถ้าโจทย์กำหนดว่า

$$x_i \geq a_i \quad \text{สำหรับทุก } i,$$

เราก็สามารถกำหนด

$$y_i = x_i - a_i$$

แล้วจะได้ว่า

$$y_1 + y_2 + \cdots + y_k = n - (a_1 + a_2 + \cdots + a_k),$$

ซึ่งกลับไปเป็นปัญหาแบบจำนวนเต็มไม่ลบเช่นเดิม

Example 13.3.7. จำนวนวิธีแจกขนม 12 ชิ้นที่เหมือนกัน ให้เด็ก 4 คนที่แตกต่างกัน โดยทุกคนต้องได้ออย่างน้อยหนึ่งชิ้น เท่ากับจำนวนคำตอบของ

$$x_1 + x_2 + x_3 + x_4 = 12 \quad \text{โดยที่ } x_i \geq 1.$$

เมื่อให้ $y_i = x_i - 1$ จะได้

$$y_1 + y_2 + y_3 + y_4 = 8 \quad \text{โดยที่ } y_i \geq 0.$$

ดังนั้นจำนวนวิธีแจกคือ

$$\binom{8+4-1}{4-1} = \binom{11}{3}.$$

จุดที่ควรเห็นคือ การหัก quota ขั้นต่ำไม่ใช่ลูกเล่นเชิงพีชคณิต แต่เป็นการ “แจกของขั้นต่ำที่ถูกบังคับไว้ก่อน” แล้วค่อยนับวิธีแจกของส่วนที่เหลือ ซึ่งเป็น intuition ที่ตรงกับสัญชาตญาณมาก

13.3.8 กรณีมีขอบเขตบน: การแยกกรณีและการตีความใหม่

กรณีที่มีขอบเขตบน เช่น

$$x_i \leq M$$

มักยากกว่ากรณีมีเพียงขอบเขตล่าง เพราะ Stars and Bars แบบตรง ๆ ใช้ไม่ได้ทันที อย่างไรก็ตาม หลักคิดสำคัญยังเหมือนเดิม: เราต้องพยายามเปลี่ยนโจทย์ให้กลับไปเป็นคำถามที่นับได้ง่ายกว่า โดยทั่วไปมักใช้หนึ่งในสองวิธีต่อไปนี้

1) การแยกกรณี ถ้าขอบเขตบนมีค่าน้อยหรือโครงสร้างของโจทย์ไม่ซับซ้อน เราสามารถแยกกรณีตามค่าที่เป็นไปได้ของตัวแปรบางตัว แล้วนับแต่ละกรณีแยกกัน ตัวอย่างเช่น ถ้าต้องการนับจำนวนคำตอบของ

$$x_1 + x_2 + x_3 = 5 \quad \text{โดยที่ } 0 \leq x_i \leq 2,$$

เราอาจแยกกรณีตามค่า x_1 แล้วนับสมการของตัวแปรที่เหลือทีละกรณี

2) การตีความใหม่ด้วยตัวแปรเสริม ถ้าขอบเขตบนเป็นแบบสม่ำเสมอ เช่น

$$0 \leq x_i \leq M$$

สำหรับทุก i , และผลรวม n อยู่ใกล้กับค่าสูงสุด kM , บางครั้งจะง่ายกว่าถ้ากำหนดตัวแปรใหม่

$$y_i = M - x_i.$$

แล้วจะได้ว่า $y_i \geq 0$ และ

$$y_1 + y_2 + \cdots + y_k = kM - n.$$

จึงแปลงโจทย์เดิมให้กลับเป็นโจทย์แบบจำนวนเต็มไม่ลบอีกครั้ง

Example 13.3.8. จงหาจำนวนคำตอบของ

$$x_1 + x_2 + x_3 = 10 \quad \text{โดยที่ } 0 \leq x_i \leq 4.$$

เนื่องจากค่าสูงสุดรวมคือ 12 ให้กำหนด

$$y_i = 4 - x_i.$$

แล้วจะได้

$$y_1 + y_2 + y_3 = 2 \quad \text{โดยที่ } y_i \geq 0.$$

ดังนั้นจำนวนคำตอบเท่ากับ

$$\binom{2+3-1}{3-1} = \binom{4}{2} = 6.$$

สิ่งที่ควรสังเกตคือ กรณีที่มีขอบเขตบนไม่ได้มีสูตรเดียวที่ใช้ครอบคลุมทุกสถานการณ์ได้ง่ายเหมือน Stars and Bars แต่สิ่งที่ยังคงเหมือนเดิมคือ เราต้องมองหาวิธีเปลี่ยน representation ของโจทย์ ให้กลับไปอยู่ในรูปที่เราอ่านได้จาก combinations หรือจากการแยกกรณีอย่างมีระบบ

13.3.9 แจกของที่เหมือนกันลงกลุ่มที่ไม่ distinguish กัน

จนถึงตอนนี้ เราสมมติว่าผู้รับหรือกล่อง *แตกต่างกัน* ดังนั้นการแจกแบบ

$$(3, 1, 2)$$

กับ

$$(1, 3, 2)$$

ถือว่าต่างกัน เพราะบอกว่าคนละคนได้จำนวนไม่เท่ากัน

แต่ถ้ากล่องหรือกลุ่ม *ไม่ distinguish กัน* สถานการณ์จะเปลี่ยนไปที่ ตัวอย่างเช่น ถ้ามีก้อนหิน 5 ก้อนที่เหมือนกัน และมีถุง 3 ใบที่เหมือนกันหมด การแจกแบบ $(3, 1, 1)$ กับ $(1, 3, 1)$ ไม่ควรถูกนับแยก เพราะจริง ๆ แล้วเป็นรูปแบบเดียวกัน ต่างกันเพียงการสลับชื่อถุงซึ่งเราไม่ได้ถือว่ามีความหมาย

ดังนั้นเมื่อกล่องไม่ distinguish กัน โจทย์การแจกของเหมือนกันจะไม่เท่ากับการนับคำตอบของสมการเชิงลำดับอีกต่อไป แต่จะกลายเป็นปัญหาของการหา **พาร์ทิชันของจำนวนเต็ม** แทน เช่น การแจกของ n ชิ้นที่เหมือนกันลงในกล่องที่ไม่ distinguish กัน k ใบ สมมูลกับการเขียน n เป็นผลบวกของจำนวนเต็มไม่ลบไม่เกิน k พจน์ โดยไม่สนใจลำดับของพจน์เหล่านั้น

Example 13.3.9. การแจกของเหมือนกัน 5 ชิ้นลงในกล่องเหมือนกัน 3 ใบ สอดคล้องกับรูปแบบต่อไปนี้:

$$5 + 0 + 0, \quad 4 + 1 + 0, \quad 3 + 2 + 0, \quad 3 + 1 + 1, \quad 2 + 2 + 1.$$

ดังนั้นมีทั้งหมด 5 วิธี

หัวข้อนี้ลึกกว่ากรณีที่กล่องแตกต่างกันมาก และโดยทั่วไปไม่มีสูตรง่าย ๆ แบบเดียวกับ Stars and Bars จึงควรมองไว้ก่อนว่าเป็นการเปิดประตูไปสู่โลกของ integer partitions มากกว่าจะพยายามรีบบหา "สูตรสากล" ตั้งแต่นั้น

13.3.10 Multisets, weak compositions, และมุมมองที่เชื่อมโยงกัน

หนึ่งในประเด็นที่สวยงามที่สุดของหัวข้อนี้คือ วัตถุหลายชนิดที่ดูต่างกันในตอนแรก แท้จริงแล้วเป็นคนละชื่อของโครงสร้างเดียวกัน

Weak compositions คำตอบจำนวนเต็มไม่ลบของสมการ

$$x_1 + x_2 + \cdots + x_k = n$$

มักเรียกว่า **weak composition** ของ n เป็น k ส่วน คำว่า “weak” เน้นว่าเรายอมให้บางส่วนเป็นศูนย์ได้

Multisets อีกมุมมองหนึ่ง การเลือกของ n ชิ้นจากชนิดของของ k ชนิด โดยอนุญาตให้เลือกซ้ำได้ ก็คือการสร้าง **multiset** ขนาด n จากชนิด k ชนิด ถ้าชนิดที่ i ถูกเลือก x_i ครั้ง เราจะได้สมการเดียวกันคือ

$$x_1 + x_2 + \cdots + x_k = n.$$

ดังนั้นข้อความต่อไปนี้จึงเป็นปัญหาเดียวกัน:

- แจกของเหมือนกัน n ชิ้นให้คนแตกต่างกัน k คน
- หาคำตอบจำนวนเต็มไม่ลบของ $x_1 + \cdots + x_k = n$
- สร้าง weak compositions ของ n เป็น k ส่วน
- เลือกแบบซ้ำได้ n ครั้งจากตัวเลือก k ชนิด

ทั้งหมดนี้ถูกนับด้วยจำนวนเดียวกันคือ

$$\binom{n+k-1}{k-1}.$$

Theorem 13.3.10. จำนวน multisets ขนาด n ที่สร้างจากชนิดของวัตถุ k ชนิด เท่ากับ

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}.$$

บทพิสูจน์. ถ้าชนิดที่ i ถูกเลือก x_i ครั้ง เราจะได้

$$x_1 + x_2 + \cdots + x_k = n \quad \text{โดยที่ } x_i \geq 0.$$

ดังนั้นจำนวน multisets ดังกล่าว เท่ากับจำนวนคำตอบจำนวนเต็มไม่ลบของสมการนี้ ซึ่งจาก Stars and Bars มีค่าเท่ากับ

$$\binom{n+k-1}{k-1}.$$

□

มุมมองนี้สำคัญมาก เพราะมันช่วยให้ผู้อ่านเลิกมองแต่ละโจทย์ว่าเป็น “สูตรคนละบท” แล้วเริ่มมองว่า หลายโจทย์ แท้จริงแล้วกำลังถามคำถามเดียวกันในคนละภาษา

13.3.11 โจทย์ประยุกต์ของการแจกของ

ในทางปฏิบัติ โจทย์การแจกของปรากฏได้หลายรูปแบบมาก และมักไม่บอกตัวเองอย่างตรงไปตรงมาว่าเป็นปัญหาการแจก ตัวอย่างเช่น

- จำนวนวิธีแจกหนังสือที่แตกต่างกันให้กับนักเรียนหลายคน
- จำนวนวิธีแจกขนมที่เหมือนกันให้เด็กหลายคน
- จำนวนวิธีเลือกสินค้า n ชิ้นจากสินค้าหลายประเภทโดยเลือกซ้ำได้
- จำนวนวิธีหาคำตอบจำนวนเต็มของสมการเชิงเส้นภายใต้เงื่อนไขไม่ลบ
- จำนวนวิธีแบ่งจำนวนเต็มหนึ่งจำนวนออกเป็นหลายส่วน

แม้บริบทจะต่างกัน แต่เมื่อแปลให้ถูก representation โจทย์เหล่านี้มักย้อนกลับไปหาเครื่องมือชุดเดิมที่เราเพิ่งพัฒนาไป

ยกตัวอย่างเช่น ถ้าต้องการเลือกไอศกรีม 6 ลูกจากรสชาติ 4 รส โดยแต่ละรสเลือกซ้ำได้ โจทย์นี้ไม่ได้เป็นเรื่อง “แจกไอศกรีมให้คน” ในถ้อยคำตรง ๆ แต่ในเชิงโครงสร้าง มันคือการเลือก multiset ขนาด 6 จากชนิด 4 ชนิด หรือเทียบเท่ากับการหาคำตอบจำนวนเต็มไม่ลบของ

$$x_1 + x_2 + x_3 + x_4 = 6.$$

ดังนั้นจำนวนวิธีจึงเป็น

$$\binom{6+4-1}{4-1} = \binom{9}{3}.$$

อีกตัวอย่างหนึ่ง ถ้าต้องการแจกของที่แตกต่างกัน 8 ชิ้นให้คน 3 คนที่แตกต่างกัน โดยคนที่หนึ่งห้ามได้อะไรเลย โจทย์นี้ลดลงทันทีเป็น

$$2^8$$

เพราะของแต่ละชิ้นเลือกผู้รับได้เพียงสองคนที่เหลือ

ในขณะที่ถ้าถามว่า จะแจกลูกบอลเหมือนกัน 10 ลูกให้กล่องแตกต่างกัน 4 ใบ โดยทุกกล่องต้องมียาอย่างน้อยหนึ่งลูก โจทย์นี้แปลเป็น

$$x_1 + x_2 + x_3 + x_4 = 10 \quad \text{โดยที่ } x_i \geq 1,$$

และเมื่อหักขั้นต่ำออกก่อน จะได้คำตอบเป็น

$$\binom{9}{3}.$$

ตัวอย่างเหล่านี้ชี้ให้เห็นชัดว่า สิ่งสำคัญในหัวข้อนี้ไม่ใช่การจำรายการสูตรให้ได้มากที่สุด แต่คือการอ่านให้ออกว่า โจทย์ควรถูกแปลไปเป็นวัตถุชนิดใด ถ้าแปลถูก representation การนับมักตามมาได้ค่อนข้างตรงไปตรงมา

Checklist ก่อนนับโจทย์การแจกของ

ก่อนลงมือแก้โจทย์การแจกของ ควรถามตัวเองอย่างน้อยสี่ข้อดังนี้:

1. ของที่แจกแตกต่างกันหรือเหมือนกัน
2. ผู้รับหรือกล่องแตกต่างกันหรือไม่
3. สนใจว่าใครได้ชิ้นไหน หรือสนใจเพียงว่าแต่ละคนได้กี่ชิ้น
4. มีเงื่อนไขขั้นต่ำหรือขอบเขตบนที่ต้องแปลงก่อนหรือไม่

ถ้าตอบสี่ข้อนี้ได้ชัด representation ของโจทย์มักจะชัดเจนตามไปด้วย

สรุปแล้ว หัวข้อการแจกของเป็นอีกตัวอย่างสำคัญของแนวคิดหลักใน combinatorics: ปัญหาที่ดูต่างกันมากในภาษามนุษย์ อาจเป็นปัญหาเดียวกันในภาษาคณิตศาสตร์ และงานของเราคือการมองให้เห็นว่า ควรแปลโจทย์นั้นเป็นฟังก์ชัน เป็นเซตย่อย เป็นลำดับ หรือเป็นคำตอบจำนวนเต็มแบบใด เมื่อมองได้ถูก เครื่องมืออย่างหลักการคูณ combinations และ Stars and Bars ก็จะเริ่มทำงานได้อย่างเป็นธรรมชาติ

13.4 สรุปภาพรวมของบท

เมื่อมองย้อนกลับไปที่บท จะเห็นว่าหัวข้อที่ผ่านมามีความเหมือนหลากรวมกันพอสมควร ตั้งแต่หลักการบวก หลักการคูณ ไปจนถึง permutations, combinations, การพิสูจน์เชิงการนับ สัมประสิทธิ์ทวินาม เส้นทางบนตาราง และโจทย์การแจกของ แต่แท้จริงแล้วทั้งหมดนี้ไม่ได้เป็นรายการสูตรที่แยกขาดจากกัน หากเป็นการพัฒนาวิธีคิดแบบเดียวกันที่ละชั้น

วิธีคิดนั้นคือ การวางแผนการนับอย่างเป็นระบบ เราต้องระบุให้ชัดก่อนว่า ผลลัพธ์หนึ่งแบบของโจทย์คืออะไร อะไรคือหน่วยที่เรากำลังนับ ลำดับมีความสำคัญหรือไม่ ควรแยกเป็นกรณีหรือควรแบ่งเป็นขั้นตอน มีการนับซ้ำหรือไม่ และมีวิธีเปลี่ยน representation ของปัญหา ให้กลายเป็นโจทย์ที่คุ้นกว่าได้หรือไม่

เพราะฉะนั้น เป้าหมายของบทนี้ไม่ใช่การฝึกให้ผู้อ่าน มองโจทย์แล้วรีบจับคู่กับสูตรสำเร็จรูปว่า ``เจอหน้าตาแบบนี้ต้องใช้สูตรนี้" แม้ว่าสูตรต่าง ๆ จะมีประโยชน์และควรรู้ แต่สูตรไม่ใช่แก่นของเรื่อง แก่นของเรื่องคือความสามารถในการจัดระเบียบความเป็นไปได้ ให้เป็นกรณีที่ไม่ทับกัน หรือเป็นลำดับขั้นที่ชัดเจน และควบคุมการนับนั้นไม่ให้ตกหล่นหรือซ้ำกัน

กล่าวอีกแบบหนึ่ง สูตรใน combinatorics ควรถูกมองว่าเป็น บันทึกย่อของกระบวนการนับที่ถูกต้อง ไม่ใช่สิ่งที่ต้องจำแบบตัดขาดจากเหตุผล ถ้าผู้อ่านลืมนสูตรบางตัว แต่ยังอ่านโครงสร้างของโจทย์ออก ยังเห็นว่าจะต้องบวกตรงไหน คูณตรงไหน ต้องเลือกตรงไหน เรียงตรงไหน หรือต้องแปลงปัญหาให้เป็นคำตอบจำนวนเต็มอย่างไร ผู้อ่านก็ยังสามารถสร้างสูตรนั้นขึ้นมาใหม่ได้ด้วยตนเอง ซึ่งในความจริงแล้วเป็นทักษะที่สำคัญกว่าการจำสูตรเสียอีก

13.4.1 แผนที่ความคิด: บวก คูณ เรียง เลือก ตัดซ้ำ รับประกันการมีอยู่

ถ้าจะวาดแผนที่ความคิดของบทนี้อย่างย่อ ผู้เขียนอยากให้มองเห็นเส้นทางต่อไปนี้

จุดเริ่มต้นของทุกอย่างคือ **หลักการบวก** และ **หลักการคูณ** ซึ่งเป็นภาษาพื้นฐานที่สุดของการนับ ถ้าโจทย์แยกเป็นหลายกรณีที่ไม่ทับกัน เรามักกำลังอยู่ในโลกของการบวก แต่ถ้าโจทย์เป็นกระบวนการหลายขั้นที่ต้องทำต่อเนื่องกัน เรามักกำลังอยู่ในโลกของการคูณ สองหลักการนี้ไม่ใช่เพียงหัวข้อแรกของบท แต่เป็นรากฐานของทุกสูตรที่ตามมาทั้งหมด

เมื่อผลลัพธ์หนึ่งแบบเป็น **ลำดับ** เราก้าวไปสู่ **permutations** ซึ่งเป็นการนับที่ลำดับมีความสำคัญ เมื่อผลลัพธ์หนึ่งแบบเป็นเพียง **กลุ่มของสิ่งที่ถูกเลือก** เราเข้าสู่ **combinations** ซึ่งเป็นการนับที่ลำดับไม่สำคัญ ดังนั้นความแตกต่างระหว่างเรียงกับเลือก จึงไม่ใช่เรื่องของการจำชื่อสูตร แต่เป็นเรื่องของการระบุให้ถูกว่า ผลลัพธ์หนึ่งแบบของโจทย์ควรถูกมองเป็นวัตถุชนิดใด

จากนั้น เราได้เห็นว่า จำนวนเดียวกันอาจถูกอธิบายได้หลายวิธี สิ่งนี้นำไปสู่ **combinatorial proofs** และ **double counting** ซึ่งสอนให้เรามองสมการไม่ใช่เป็นเพียงนิพจน์ที่ต้องจัดรูป แต่เป็นข้อความที่บอกว่า “สองข้างนี้กำลังนับสิ่งเดียวกันอยู่คนละทาง” มุมมองนี้สำคัญมาก เพราะทำให้ **combinatorics** เชื่อมกับการพิสูจน์อย่างชัดเจน

ถัดมา **binomial coefficients** และ **binomial theorem** แสดงให้เห็นว่าจำนวนวิธีเลือก สามารถปรากฏใหม่อีกครั้งในฐานะสัมประสิทธิ์ของพหุนาม ดังนั้น $\binom{n}{r}$ จึงไม่ได้เป็นเพียงจำนวนวิธีเลือกสมาชิก r ตัว แต่ยังเป็นจำนวนวิธีเลือกที่จะหยิบพจน์ชนิดใดจากแต่ละปัจจัยในการกระจายพหุนามด้วย ตรงนี้เป็นจุดที่ **combinatorics** กับ **algebra** มาบรรจบกันอย่างสวยงาม

หัวข้อ **เส้นทางบนตาราง** และ **โจทย์การแจกของ** ช่วยขยายภาพออกไปอีกว่า ปัญหาที่ดูไม่เหมือน combination เลยในตอนแรก แท้จริงแล้วอาจเป็นโจทย์การเลือกในอีก **representation** หนึ่ง เส้นทางบนตารางกลายเป็นการเลือกตำแหน่งของก้าว การแจกของเหมือนกันกลายเป็นการหาคำตอบจำนวนเต็มไม่ลบ และการเลือกซ้ำจากของหลายชนิดก็กลายเป็นปัญหาแบบ **multiset** ดังนั้นความก้าวหน้าที่นี่แท้จริงของบทนี้ จึงไม่ใช่การมีสูตรเพิ่มขึ้นเรื่อย ๆ แต่คือการมองเห็นว่าโจทย์ต่างชนิดกัน สามารถถูกยุบกลับมาเป็นโครงสร้างพื้นฐานเดียวกันได้

หากจะสรุปให้สั้นที่สุด ผู้เขียนอยากให้จำว่า

combinatorics ไม่ใช่วิชาของการลิสต์สูตร แต่เป็นวิชาของการมองโครงสร้างของความเป็นไปได้
แล้วนับมันอย่างมีระเบียบ

13.4.2 เทคนิคที่ควรเลือกใช้ในสถานการณ์ต่าง ๆ

แทนที่จะพยายามจำว่า “เจอโจทย์แบบนี้ต้องใช้สูตรอะไร” ผู้เขียนอยากเสนอให้ใช้ชุดคำถามต่อไปนี้เป็นเครื่องมือตัดสินใจ ก่อนลงมือคำนวณทุกครั้ง

1) ผลลัพธ์หนึ่งแบบของโจทย์คืออะไร นี่เป็นคำถามที่สำคัญที่สุด เพราะถ้ายังไม่รู้ว่าเรากำลังนับวัตถุชนิดใด การใช้สูตรใด ๆ ก็เสี่ยงจะผิดตั้งแต่ต้น ผลลัพธ์หนึ่งแบบอาจเป็น ลำดับ เซตย่อย การแบ่งกลุ่ม ฟังก์ชันจากของไปยังผู้รับ หรือคำตอบจำนวนเต็มของสมการ การระบุจุดนี้ให้ชัด มักทำให้แนวทางของโจทย์ชัดเจนขึ้นทันที

2) ลำดับมีความสำคัญหรือไม่ ถ้าการสลับตำแหน่งทำให้ผลลัพธ์เปลี่ยน เรามักอยู่ในโลกของ permutations แต่ถ้าการสลับลำดับไม่เปลี่ยนผลลัพธ์ เรามักอยู่ในโลกของ combinations คำถามนี้ดูเรียบง่าย แต่เป็นเส้นแบ่งที่สำคัญที่สุดเส้นหนึ่งของทั้งบท

3) ควรแยกกรณีหรือควรแบ่งเป็นขั้นตอน ถ้าโจทย์มีธรรมชาติเป็น ``อย่างใดอย่างหนึ่ง" และกรณีไม่ทับกัน มักเหมาะกับหลักการบวก แต่ถ้าโจทย์มีธรรมชาติเป็น ``ทำสิ่งนี้แล้วทำสิ่งนั้นต่อ" มักเหมาะกับหลักการคูณ หลายโจทย์ใช้ทั้งสองอย่างร่วมกัน ดังนั้นทักษะสำคัญคือการมองให้ออกว่า ส่วนใดควรบวกและส่วนใดควรคูณ

4) มีการนับซ้ำหรือมีกรณีตกหล่นหรือไม่ นี่เป็นคำถามตรงงานที่สำคัญมาก โจทย์ combinatorics จำนวนมากผิด ไม่ใช่เพราะจัดสูตรไม่เป็น แต่เพราะนับผลลัพธ์หนึ่งแบบซ้ำหลายครั้ง หรือแบ่งกรณีไม่ครบ ดังนั้นหลังได้คำตอบแล้ว ควรถอยกลับมาตรวจสอบว่า ทุกแบบถูกนับหนึ่งครั้งพอดีจริงหรือไม่

5) เปลี่ยน representation ของปัญหาได้หรือไม่ หลายโจทย์จะง่ายขึ้นมากเมื่อเปลี่ยนภาษา เช่น

- เปลี่ยนการเดินบนตารางให้เป็นการเลือกตำแหน่งของก้าว
- เปลี่ยนการแจกของเหมือนกันให้เป็นคำตอบจำนวนเต็ม
- เปลี่ยนการเลือกซ้ำให้เป็น multiset
- เปลี่ยนสมการของสัมประสิทธิ์ทวินามให้เป็นการนับสิ่งเดียวกันสองวิธี

ความสามารถในการเปลี่ยน representation คือหนึ่งในทักษะที่มีค่าที่สุดของบทนี้ และเป็นจุดที่ทำให้ผู้เรียนเริ่ม ``คิดเป็น" แทนที่จะเพียง ``แทนสูตรเป็น"

หากตอบคำถามทั้งห้าข้อนี้ได้ เรามักจะเลือกเทคนิคได้ถูกโดยไม่ต้องอาศัยการเดารูปโจทย์ และนี่คือจุดที่ผู้เขียนอยากเน้นที่สุด: ทักษะสำคัญของบทนี้ไม่ใช่การจำสูตรจำนวนมาก แต่คือการสร้างแผนการนับจากหลักการพื้นฐานให้ได้

ตัวอย่างเช่น ถ้าลิมสูตร

$$P(n, r) = \frac{n!}{(n-r)!},$$

แต่ยังเข้าใจว่าการเรียงสับเปลี่ยนคือการเติมของลงในตำแหน่งที่ละตำแหน่ง เราก็สามารถสร้างสูตรนี้ขึ้นมาใหม่ได้จาก

$$n(n-1) \cdots (n-r+1).$$

ถ้าลิมสูตร

$$\binom{n}{r} = \frac{n!}{r!(n-r)!},$$

แต่ยังเข้าใจว่าการจัดกลุ่มคือการเลือกโดยไม่สนลำดับ เราก็ก่อนจาก permutations แล้วหารด้วย $r!$ เพื่อชดเชยการนับซ้ำได้

ถ้าลิมสูตร Stars and Bars แต่ยังเข้าใจว่าการแจกของเหมือนกันให้คนแตกต่างกัน คือการหาคำตอบจำนวนเต็มไม่ลบของ

$$x_1 + \cdots + x_k = n,$$

เราก็สามารถสร้างภาพดาวและแท่งขึ้นมาใหม่ แล้วนับเป็นการเลือกตำแหน่งของแท่งตัวเอง

และถ้าลิมว่าทำไมสัมประสิทธิ์ของ $x^{n-r}y^r$ ใน $(x+y)^n$ จึงเป็น $\binom{n}{r}$ เราก็ก่อนสร้างคำตอบขึ้นมาใหม่ได้ โดยคิดว่าเรากำลังเลือกจาก n ปัจจัยว่า จะหยิบ y มาจากปัจจัยใดบ้างจำนวน r ปัจจัย

ตัวอย่างเหล่านี้สะท้อนสิ่งเดียวกันทั้งหมด: สูตรที่ดีเป็นผลลัพธ์ของการคิดอย่างเป็นระบบ ดังนั้นถ้าระบบการคิดยังอยู่ สูตรก็สามารถถูกสร้างขึ้นใหม่ได้เสมอ

สิ่งที่ผู้เขียนอยากให้จำจากบทนี้ที่สุด

ถ้าผู้อ่านอ่านจบบทนี้แล้วจำสูตรได้ไม่ครบ นั่นยังไม่ใช่วิธีแก้ปัญหาใหญ่ แต่ถ้าผู้อ่านเริ่มมองโจทย์การนับได้ว่า ต้องแยกกรณีตรงไหน ต้องแบ่งขั้นตอนตรงไหน อะไรคือสิ่งที่กำลังนับ อะไรคือต้นเหตุของการนับซ้ำ และควรเปลี่ยน representation ของโจทย์อย่างไร นั่นถือว่าผู้อ่านได้เป้าหมายสำคัญของบทนี้แล้ว

ดังนั้น บทนี้ควรถูกอ่านในฐานะบทฝึก วินัยของการนับ ไม่ใช่บทสะสมสูตร เพราะในท้ายที่สุด คนที่แก้โจทย์ combinatorics ได้ดี ไม่ใช่คนที่จำสูตรได้มากที่สุดเสมอไป แต่คือคนที่สามารถจัดระเบียบความเป็นไปได้ และให้เหตุผลกับกระบวนการนับของตนเองได้อย่างชัดเจน

เมื่อเดินทางมาถึงตอนท้ายของสองบทนี้ ผู้เขียนอยากให้ผู้อ่านย้อนกลับไปมองภาพใหญ่ของ combinatorics อีกครั้ง เราเริ่มจากกฎพื้นฐานอย่างการบวกและการคูณ ขยับไปสู่การเรียงและการเลือก จากนั้นจึงต่อยอดไปสู่การพิสูจน์เชิงการนับ สัมประสิทธิ์ทวินาม เส้นทางบนตาราง และโจทย์การแจกของ แม้หัวข้อเหล่านี้จะดูแตกต่างกันมากในตอนแรก แต่สิ่งที่เชื่อมทุกส่วนเข้าด้วยกันคือวิธีคิดแบบเดียวกัน: การจัดระเบียบความเป็นไปได้ให้ชัด แล้วนับมันอย่างครบถ้วนและไม่ซ้ำ

ด้วยเหตุนี้ สิ่งที่สำคัญที่สุดที่บท combinatorics ทั้งสองบทพยายามฝึก จึงไม่ใช่การทำให้ผู้อ่านจำสูตรได้มากที่สุด แต่คือการทำให้ผู้อ่านมี ระบบการนับ ที่แม่นยำ จนสามารถสร้างสูตรขึ้นมาใหม่ได้เองเมื่อจำเป็น ถ้าลิมสูตร permutations แต่ยังมองเห็นการเติมของลงในตำแหน่งที่ละตำแหน่ง เราก็ก่อนสร้างมันขึ้นมาใหม่ได้ ถ้าลิมสูตร combinations แต่ยังเข้าใจว่ามันคือการเลือกโดยไม่สนลำดับ เราก็ก่อนกลับไปเริ่มจาก permutations แล้วหารการนับซ้ำได้ ถ้าลิม Stars and Bars แต่ยังเข้าใจว่าโจทย์กำลังถามถึงคำตอบจำนวนเต็มไม่ลบของสมการ เราก็ก่อน

สร้าง representation ของดาวและแท่งขึ้นมาใหม่ได้อีกครั้ง

นี่คือสาระที่ผู้เขียนอยากเน้นมากที่สุด:

combinatorics ไม่ใช่วิชาของการจับคู่โจทย์กับสูตร แต่เป็นวิชาของการวางแผนการนับอย่างมีระบบ

สูตรที่ดีเป็นผลของการคิดที่ดี ไม่ใช่สิ่งที่แทนความคิดนั้นได้ทั้งหมด ดังนั้นต่อให้ในอนาคตผู้อ่านจำสูตรบางตัวไม่ได้ ถ้าระบบการตั้งคำถามยังอยู่ ถ้ายังมองออกว่าโจทย์ต้องแยกกรณีหรือแบ่งเป็นขั้นตอน ถ้ายังรู้ว่าควรเปลี่ยน representation ของปัญหาเมื่อใด ผู้อ่านก็ยังมีเครื่องมือสำคัญที่สุดของ combinatorics ติดตัวอยู่

หากจะสรุปสองบทนี้ในประโยคเดียว ผู้เขียนอยากฝากไว้ว่า

การนับที่ดีไม่เริ่มจากการถามว่า “ต้องใช้สูตรอะไร” แต่เริ่มจากการถามว่า “สิ่งที่กำลังนับคืออะไร” และ “จะจัดระเบียบความเป็นไปได้เหล่านั้นอย่างไร”

ถ้าผู้อ่านเริ่มตอบสองคำถามนี้ได้ชัดขึ้นหลังอ่านจบ บท combinatorics ทั้งหมดนี้ก็ได้บรรลุเป้าหมายสำคัญของมันแล้ว

ภาค VI

ส่งท้าย

Final Draft
Handout version
Phaphontee Yamchote

บทส่งท้าย

เมื่อเดินทางมาถึงตอนท้ายของหนังสือเล่มนี้ ผู้อ่านอาจรู้สึกว่เนื้อหาที่ผ่านมาแตกแขนงออกไปหลายทิศทางพอสมควร เราเริ่มจากภาษาและตรรกศาสตร์ ไปสู่การพิสูจน์ เซต ความสัมพันธ์ และฟังก์ชัน จากนั้นจึงเข้าสู่ recursion, induction, การพิสูจน์ความถูกต้องของอัลกอริทึม, ความสัมพันธ์เวียนเกิด, กราฟ, และ combinatorics ถ้ามองเพียงจากรายชื่อหัวข้อ หนังสือเล่มนี้อาจดูเหมือนเป็นการรวบรวมเนื้อหาหลายบทของวิชาคณิตศาสตร์เข้าด้วยกัน แต่ถ้ามองให้ลึกลงไป สิ่งที่เชื่อมทุกบทเข้าหากันไม่ใช่เพียงการอยู่ในรายวิชาเดียวกัน แต่คือวิธีคิดแบบเดียวกันที่ค่อย ๆ ซัดขึ้นตลอดทาง

วิธีคิดนั้นคือ คณิตศาสตร์ไม่ต่อเนื่องไม่ได้มีหัวใจอยู่ที่การแทนค่าสูตรให้เร็วที่สุด และไม่ได้มีเป้าหมายหลักอยู่ที่การจำเทคนิคเฉพาะข้อให้ได้มากที่สุด แต่มีหัวใจอยู่ที่การนิยามสิ่งที่กำลังพูดถึงให้ชัด อ่านโครงสร้างของปัญหาให้ออก และให้เหตุผลกับสิ่งที่เรากำลังอ้างอย่างตรวจสอบได้ ในบางบท วิธีคิดนี้ปรากฏในรูปของการอ่านอิมพลีเคชันให้ถูก ในบางบทปรากฏในรูปของการแยกสมมติฐานออกจากข้อสรุป ในบางบทปรากฏในรูปของการตั้ง recursion ให้สะท้อนโครงสร้างของปัญหา และในบางบทปรากฏในรูปของการวางแผนการนับอย่างเป็นระบบ แม้บริบทจะเปลี่ยนไป แต่ระเบียบวิธีพื้นฐานกลับเป็นเรื่องเดียวกันเสมอ

สิ่งที่ผู้เขียนอยากให้ผู้อ่านเก็บกลับไปจากหนังสือเล่มนี้ จึงไม่ใช่เพียงรายการคำจำกัดความ ทฤษฎีบท หรือสูตรต่าง ๆ แม้ว่าสิ่งเหล่านั้นจะสำคัญก็ตาม แต่คือความสามารถในการถามคำถามที่ถูกต้องกับปัญหาตรงหน้า เช่น ตอนนี้เรากำลังทำงานกับวัตถุชนิดใด นิยามที่เกี่ยวข้องคืออะไร ข้อความนี้มีรูปแบบใด ถ้าจะพิสูจน์ควรเริ่มจากสมมติอะไร ถ้านับควรแยกกรณีหรือควรแบ่งเป็นขั้นตอน ถ้าปัญหาคับข้องเกินไป มี representation แบบอื่นที่ทำให้มันง่ายขึ้นหรือไม่ คำถามเหล่านี้ต่างหากที่เป็นทักษะระยะยาว และเป็นสิ่งที่ทำให้เนื้อหาในหนังสือไม่กลายเป็นเพียงความรู้เฉพาะวิชาที่จบลงพร้อมภาคการศึกษา

สิ่งที่หนังสือเล่มนี้พยายามฝึก

ถ้าจะสรุปเป้าหมายของหนังสือเล่มนี้ในภาพกว้าง ผู้เขียนคิดว่ามีอยู่สี่เรื่องหลัก

เรื่องแรกคือการฝึกให้ผู้อ่านมองคณิตศาสตร์เป็น ภาษา ไม่ใช่เพียงคลังสูตร ภาษาในที่นี้หมายถึงระบบของสัญลักษณ์ นิยาม ข้อกำหนด และรูปแบบของข้อความที่ต้องอ่านให้เป็น เมื่ออ่านภาษาออก เราจะเริ่มเห็นว่าความ

แตกต่างระหว่าง ``ยกตัวอย่าง" กับ ``พิสูจน์" ต่างกันอย่างไร หรือความแตกต่างระหว่าง ``มีอยู่บางตัว" กับ ``เป็นจริงสำหรับทุกตัว" เปลี่ยนโครงสร้างของคำตอบอย่างไร

เรื่องที่สองคือการฝึกให้ผู้อ่านมองคณิตศาสตร์เป็น *โครงสร้าง* ไม่ใช่เพียงชุดของตัวเลข ในบทเรื่องเซต เราเรียนรู้การจัดวางความคิดเป็นกลุ่มและความสัมพันธ์ ในบทเรื่องกราฟ เราเรียนรู้การมองการเชื่อมโยงในฐานะวัตถุทางคณิตศาสตร์ ในบทเรื่อง recursion และ recurrence เราเรียนรู้การมองกระบวนการที่สร้างตัวเองจากกรณีเล็กกว่า ส่วนใน combinatorics เราเรียนรู้การมองความเป็นไปได้จำนวนมาก ให้กลายเป็นวัตถุที่นับได้อย่างเป็นระบบ ทุกบทจึงกำลังฝึกสายตาแบบเดียวกัน: มองสิ่งที่ดูวุ่นวาย แล้วค่อย ๆ หาโครงสร้างที่ซ่อนอยู่ใต้รายละเอียดเหล่านั้น

เรื่องที่สามคือการฝึกให้ผู้อ่านมองคณิตศาสตร์เป็น *การให้เหตุผล* ไม่ใช่เพียงการได้คำตอบ หนังสือเล่มนี้ให้ความสำคัญมากกับคำถามว่า เรารู้ได้อย่างไรว่าสิ่งที่เขียนนั้นจริง ข้อพิสูจน์ไม่ได้ถูกใส่เข้ามาเพื่อให้เนื้อหาดูเป็นวิชาการมากขึ้น แต่เพราะการพิสูจน์คือรูปแบบสูงสุดของการอธิบายว่า ทำไมข้อสรุปหนึ่งจึงตามมาจากสิ่งที่รู้ก่อนหน้าได้จริง ถ้าผู้อ่านอ่านหนังสือเล่มนี้แล้ว เริ่มรู้สึกไม่พอใจกับคำตอบที่ ``ดูเหมือนจะใช่" แต่ยังไม่รู้ว่ามันใช่เพราะอะไร ผู้เขียนถือว่าหนังสือเล่มนี้ทำหน้าที่ของมันได้แล้วส่วนหนึ่ง

เรื่องสุดท้ายคือการฝึกให้ผู้อ่านมองคณิตศาสตร์เป็น *การออกแบบกระบวนการคิด* นี่คือนิสัยที่เด่นมากในบท recursion และ combinatorics แต่จริง ๆ แล้วปรากฏอยู่ตลอดทั้งเล่ม เราต้องออกแบบว่าจะเริ่มพิสูจน์จากตรงไหน ต้องออกแบบว่าจะเลือกตัวแทนทั่วไปหรือเลือกพยาน ต้องออกแบบว่าจะลดปัญหาเดิมลงเป็นกรณีเล็กกว่าอย่างไร หรือจะวางแผนการนับให้ครบและไม่ซ้ำได้อย่างไร ทักษะเช่นนี้มีคุณค่าเกินกว่าบริบทของวิชาดีสครีต เพราะมันคือทักษะของการจัดการปัญหาที่ซับซ้อนให้กลายเป็นลำดับขั้นของเหตุผลที่ทำงานได้จริง

สิ่งที่ควรติดตัวออกไปหลังอ่านจบ

เมื่อปิดหนังสือเล่มนี้ ผู้เขียนไม่ได้คาดหวังว่าผู้อ่านทุกคนจะจำทฤษฎีบททุกข้อได้ หรือจำสูตรทุกบทได้ครบถ้วน สิ่งนั้นอาจเกิดขึ้นได้กับบางคน แต่ไม่ใช่เป้าหมายสูงสุดของการเรียน สิ่งที่สำคัญกว่าคือ ผู้อ่านควรเริ่มมีนิสัยทางคณิตศาสตร์บางอย่างติดตัวไป

นิสัยแรกคือ ก่อนตอบคำถามใด ให้พยายามนิยามสิ่งที่กำลังพูดให้ชัดเจนเสมอ หลีกเลี่ยงความผิดพลาดในคณิตศาสตร์ และในความจริงแล้วรวมถึงในงานด้านคอมพิวเตอร์ด้วย ไม่ได้เกิดจากการคำนวณผิด แต่เกิดจากการที่สิ่งที่เรากำลังพูดถึงยังไม่ถูกกำหนดให้ชัดเจนตั้งแต่ต้น

นิสัยที่สองคือ เมื่อเจอข้อความหนึ่ง ให้พยายามอ่านรูปของมันให้ออกก่อนว่า มันเป็นข้อความแบบใด มีตัวบ่งปริมาณอะไร มีสมมติฐานหรือข้อสรุปอย่างไร และรูปของข้อความนั้นกำลังบอกวิธีเริ่มพิสูจน์หรือวิธีโต้แย้งมันอย่างไร นิสัยนี้จะทำให้ผู้อ่านไม่มองโจทย์เป็นก้อนที่อึดอัดต่อไป แต่เริ่มแตกมันออกเป็นส่วนที่จัดการได้

นิสัยที่สามคือ เมื่อเจอปัญหาที่ดูยากเกินไปในรูปเดิม ให้ถามว่าเปลี่ยน representation ได้หรือไม่ ในหนังสือเล่มนี้ เราเห็นตัวอย่างเช่นนี้อยู่หลายครั้ง ความสัมพันธ์ถูกมองเป็นกราฟมีทิศทาง กราฟถูกมองผ่าน adjacency matrix

การแจกของเหมือนกันถูกมองเป็นคำตอบจำนวนเต็ม และเส้นทางบนตารางถูกมองเป็นการเลือกตำแหน่งของก้าว
หลายครั้งปัญหาไม่ได้ยากเพราะตัวมันเองซับซ้อนเกินไป แต่ยากเพราะเรากำลังมองมันในภาษาที่ไม่เหมาะกับมัน
ต่างหาก

นิสยที่สี่คือ อย่าพอใจกับการตอบว่า “ใช้สูตรนี้” โดยไม่รู้ว่สูตรนั้นมาจากไหน ถ้าระบบการคิดยังอยู่ แม้สูตรจะ
หล่นหายไปชั่วคราว เรามักสร้างมันกลับขึ้นมาใหม่ได้ แต่ถ้ามีแต่สูตรโดยไม่มีเหตุผลรองรับ ความรู้ทั้งหมดนั้นจะ
เปราะบางมาก และมักใช้ไม่ได้ทันทีเมื่อโจทย์เปลี่ยนหน้าตาไปเพียงเล็กน้อย

จากทฤษฎีไปสู่วิทยาการคอมพิวเตอร์

แม้หนังสือเล่มนี้จะเป็นหนังสือคณิตศาสตร์ แต่ผู้เขียนเชื่อว่าผู้อ่านจำนวนมากอ่านมันด้วยแรงจูงใจที่เชื่อมโยงไปยัง
โลกของวิทยาการคอมพิวเตอร์ และนี่ไม่ใช่เรื่องบังเอิญ เพราะคณิตศาสตร์ทฤษฎีเป็นหนึ่งในภาษาพื้นฐานของวิธี
คิดแบบวิทยาการคอมพิวเตอร์อย่างแท้จริง

ตรรกศาสตร์เชื่อมโยงไปสู่การออกแบบเงื่อนไข การอ่านสเปก และการให้เหตุผลกับโปรแกรม เซตและฟังก์ชันเชื่อมโยงไป
สู่การออกแบบชนิดข้อมูล ความสัมพันธ์ และการสร้าง abstraction ที่ชัดเจน กราฟเชื่อมโยงไปสู่เครือข่าย dependency
การค้นหา และการแทนโครงสร้างที่มีความเชื่อมโยง recursion และ induction เชื่อมไปสู่การออกแบบอัลกอริ
ทึมแบบ divide-and-conquer และการพิสูจน์ความถูกต้องของโปรแกรม ส่วน combinatorics เชื่อมไปสู่การ
วิเคราะห์ search space ความเป็นไปได้ การเข้ารหัส และการประเมินความซับซ้อนของปัญหา

ดังนั้นถ้าจะพูดให้ชัดที่สุด คุณค่าของหนังสือเล่มนี้อาจไม่ได้อยู่ที่การทำให้ผู้อ่าน “เก่งคณิตศาสตร์ทฤษฎี” เท่านั้น
แต่อยู่ที่การทำให้ผู้อ่านค่อย ๆ พัฒนาวินัยทางความคิด ที่จำเป็นต่อการเรียนต่อในรายวิชาอื่นด้วย ไม่ว่าจะเป็นอัล
กอริทึม โครงสร้างข้อมูล ออโตมาตา ฐานข้อมูล หรือแม้แต่การเขียนโปรแกรมในระดับที่ต้องอธิบายความถูกต้อง
ของสิ่งที่ตนเองทำ

หนังสือเล่มนี้ควรพาไปถึงตรงไหน

มีคำถามหนึ่งที่ผู้เรียนมักถามโดยตรงหรือโดยนัยอยู่เสมอว่า เมื่ออ่านหนังสือเล่มนี้จบแล้ว “ควรจะทำอะไรได้” ผู้
เขียนคิดว่าคำตอบที่เหมาะสมไม่ได้อยู่ในรูปของรายการหัวข้อที่จำได้ แต่อยู่ในรูปของความสามารถบางอย่างที่
ควรเกิดขึ้น

ผู้อ่านควรเริ่มอ่านนิยามทางคณิตศาสตร์ได้คล่องขึ้น และรู้ว่าต้องแกะนิยามอย่างไรเมื่อต้องใช้ในการพิสูจน์

ผู้อ่านควรเริ่มอ่านข้อความที่มีตัวบ่งปริมาณ อิมพลีเคชัน หรือไบคอนดิชันนัลได้ดีขึ้น และรู้อารมณ์ของข้อความนั้น
กำลังบังคับรูปของคำตอบไว้อย่างไร

ผู้อ่านควรเริ่มแยกให้ออกว่า อะไรคือการยกตัวอย่าง อะไรคือการให้ intuition และอะไรคือการพิสูจน์ที่สมบูรณ์

จริง

ผู้อ่านควรเริ่มวาง recursion ได้อย่างระมัดระวังมากขึ้น และรู้ว่าการเขียน recursive step ให้ได้ ยังไม่เท่ากับการพิสูจน์ว่ามันถูกต้อง

ผู้อ่านควรเริ่มใช้อุปนัยได้อย่างมีสติว่า กำลังสมมติอะไร และกำลังพยายามพิสูจน์ขั้นถัดไปจากอะไร

ผู้อ่านควรเริ่มมองโจทย์การนับได้ว่า มันเป็นปัญหาของการจัดระเบียบความเป็นไปได้ ไม่ใช่เพียงปัญหาของการหาสูตรสำเร็จรูปให้เจอ

ถ้าอ่านจบแล้วทักษะเหล่านี้เริ่มชัดขึ้น แม้ยังไม่สมบูรณ์ ผู้เขียนก็คิดว่าหนังสือเล่มนี้ทำหน้าที่ของมันได้อย่างมีความหมายแล้ว

ขอปิดท้ายหนังสือ Discrete Math for Computer Students เล่มนี้

สุดท้ายนี้ ผู้เขียนอยากชวนให้ผู้อ่านมองการเรียนคณิตศาสตร์ดิสครีต ไม่ใช่ในฐานะการผ่านวิชาหนึ่งวิชาให้จบ แต่ในฐานะการฝึกตนเองให้คิดอย่างมีระเบียบมากขึ้น บางครั้งการฝึกเช่นนี้อาจช้า อาจรู้สึกว่าการเข้าใจช้ากว่าการจำสูตร หรืออาจรู้สึกว่าการเขียนพิสูจน์ที่ละเอียดละเอียดยากเกินไป แต่การฝึกแบบนี้มีคุณค่าในระยะยาวมาก เพราะมันเปลี่ยนไม่เพียงวิธีตอบโจทย์ แต่เปลี่ยนวิธีที่เรามองปัญหาโดยรวม

ถ้าจะสรุปหนังสือเล่มนี้ในประโยคเดียว ผู้เขียนอยากฝากไว้ว่า

คณิตศาสตร์ดิสครีตไม่ใช่ศิลปะของการหีบสูตรให้ถูกข้อเพื่อนำไปแก้โจทย์เฉพาะเจาะจง แต่เป็นศิลปะของการมองโครงสร้างให้ออก นิยามให้ชัด และให้เหตุผลอย่างตรวจสอบได้

หากผู้อ่านเริ่มรู้สึกกับปัญหาทางคณิตศาสตร์และปัญหาทางคอมพิวเตอร์ว่า ก่อนจะตอบ เราควรนิยามก่อน ควรแยกโครงสร้างก่อน ควรตรวจสอบเหตุผลก่อน และควรถามให้ชัดก่อนว่าเรารู้อะไรอยู่แล้วกับสิ่งใดที่ยังต้องพิสูจน์ นั่นคือสัญญาณที่ดีมากกว่า หนังสือเล่มนี้ได้ทำงานของมันแล้ว

จากตรงนี้ไป เนื้อหาในเล่มอาจจบลง แต่ระเบียบวิธีคิดที่หนังสือพยายามฝึก ไม่ควรจบลงพร้อมกัน เพราะไม่ว่าผู้อ่านจะเดินต่อไปสู่รายวิชาใด หรือปัญหาแบบใด ภาษา โครงสร้าง และการให้เหตุผล จะยังคงเป็นเครื่องมือพื้นฐานที่ติดตัวไปเสมอ

Final Draft
Handout version
Phaphontee Yamchote

เกี่ยวกับผู้เขียน

ปพนธีร์ แยมโซติ เป็นอาจารย์และนักวิจัยที่ทำงานอยู่บนรอยต่อระหว่าง คณิตศาสตร์ วิทยาการคอมพิวเตอร์ และวิทยาการข้อมูล ความสนใจทางวิชาการของเขาครอบคลุมทั้งการให้เหตุผลเชิงคณิตศาสตร์ โครงสร้างไม่ต่อเนื่อง การเขียนโปรแกรม ตลอดจน machine learning, deep learning, และ graph neural networks โดยเฉพาะในบริบทที่ต้องอธิบายทั้งโครงสร้างของปัญหา และความถูกต้องของวิธีการแก้ปัญหาอย่างเป็นระบบ

ปัจจุบันเขากำลังศึกษาระดับปริญญาเอกสาขา Computer Science ที่คณะ ICT มหาวิทยาลัยมหิดล โดยมุ่งเน้นหัวข้อด้าน machine learning และ deep learning สำหรับงานด้าน data science ก่อนหน้านั้นเขาสำเร็จการศึกษาระดับปริญญาโทและปริญญาตรีสาขาคณิตศาสตร์ จากคณะวิทยาศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย พื้นฐานทางคณิตศาสตร์ที่เข้มแข็งนี้มีบทบาทสำคัญต่อวิธีคิดและวิธีอธิบายของเขา โดยเฉพาะการเชื่อมโยงระหว่างนิยาม โครงสร้าง และการพิสูจน์ เข้ากับการออกแบบอัลกอริทึมและการเขียนโปรแกรม

ในด้านการสอน ปพนธีร์ แยมโซติ ปฏิบัติงานในตำแหน่ง Lecturer in Computer Science and Engineering ที่คณะบริหารธุรกิจ มหาวิทยาลัยเอเชียอาคเนย์ และมีประสบการณ์สอนทั้งคณิตศาสตร์ วิทยาการคอมพิวเตอร์ การเขียนโปรแกรม ตลอดจนหัวข้อประยุกต์ด้าน data science และ AI แนวทางการสอนของเขาให้ความสำคัญกับการอธิบายอย่างค่อยเป็นค่อยไป การจัดระเบียบความคิดให้เห็นโครงสร้างของปัญหา และการทำให้ผู้เรียนเข้าใจเหตุผลเบื้องหลังนิยาม ทฤษฎีบท และวิธีพิสูจน์ มากกว่าการจดจำสูตรหรือขั้นตอนแบบแยกส่วน

นอกจากงานสอนแล้ว เขายังมีประสบการณ์ทำงานในสายข้อมูลและ AI ทั้งในงานวิจัยและงานอุตสาหกรรม ไม่ว่าจะเป็น data engineering, machine learning consulting, การพัฒนาโมเดลเชิงพยากรณ์, การทำ optimization, และการฝึกอบรมด้าน machine learning ให้กับองค์กร ประสบการณ์จากทั้งโลกวิชาการและโลกปฏิบัตินี้ ส่งผลโดยตรงต่อวิธีเรียบเรียงหนังสือเล่มนี้ ซึ่งพยายามทำให้คณิตศาสตร์ที่สละสลวยไม่ใช่เพียงวิชาทฤษฎีที่แยกขาดจากการใช้งานจริง แต่เป็นภาษาพื้นฐานของการคิดเชิงคอมพิวเตอร์อย่างแท้จริง

หนังสือเล่มนี้จึงสะท้อนความตั้งใจสำคัญของผู้เขียน คือการทำให้ผู้เรียนมองเห็นว่า คณิตศาสตร์ที่สละสลวยไม่ใช่เพียงการเรียนรู้สัญลักษณ์หรือสูตร แต่เป็นการฝึกนิยามให้ชัด อ่านโครงสร้างให้ออก และให้เหตุผลอย่างตรวจสอบได้ ซึ่งเป็นทักษะพื้นฐานที่สำคัญต่อการเรียนรู้ในศาสตร์คอมพิวเตอร์โดยรวม